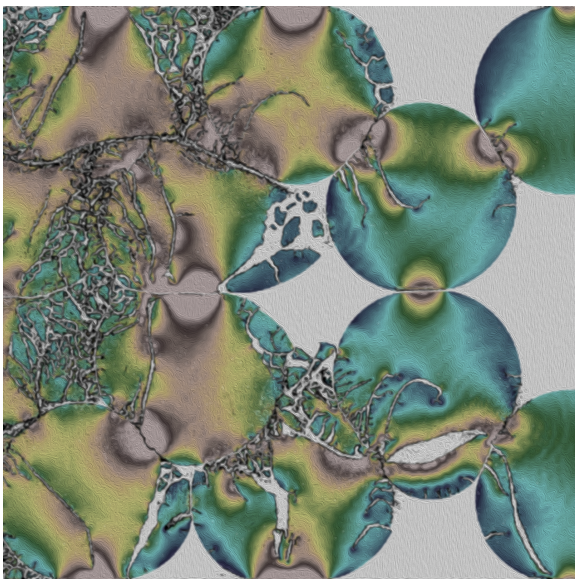
GEOS-MPM Manual

Material Point Method theory, implementation, and ParticleFileWriter reference

Source version: `VERSION_ID = v1.1.0` Generated on 2026-06-03



Stylized two-dimensional disk-compaction fragmentation result.

Core GEOS-MPM development
Michael A. Homel, Ph.D. (LLNL) - GEOS-MPM LEAD Developer
Cameron M. Crook, Ph.D. (LLNL) - GEOS-MPM Developer
Stefan J. Povolny, Ph.D. (LLNL) - GEOS-MPM Developer

GEOS-MPM Contributors:
Jay Appleton, Sohanjit Ghosh, Margariete Malenda, Jacob Nuttal

GEOS Infrastructure Support:
Randy Settgast, Josh White, Matteo Cusini, Ben Corbett, Chris Sherman, Thomas Gazzola, Nicola Castalletto

Programmatic Support:
Eric Herbold, Alan DeHope, Andrew Saab, Kyle Sullivan

Designed for regeneration and future Sphinx conversion

Preface

This starter manual documents the GEOS Material Point Method implementation as it appears in the uploaded source archive. It is designed to be regenerated as the code changes: narrative chapters provide the stable structure, while generated appendices collect solver attributes, ParticleFileWriter parameters, particle fields, event attributes, geometry objects, materials, suites, and post-processing tools extracted from the code and schema documentation.

Generation status. The source archive reports `VERSION_ID = v1.1.0` in `src/VERSION`. The uploaded zip did not include a Git commit hash, so this manual records the archive content rather than a specific commit. Generated tables were built from the C++ wrapper registrations, schema-generated RST files, ParticleFileWriter Python modules, and the suite directories.

What is included. The manual focuses on `SolidMechanics_MPM`, MPM events, the particle mesh and particle-file format, ParticleFileWriter input generation, diagnostics, VisIt/ParaView output pathways, and the existing verification, validation, and example suite reports. LLNL-specific external material-model reports are represented by a placeholder link until those private or out-of-tree reports are added to the documentation tree.

Table 1: Code-derived content counts in this starter manual.

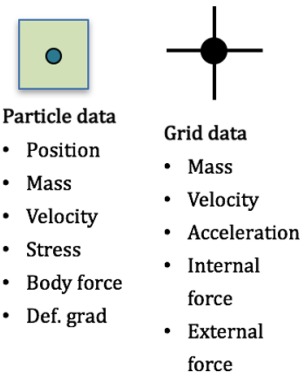
Topic	Notes
Solver attributes	157
MPM event types	18
PFW parameters	112
Particle file fields	44
Geometry objects	42
Geometry wrappers	22
Post-processing scripts	40

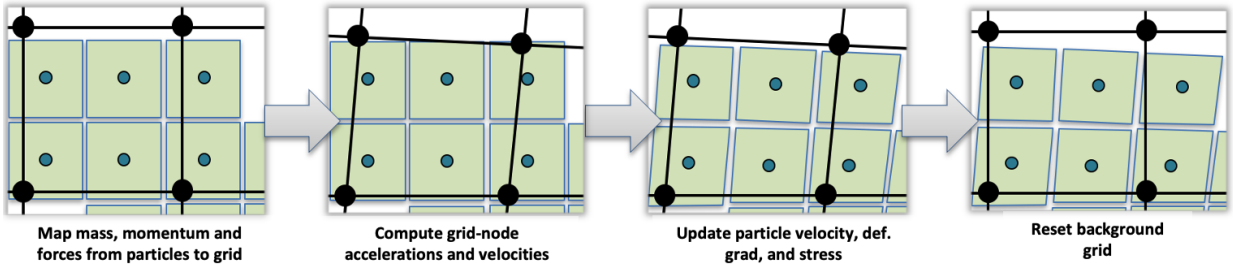
Contents

Introduction

The Material Point Method (MPM) is a hybrid Eulerian–Lagrangian discretization for continuum mechanics. Material is represented by Lagrangian particles that carry mass, position, volume, velocity, stress, deformation gradient, damage, temperature, and other history-dependent internal variables. A background grid is used only as a computational scratchpad for the momentum balance. In a typical explicit step, particle mass, momentum, stress divergence, and body forces are mapped to grid nodes; nodal accelerations and velocities are advanced subject to contact and boundary conditions; particle kinematics and material state are updated; and the grid is reset before the next step. This organization retains the history-tracking advantages of a Lagrangian method while avoiding the mesh entanglement that limits conventional finite elements in very large deformation, evolving contact, fracture, fragmentation, and granular-flow problems.

- Simplified model creation from voxelized data to define particles.
- Motion is solved on a fixed grid, and is robust for large deformation
- Material state is tracked at Lagrangian particles, so complex constitutive models can be used without advection errors in the material state
- The single velocity field automatically produces a non-interpenetrating, no-slip contact condition.





High-level MPM data model and update cycle. Particles carry material state, while the background grid supplies a temporary computational frame for mass, momentum, force, acceleration, boundary-condition, and contact updates.

Relationship to the main GEOS project

The main GEOS open-source project is hosted at the GEOS-DEV repository, https://github.com/GEOS-DEV/GEOS, and its public user/developer documentation is available through the GEOS documentation site, https://geosx-geosx.readthedocs-hosted.com/en/latest/. GEOS-MPM is developed in the same software ecosystem, but this manual is written for the MPM-focused branch rather than for the current public release/develop branch of the main GEOS project. At the time this manual was prepared, the main GEOS release and develop branches contain only an early MPM solver snapshot relative to the code documented here.

The MPM branch intentionally reuses the pieces of GEOS that are valuable for large-deformation particle mechanics: the XML/object catalog and data-repository model, the GEOS internal Cartesian mesh generator, MPI domain decomposition and ghosting infrastructure, event scheduling, restart/output utilities, constitutive-model registration, unit/integrated-test conventions, and the CMake-based build system. Conversely, a lightweight MPM build does not need many finite-element, finite-volume, reservoir-flow, well, compositional-fluid, and solver-stack components required by the GEOS flow, fracture, and poromechanics applications. This branch is therefore maintained to support MPM-specific build tools that remove third-party libraries not required by MPM, while also providing detailed MPM and ParticleFileWriter documentation that would be too specialized for most finite-element/finite-volume GEOS users.

Brief MPM history

The MPM descends from particle-in-cell (PIC) and FLIP ideas developed for computational fluid dynamics, in which particles carry advected state and a grid is used to solve the equations of motion [26, 20]. Sulsky, Chen, and Schreyer introduced the modern MPM formulation for history-dependent solid materials, and Sulsky, Zhou, and Schreyer established its solid-mechanics interpretation for large-deformation calculations [1, 19]. Subsequent developments reduced grid-cell-crossing and quadrature error, generalized particle-to-grid interpolation, and improved contact and fracture treatment. Examples important to GEOS-MPM include GIMP [2], convected particle domain interpolation (CPDI) and CPDI2 [3, 4], Bardenhagen–Guilkey multi-field contact [9], Nairn's explicit-crack and later contact/update algorithms [7, 10], Homel–Herbold damage-field-gradient partitioning [6], and Crook–Homel cohesive-zone treatment for large deformation and damage [23].

GEOS-MPM follows this lineage but is organized around GEOS-style input generation and multi-physics extensibility. The code emphasizes voxelized or image-derived geometries, large-deformation solid dynamics, frictional contact, continuum damage, cohesive fracture, CPDI-domain treatments, and verification through generated particle files and repeatable simulation suites.

Application scope

MPM and MPM-derived methods have been used across several application classes that motivate GEOS-MPM features:

- **Porous and heterogeneous geomaterials.** Image-derived and mesoscale material descriptions can represent pores, inclusions, grains, weak interfaces, and material heterogeneity directly by particles or particle-associated state variables [30, 31].
- **Large deformation with CPDI domain scaling.** CPDI domain scaling controls the onset of numerical fracture in massively deforming, parallel MPM calculations, a key issue for compaction, indentation, fracture, and high-distortion solid-mechanics simulations [5].
- **Fracture and frictional self-contact.** Damage-field-gradient partitioning creates local kinematic enrichment without predefining every contact pair. The method supports dynamic crack propagation,

self-contact, frictional sliding, granular flow, porous compaction, fragmentation, and comminution of brittle materials [6].

- **Mesoscale porous and heterogeneous materials.** DFG-based MPM supports simulations aimed at mesh-independent particle-size distributions in comminution and fragmentation problems, as well as damage evolution in heterogeneous solids [31].
- **Large-deformation cohesive fracture and exact-surface contact.** The Crook–Homel cohesive-zone formulation adds explicit surface geometry, cohesive tractions, and exact-surface contact data paths for problems in which curved interfaces and under-resolved binder or interface layers are important [23].
- **Image-based mesoscale damage in concrete and related composites.** Mesoscale modeling together with X-ray computed micro-tomography motivates the voxelized-data and image-based mesostructure workflows supported by PFW [33].

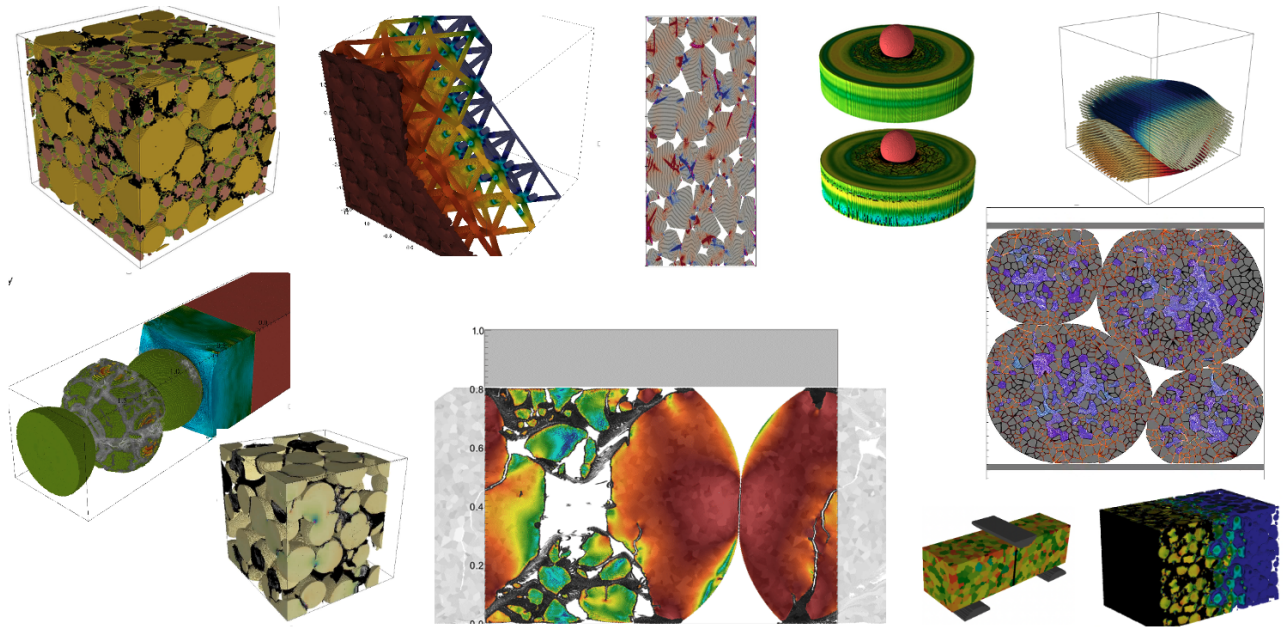


Figure 1: Examples of the type of large-deformation solid-mechanics simulations for which the Material Point Method is well suited: voxelized and image-derived mesostructures, heterogeneous compaction, fracture, contact, indentation, torsion, and coupled damage/flow of fragmented solids.

This manual therefore treats GEOS-MPM as both a production code and a research platform: the same particle-grid structure supports classical MPM, CPDI and B-spline mapping, PIC/FLIP/XPIC/FMPM update options, multi-field and DFG contact, cohesive-zone surfaces, and robustness controls for deletion, deformation-gradient reset, and particle splitting.

Chapter 1

Getting started and install

1.1 Recommended source-tree entry points

The MPM code spans the GEOS solver, the MPM event package, particle mesh generation, the Python ParticleFileWriter, and suite tooling. Table 7.1 lists the source paths used while generating this manual. The most useful initial files are:

- `src/coreComponents/physicsSolvers/solidMechanics/SolidMechanicsMPM.cpp` for solver input wrappers, the explicit step sequence, output controls, and diagnostics.
- `src/coreComponents/physicsSolvers/solidMechanics/kernels/ExplicitMPM.hpp` for the low-level nodal and particle update kernels.
- `src/coreComponents/events/mpmEvents/` for time-dependent MPM events such as deformation updates, stress initialization, pressure loading, material swaps, healing, and cohesive-zone insertion.
- `scripts/preProcessing/materialPointMethodParticleFileWriter/particleFileWriter.py` for generated XML, particle files, batch scripts, and PFW input normalization.
- `scripts/preProcessing/materialPointMethodParticleFileWriter/pfw_geometryObjects.py` and `pfw_materials.py` for geometry and material dictionaries.
- `inputFiles/materialPointMethod/` for direct XML examples outside the PFW workflow.

1.2 LLNL convenience build with `setupMPM`

The archive includes a top-level `setupMPM` script for LLNL Dane and Tuolumne. It can infer the machine from the hostname or accept `-dane` or `-tuolumne`. The script creates local MPM convenience files, configures the machine-specific minimal-TPL host config, and builds the `geosx` target.

Listing 1.1: Typical LLNL setupMPM usage.

```
# Configure and build on Dane.
./setupMPM --dane --bank <lc_bank>

# Configure and build on Tuolumne.
./setupMPM --tuolumne --bank <lc_bank>

# Enable VTK output support in the otherwise minimal MPM build.
./setupMPM --dane --enable-vtk

# Attach out-of-tree/internal constitutive models.
./setupMPM --dane --external-material-models /path/to/material-models

# Rebuild an existing configured tree after source/header edits.
```

```
./setupMPM --dane --rebuild
```

When external material models are used, the external directory should provide either the external-model CMake hook file or a `CMakeLists.txt` that registers the model library. The setup script forwards that location to CMake through the external constitutive-model directory variable.

Listing 1.2: External constitutive-model CMake hook names.

```
GEOSExternalConstitutiveModels.cmake
CMakeLists.txt
geos_register_external_constitutive_models()
GEOS_EXTERNAL_CONSTITUTIVE_MODELS_DIR
```

1.3 Direct CMake build pattern

The suite README also records the Dane minimal-TPL build pattern. Adjust build and install roots to local workspace policy.

Listing 1.3: Manual GEOS-MPM build pattern from the suite workflow.

```
python3 scripts/config-build.py \
  -hc host-configs/LLNL/dane-toss_4_x86_64_ib-gcc@12.1.1-mpm-minimal-tpl.cmake \
  -bt Release \
  -br /usr/workspace/$USER/geos-builds \
  -ir /usr/workspace/$USER/geos-installs

cmake --build \
  /usr/workspace/$USER/geos-builds/build-dane-toss_4_x86_64_ib-gcc@12.1.1-mpm-minimal-tpl-
    release \
  -j

export GEOS_EXECUTABLE=/usr/workspace/$USER/geos-builds/build-dane-toss_4_x86_64_ib-gcc@12
    .1.1-mpm-minimal-tpl-release/bin/geosx
export GEOS_BANK=<your_lc_bank>
```

1.4 Running a PFW-generated case

The ParticleFileWriter workflow creates particle files, GEOS XML inputs, and optionally run scripts. The basic workflow is:

1. Create a user definition file such as `userDefs_$USER.py` with paths to the GEOS executable, run directories, and scheduler defaults.
2. Create a case input named `pfw_input_<case>.py`. The input defines a Python dictionary named `pfw` and typically imports `pfw_geometryObjects.py` and `pfw_materials.py`.
3. Create or use a `runClean_$USER.sh` wrapper and set `fileNames`, `fileLocation`, and `runLocation`.
4. Execute `./runClean_$USER.sh`. Passing an integer requests that the particle file writer use that many MPI tasks under Slurm.

Listing 1.4: Minimal ParticleFileWriter run pattern.

```
cd scripts/preProcessing/materialPointMethodParticleFileWriter
./runClean_$USER.sh        # serial PFW generation and job setup
./runClean_$USER.sh 8      # request eight MPI tasks for PFW generation
```

1.5 Running the example, verification, and validation suites

The parent suite wrappers are `examples/runExamplesSuite`, `verification/runVerificationSuite`, and `validation/runValidationSuite`. They support preflight checks, case listing, run preparation, optional submission, and status/report refresh.

Listing 1.5: Common suite commands.

```
# Static compile/deprecated-keyword check only.
./verification/runVerificationSuite preflight

# List discovered cases; template inputs with XXXX/YYYY/ZZZZ are skipped by default.
./verification/runVerificationSuite list

# Prepare run directories and invoke PFW, but do not submit GEOS jobs.
./verification/runVerificationSuite run \
  --run-root /p/lustre1/$USER/geos-mpm-suite-runs \
  --geos-path "$GEOS_EXECUTABLE" \
  --bank "$GEOS_BANK" \
  --partition pdebug \
  --output-type silo \
  --keep-going

# Refresh status/report after jobs have run.
./verification/runVerificationSuite status \
  --run-root /p/lustre1/$USER/geos-mpm-suite-runs \
  --geos-path "$GEOS_EXECUTABLE"
```

Reports are written to `<run-root>/<suite-name>/suite_report.md` and `suite_report.json`. The source archive also includes generated PDF reports linked in Chapter 5.

Chapter 2

GEOS-MPM theory and solver architecture

2.1 Scope of this chapter

This chapter describes the implementation-level theory needed to read the code and configure cases. It is not a replacement for an MPM textbook. It documents the solver's organization, explicit step sequence, event framework, constitutive-model attachment points, contact/cohesive controls, and output lifecycle as extracted from the current source.

2.2 MPM data model

GEOS-MPM evolves material state on particles while using a background finite-element grid to solve momentum balance. The PFW-generated input normally creates two meshes: an `InternalMesh` named `background` and a `ParticleMesh` named `particles`. The solver target regions include the background mesh region and one or more `ParticleRegion` blocks that bind particle blocks to constitutive material names.

At runtime, particles carry mass, volume, position, velocity, deformation information, material identity, particle type, contact group, surface/cohesive flags, thermal fields, and optional material directions. Grid nodes receive mapped mass, momentum/velocity, internal and external forces, damage gradients, contact/cohesive terms, and optional diagnostic fields. Particle-to-grid and grid-to-particle transfers are governed by particle type, interpolation/update method, CFL and time-step controls, contact settings, and boundary conditions.

2.3 Current time integration status

The solver enumerates quasi-static, implicit-dynamic, and explicit-dynamic modes, but the implemented `solverStep` path in the reviewed source accepts the explicit-dynamic mode. The explicit path controls the stable time step from the CFL factor, material wave speeds, particle/grid geometry, and additional solver limits such as maximum particle velocity and optional artificial viscosity.

2.4 Governing equations at solver level

The core solver loop represents the standard MPM split:

$$\begin{aligned} \dot{\rho} + \rho \nabla \cdot \mathbf{v} &= 0, \\ \rho \dot{\mathbf{v}} &= \nabla \cdot \boldsymbol{\sigma} + \rho \mathbf{b}, \\ \dot{\mathbf{F}} &= \mathbf{L}\mathbf{F}. \end{aligned}$$

Particles store constitutive state and deformation measures; grid nodes provide a transient computational scaffold for velocities, accelerations, forces, and boundary/contact constraints. Material response is supplied through GEOS constitutive models registered in the input `Constitutive` block and assigned through `ParticleRegion` material lists.

2.5 Solver steps

The explicit solver step is the implementation spine of the GEOS-MPM solver. In the reviewed source the public `solverStep` entry point dispatches to an explicit dynamic update; the explicit step then performs the ordered operations in Table 2.1. The ordering follows the standard MPM cycle of particle-grid projection, grid solution, and grid-particle projection introduced by Sulsky, Chen, and Schreyer, with extensions for generalized interpolation, CPDI particle domains, field-gradient partitioning, contact, cohesive zones, prescribed deformation, and GEOS constitutive-model dispatch [1, 2, 3, 5, 6].

Throughout this section, particles are indexed by p, background-grid nodes by I, and multi-material or contact velocity fields by α. The particle position, mass, current volume, velocity, stress, deformation gradient, and velocity gradient are denoted by $\mathbf{x}_p$, m_p, V_p, $\mathbf{v}_p$, $\boldsymbol{\sigma}_p$, $\mathbf{F}_p$, and $\mathbf{L}_p$. The particle-to-node basis value and gradient used by the selected particle type are denoted by N_{Ip} and ∇N_{Ip}. For CPDI particles these symbols represent domain-averaged quantities rather than point samples. The nodal mass, momentum, velocity, acceleration, internal force, and external force are denoted by m_I, $\mathbf{q}_I$, $\mathbf{v}_I$, $\mathbf{a}_I$, $\mathbf{f}_I^{\text{int}}$, and $\mathbf{f}_I^{\text{ext}}$.

Table 2.1: Implementation-level explicit MPM solver-step sequence.

Step	Name	Purpose
1	Resolve explicit-step managers	Acquire mesh, particle, node, partition, constitutive, field, and communication managers used by the step.
2	Initialize and reset explicit-step state	On the first cycle perform one-time solver initialization; every cycle reset nodal and temporary particle fields.
3	Prepare particle topology	Trigger pre-transfer events, optionally split particles, refresh particle maps, ghost data, active-particle lists, and periodic positions.
4	Build neighborhood and contact state	Build neighbor lists, update surface/contact flags, update boundary-condition tables, and initialize CPDI scaling as needed.
5	Populate particle-grid mapping	Compute particle-grid supports, basis values, gradients, CPDI/B-spline data, and compact particle-node maps.
6	Update damage and surface fields	Refresh damage-gradient, crack-tip, SPH-Jacobian, surface-normal, and surface-position data used by fracture/contact models.
7	Update cohesive-zone state	Initialize cohesive reference configuration and enforce cohesive laws, producing particle cohesive forces.

Step	Name	Purpose
8	Compute particle loads and background fields	Evaluate body forces, manufactured-solution loads, surface-tension terms, and prescribed background stresses.
9	Map particle state to grid	Transfer mass, volume, momentum, damage, forces, surface fields, and first moments from particles to active grid fields.
10	Synchronize grid fields across MPI ranks	Communicate nodal reductions on partition boundaries and determine the active nodal field mask.
11	Enforce grid symmetry and normalize grid fields	Apply symmetry projection and normalize surface normals, surface positions, and first moments.
12	Update grid dynamics and contact	Advance nodal momentum/velocity/acceleration and resolve contact between velocity fields.
13	Apply prescribed deformation and boundary conditions	Apply F-table, stress-control, moving/symmetry/contact/outflow boundary, and FMPM boundary corrections.
14	Map grid state back to particles	Update particle velocity, position increment, and velocity gradient using FLIP, PIC, XPIC, or FMPM transfer.
15	Update particle kinematics	Advance deformation gradients, volumes, densities, material directions, surface measures, and kinetic energy.
16	Update constitutive and thermal state	Call GEOS constitutive kernels, update stress/internal variables, thermal state, and wave-speed dependencies.
17	Write optional outputs and compute next stable time step	Emit scheduled histories/plot data and compute the CFL-limited candidate time step.
18	Resize grid and clean particle state	Resize moving domains, remove failed/out-of-range particles, repartition, and reset requested state variables.

The high-level algorithm can be written compactly as Algorithm 2.1. The following subsections expand each line with the implementation-specific equations used in the current solver.

Listing 2.1: High-level explicit GEOS-MPM solver step.

```
explicitStep(t_n, dt, cycle):
  1.  managers = getExplicitStepManagers(domain)
  2.  initializeAndResetExplicitStepState(cycle, managers)
  3.  prepareParticleTopologyForExplicitStep(dt, t_n, cycle, managers)
  4.  buildNeighborhoodAndContactStateForExplicitStep(dt, t_n, cycle, managers)
  5.  populateParticleGridMappingForExplicitStep(particles, nodes)
  6.  updateDamageAndSurfaceFieldsForExplicitStep(domain, particles, nodes)
  7.  updateCohesiveZonesForExplicitStep(dt, domain, particles, nodes)
  8.  computeParticleLoadsAndBackgroundFieldsForExplicitStep(t_n, particles, nodes)
  9.  performParticleToGridForExplicitStep(t_n, cycle, particles, nodes)
  10. syncGridFieldsForExplicitStep(domain, nodes, mesh)
      computeActiveGridFieldsForExplicitStep(domain, nodes, mesh)
  11. enforceGridFieldSymmetryAndNormalize(nodes)
  12. updateGridDynamicsAndContactForExplicitStep(dt, particles, nodes)
  13. applyPrescribedDeformationAndBoundaryConditionsForExplicitStep(dt, t_n, cycle)
  14. gridToParticle(dt, particles, nodes, domain, mesh)
  15. updateParticleKinematicsForExplicitStep(dt, t_n, particles, partition)
  16. updateConstitutiveAndThermalStateForExplicitStep(dt, particles)
  17. dt_next = writeOutputsAndComputeStableTimeStepForExplicitStep(t_n, dt, cycle)
  18. resizeGridAndCleanParticlesForExplicitStep(dt, domain, particles, nodes, partition)
  return dt_next
```

2.5.1 Step 1: resolve explicit-step managers

The first step is a bookkeeping stage that binds the solver object to the concrete GEOS objects needed by the explicit update: the partition, the particle mesh body, the background mesh level, the `ParticleManager`, and the background `NodeManager`. It also determines whether periodic partitioning is active. Although this stage does not modify physics state, it establishes the discrete spaces for the rest of the step:

$$\mathcal{P}_h = \{p\}_{p=1}^{N_p}, \qquad \mathcal{G}_h = \{I\}_{I=1}^{N_I}, \qquad \mathcal{V}_h = \{(I,\alpha) : I \in \mathcal{G}_h,\ \alpha \in \mathcal{A}_I\}, \tag{2.1}$$

where $\mathcal{A}_I$ is the set of active velocity/contact fields stored at node I. The multi-field view is important for contact and fracture: conventional single-field MPM enforces velocity continuity through the grid, while crack/contact algorithms introduce distinct local velocity fields to permit discontinuities [7, 6].

Listing 2.2: Manager resolution.

```
resolve_managers(domain):
  partition = domain.partition
  particleMesh = find mesh body that owns the particle region
  meshLevel = particleMesh.background_mesh_level
  particles = meshLevel.particle_manager
  nodes = meshLevel.node_manager
  periodic = partition.has_periodic_partitions()
  return partition, particles, meshLevel, nodes, periodic
```

2.5.2 Step 2: initialize and reset explicit-step state

On the first cycle, the solver performs one-time setup. This includes construction of neighbor-list storage, local and global domain extents, element spacings, boundary and buffer node sets, particle fields, history-file handles, contact velocity-field counts, constitutive-model dependency arrays, and material/contact coefficient tables. If a neighbor radius is not supplied, the implementation chooses a grid-diagonal length scale, with a plane-strain specialization when appropriate. This initialization fixes the discrete length scales later used by neighbor searches, surface detection, contact, and CFL estimates.

Every cycle, all grid-resident temporary quantities are reset before the particle-to-grid transfer. The reset implements the assumption that the background grid is a transient computational scaffold rather than a history-carrying mesh:

$$m_{I\alpha} = 0, \quad \mathbf{q}_{I\alpha} = \mathbf{0}, \quad \mathbf{f}_{I\alpha}^{\text{int}} = \mathbf{0}, \quad \mathbf{f}_{I\alpha}^{\text{ext}} = \mathbf{0}, \quad \mathbf{a}_{I\alpha} = \mathbf{0}, \quad \mathbf{v}_{I\alpha} = \mathbf{0}. \tag{2.2}$$

Auxiliary fields such as material volume, damage, maximum damage, surface normal, surface position, center of mass, center of volume, and contact force are similarly cleared or marked inactive. This follows the classical MPM practice in which particles carry the material state and the grid is reinitialized each time step [1, 14].

Listing 2.3: State reset.

```
initialize_and_reset(cycle):
  if cycle == 0:
    initialize_neighbor_lists_and_extents()
    initialize_particle_fields_and_history_files()
    initialize_constitutive_dependencies()
    allocate_contact_velocity_fields()
  set_grid_field_labels_from_solver_configuration()
  zero_grid_mass_momentum_forces_and_auxiliary_fields()
```

2.5.3 Step 3: prepare particle topology

Before particle-grid transfers, the solver refreshes the topology of the particle set. Time-windowed MPM events are triggered when their activation criteria are satisfied, particle subdivision is performed when the particle-splitting option is enabled, and the particle manager updates internal maps and block ownership. In parallel runs, particles needed by neighbor-list or contact operations are ghosted across subdomain boundaries. The active-particle index set

$$\mathcal{P}_{\text{act}} = \{p \in \mathcal{P}_h : p \text{ is not deleted and belongs to an active region}\} \tag{2.3}$$

is then reconstructed. Periodic ghost particles have their centers corrected so that distances used by mapping, neighbor lists, and contact are computed in the intended periodic image.

This step is algorithmically separate from shape-function construction because the support of CPDI and B-spline particles depends on both particle topology and the background cell topology. The separation also keeps event-driven operations, such as material swaps or particle transformations, ahead of force and mapping calculations.

Listing 2.4: Particle topology preparation.

```
prepare_particle_topology(dt, t_n, cycle):
  if events are enabled:
    trigger_events(t_n, dt, cycle)
  if particle_splitting is enabled:
    split_particles_that_satisfy_refinement_criteria()
  particles.updateMaps()
  if parallel_neighborhood_data_are_needed:
    ghost_particles_and_periodic_images()
  activeParticles = compute_active_particle_indices()
```

2.5.4 Step 4: build neighborhood and contact state

Classical MPM formulations deliberately avoid the persistent element connectivity of a purely Lagrangian finite-element mesh. Particles carry mass, momentum, deformation, and internal variables, while the Eulerian background grid is reset each step; the only coupling required by the basic method is the compact particle-to-grid and grid-to-particle support of the selected shape functions [1, 2]. GEOS-MPM adds a particle-particle neighbor relation when algorithms require nonlocal particle information. This relation should not be interpreted as a material mesh: it is a transient graph rebuilt from particle positions and ghost particles when needed. Its primary motivation is damage-field-gradient (DFG) partitioning, in which a nonlocal damage gradient is used to split particles and grid nodes into opposite velocity fields so that damaged material can slip, separate, and later contact automatically [6].

For an active particle p, the neighbor graph is

$$\mathcal{N}_p = \{q \in \mathcal{P}_{\text{act}} : \|\mathbf{x}_q - \mathbf{x}_p\| \le r_n\}, \tag{2.4}$$

where r_n is the solver neighbor radius. In parallel runs, the set also includes ghost particles whose owners lie on neighboring MPI ranks, and periodic images are shifted before the distance test so that the closest periodic copy is used. The graph is stored as an ordered variable-to-many particle relation. Each entry records the neighbor particle's region, subregion, and local particle index, so kernels can access fields from different particle subregions without copying all particle data into a single flat material array.

Use in damage-field-gradient contact. DFG contact first constructs a particle-scale scalar damage field and then computes its normalized kernel gradient over the neighbor graph. In the implementation, explicit surface flags and cohesive-zone flags may be treated as unit damage in this gradient construction,

so painted surfaces and inserted cohesive surfaces can participate in the same partitioning logic. Abstractly, for a scalar indicator d_q and compact kernel $W(r; r_n)$, the particle damage gradient is evaluated in the normalized form

$$\nabla d_p \approx \frac{\sum_{q\in\mathcal{N}_p} V_q d_q \nabla W_{pq}}{\sum_{q\in\mathcal{N}_p} V_q W_{pq}} - \frac{\sum_{q\in\mathcal{N}_p} V_q d_q W_{pq}}{\left(\sum_{q\in\mathcal{N}_p} V_q W_{pq}\right)^2} \sum_{q\in\mathcal{N}_p} V_q \nabla W_{pq}, \qquad W_{pq} = W(\|\mathbf{x}_q - \mathbf{x}_p\|; r_n). \tag{2.5}$$

The grid receives a representative damage-gradient vector, and particle-to-node mappings are assigned to one of the two DFG velocity fields according to the sign of the alignment between the particle mapping normal and the grid damage gradient. This converts an evolving diffuse damage band, or a painted explicit surface, into a local kinematic enrichment without requiring the user to enumerate every possible contact pair in advance. The same neighbor infrastructure therefore supports automatic self-contact, automatic fracture insertion with slip and separation, nonlocal surface detection and healing, crack-tip metrics, SPH-style Jacobian or overlap corrections, L-bar and directional overlap-correction options, and surface-tension-style particle interactions.

Kernel radius and related length scales. The default neighbor radius is the diagonal of one background cell, using the in-plane diagonal for plane-strain calculations,

$$r_{n,0} = \begin{cases} \sqrt{h_x^2 + h_y^2}, & \text{plane strain,} \\ \sqrt{h_x^2 + h_y^2 + h_z^2}, & \text{3D.} \end{cases} \tag{2.6}$$

The user input `neighborRadius` is interpreted as follows: zero selects this default, a positive value is used directly, and a negative value $-a$ requests $a r_{n,0}$. Thus `neighborRadius=-1.5` gives a radius equal to 1.5 cell diagonals. The same radius is used in several parts of the MPM solver: it defines the halo thickness for particle ghosting, the distance cutoff for particle-neighbor graph construction, the support of the compact nonlocal kernel used by DFG and SPH-type operations, the periodic ghost-image correction offset, and several dimensionless thresholds that are expressed relative to the neighborhood length. Choosing r_n therefore changes both cost and regularization length; it should usually be varied only as part of a resolution or sensitivity study.

The compact kernel used by these nonlocal particle operations is

$$W(r; r_n) = C_d \left[1 - 3\left(\frac{r}{r_n}\right)^2 + 2\left(\frac{r}{r_n}\right)^3\right], \quad 0 \le r < r_n, \qquad W = 0 \quad r \ge r_n, \tag{2.7}$$

with a two-dimensional plane-strain normalization and a three-dimensional normalization. Because the code normalizes scalar fields by the local kernel-weighted particle volume, the method is less sensitive to missing neighbors near free surfaces than an unnormalized convolution, although surface particles still require careful interpretation.

Binning construction. A naive all-pairs neighbor search is unacceptable for production calculations because it scales as $O(N_p^2)$ per rank. GEOS-MPM therefore performs a bin sort on the local partition expanded by r_n. The bin width is

$$h_{\text{bin}} = m_b r_n, \tag{2.8}$$

where `binSizeMultiplier` $= m_b \ge 1$. Larger bins reduce the number of bins but increase the number of candidate particles per bin; smaller bins reduce candidates but increase bin-management overhead. In plane strain the binning has one bin through the out-of-plane direction; otherwise the bin lattice is three-dimensional.

The implementation first counts particles per bin, resizes the bin arrays, and then populates the bins. It then performs two passes over active particles. The first pass counts the number of neighbors of each active particle so the variable-length neighbor relation can be resized exactly. The second pass repeats the same local bin query and fills the region, subregion, and particle-index arrays. The search for particle p only visits bins intersecting the sphere of radius r_n about $\mathbf{x}_p$, and each candidate is accepted by the exact distance test $\|\mathbf{x}_q - \mathbf{x}_p\|^2 \leq r_n^2$.

Listing 2.5: Particle-neighbor list construction by radius-binning.

```
build_particle_neighbor_lists(radius = r_n):
  exchange or wrap ghost particles over a halo of thickness r_n
  define an expanded local search box [x_min - r_n, x_max + r_n]
  bin_width = binSizeMultiplier * r_n
  create a 2D or 3D bin lattice over the expanded box

  for each particle q in each particle subregion:
      append q to its spatial bin

  for each active particle p:
      count candidates q in bins intersecting the radius-r_n ball about p
      keep candidates satisfying |x_q - x_p|^2 <= r_n^2
      resize neighbor relation for p to the counted capacity

  for each active particle p:
      repeat the same bin query and fill
      (neighborRegion, neighborSubRegion, neighborIndex)
```

Parallel and GPU scalability. The neighbor list is one of the few steps in GEOS-MPM that introduces particle-particle rather than particle-grid communication. In MPI, the dominant additional requirements are the exchange of ghost particles within r_n of partition boundaries and the use of periodic image corrections before distances are evaluated. Larger r_n values increase ghost volume, memory footprint, and candidate counts. On GPUs, the main challenges are variable-length neighbor lists, irregular memory access across particle subregions, dynamic resizing of array-of-arrays storage, and race-free bin population. The current approach favors a robust two-pass construction: determine capacities first, then fill. This avoids over-allocation and makes the downstream DFG/contact kernels operate on compact neighbor arrays, at the cost of rebuilding and traversing the bins every step for neighbor-dependent calculations. The added cost is justified when DFG, automatic contact, automatic fracture, SPH corrections, or related nonlocal options are active; for standard single-field MPM without these features the neighbor list can remain disabled.

The contact state is updated after the neighbor graph is available. Boundary-condition tables are advanced, contact flags and contact-group metadata are refreshed, damaged-particle surface data may be suppressed, and initial CPDI domain-scaling factors may be initialized. Domain scaling is included in this step because CPDI particle domains can become distorted in large tensile deformation; the Homel-Brannon-Guilkey domain-scaling strategy controls the onset of numerical fracture in parallel CPDI calculations [5].

Listing 2.6: Neighborhood and contact state update.

```
build_neighborhood_and_contact_state():
  if neighbor_lists_are_required:
    build_particle_neighbor_lists(radius = neighborRadius)
  if surface_detection_or_healing_requires_neighbors:
    update_surface_flags_from_kernel_volume_neighborhoods()
```

```
  update_boundary_condition_tables()
  update_contact_flags_and_contact_group_state()
  disable_inappropriate_surface_data_on_failed_particles()
  if cycle == 0 and CPDI_domain_scaling_enabled:
    initialize_CPDI_domain_scaling()
```

2.5.5 Step 5: populate particle-grid mapping

For each active particle, the solver computes a compact list of grid nodes in the particle support together with N_{Ip} and ∇N_{Ip}. These arrays are the quadrature bridge between Lagrangian material points and the Eulerian background grid. They determine mass projection, force integration, velocity-gradient reconstruction, and particle advection.

For a single-point particle in a hexahedral background cell, the implementation uses the standard trilinear basis. If $\boldsymbol{\xi}_p = (\xi_p, \eta_p, \zeta_p)$ is the local coordinate of the particle in the cell and I corresponds to signs $s_I^d \in \{-1, 1\}$, then

$$N_{Ip} = \frac{1}{8}(1 + s_I^1\xi_p)(1 + s_I^2\eta_p)(1 + s_I^3\zeta_p), \qquad \nabla N_{Ip} = \mathbf{J}^{-T}\nabla_{\boldsymbol{\xi}} N_I(\boldsymbol{\xi}_p). \tag{2.9}$$

The B-spline option uses a tensor product of cubic cardinal B-splines over a $4 \times 4 \times 4$ node stencil. Its smoother basis reduces cell-crossing noise and quadrature artifacts relative to piecewise-linear point sampling, consistent with the GIMP and implementation-choice literature [2, 13, 14].

For CPDI, the code interprets each particle as a convected parallelepiped with corner positions

$$\mathbf{x}_{pc} = \mathbf{x}_p + s_c^1\mathbf{r}_{p1} + s_c^2\mathbf{r}_{p2} + s_c^3\mathbf{r}_{p3}, \qquad s_c^d \in \{-1, 1\}. \tag{2.10}$$

The effective CPDI basis value is the average of background-grid basis values at the particle corners,

$$\bar{N}_{Ip} = \frac{1}{8}\sum_{c=1}^{8} N_I(\mathbf{x}_{pc}), \tag{2.11}$$

while the effective gradient is assembled from corner weights

$$\bar{\nabla N}_{Ip} = \sum_{c=1}^{8} N_I(\mathbf{x}_{pc})\,\boldsymbol{\alpha}_{pc}, \tag{2.12}$$

$$\boldsymbol{\alpha}_{pc} = \frac{s_c^1(\mathbf{r}_{p2} \times \mathbf{r}_{p3}) + s_c^2(\mathbf{r}_{p3} \times \mathbf{r}_{p1}) + s_c^3(\mathbf{r}_{p1} \times \mathbf{r}_{p2})}{8\,\mathbf{r}_{p1} \cdot (\mathbf{r}_{p2} \times \mathbf{r}_{p3})}. \tag{2.13}$$

This is the implementation form of a domain-averaged CPDI transfer; it tracks the convected particle domain more accurately than undeformed point-sample MPM for massive deformation [3]. Duplicate node entries generated from multiple particle corners are coalesced by summing weights and gradients before the transfer kernels are launched.

Listing 2.7: Particle-grid mapping.

```
for p in activeParticles:
  if particleType[p] == SinglePoint:
    compute 8 trilinear basis values and gradients
  else if particleType[p] == SinglePointBSpline:
    compute 64 cubic B-spline values and gradients
  else if particleType[p] == CPDI:
    compute corner positions and CPDI-averaged basis/gradient
  coalesce_duplicate_grid_nodes_in_particle_support(p)
```

2.5.6 Step 6: update damage and surface fields

This step computes fields that mediate dynamic fracture, free-surface detection, and contact partitioning. When damage partitioning is active, a kernel-smoothed damage field $d(\mathbf{x})$ is constructed from particle damage variables d_p and differentiated to obtain a local material-interface indicator,

$$d(\mathbf{x}) \approx \frac{\sum_p m_p d_p W(\mathbf{x}-\mathbf{x}_p,h)}{\sum_p m_p W(\mathbf{x}-\mathbf{x}_p,h)}, \qquad \nabla d(\mathbf{x}) \approx \frac{\sum_p m_p d_p \nabla W(\mathbf{x}-\mathbf{x}_p,h)}{\sum_p m_p W(\mathbf{x}-\mathbf{x}_p,h)}. \tag{2.14}$$

The resulting gradient is projected to the grid and used by field-gradient partitioning logic to split nodal velocity fields along damage-induced discontinuities. This follows the Homel-Herbold idea that the gradient of a kernel-based damage field can define dynamically generated contact pairs without constructing explicit crack surfaces [6]. When explicit crack-style data are used, the design is conceptually related to Nairn's CRAMP approach, in which special nodes carry multiple velocity fields to represent displacement discontinuities [7].

Surface normals and surface positions are updated from particle and grid data. A representative MPM free-surface normal estimate is

$$\mathbf{n}_p^{\mathrm{raw}} \propto -\sum_{q\in\mathcal{N}_p} V_q \nabla W(\mathbf{x}_p-\mathbf{x}_q,h), \qquad \mathbf{n}_p = \frac{\mathbf{n}_p^{\mathrm{raw}}}{\|\mathbf{n}_p^{\mathrm{raw}}\|}, \tag{2.15}$$

with solver-specific thresholds determining whether the particle is considered a free-surface, damaged-surface, or interior particle. The code also computes SPH-style Jacobian data and crack-tip distances when the corresponding fracture options are active.

Listing 2.8: Damage and surface field update.

```
if damage_partitioning_enabled:
  compute_particle_damage_gradient_from_neighbors()
if crack_tip_detection_enabled:
  compute_distance_to_crack_tip()
if surface_or_tension_model_requires_SPH_data:
  compute_particle_SPH_jacobian()
project_damage_gradient_to_grid()
compute_particle_field_mappings()
if surface_detection_enabled:
  update_particle_surface_normals_and_positions()
```

2.5.7 Step 7: update cohesive-zone state

If cohesive zones are configured, the solver resets the particle cohesive force and then initializes or updates cohesive interfaces. During initialization, surface flags and normals are projected to the grid, and a reference cohesive configuration is established. During the update, the cohesive law maps an interface opening/sliding displacement to a traction and accumulates equal-and-opposite particle forces.

A generic cohesive-zone update can be written as

$$\boldsymbol{\delta}_p^{\mathrm{cz}} = [\![\mathbf{u}]\!]_p\,, \tag{2.16}$$

$$\mathbf{t}_p^{\mathrm{cz}} = \mathcal{T}\Big(\boldsymbol{\delta}_p^{\mathrm{cz}}, \boldsymbol{\kappa}_p;\ \text{law parameters}\Big)\,, \tag{2.17}$$

$$\mathbf{f}_p^{\mathrm{cz}} = A_p^{\mathrm{cz}} \mathbf{t}_p^{\mathrm{cz}}, \tag{2.18}$$

where $\boldsymbol{\kappa}_p$ denotes cohesive history variables and A_p^{cz} is an effective interface area. The exact law is supplied by the configured cohesive-zone model, while the solver controls when the reference configuration is created and how its forces are added to the external-force transfer.

Listing 2.9: Cohesive-zone update.

```
set particleCohesiveForce[p] = 0 for all active p
if cohesive zones have uninitialized regions:
  reset cohesive surface flags
  project particle surface normals to the grid
  initialize cohesive reference configuration
if cohesive zones are active:
  enforce cohesive law and accumulate particle cohesive forces
```

2.5.8 Step 8: compute particle loads and background fields

The particle load stage assembles force-like quantities that will be transferred to the grid in Step 9. The solver computes prescribed body forces, manufactured-solution body-force terms, surface-tension curvature terms, cohesive forces from Step 7, and background stresses associated with controls such as borehole pressure or confining pressure. At the particle level, the non-internal force contribution can be viewed as

$$\mathbf{f}_p^{\mathrm{load}} = m_p \mathbf{b}_p + \mathbf{f}_p^{\mathrm{traction}} + \mathbf{f}_p^{\mathrm{cz}} + \mathbf{f}_p^{\mathrm{st}}, \tag{2.19}$$

where $\mathbf{f}_p^{\mathrm{st}}$ may contain curvature-dependent surface-tension forces. Background stress fields enter the internal-force calculation through an effective stress difference. This allows the same MPM divergence operator to represent material stress, manufactured solutions, and selected externally prescribed stress fields.

Listing 2.10: Particle load calculation.

```
compute_body_force_on_particles()
if manufactured_solution_enabled:
  add_manufactured_solution_body_force(t_n)
if surface_tension_enabled:
  compute_curvature_and_surface_tension_force()
if background_pressure_or_stress_enabled:
  compute_background_stress_field()
```

2.5.9 Step 9: map particle state to grid

The particle-to-grid transfer is the discrete weak-form assembly for the explicit grid solve. For each active particle, each mapped node I, and each active velocity field α, the code accumulates lumped nodal mass, volume, momentum, damage, forces, surface fields, and first moments. In the simplest single-field notation,

$$m_I = \sum_p m_p N_{Ip}, \tag{2.20}$$

$$V_I = \sum_p V_p N_{Ip}, \tag{2.21}$$

$$\mathbf{q}_I = \sum_p m_p \mathbf{v}_p N_{Ip}, \tag{2.22}$$

$$d_I^{\mathrm{mass}} = \sum_p m_p d_p N_{Ip}. \tag{2.23}$$

The force assembly is

$$\mathbf{f}_I^{\text{ext}} = \sum_p \mathbf{f}_p^{\text{load}} N_{Ip}, \tag{2.24}$$

$$\mathbf{f}_I^{\text{int}} = -\sum_p V_p \left(\boldsymbol{\sigma}_p - \boldsymbol{\sigma}_p^{\text{bg}} - q_p \mathbf{I} \right) \nabla N_{Ip}, \tag{2.25}$$

where q_p denotes the artificial-viscosity pressure-like term when enabled. Equation (2.25) is the MPM quadrature form of the stress-divergence contribution in the weak momentum balance. It is the same fundamental particle quadrature used in classical MPM, GIMP, and CPDI, with differences only in the definition of N_{Ip} and ∇N_{Ip} [1, 2, 3, 13].

The solver additionally maps surface normals, surface positions, maximum damage, centers of mass, and centers of volume. In parallel/GPU execution, these are accumulated with atomic additions or reductions. If compact coalesced support arrays are enabled, duplicate particle-node contributions are summed first to reduce redundant atomic operations.

Listing 2.11: Particle-to-grid transfer.

```
for p in activeParticles:
  for each mapped node I in support(p):
    for each active velocity field alpha of particle p:
      m[I,alpha]      += m_p * N_Ip
      volume[I,alpha] += V_p * N_Ip
      q[I,alpha]      += m_p * v_p * N_Ip
      f_ext[I,alpha]  += particle_load_p * N_Ip
      f_int[I,alpha]  -= V_p * effective_stress_p * grad_N_Ip
      damage[I,alpha] += m_p * damage_p * N_Ip
      map_surface_and_moment_fields_if_enabled()
```

2.5.10 Step 10: synchronize grid fields across MPI ranks

After local assembly, nodal quantities shared by neighboring MPI ranks are reduced. Additive fields, such as mass, material volume, momentum, internal force, external force, contact-surface mass, surface normals, surface positions, centers of mass, and centers of volume, use sum reductions. Maximum-damage fields use maximum reductions. The global active-field mask is then computed from the reduced mass:

$$\chi_{I\alpha} = \begin{cases} 1, & m_{I\alpha} > m_{\min}, \\ 0, & \text{otherwise}, \end{cases} \tag{2.26}$$

where $m_{\min}$ is the solver's small-mass threshold. The mask is also synchronized so that every rank that touches a shared node has a consistent view of active velocity fields. This reduction stage is essential for the conservation properties of MPM in parallel because mass, momentum, and force contributions are assembled from particle ownership rather than from a single owner of each nodal support.

Listing 2.12: Grid synchronization.

```
sum_reduce(gridMass, gridVolume, gridMomentum,
           gridInternalForce, gridExternalForce,
           gridSurfaceMass, gridSurfaceNormal, gridSurfacePosition,
           gridCenterOfMass, gridCenterOfVolume)
max_reduce(gridMaxDamage)
active[I,alpha] = gridMass[I,alpha] > smallMass
max_reduce(active)
```

2.5.11 Step 11: enforce grid symmetry and normalize grid fields

Before advancing nodal dynamics, the solver projects selected vector fields onto symmetry-compatible subspaces and normalizes accumulated quantities. Symmetry projection removes the forbidden normal component on symmetry boundaries,

$$\mathbf{w}_I \leftarrow \mathbf{w}_I - (\mathbf{w}_I \cdot \mathbf{n}_\Gamma)\mathbf{n}_\Gamma, \qquad I \in \Gamma_{\text{sym}}, \tag{2.27}$$

for fields such as surface normals, center of mass, and center of volume. Surface normals and positions are then normalized:

$$\mathbf{n}_I = \frac{\mathbf{n}_I^{\text{raw}}}{\|\mathbf{n}_I^{\text{raw}}\|}, \qquad s_I = \frac{s_I^{\text{raw}}}{m_I^{\text{surf}}}, \tag{2.28}$$

with zero values assigned below threshold. Centers of mass and centers of volume are similarly divided by the appropriate mass or material-volume measures. This operation turns conserved sums from Step 9 into nodal geometric fields used by contact, surface tension, and diagnostics.

Listing 2.13: Symmetry projection and normalization.

```
project_surface_normals_and_moments_on_symmetry_boundaries()
for each active grid field:
  if norm(surfaceNormalRaw) > tolerance:
    surfaceNormal = surfaceNormalRaw / norm(surfaceNormalRaw)
  if surfaceMass > tolerance:
    surfacePosition = surfacePositionRaw / surfaceMass
  normalize_center_of_mass_and_center_of_volume()
```

2.5.12 Step 12: update grid dynamics and contact

The grid trial update advances each active nodal velocity field using the assembled forces:

$$\mathbf{a}^*_{I\alpha} = \frac{\mathbf{f}^{\text{int}}_{I\alpha} + \mathbf{f}^{\text{ext}}_{I\alpha}}{m_{I\alpha}}, \tag{2.29}$$

$$\Delta\mathbf{v}^*_{I\alpha} = \Delta t\, \mathbf{a}^*_{I\alpha}, \tag{2.30}$$

$$\mathbf{q}^*_{I\alpha} = \mathbf{q}^n_{I\alpha} + \Delta t \left(\mathbf{f}^{\text{int}}_{I\alpha} + \mathbf{f}^{\text{ext}}_{I\alpha}\right), \tag{2.31}$$

$$\mathbf{v}^*_{I\alpha} = \frac{\mathbf{q}^*_{I\alpha}}{m_{I\alpha}}. \tag{2.32}$$

This is the explicit lumped-mass update underlying FLIP-MPM methods [1, 12]. Inactive fields are zeroed except where momentum is intentionally preserved for conservation checks.

Contact is then evaluated between pairs of velocity fields (α, β) at a node. The solver first determines whether the pair should be treated as separable, cohesive, self-contact, or no-slip based on mass, surface normals, damage fields, damage-gradient partitioning, contact groups, cohesive-zone flags, and quality thresholds. Contact normals may be constructed from surface-normal differences, mass-weighted normals, larger-mass normals, mixed rules, aligned rules, or classifier-like logistic rules. Field-gradient partitioning and explicit-crack MPM both motivate this multi-field contact view [7, 6].

For a pair of active fields, define the common nodal velocity

$$\mathbf{v}_I^{\alpha\beta} = \frac{m_{I\alpha}\mathbf{v}^*_{I\alpha} + m_{I\beta}\mathbf{v}^*_{I\beta}}{m_{I\alpha} + m_{I\beta}}. \tag{2.33}$$

The force that would bring field α to the common velocity over one step has normal and tangential components

$$f_n = \frac{m_{I\alpha}}{\Delta t}\left(\mathbf{v}_I^{\alpha\beta} - \mathbf{v}_{I\alpha}^{*}\right)\cdot\mathbf{n}, \tag{2.34}$$

$$f_{t,k} = \frac{m_{I\alpha}}{\Delta t}\left(\mathbf{v}_I^{\alpha\beta} - \mathbf{v}_{I\alpha}^{*}\right)\cdot\mathbf{s}_k, \qquad k = 1, 2. \tag{2.35}$$

If the pair is not separable, the full common velocity is enforced. If the pair is separable, normal contact is activated by the configured approach/gap criterion, overlap correction may add a limited compressive correction, and tangential force is bounded by the Coulomb cone

$$\|\mathbf{f}_t\| \le \mu |f_n|. \tag{2.36}$$

The contact force is applied as an equal-and-opposite pair, preserving pairwise nodal momentum. This design follows the Bardenhagen-Brackbill-Sulsky and Bardenhagen-Guilkey contact algorithms, including normal-traction-based separation logic, frictional sliding, and impulse limiting for suspect grid information [8, 9].

Listing 2.14: Grid dynamics and contact.

```
for each active field (I, alpha):
  totalForce = internalForce[I,alpha] + externalForce[I,alpha]
  acceleration[I,alpha] = totalForce / mass[I,alpha]
  momentum[I,alpha] += dt * totalForce
  velocity[I,alpha] = momentum[I,alpha] / mass[I,alpha]

for each node I and each active field pair (alpha, beta):
  determine_contact_normal_and_separability()
  compute_common_velocity_and_trial_contact_impulse()
  apply_gap_overlap_and_friction_limits()
  apply_equal_and_opposite_contact_forces()
  update nodal acceleration, momentum, deltaVelocity, and velocity
```

2.5.13 Step 13: apply prescribed deformation and boundary conditions

After the unconstrained grid/contact update, prescribed kinematics and essential boundary conditions are imposed. The solver interpolates F-table and prescribed-F-table data unless all active directions are under stress control. A prescribed macroscopic deformation gradient $\bar{\mathbf{F}}(t)$ or velocity gradient $\bar{\mathbf{L}}(t)$ induces affine boundary velocities of the form

$$\mathbf{v}^{\mathrm{bc}}(\mathbf{X}, t) = \bar{\mathbf{L}}(t)\,(\mathbf{x} - \mathbf{x}_0) + \mathbf{v}_0(t), \tag{2.37}$$

where the exact origin and direction masks are set by solver controls. Stress control interpolates the stress table and updates the prescribed domain deformation measures using PID-like gains supplied in the input. Moving, symmetry, outflow, and contact boundary conditions are then enforced on the nodal fields. For FMPM, the code applies an additional uncontacted velocity-boundary correction because the filtered transfer needs to remain compatible with essential constraints.

The timing of boundary-condition imposition is significant: it occurs after trial grid dynamics and contact so that the grid-to-particle transfer sees velocities that already satisfy prescribed domain motion and essential constraints. This is consistent with explicit MPM practice, where nodal accelerations/velocities are the constrained solution variables for the step [12, 14].

Listing 2.15: Prescribed deformation and essential boundary conditions.

```
if F_table_or_stress_control_enabled:
  interpolate_prescribed_F_or_stress_table(t_n, dt)
  update_prescribed_domain_deformation_measures()
apply_essential_boundary_conditions(dt, t_n)
if FMPM_update_is_enabled:
  apply_FMPM_uncontacted_velocity_boundary_conditions()
write_profile_outputs_if_scheduled()
```

2.5.14 Step 14: map grid state back to particles

The grid-to-particle transfer updates particle velocity, position increment, and velocity gradient. For the FLIP update, the code gathers nodal acceleration increments and the time-centered grid velocity:

$$\mathbf{v}_p^{n+1} = \mathbf{v}_p^n + \Delta t \sum_I N_{Ip} \mathbf{a}_I, \tag{2.38}$$

$$\mathbf{x}_p^{n+1} = \mathbf{x}_p^n + \Delta t \sum_I N_{Ip} \left(\mathbf{v}_I - \frac{1}{2} \Delta t \, \mathbf{a}_I \right), \tag{2.39}$$

$$\mathbf{L}_p^{n+1} = \sum_I \mathbf{v}_I \otimes \nabla N_{Ip}. \tag{2.40}$$

For PIC, the particle velocity is reset to the interpolated grid velocity,

$$\mathbf{v}_p^{n+1} = \sum_I N_{Ip} \mathbf{v}_I, \tag{2.41}$$

while the same style of position and velocity-gradient gather is used. The FLIP/PIC distinction is the standard tradeoff between lower numerical dissipation and stronger grid-scale filtering; the role of velocity projection in MPM accuracy has been analyzed by Wallstedt and Guilkey [11].

The XPIC option performs a recursive particle-grid velocity filtering step. In operator notation, let $\mathbf{S}$ scatter particle velocities to lumped grid velocities and $\mathbf{S}^+$ gather nodal velocities back to particles. An XPIC/FMPM-style filtered velocity can be described by a truncated series involving repeated applications of $\mathbf{S}^+\mathbf{S}$ to damp unresolved particle-grid modes. The FMPM implementation constructs a first-order lumped grid velocity, then recursively applies

$$\mathbf{v}_{\text{next}}^{(\ell)} = \mathbf{S}^+ \mathbf{S} \mathbf{v}_{\text{rem}}^{(\ell-1)}, \tag{2.42}$$

$$\mathbf{v}_{\text{rem}}^{(\ell)} = \mathbf{v}_{\text{rem}}^{(\ell-1)} - \mathbf{v}_{\text{next}}^{(\ell)}, \tag{2.43}$$

$$\mathbf{v}^{\text{FMPM}} = \mathbf{v}^{\text{FMPM}} + \mathbf{v}_{\text{rem}}^{(\ell)}, \tag{2.44}$$

with boundary and material-contact corrections inserted into the recursive update. The goal is to improve the fidelity of particle-grid-particle transfer while retaining the explicit MPM update structure [16, 17].

Listing 2.16: Grid-to-particle transfer options.

```
if updateMethod == FLIP:
  gather acceleration increments to particle velocity
  gather time-centered grid velocity to particle position
  gather grid velocity gradient
else if updateMethod == PIC:
  set particle velocity to gathered grid velocity
  gather position increment and velocity gradient
else if updateMethod == XPIC:
```

```
  apply recursive velocity projection filter, then gather kinematics
else if updateMethod == FMPM:
  build filtered velocity by recursive scatter-gather corrections
  apply boundary/contact corrections and gather kinematics
```

2.5.15 Step 15: update particle kinematics

After grid-to-particle transfer, particles carry updated velocity-gradient estimates. If prescribed deformation is active, a superposed macroscopic velocity gradient is applied. The deformation gradient is then advanced by

$$\dot{\mathbf{F}}_p = \mathbf{L}_p \mathbf{F}_p. \tag{2.45}$$

The implementation supports subcycling the explicit deformation-gradient update:

$$\mathbf{F}_p^{k+1} = \mathbf{F}_p^k + \Delta t_{\text{sub}}\, \mathbf{L}_p \mathbf{F}_p^k, \qquad \Delta t_{\text{sub}} = \frac{\Delta t}{N_F}. \tag{2.46}$$

When exact Jacobian integration is enabled, the determinant update follows

$$J_p^{n+1} = J_p^n \exp\left(\Delta t \,\mathrm{tr}\, \mathbf{L}_p\right), \tag{2.47}$$

and $\mathbf{F}_p$ is rescaled to match the exact determinant. The code computes $\dot{\mathbf{F}}_p \approx (\mathbf{F}_p^{n+1} - \mathbf{F}_p^n)/\Delta t$ for constitutive models requiring that rate.

The particle kinematic update then checks failure/deletion criteria, determinant bounds, and velocity limits; updates current volume and density,

$$V_p^{n+1} = J_p^{n+1} V_{p0}, \qquad \rho_p^{n+1} = \frac{m_p}{V_p^{n+1}}; \tag{2.48}$$

updates CPDI domain vectors and material directions; transforms surface normals, surface positions, and surface tractions; and computes kinetic energy. These operations keep the particle as the carrier of history-dependent state, which is a central advantage of MPM for large deformation with inelastic constitutive behavior [1, 3].

Listing 2.17: Particle kinematic update.

```
if prescribed_domain_deformation_enabled:
  add macroscopic velocity gradient to particle L
if transform_particle_events_are_active:
  apply particle transformations
update deformation gradient using F_subcycles
if exact_J_update_enabled:
  rescale F to match J_old * exp(trace(L) * dt)
update particle volume, density, CPDI vectors, material directions,
       surface fields, deletion flags, and kinetic energy
```

2.5.16 Step 16: update constitutive and thermal state

The constitutive update dispatches through GEOS material models assigned to particle regions. Artificial viscosity, when enabled, is computed before stress update and enters Step 9 as an isotropic stress-like contribution in the next cycle. Constitutive dependencies are synchronized into model-specific views, and each material updates stress and internal variables from the particle kinematic state:

$$\left(\boldsymbol{\sigma}_p^{n+1}, \boldsymbol{\kappa}_p^{n+1}, c_p^{n+1}\right) = \mathcal{M}\left(\boldsymbol{\sigma}_p^n, \boldsymbol{\kappa}_p^n, \mathbf{F}_p^{n+1}, \dot{\mathbf{F}}_p, \mathbf{L}_p, \Delta t, T_p^n, \ldots\right), \tag{2.49}$$

where $\boldsymbol{\kappa}_p$ denotes material history variables and c_p is the material wave speed used in the CFL estimate. Hyperelastic materials may consume $\mathbf{F}_p$ directly, while rate/inelastic materials generally use $\mathbf{L}_p$, $\dot{\mathbf{F}}_p$, and their own history variables.

When a material model does not own the internal-energy/temperature update, the solver supplies a fallback thermo-mechanical update based on stress power. A representative form is

$$\Delta e_p \approx \frac{\Delta t}{\rho_p} \bar{\boldsymbol{\sigma}}_p : \mathbf{L}_p - \Delta e_p^{\text{visc}}, \qquad \Delta T_p = \frac{\Delta e_p}{C_{v,p}}, \tag{2.50}$$

where $\bar{\boldsymbol{\sigma}}_p$ is an average stress over the step and $C_{v,p}$ is heat capacity. After the material update, solver dependency arrays such as damage, plastic strain, temperature, temperature rate, heat capacity, update flags, and wave speed are refreshed from the constitutive model views.

Listing 2.18: Constitutive and thermal update.

```
if artificial_viscosity_enabled:
  compute_particle_artificial_viscosity()
update_constitutive_dependencies_before_stress_update()
for each particle region and material model:
  call material stress/update kernel with F, Fdot, L, stress, dt, state
if material_does_not_update_temperature:
  update internal energy and temperature from stress power
update_solver_dependencies_from_constitutive_state()
```

2.5.17 Step 17: write optional outputs and compute the next stable time step

At the end of the physical update, scheduled diagnostics are written. These include box averages, particle-history files, tracer outputs, reaction histories, profiles, plot/restart data, and other histories configured in the input/PFW workflow. Diagnostic quantities are evaluated after the constitutive update so that histories reflect the end-of-step particle state.

The next explicit time step is computed from a CFL estimate. The reviewed implementation forms the maximum particle signal speed

$$s_{\max} = \max_{p \in \mathcal{P}_{\text{act}}} \left(c_p + \|\mathbf{v}_p\| \right), \tag{2.51}$$

uses a characteristic grid length $h_{\min}$, and returns

$$\Delta t_{\text{CFL}} = C_{\text{CFL}} \frac{h_{\min}}{s_{\max}}. \tag{2.52}$$

If no active particles contribute, the function returns a very large sentinel value. The CFL structure is consistent with explicit MPM/GIMP time integration analyses, where stability depends on wave speed, particle/grid discretization, and update scheme [12, 15].

Listing 2.19: Output and stable time step.

```
if scheduled:
  write_box_average_history()
  write_particle_history()
  write_tracer_history()
  write_plot_or_restart_outputs()

s_max = max_p( waveSpeed[p] + norm(velocity[p]) )
if s_max > 0:
  dt_next = cflFactor * min_grid_spacing / s_max
else:
  dt_next = huge_value
```

2.5.18 Step 18: resize grid and clean particle state

The final step performs post-update domain management. If prescribed boundary deformation, prescribed domain deformation, or stress control is active, the background grid/domain measures are resized. In affine form, a nodal reference position may be updated by a domain velocity-gradient increment such as

$$\mathbf{x}_I^{\mathrm{ref},n+1} \approx \left(\mathbf{I} + \Delta t\, \bar{\mathbf{L}}\right) \mathbf{x}_I^{\mathrm{ref},n}, \tag{2.53}$$

with corresponding updates to element spacing, local/global extents, and diagnostic box-average bounds. CPDI domain vectors are scaled or unscaled around this resize depending on plotting and domain-scaling options.

Particles flagged as failed, out of range, or otherwise invalid are deleted or compacted. In parallel runs, the partition is updated, particles are redistributed, and periodic corrections are reapplied. If reset-deformation-gradient events are active, the relevant particle deformation measures are reset after output and before the next step begins. This cleanup ensures that the next cycle starts from a consistent particle set, updated domain geometry, and zeroed transient grid state.

Listing 2.20: Grid resize and cleanup.

```
if prescribed_boundary_F_or_stress_control_requires_resize:
  resize_background_grid_and_domain_bounds()
  update_box_average_extents_if_needed()
if CPDI_domain_scaling_enabled:
  apply_or_remove_CPDI_plot_scaling_as_requested()
flag_particles_outside_valid_domain()
delete_or_compact_bad_particles()
if running_in_parallel:
  repartition_and_update_periodic_particle_positions()
if reset_deformation_gradient_requested:
  reset_selected_particle_deformation_gradients()
```

2.6 Events

MPM events are time-windowed run-time operations parsed under the solver's `MPMEvents` block. In the current implementation, the event manager is visited near the beginning of each explicit time step, before particle subdivision, particle-map refresh, ghost construction, and active-particle index reconstruction. Most events therefore modify particle topology, particle state, solver controls, or auxiliary managers before the particle-to-grid stage of the same step. The exception is `TransformParticles`, which is handled by a later event pass after the deformation-gradient update.

2.6.1 Event-manager semantics and common inputs

Every event type derives from `MPMEventBase`. The base class registers the required start time, an optional end time, and an internal completion flag:

Required input.

`startTime`: time at which the event first becomes eligible for execution.

Optional input.

`endTime`: time at which the event window closes; the default is `DBL_MAX`. The base class rejects input for which `startTime > endTime`.

Internal state.

`isComplete`: a non-input flag used to prevent one-shot events from firing repeatedly.

At time step n, with step size Δt and solver time t^n, a non-complete event is considered active if

$$t_s - \frac{\Delta t}{2} \le t^n \le t_e + \frac{\Delta t}{2}, \tag{2.54}$$

where t_s and t_e are the event's `startTime` and `endTime`. The half-step tolerance makes event activation robust to time-step discretization and roundoff. In algorithmic form, the event loop is

Listing 2.21: Generic MPM event-manager loop.

```
for event in MPMEvents:
  start = event.startTime
  end   = event.endTime
  if start - 0.5*dt <= time_n <= end + 0.5*dt and not event.isComplete:
    dispatch event by catalogName
    if event is a one-shot operation:
      event.isComplete = true
```

This structure is consistent with the MPM view that material history variables, damage variables, contact state, and boundary data are carried by particles and grid fields and may be changed between time steps before the next particle-grid projection [1, 19, 14]. Event-specific inputs and generated schema tables are listed in Appendix B; the subsections below describe the algorithms implemented by each registered event.

For several ramp-like events, GEOS-MPM uses a scalar interpolation helper. Given a scalar input x, interval $[x_0, x_1]$, endpoint values y_0, y_1, and $\lambda = (x - x_0)/(x_1 - x_0)$, the returned value is

$$I(x) = \begin{cases} y_0, & x < x_0 \text{ or } x_1 \le x_0, \\ y_1, & x > x_1, \\ y_0(1-\lambda) + y_1\lambda, & \text{linear}, \\ y_0 - \dfrac{1}{2}(y_1 - y_0)\left[\cos(\pi\lambda) - 1\right], & \text{cosine}, \\ y_0 + (y_1 - y_0)\left(10\lambda^3 - 15\lambda^4 + 6\lambda^5\right), & \text{smoothstep}. \end{cases} \tag{2.55}$$

The cosine and smoothstep options are useful when an event should avoid an impulsive endpoint derivative.

2.6.2 `Anneal`

Inputs.
Required.
`startTime`; `targetRegion`, the particle region name to anneal, or `all`.
Optional.
`endTime`; inherited default `DBL_MAX`.

Algorithm. The `Anneal` event reduces the deviatoric part of the constitutive-model history stress in selected particle regions while preserving the hydrostatic component. For each matching `ParticleRegion`, the event obtains the material model associated with each subregion and requires that it expose an `oldStress` wrapper. The event then visits every constitutive point, decomposes the stored stress into mean and deviatoric invariants, scales only the J_2 magnitude, and recomposes the stress. In Voigt notation, let

$$p = \frac{1}{3}\operatorname{tr}\boldsymbol{\sigma}, \qquad \mathbf{s} = \boldsymbol{\sigma} - p\mathbf{I}, \qquad q = \sqrt{\frac{3}{2}\mathbf{s} : \mathbf{s}}. \tag{2.56}$$

The code computes a knockdown factor

$$k^n = \begin{cases} 0, & t^n - \Delta t/2 \le t_e < t^n + \Delta t/2, \\ \max\left(0, 1 - \dfrac{20\Delta t(t^n - t_s)}{(t_e - t_s)^2}\right), & \text{otherwise}, \end{cases} \tag{2.57}$$

then writes

$$\boldsymbol{\sigma}_{\text{old}}^{\text{new}} = p\mathbf{I} + k^n \mathbf{s}. \tag{2.58}$$

The implementation describes this as a gradual annealing of deviatoric stress. Because it acts on a history variable rather than solving a thermal relaxation equation, it should be interpreted as an algorithmic material-state operation. This is compatible with history-dependent MPM formulations in which particles carry constitutive state variables between grid projection phases [1, 19].

Listing 2.22: Anneal event pseudocode.

```
for region in particleRegions:
  if region.name == targetRegion or targetRegion == "all":
    for subRegion in region.subRegions:
      material = subRegion.solidMaterial
      require material.has("oldStress")
      for p in material.points:
        sigma = material.oldStress[p]
        (meanStress, q, deviator) = stressDecomposition(sigma)
        k = annealKnockdown(time_n, dt, startTime, endTime)
        material.oldStress[p] = stressRecomposition(meanStress, k*q, deviator)
```

Implementation note. The current runtime branch does not explicitly set `isComplete` for `Anneal`; the event naturally stops being active after its time window. A future implementation may choose to mark it complete at the final event step for restart clarity.

2.6.3 `BodyForceUpdate`

Inputs.
Required.
`startTime`.
Optional.
`endTime`; `bodyForce`, a length-three vector. If omitted, the vector is initialized to zero.

Algorithm. This one-shot event overwrites the solver-level body-force vector `m_bodyForce`. Subsequent particle force construction assigns this vector to each particle, limited by the active spatial dimension. In the semidiscrete balance of momentum, the event changes the body-force contribution

$$\mathbf{f}_I^{\text{body}} = \sum_p m_p N_I(\mathbf{x}_p)\, \mathbf{b}, \tag{2.59}$$

where N_I is the grid basis function and $\mathbf{b}$ is the updated acceleration-like body-force vector. This is the usual MPM weak-form load projection from material points to the background grid [1, 19].

Listing 2.23: BodyForceUpdate event pseudocode.

```
if event active:
  solver.bodyForce = event.bodyForce
  event.isComplete = true
```

2.6.4 `BoreholePressure`

Inputs.
Required.
`startTime`; `boreholeRadius`; `startPressure`; `endPressure`.
Optional.
`endTime`; `interpType`, with options `0` linear, `1` cosine, and `2` smoothstep. The default is the event's initialized interpolation type, which is cosine in the current source. The radius must be non-negative.

Algorithm. `BoreholePressure` imposes a virtual fluid pressure in void space inside an infinite cylinder aligned with the z axis and centered at the origin in the x-y plane. At every active step, the event evaluates

$$p_b(t^n) = I(t^n; t_s, t_e, p_s, p_e), \tag{2.60}$$

using Eq. (2.55), enables the borehole-pressure flag, stores the radius, and writes the background stress

$$\boldsymbol{\sigma}_b^{\mathrm{bg}} = -p_b \mathbf{I}. \tag{2.61}$$

During the grid-background-stress update, every grid node satisfying

$$X_I^2 + Y_I^2 < R_b^2 \tag{2.62}$$

receives this background stress. The internal-force projection then uses the background-stress-corrected stress. In abstract notation,

$$\mathbf{f}_I^{\mathrm{int}} = -\sum_p V_p \left(\boldsymbol{\sigma}_p - \boldsymbol{\sigma}_I^{\mathrm{bg}} - q_p \mathbf{I}\right) \nabla N_I(\mathbf{x}_p), \tag{2.63}$$

where q_p denotes the particle-level artificial viscosity or pressure-like correction where active. This converts a prescribed pressure region into the same weak-form force balance used by the standard MPM update, avoiding explicit surface tracking for this pressure boundary. The approach is algorithmic rather than a general immersed-boundary formulation, but it fits the particle-grid balance framework of MPM [1, 19].

Listing 2.24: BoreholePressure event pseudocode.

```
if event active:
  p = interpolate(time_n, startTime, endTime, startPressure, endPressure, interpType)
  solver.enableBoreholePressure = true
  solver.boreholeRadius = boreholeRadius
  solver.boreholeStress = [-p, -p, -p, 0, 0, 0]

for grid node I:
  if X[I]^2 + Y[I]^2 < boreholeRadius^2:
    gridBackgroundStress[I] = solver.boreholeStress
```

Implementation note. The current event does not reset the borehole-pressure flag or stress after `endTime`; after the event window closes, the most recently assigned background stress remains in the solver state unless a later event or solver initialization changes it.

2.6.5 `CohesiveZone`

Inputs.
Required.

`startTime`; `regionNames`; `constitutiveModels`; `czTags`. The three arrays must be non-empty and must have identical lengths.

Optional.

`endTime`; `czVolumeNormalization`, default `1`; `computeNormalsAndPositions`, default `0`; `normalsAndPositionsMethod`, default `LogisticRegression`; `czSurfaceDisplacementUpdate`, default `TypeB`. The two Boolean-like flags `czVolumeNormalization` and `computeNormalsAndPositions` must be either `0` or `1`.

Algorithm. The `CohesiveZone` event dynamically creates and later removes cohesive-zone regions. Each entry of the input arrays defines one cohesive-zone region name, one cohesive constitutive model, and one tag identifying the particle/interface set. At the start of the event window, the global particle-surface normal and position computation controls are copied from the event to the solver. For each requested cohesive-zone region not already present, the event creates a `CohesiveZoneRegion`, stores its tag and algorithmic controls, clones the named cohesive constitutive relation from the domain constitutive manager, allocates one quadrature point per cohesive-zone entry, and registers the material model on the new region. When $t^n \geq t_e - \Delta t/2$, the region is removed and the event is marked complete.

The event is coupled to the dedicated cohesive-zone update in the solver step sequence. When a newly created cohesive-zone region is detected, the solver resets cohesive surface flags, projects surface normals to the grid, and initializes the cohesive reference configuration. During active cohesive-zone evolution, the solver forms a displacement jump across tagged surfaces and evaluates the cohesive traction through the region's constitutive model. In generic notation,

$$[\![\mathbf{u}]\!] = \mathbf{u}^+ - \mathbf{u}^-, \qquad \mathbf{t}_{\mathrm{cz}} = \mathcal{T}([\![\mathbf{u}]\!], \mathbf{n}_0, \boldsymbol{\alpha}), \tag{2.64}$$

where $\mathbf{n}_0$ is the reference surface normal and $\boldsymbol{\alpha}$ denotes cohesive history variables. The resulting cohesive force is mapped back to particles/nodes by the same particle-grid machinery used elsewhere in the solver. The use of explicitly tracked surface/interface state is closely related to MPM treatments of contact, cracks, and field-gradient partitioning by Bardenhagen, Nairn, and Homel, where material discontinuities are represented without conforming remeshing of a Lagrangian mesh [9, 7, 6].

Listing 2.25: CohesiveZone event pseudocode.

```
if active near startTime:
  solver.computeNormalsAndPositions = event.computeNormalsAndPositions
  solver.normalsAndPositionsMethod  = event.normalsAndPositionsMethod

for k in range(numberOfCohesiveZones):
  name  = regionNames[k]
  model = constitutiveModels[k]
  tag   = czTags[k]
  if not cohesiveZoneManager.hasRegion(name):
    region = cohesiveZoneManager.addRegion(name)
    region.tag = tag
    region.czVolumeNormalization = czVolumeNormalization
    region.surfaceDisplacementUpdate = czSurfaceDisplacementUpdate
    region.material = cloneConstitutiveModel(model)
    region.material.allocateConstitutiveData(region, oneQuadraturePoint)
  if time_n >= endTime - 0.5*dt:
    cohesiveZoneManager.removeRegion(name)
    event.isComplete = true
```

Publication note. A paper-level description should separate the event, which creates/removes cohesive-zone regions, from the cohesive-zone constitutive law, which supplies $\mathcal{T}$ in Eq. (2.64). The latter belongs in the constitutive-model chapter or linked material-model report.

2.6.6 `ConfiningPressure`

Inputs.
Required.

`startTime`; `confiningPressureBoxMin`, a length-three vector; `confiningPressureBoxMax`, a length-three vector; `startPressure`; `endPressure`.

Optional.

`endTime`; `interpType`, with options `0` linear, `1` cosine, and `2` smoothstep. The default is the event's initialized interpolation type, which is cosine in the current source.

Algorithm. `ConfiningPressure` applies a virtual hydrostatic pressure to grid nodes outside a user-specified axis-aligned box. Let $\mathbf{X}^{\min}$ and $\mathbf{X}^{\max}$ denote the two box corners. During each active event step, the pressure

$$p_c(t^n) = I(t^n; t_s, t_e, p_s, p_e) \tag{2.65}$$

sets

$$\boldsymbol{\sigma}_c^{\mathrm{bg}} = -p_c \mathbf{I}. \tag{2.66}$$

A grid node is assigned this stress if any coordinate lies outside the box,

$$X_{I,j} < X_j^{\min} \quad \text{or} \quad X_{I,j} > X_j^{\max} \quad \text{for at least one } j \in \{1,2,3\}. \tag{2.67}$$

The background stress then enters the internal-force calculation through Eq. (2.63). This event is useful for triaxial-type calculations and pressure initialization workflows because it can impose a pressure-like exterior traction without explicit contact facets.

Listing 2.26: ConfiningPressure event pseudocode.

```
if event active:
  p = interpolate(time_n, startTime, endTime, startPressure, endPressure, interpType)
  solver.enableConfiningPressure = true
  solver.confiningPressureBoxMin = event.confiningPressureBoxMin
  solver.confiningPressureBoxMax = event.confiningPressureBoxMax
  solver.confiningStress = [-p, -p, -p, 0, 0, 0]

for grid node I:
  if any_coordinate_outside_box(X[I], boxMin, boxMax):
    gridBackgroundStress[I] = solver.confiningStress
```

Implementation note. Like `BoreholePressure`, this event is not marked complete at `endTime` in the current runtime branch and does not explicitly reset the confining-pressure flag.

2.6.7 `CrystalHeal`

Inputs.
Required.

`startTime`; `targetRegion`, a particle region name or `all`; `healType`, an integer selecting the healing rule.

Optional.

`endTime`.

Internal state.

`markedParticlesToHeal`, a non-input flag used to indicate that candidate particles have been identified.

Algorithm. `CrystalHeal` is a damage-state event for regions containing particle damage, particle damage gradients, particle stresses, deformation gradients, and CPDI particle vectors. It first identifies candidate particles to heal and then gradually reduces their damage over the event window. If the constitutive model exposes `crackSpeed`, the code multiplies it by 10^{-100} while the healing process is active and restores it by multiplying by 10^{100} after completion.

For a particle with damage d_p, damage-gradient vector $\mathbf{g}_p$, stress $\boldsymbol{\sigma}_p$, and deformation gradient $\mathbf{F}_p$, the event computes

$$\sigma_{n,p} = \mathbf{g}_p \cdot \mathrm{sym}(\boldsymbol{\sigma}_p)\mathbf{g}_p, \qquad J_p = \det \mathbf{F}_p. \tag{2.68}$$

A particle is marked if $d_p > 0$ and one of the following conditions holds: `healType` is `1`; `healType` is `3`; or `healType` is `0` and either $\sigma_{n,p} < 0$ or $J_p < 1$. If `healType` is `3` and $J_p > 1$, the code also rescales the deformation gradient and CPDI domain vectors. Let

$$a_p = J_p^{\beta}, \qquad \beta = \begin{cases} 1/2, & \text{plane strain,} \\ 1/3, & \text{three dimensions.} \end{cases} \tag{2.69}$$

The event applies

$$\mathbf{F}_p \leftarrow a_p^{-1}\mathbf{F}_p, \qquad \mathbf{r}_{p,i} \leftarrow a_p \mathbf{r}_{p,i}, \qquad V_p^0 \leftarrow J_p V_p^0, \tag{2.70}$$

with the third CPDI vector omitted in plane strain. On subsequent active steps, marked particles have their damage reduced recursively as

$$d_p^{n+1} = d_p^n \max\left(0, 1 - \frac{20\Delta t(t^n - t_s)}{(t_e - t_s)^2}\right), \tag{2.71}$$

with exact zero enforced when the step crosses `endTime`. This event is algorithmically connected to MPM fracture and discontinuity treatments in which cracks, damage, and material state are particle data rather than mesh data [7, 6, 5].

Listing 2.27: CrystalHeal event pseudocode.

```
for p in target particles:
  if not markedParticlesToHeal:
    if material.has("crackSpeed"):
      material.crackSpeed *= 1e-100
    normalStress = damageGradient[p] dot sym(stress[p]) damageGradient[p]
    J = det(F[p])
    if damage[p] > 0 and healingCriterion(healType, normalStress, J):
      crystalHealFlag[p] = 1
      if healType == 3 and J > 1:
        scale F[p], CPDI r-vectors, and reference volume
    else:
      crystalHealFlag[p] = 0
  else:
    if crossing endTime:
      damage[p] = 0 for flagged particles
      event.isComplete = true
    else:
      damage[p] *= recursiveKnockdown(time_n, dt, startTime, endTime)
```

2.6.8 `DeformationUpdate`

Inputs.
Required.
`startTime`.
Optional.
`endTime`; `prescribedFTable`, default `0`; intended `prescribedBoundaryFTable`, default `0`; `stressControl`, a length-three integer vector with one flag per coordinate direction. If `stressControl` is supplied, it must have length three.

Algorithm. `DeformationUpdate` switches global deformation and stress-control modes during a run. When active, it copies the event's flags to the solver:

$$\texttt{m\_prescribedFTable} \leftarrow \texttt{prescribedFTable}, \qquad \texttt{m\_prescribedBoundaryFTable} \leftarrow \texttt{prescribedBoundaryFTable}, \tag{2.72}$$

These flags are consumed by the solver's prescribed-deformation and stress-control logic, including the update of domain deformation measures and, where enabled, background-grid resizing and boundary kinematics. In publication notation, the event modifies which components of the macroscopic deformation gradient $\overline{\mathbf{F}}$ are prescribed and which components are adjusted by stress-control feedback, but the feedback law and the time-table interpolation remain solver-level controls rather than event-specific algorithms.

Listing 2.28: DeformationUpdate event pseudocode.

```
if event active:
  solver.prescribedFTable = event.prescribedFTable
  solver.prescribedBoundaryFTable = event.prescribedBoundaryFTable
  solver.stressControl = event.stressControl
```

Implementation note. In the reviewed source, both `prescribedFTableString()` and `prescribedBoundaryFTableString()` return the XML key `prescribedFTable`. The generated schema therefore shows two optional entries with the same key. This appears to be an implementation typo; the intended second key is documented here as `prescribedBoundaryFTable` so the manual can flag the ambiguity for code maintenance.

2.6.9 `FrictionCoefficientSwap`

Inputs.
Required.
`startTime`.
Optional.
`endTime`; `frictionCoefficient`, default `-1`; `frictionCoefficientTable`, a table for group-dependent friction coefficients.

Algorithm. This one-shot event updates the global friction coefficient and optional coefficient table, then calls the solver's friction-coefficient initialization routine. The resulting coefficients are subsequently used by contact and boundary algorithms. A scalar Coulomb-type coefficient enters contact algorithms through a tangential slip limit of the form

$$\|\mathbf{t}_T\| \le \mu \max(0, -t_N), \tag{2.73}$$

where t_N and $\mathbf{t}_T$ are normal and tangential contact tractions. The exact contact update belongs to the contact section; the event's role is to change the coefficients used by that update. The event is related to the family of multi-material MPM contact methods described by Bardenhagen and co-workers [9, 8].

Listing 2.29: FrictionCoefficientSwap event pseudocode.

```
if event active:
  solver.frictionCoefficient = event.frictionCoefficient
  solver.frictionCoefficientTable = event.frictionCoefficientTable
  solver.initializeFrictionCoefficients()
  event.isComplete = true
```

2.6.10 `Heal`

Inputs.
Required.
`startTime`; `targetRegion`.
Optional.
`endTime`.

Algorithm. `Heal` is a one-shot surface-state reset. It sets the solver's surface-healing flag and visits all particle subregions. For each active particle, if the surface flag is less than `3`, it resets the flag to zero. Flags `3` and `4`, which denote cohesive surface states in the current code comments, are intentionally preserved. The operation can be interpreted as removing ordinary free-surface/contact classifications while leaving cohesive interfaces intact.

Listing 2.30: Heal event pseudocode.

```
if event active:
  solver.surfaceHealing = true
  for subRegion in allParticleSubRegions:
    for p in activeParticles(subRegion):
      if particleSurfaceFlag[p] < 3:
        particleSurfaceFlag[p] = 0
  event.isComplete = true
```

Implementation note. Although `targetRegion` is registered as a required input, the dynamic cast to `HealMPMEvent` is commented out in the reviewed runtime branch. The current event therefore applies to all particle subregions, not only to the named target region.

2.6.11 `InitializeStress`

Inputs.
Required.
`startTime`; `pressure`; `targetRegion`, a particle region name or `all`.
Optional.
`endTime`.

Algorithm. This one-shot event initializes a hydrostatic particle stress and the matching constitutive-model history stress. The code uses compression-negative stress convention,

$$\sigma_0 = -p_0, \qquad \boldsymbol{\sigma}_p = \sigma_0 \mathbf{I}. \tag{2.74}$$

For each selected particle subregion, the event writes `particleStress` to $[\sigma_0, \sigma_0, \sigma_0, 0, 0, 0]$ in Voigt order. It then obtains the subregion's constitutive model and requires the `oldStress` wrapper; that wrapper is initialized to the same hydrostatic state at all constitutive points. Consistently initializing

both particle stress and material history is important in MPM because particles carry history data between grid solves [1, 19].

Listing 2.31: InitializeStress event pseudocode.

```
initialMeanStress = -pressure
for region in particleRegions:
  if region.name == targetRegion or targetRegion == "all":
    for subRegion in region.subRegions:
      for p in activeParticles(subRegion):
        particleStress[p] = [initialMeanStress, initialMeanStress,
                             initialMeanStress, 0, 0, 0]
      material = subRegion.solidMaterial
      require material.has("oldStress")
      for q in material.points:
        material.oldStress[q] = [initialMeanStress, initialMeanStress,
                                 initialMeanStress, 0, 0, 0]
  event.isComplete = true
```

Modeling note. The source comments emphasize that this is a simple hydrostatic initialization. A general nonhydrostatic state, an inelastic state, or a hyperelastic state may require additional constitutive variables and equilibrium boundary conditions to be initialized consistently.

2.6.12 `InsertPeriodicContactSurfaces`

Inputs.
Required.
`startTime`.
Optional.
`endTime`.

Algorithm. This one-shot event tags particles near periodic domain faces as contact surfaces. It reads periodicity flags from the spatial partition, grid cell sizes h_j, and global bounds $X_j^{\min}, X_j^{\max}$. A particle is tagged if, in at least one periodic coordinate direction,

$$x_{p,j} - X_j^{\min} < h_j \quad \text{or} \quad X_j^{\max} - x_{p,j} < h_j. \tag{2.75}$$

When this condition is met, `particleSurfaceFlag[p]` is set to `2`. This provides a simple way to seed surface/contact information on particles adjacent to periodic faces before subsequent contact and neighborhood updates.

Listing 2.32: InsertPeriodicContactSurfaces event pseudocode.

```
periodic = partition.periodicFlags()
for p in all active particles:
  for direction j in 0,1,2:
    nearLower = particlePosition[p][j] - globalMin[j] < h[j]
    nearUpper = globalMax[j] - particlePosition[p][j] < h[j]
    if periodic[j] and (nearLower or nearUpper):
      particleSurfaceFlag[p] = 2
      break
if event active:
  event.isComplete = true
```

2.6.13 `MachineSample`

Inputs.
Required.
`startTime`; `sampleType`.
Optional.
`endTime`; `filletRadius`, default `-1`; `gaugeLength`, default `-1`; `gaugeRadius`, default `-1`; `diskRadius`, default `-1`.
Conditional requirements.
`sampleType="dogbone"` requires non-negative `filletRadius`, `gaugeLength`, and `gaugeRadius`; `sampleType="brazilianDisk"` requires non-negative `diskRadius`; `sampleType="cylinder"` requires non-negative `gaugeRadius`.

Algorithm. `MachineSample` flags particles for deletion to machine a generated particle block into a common test-specimen shape. The event computes the domain center

$$\mathbf{c} = \frac{1}{2}\left(\mathbf{X}^{\min} + \mathbf{X}^{\max}\right) \tag{2.76}$$

and uses geometric tests to set `particleDeleteFlag[p] = 1`. The actual compaction/deletion of flagged particles occurs later in the explicit-step cleanup logic.

For `sampleType="dogbone"`, machining is aligned with the y direction. Let L_g be the gauge length, R_g the gauge radius, R_f the fillet radius, and $L_m = 2R_f + L_g$ the machined-zone length. Inside the central machined span, the radial distance is

$$r_{xz}^2 = (x_p - c_x)^2 + (z_p - c_z)^2. \tag{2.77}$$

Particles inside the gauge section are deleted if $r_{xz} > R_g$. In the fillet sections, with

$$y_f = |y_p - c_y| - \frac{L_g}{2}, \qquad R(y_f) = R_f - \sqrt{R_f^2 - y_f^2} + R_g, \tag{2.78}$$

particles are deleted if $r_{xz} > R(y_f)$. For `sampleType="brazilianDisk"`, particles outside the sphere of radius `diskRadius` centered at $\mathbf{c}$ are deleted. For `sampleType="cylinder"`, particles with $r_{xz} > R_g$ are deleted.

Listing 2.33: MachineSample event pseudocode.

```
center = 0.5*(globalMin + globalMax)
for p in active particles:
  if sampleType == "dogbone":
    if p is within machined y-span:
      r2 = (x[p]-center.x)^2 + (z[p]-center.z)^2
      if p is within gauge section:
        delete if r2 > gaugeRadius^2
      else:
        y = abs(y[p]-center.y) - 0.5*gaugeLength
        radius = filletRadius - sqrt(filletRadius^2 - y^2) + gaugeRadius
        delete if r2 > radius^2
  if sampleType == "brazilianDisk":
    delete if norm(x[p]-center)^2 > diskRadius^2
  if sampleType == "cylinder":
    delete if (x[p]-center.x)^2 + (z[p]-center.z)^2 > gaugeRadius^2
event.isComplete = true
```

2.6.14 `MaterialSwap`

Inputs.
Required.
`startTime`; `sourceRegion`; `destinationRegion`.
Optional.
`endTime`.

Algorithm. `MaterialSwap` moves all particles and particle-carried state from one particle region to another. It assumes that the source and destination particle regions have aligned subregion lists representing corresponding particle types. For each subregion pair, the destination is resized to the source size, particle wrappers whose names begin with `particle` are copied, and the source subregion is subsequently resized to zero. Most particle fields are copied directly. Two fields are recomputed from the destination constitutive-model reference density ρ^0_{dst}:

$$m_p^{\mathrm{dst}} = \rho^0_{\mathrm{dst}} V^0_{p,\mathrm{src}}, \qquad \rho_p^{\mathrm{dst}} = \frac{\rho^0_{\mathrm{dst}} V^0_{p,\mathrm{src}}}{V_{p,\mathrm{src}}}. \tag{2.79}$$

The event also copies the source constitutive `oldStress` into the destination constitutive `oldStress`. After the swap, active particle indices are refreshed. Because MPM material history is particle-resident, the event can be viewed as changing the material assignment while preserving or reconstructing selected history variables [1, 14].

Listing 2.34: MaterialSwap event pseudocode.

```
source = particleManager.region(sourceRegion)
destination = particleManager.region(destinationRegion)
for each corresponding sourceSubRegion, destinationSubRegion:
  destinationSubRegion.resize(sourceSubRegion.size)
  for wrapper in sourceSubRegion.wrappers:
    if wrapper.name starts with "particle":
      if wrapper.name in {particleMass, particleDensity}:
        recompute using destination reference density
      else:
        copy source wrapper to destination wrapper
  copy sourceMaterial.oldStress to destinationMaterial.oldStress
  sourceSubRegion.resize(0)
refresh active particle indices
event.isComplete = true
```

2.6.15 `PolymerHeal`

Inputs.
Required.
`startTime`; `targetRegion`, a particle region name or `all`.
Optional.
`endTime`.

Algorithm. `PolymerHeal` is a one-shot state reset for subregions whose constitutive catalog name is `StrainHardeningPolymer`. For each selected particle with `particleDamage` equal to one within a tolerance of 10^{-16}, the event sets the damage to zero, resets the particle deformation gradient to the

identity, sets the particle reference volume equal to its current volume, and copies current CPDI particle vectors to reference CPDI particle vectors:

$$d_p \leftarrow 0, \qquad \mathbf{F}_p \leftarrow \mathbf{I}, \qquad V_p^0 \leftarrow V_p, \qquad \mathbf{r}_{p,i}^0 \leftarrow \mathbf{r}_{p,i}. \tag{2.80}$$

The code comments mention retaining rotation as a possible future refinement, but the current implementation resets $\mathbf{F}_p$ exactly to the identity. The event is material-model-specific and should be documented together with the strain-hardening polymer law in the LLNL-specific or material-model reports.

Listing 2.35: PolymerHeal event pseudocode.

```
for region in particleRegions:
  if region.name == targetRegion or targetRegion == "all":
    for subRegion in region.subRegions:
      material = subRegion.solidMaterial
      if material.catalogName == "StrainHardeningPolymer":
        for p in activeParticles(subRegion):
          if abs(particleDamage[p] - 1.0) <= 1e-16:
            particleDamage[p] = 0
            F[p] = identity
            referenceVolume[p] = volume[p]
            referenceRVectors[p] = rVectors[p]
event.isComplete = true
```

2.6.16 `ResetDeformationGradient`

Inputs.
Required.
`startTime`.
Optional.
`endTime`.

Algorithm. This one-shot event does not directly loop over particles in the event manager. Instead, it sets the solver flag `m_resetDefGradForScaledSurfaceParticles = 1` and marks the event complete. The explicit-step cleanup and grid-resize phase later checks this flag and resets selected deformation gradients associated with scaled surface particles. The separation is important: the event schedules a reset, but the reset is applied after the solver has completed the output, resize, and particle-cleanup logic for the step.

Listing 2.36: ResetDeformationGradient event pseudocode.

```
if event active:
  solver.resetDefGradForScaledSurfaceParticles = true
  event.isComplete = true

later in cleanup step:
  if solver.resetDefGradForScaledSurfaceParticles:
    reset selected deformation gradients for scaled surface particles
```

2.6.17 `TemperatureProfile`

Inputs.
Required.
`startTime`.

Optional.
`endTime`.

Solver-level dependencies.
The event has no event-specific temperature values. It uses the solver's `temperatureTable` and `temperatureTableInterpType` controls.

Algorithm. When active, `TemperatureProfile` interpolates the solver-level temperature table to obtain a domain temperature $T(t^n)$ and temperature rate $\dot{T}(t^n)$. It then assigns these values to every active particle and writes the temperature to every cohesive-zone entry:

$$T_p \leftarrow T(t^n), \qquad \dot{T}_p \leftarrow \dot{T}(t^n), \qquad T_{\mathrm{cz}} \leftarrow T(t^n). \tag{2.81}$$

This is a spatially uniform thermal loading event; any spatially varying thermal field would require a different event or constitutive coupling.

Listing 2.37: TemperatureProfile event pseudocode.

```
if event active:
  (domainTemperature, domainTemperatureRate) = interpolateTemperatureTable(time_n, dt)
  for p in all active particles:
    particleTemperature[p] = domainTemperature
    particleTemperatureRate[p] = domainTemperatureRate
  for cz in all cohesive-zone entries:
    czTemperature[cz] = domainTemperature
```

Implementation note. The current branch does not set `isComplete` for `TemperatureProfile`; it remains active only while the event time window is satisfied.

2.6.18 `TemperatureRamp`

Inputs.
Required.
`startTime`; `startTemperature`; `endTemperature`. The two temperatures must be non-negative.
Optional.
`endTime`; `interpType`, with options `0` linear, `1` cosine, and `2` smoothstep/sigmoid according to the source description. The default is the event's initialized interpolation type.

Registered behavior and current runtime status. `TemperatureRamp` is registered as an event input class and validates the start/end temperatures and interpolation type, but the reviewed solver's `triggerEvents` dispatch does not contain a `TemperatureRamp` branch. Therefore, in the current codebase, a `TemperatureRamp` block can be parsed but does not change particle or cohesive-zone temperatures during a run. Users should use `TemperatureProfile` with a solver-level `temperatureTable` for executable temperature histories unless the runtime dispatch is extended.

Intended algorithm if activated. The natural event-level ramp would evaluate

$$T(t^n) = I(t^n; t_s, t_e, T_s, T_e) \tag{2.82}$$

and then assign T_p and possibly $\dot{T}_p$ analogously to Eq. (2.81). This is not performed by the current solver branch.

Listing 2.38: TemperatureRamp registered-input pseudocode.

```
# Current source:
#   parse startTemperature, endTemperature, interpType
#   validate temperatures are non-negative
#   no triggerEvents dispatch branch is present
```

2.6.19 `TransformParticles`

Inputs.
Required.
`startTime`.
Optional.
`endTime`.

Algorithm. `TransformParticles` is handled by a separate event pass after the deformation-gradient update. In the current source, it applies a fixed rotation of angle π about the z axis to every active particle position and deformation gradient:

$$\mathbf{R}_z(\pi) = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{x}_p \leftarrow \mathbf{R}_z(\pi)\mathbf{x}_p, \qquad \mathbf{F}_p \leftarrow \mathbf{R}_z(\pi)\mathbf{F}_p. \tag{2.83}$$

The event is one-shot and is marked complete after the transform. It currently has no inputs for target region, rotation angle, rotation axis, or translation; those would be natural extensions if the event is generalized.

Listing 2.39: TransformParticles event pseudocode.

```
if event active in post-deformation-gradient event pass:
  R = rotation matrix for angle pi about z-axis
  for p in all active particles:
    particlePosition[p] = R * particlePosition[p]
    particleDeformationGradient[p] = R * particleDeformationGradient[p]
  event.isComplete = true
```

2.6.20 Event-input summary

Table 2.2 summarizes the user-facing inputs discussed above. Appendix B remains the code-derived source of truth for attribute names discovered directly from the source archive.

Table 2.2: Manual summary of event inputs and execution status.

Event	Required inputs beyond inherited fields	Optional inputs and notes
`Anneal`	`targetRegion`	Active-window deviatoric-stress knockdown; does not explicitly mark complete.
`BodyForce Update`	None	`bodyForce`; one-shot.
`Borehole Pressure`	`boreholeRadius`, `startPressure`, `endPressure`	`interpType`; active-window pressure update; leaves last value in solver state.

Continued on next page

Event	Required inputs beyond inherited fields	Optional inputs and notes
`Cohesive Zone`	`regionNames`, `constitutiveModels`, `czTags`	`czVolumeNormalization`, `computeNormalsAndPositions`, `normalsAndPositionsMethod`, `czSurfaceDisplacementUpdate`.
`Confining Pressure`	`confiningPressureBoxMin`, `confiningPressureBoxMax`, `startPressure`, `endPressure`	`interpType`; active-window pressure update.
`Crystal Heal`	`targetRegion`, `healType`	Internal `markedParticlesToHeal`; damage decay until complete.
`Deformation Update`	None	`prescribedFTable`, intended `prescribedBoundaryFTable`, `stressControl`.
`Friction Coefficient Swap`	None	`frictionCoefficient`, `frictionCoefficientTable`; one-shot.
`Heal`	`targetRegion`	Current runtime applies to all subregions; one-shot.
`Initialize Stress`	`pressure`, `targetRegion`	One-shot hydrostatic stress initialization.
`InsertPeriodic Contact Surfaces`	None	One-shot periodic-face surface tagging.
`Machine Sample`	`sampleType`	Shape-dependent radii/lengths: `filletRadius`, `gaugeLength`, `gaugeRadius`, `diskRadius`.
`Material Swap`	`sourceRegion`, `destinationRegion`	One-shot particle/material-state transfer.
`Polymer Heal`	`targetRegion`	One-shot reset for `StrainHardeningPolymer` damaged particles.
`Reset Deformation Gradient`	None	Schedules later deformation-gradient reset; one-shot.
`Temperature Profile`	None	Uses solver-level `temperatureTable`; active-window uniform temperature update.
`Temperature Ramp`	`startTemperature`, `endTemperature`	`interpType`; registered but not triggered in current solver dispatch.
`Transform Particles`	None	Fixed π rotation about z; handled in later event pass; one-shot.

PFW can generate the XML `MPMEvents` block from `pfw["mpmEventsString"]`. The PFW layer also contains compatibility normalization for several legacy cohesive-zone snippets; however, event semantics at run time are governed by the solver dispatch described in this section.

2.7 Constitutive models

This section documents the continuum constitutive models used by the MPM solver. Cohesive-zone material laws are documented separately in Section 2.10, because they operate on interface opening/sliding states rather than on ordinary particle volume states. The continuum models described here are defined in the XML `Constitutive` block and assigned to particles through `ParticleRegion` material lists. PFW assigns the particle material index as described in Section 3.6; during the explicit step, that index selects the GEOS constitutive object used at Step 16, Section 2.5.16.

For a continuum particle, the solver supplies the material update with the current or incremental kinematic state, density, volume, temperature, porosity, damage, material direction, and any model-specific history fields. The constitutive model returns the updated Cauchy stress, updated history variables, material wave-speed data used by the CFL estimate, and a constitutive update flag that may

be consumed by the robustness controls in Section 2.12. The common notation used below is

$$\mathbf{F}_p^{n+1} = \text{deformation gradient}, \qquad J_p = \det \mathbf{F}_p^{n+1}, \tag{2.84}$$

$$\Delta\boldsymbol{\epsilon}_p = \text{corotational strain increment}, \qquad \mathbf{D}_p = \frac{1}{2}\left(\mathbf{L}_p + \mathbf{L}_p^T\right), \tag{2.85}$$

$$p = -\frac{1}{3}\operatorname{tr}\boldsymbol{\sigma}, \qquad \mathbf{s} = \boldsymbol{\sigma} + p\mathbf{I}, \qquad q = \sqrt{\frac{3}{2}\mathbf{s} : \mathbf{s}}. \tag{2.86}$$

The sign convention is the GEOS stress convention used by the material implementation; the equations below are written to expose the algorithmic structure rather than to replace the source code for calibration.

2.7.1 Solid models

The MPM-connected solid models share a common explicit-update skeleton. Depending on the model, the update may be a small-strain corotational update, a finite-deformation hyperelastic update based directly on $\mathbf{F}$, or an elastic predictor followed by an inelastic/damage corrector. The following algorithm is the common solver-side pattern; individual models replace the `materialUpdate` line with the specialized laws described in the following subsections.

Listing 2.40: Common explicit MPM continuum material update pattern.

```
for active particle p:
  read material index mat[p]
  gather F[p], J[p], strainIncrement[p], velocityGradient[p]
  gather optional fields: temperature, porosity, damage,
                          strengthScale, materialDirection,
                          distanceToCrackTip
  (stress[p], state[p], waveSpeed[p], updateFlag[p]) =
      materialUpdate(mat[p], stressOld[p], stateOld[p], kinematics[p])
  store stress[p] and state[p]
```

2.7.1.1 `ElasticIsotropic`

`ElasticIsotropic` is the baseline small-strain isotropic elastic material. It is useful for mapping tests, wave-speed verification, contact/boundary tests, and reference calculations when the material deformation is small enough that a corotational elastic increment is adequate. The model is the standard Hooke law form documented in continuum mechanics texts such as Malvern [34].

With bulk modulus K, shear modulus G, and strain increment $\Delta\boldsymbol{\epsilon}$, the update is

$$\boldsymbol{\sigma}^{n+1} = \boldsymbol{\sigma}^n + K\operatorname{tr}(\Delta\boldsymbol{\epsilon})\mathbf{I} + 2G\operatorname{dev}(\Delta\boldsymbol{\epsilon}). \tag{2.87}$$

The wave speed used in explicit time-step estimation is the compressional elastic speed

$$c = \sqrt{\frac{K + 4G/3}{\rho}}. \tag{2.88}$$

Listing 2.41: ElasticIsotropic stress update.

```
trde = trace(strainIncrement)
devde = strainIncrement - trde/3 * I
stress = oldStress + K * trde * I + 2 * G * devde
waveSpeed = sqrt((K + 4*G/3) / density)
```

2.7.1.2 `Hyperelastic`

`Hyperelastic` is the finite-deformation elastic baseline. It is used when the stress should be evaluated from the deformation gradient instead of from an accumulated infinitesimal strain increment. The implementation uses an isochoric-volumetric split typical of compressible neo-Hookean finite elasticity [35, 36]. Let

$$\mathbf{B} = \mathbf{F}\mathbf{F}^T, \qquad J = \det \mathbf{F}, \qquad \bar{\mathbf{B}} = J^{-2/3}\mathbf{B}. \tag{2.89}$$

The current implementation evaluates the Cauchy stress as

$$\boldsymbol{\sigma} = GJ^{-5/3}\mathbf{B} + \left[K(J-1) - \frac{G}{3}J^{-5/3}\operatorname{tr}\mathbf{B}\right]\mathbf{I}. \tag{2.90}$$

This form is equivalent to a compressible neo-Hookean-style response with a volumetric penalty and an isochoric deviatoric contribution. It is a convenient large-deformation elastic model for solver verification, prescribed deformation, and simple contact calculations.

Listing 2.42: Hyperelastic stress update.

```
F = I + FminusI
B = F * transpose(F)
J = det(F)
x1 = G / J^(5/3)
x2 = K * (J - 1) - G * trace(B) / (3 * J^(5/3))
stress = x1 * B + x2 * I
```

2.7.1.3 `HyperelasticMMS`

`HyperelasticMMS` is a manufactured-solution variant of the finite-deformation hyperelastic path. It should be treated as a verification material rather than as a production constitutive law. Its purpose is to pair a known analytical displacement/velocity/stress field with body forces and boundary data so that the particle-grid mapping, explicit integration, and boundary treatment can be tested in a controlled setting. The method of manufactured solutions is a standard code-verification approach [39].

The current MMS hyperelastic stress uses

$$\boldsymbol{\sigma} = \frac{\lambda \ln J}{J}\mathbf{I} + \frac{G}{J}(\mathbf{B} - \mathbf{I}), \tag{2.91}$$

where λ is the first Lame constant. Compared with Eq. (2.90), this form is closer to the commonly used compressible neo-Hookean Cauchy stress.

Listing 2.43: HyperelasticMMS stress update.

```
F = I + FminusI
B = F * transpose(F)
J = det(F)
stress = lambda * log(J) / J * I + G / J * (B - I)
waveSpeed = sqrt((K + 4*G/3) / density)
```

2.7.1.4 `ElasticTransverseIsotropic`

`ElasticTransverseIsotropic` is the baseline anisotropic elastic solid model. It represents a material with one preferred axis, such as a fiber direction, bedding normal, crystal axis, or platelet normal. PFW can assign the particle material basis using `MaterialDirection`, as described in Section 3.6. The theory follows standard transversely isotropic elasticity [37].

In the local material frame, the Voigt stiffness is

$$\mathbf{C}_{\mathrm{TI}} = \begin{bmatrix} c_{11} & c_{12} & c_{13} & 0 & 0 & 0 \\ c_{12} & c_{11} & c_{13} & 0 & 0 & 0 \\ c_{13} & c_{13} & c_{33} & 0 & 0 & 0 \\ 0 & 0 & 0 & c_{44} & 0 & 0 \\ 0 & 0 & 0 & 0 & c_{44} & 0 \\ 0 & 0 & 0 & 0 & 0 & c_{66} \end{bmatrix}, \qquad c_{12} = c_{11} - 2c_{66}. \tag{2.92}$$

The implementation rotates the local stiffness to the current material direction before applying the strain increment,

$$\Delta\boldsymbol{\sigma}^V = \mathbf{M}(\mathbf{R})\mathbf{C}_{\mathrm{TI}}\mathbf{M}(\mathbf{R})^T\Delta\boldsymbol{\epsilon}^V, \tag{2.93}$$

where $\mathbf{M}(\mathbf{R})$ is the Voigt transformation associated with the rotation from the local transverse-isotropy axis to the particle material direction.

Listing 2.44: Transversely isotropic elastic update.

```
a = normalize(materialDirection[p])
R = rotation that maps local z-axis to a
M = VoigtRotationMatrix(R)
Cti = localTransverseIsotropicStiffness(c11, c33, c13, c44, c66)
Crot = M * Cti * transpose(M)
stress = oldStress + Crot * strainIncrement
```

2.7.1.5 `VonMisesJ`

`VonMisesJ` is a pressure-independent J_2 elastoplastic model. It is appropriate when yielding is controlled primarily by deviatoric stress and a constant yield strength is sufficient. The implementation computes volumetric response from the deformation-gradient Jacobian, updates deviatoric stress in a corotational frame, and performs a radial return when the von Mises invariant exceeds the yield strength. This is the standard return-mapping structure for J_2 plasticity [38].

The yield function is

$$f(\boldsymbol{\sigma}) = q - Y, \qquad q = \sqrt{\frac{3}{2}\mathbf{s} : \mathbf{s}}, \tag{2.94}$$

where Y is the current yield strength. In the current implementation, the pressure-like volumetric part is evaluated from

$$p = -K \ln J, \tag{2.95}$$

while the trial deviatoric stress is advanced from the corotational deviatoric rate. If $q^{\mathrm{tr}} > Y$, the stress is recomposed with the original trial pressure and the capped deviatoric invariant

$$\boldsymbol{\sigma}^{n+1} = -p^{\mathrm{tr}}\mathbf{I} + \sqrt{\frac{2}{3}}Y\hat{\mathbf{n}}^{\mathrm{tr}}, \qquad \hat{\mathbf{n}}^{\mathrm{tr}} = \frac{\mathbf{s}^{\mathrm{tr}}}{\|\mathbf{s}^{\mathrm{tr}}\|}. \tag{2.96}$$

Listing 2.45: VonMisesJ radial-return structure.

```
rotate old deviatoric stress to corotational frame
p_trial = -K * log(det(F))
s_trial = old_s + 2 * G * dev(D) * dt
q_trial = sqrt(3/2 * inner(s_trial, s_trial))
if inelasticity disabled or q_trial <= Y:
  stress = -p_trial * I + s_trial
else:
```

```
  n_trial = s_trial / norm(s_trial)
  s_returned = sqrt(2/3) * Y * n_trial
  stress = -p_trial * I + s_returned
  update plasticStrain from elastic/plastic strain split
```

2.7.1.6 `PerfectlyPlastic`

`PerfectlyPlastic` is a simpler elastic-perfectly-plastic isotropic model. It uses the same pressure-independent invariant check as a von Mises material, but does not include hardening. The elastic predictor is the `ElasticIsotropic` update. If the trial stress lies outside the yield surface, the pressure is retained and the deviatoric stress is recomposed with $q = Y$ [38].

$$\boldsymbol{\sigma}^{\mathrm{tr}} = \boldsymbol{\sigma}^{n} + K\,\mathrm{tr}(\Delta\boldsymbol{\epsilon})\mathbf{I} + 2G\,\mathrm{dev}(\Delta\boldsymbol{\epsilon}), \qquad f^{\mathrm{tr}} = q^{\mathrm{tr}} - Y. \tag{2.97}$$

If $f^{\mathrm{tr}} \le 0$, $\boldsymbol{\sigma}^{n+1} = \boldsymbol{\sigma}^{\mathrm{tr}}$. Otherwise,

$$\boldsymbol{\sigma}^{n+1} = -p^{\mathrm{tr}}\mathbf{I} + \sqrt{\frac{2}{3}}Y\hat{\mathbf{n}}^{\mathrm{tr}}. \tag{2.98}$$

Listing 2.46: PerfectlyPlastic stress update.

```
stress_trial = elasticIsotropicUpdate(oldStress, strainIncrement)
(p_trial, q_trial, s_trial) = stressInvariants(stress_trial)
if q_trial <= yieldStress or inelasticity disabled:
  stress = stress_trial
else:
  stress = recomposeStress(p_trial, yieldStress, s_trial)
```

2.7.1.7 `StrainHardeningPolymer`

`StrainHardeningPolymer` is a corotational polymer plasticity model for pressure-independent flow with temperature-dependent elastic properties, shear softening, stretch hardening, and stretch-based failure. The model is related to the large-deformation glassy-polymer framework of Boyce, Parks, and Argon and to the stretch-hardening ideas of Arruda and Boyce [40, 41]. In the current implementation, the elastic predictor is small-strain/corotational, while the stretch-hardening driver is obtained from the right-stretch information associated with the deformation gradient.

The elastic trial stress is

$$\boldsymbol{\sigma}^{\mathrm{tr}} = \boldsymbol{\sigma}^{n} + K(T)\,\mathrm{tr}(\Delta\boldsymbol{\epsilon})\mathbf{I} + 2G(T)\,\mathrm{dev}(\Delta\boldsymbol{\epsilon}). \tag{2.99}$$

The implementation uses logistic thermal scalings of the form

$$S_T(T; T_0, A, B) = 1 + \frac{A}{1 + \exp[B(T - T_0)]}, \tag{2.100}$$

with parameter-specific values of T_0, A, and B. The flow strength is updated from a base yield term, a decaying shear-softening term, and a tensile stretch-hardening term,

$$Y = Y_0(T) + S(T)\exp\left[-\left(\frac{\gamma_p}{r_1}\right)^{r_2}\right] + H(T)\left(\bar{\lambda}^2 - \frac{1}{\bar{\lambda}}\right), \qquad \bar{\lambda} = \max(\lambda_{\max}, 1). \tag{2.101}$$

If the maximum principal stretch exceeds a temperature-scaled failure stretch, the model sets the particle damage to one.

Listing 2.47: StrainHardeningPolymer update structure.

```
rotate old plastic strain and deformation gradient to corotational frame
compute stretch eigenvalues; lambda_max = max(stretch)
if lambda_max > failureStretch(temperature):
  damage = 1
compute trial stress and trial q
for fixed-point iteration:
  gamma_p = norm(plasticStrainOld + plasticStrainIncrement)
  Y = yield0(T) + softening(T, gamma_p) + stretchHardening(T, lambda_max)
  if q_trial <= Y and first iteration: break elastic
  return deviatoric stress to q = min(q_trial, max(Y, 0))
  recompute plasticStrainIncrement from elastic/plastic split
store stress, plasticStrain, yieldStrength, damage
```

2.7.1.8 `CeramicDamage`

`CeramicDamage` is a pressure-dependent brittle damage model for materials whose intact strength depends on confinement and whose failed state should behave more like frictional granular material than like a stress-free void. It is intended primarily for use with damage-field-gradient (DFG) partitioning (Section 2.11.3). In that mode the continuum damage field identifies a fracture surface, DFG inserts the kinematic discontinuity, and the contact algorithm then permits slip, separation, and dilation on the fracture surfaces. The model therefore does not attempt to represent all post-fracture dilatation as a continuum porosity increment. When continuum pore volume, pore collapse, or pressure-sensitive compaction of the bulk material is the central physics, the `Geomechanics` model in Section 2.7.1.11 is the more appropriate starting point.

The model is a hybrid elastic-plastic/damage law. The volumetric response is hyperelastic in the Jacobian, while the deviatoric response is limited by a damage-dependent pressure-dependent strength surface. The implementation is related to the continuum-damage regularization framework described by Homel, Crook, and Appleton for under-resolved brittle crack tips and to earlier MPM-DFG mesoscale brittle-material calculations, including the concrete mesoscale work of Homel, Iyer, Semnani, and Herbold [32, 33, 6]. The model is deliberately simple compared with full cap/plasticity or micromechanics models: it uses a small number of strength parameters and relies on DFG contact, rather than continuum porosity growth, to represent the kinematics of opened or sliding fractures.

Primary input parameters and state. The required strength parameters are the unconfined tensile strength Y_t, the unconfined compressive strength Y_c, and the maximum high-pressure shear strength $Y_{\max}$. The optional residual friction slope μ is controlled by `damagedMaterialFrictionSlope`; the optional third-invariant correction is enabled by `thirdInvariantDependence`. Damage is stored as the scalar particle state $D \in [0, 1]$. The model also owns or reads `jacobian`, `lengthScale`, `strengthScale`, `porosity`, `referencePorosity`, `plasticStrain`, `distanceToCrackTip`, `surfaceFlag`, `crackTipStressConcentration`, and accumulated work diagnostics. The most important user-visible controls are summarized in Table 2.3.

Table 2.3: Main `CeramicDamage` controls and state variables.

Name	Role	Notes
`tensileStrength`	input	Unconfined tensile strength scale Y_t.
`compressiveStrength`	input	Unconfined compressive strength scale Y_c; must exceed Y_t.
`maximumStrength`	input	High-pressure limiting shear strength $Y_{\max}$; must exceed Y_c.

Name	Role	Notes
`damagedMaterialFrictionSlope`	input	Residual damaged-material friction slope μ. The brittle-ductile transition pressure is approximately $p_2 = Y_{\max}/\mu$.
`thirdInvariantDependence`	input	Enables Lode-angle scaling so the low-pressure surface approaches a maximum-principal-stress-like tensile criterion while the high-pressure response approaches a shear criterion.
`crackSpeed`	input	Time-to-failure regularization speed used when `enableEnergyFailureCriterion=0`.
`enableEnergyFailureCriterion`	input	Enables fracture-energy regularization using `fractureEnergyReleaseRate/lengthScale`.
`fractureEnergyReleaseRate`	input	Mode-I-style fracture energy per created area used by the energy regularization.
`enableCrackTipStressConcentration`	input	Enables the DFG-based crack-tip stress correction described in Section 2.7.3.2.
`fractureToughness`	input	Toughness used to compute the fracture-process-zone radius for crack-tip scaling.
`strengthScale`	particle field	Multiplies Y_t and Y_c; see the Weibull and spatial-strength discussion in Section 2.7.3.1.
`porosity`	particle field	Reduces Y_t and Y_c as a local strength factor in the current implementation; it is not the primary mechanism for fracture dilation.
`surfaceFlag`	particle field	Set by the model when the kinematic regularization has dissipated the target energy; DFG can then treat the particle as part of an inserted surface.

Strength scaling, porosity scaling, and pressure. At the beginning of the material update, the particle tensile and compressive strengths are scaled by the particle-level heterogeneity and porosity fields,

$$\widehat{Y}_t = s_p(1-\phi_p)Y_t, \qquad \widehat{Y}_c = s_p(1-\phi_p)Y_c, \tag{2.102}$$

where $s_p =$ `strengthScale` and $\phi_p =$ `porosity`. The scaled strengths are then capped so that the compressive branch remains below $Y_{\max}$ and the tensile/compressive strength ratio remains bounded. If the third-invariant correction is active, the implementation also computes a tensile-strength parameter Y_{t0} that gives the requested tensile strength after the Lode-angle scaling is applied. In code form,

$$Y_{t0} \approx \operatorname{clip}\left[\frac{3\widehat{Y}_c\widehat{Y}_t}{2\widehat{Y}_c+\widehat{Y}_t}, \frac{1}{2}\widehat{Y}_t, 2\widehat{Y}_t\right], \qquad Y_{t0} \le \frac{(3-\mu)\widehat{Y}_c}{3+\mu}, \tag{2.103}$$

with $Y_{t0} = \widehat{Y}_t$ if the third-invariant option is disabled. The second inequality prevents damage from producing an artificial hardening branch relative to the residual friction slope.

The model computes a hyperelastic pressure from the stored deformation Jacobian,

$$p^{\mathrm{tr}} = -K_{\mathrm{eff}} \ln J, \qquad K_{\mathrm{eff}} = \begin{cases} K, & J \le 1, \\ \max(10^{-9}K, (1-D)K), & J > 1, \end{cases} \tag{2.104}$$

so tensile volumetric stiffness is reduced as damage grows while compressive stiffness remains intact. The undamaged tensile vertex pressure is

$$p_0 = -\frac{2\widehat{Y}_c Y_{t0}}{3(\widehat{Y}_c - Y_{t0})}, \qquad p_{\min}(D) = (1-D)p_0. \tag{2.105}$$

If $p^{\mathrm{tr}} < p_{\min}(D)$, the return algorithm moves the stress to the tensile vertex $\boldsymbol{\sigma} = -(1-D)p_0\mathbf{I}$. Otherwise the pressure is retained and the deviatoric stress is returned to the pressure-dependent shear strength.

Yield surface and third-invariant dependence. The yield function is evaluated in terms of the von Mises equivalent shear measure $q = \sqrt{3J_2}$, pressure p, damage D, and optionally the third invariant J_3:

$$f = q - \frac{1}{\xi_{\text{tip}}}\,\Gamma^{-1}(J_2, J_3, \partial Y_0/\partial p)\, Y_0(p, D), \tag{2.106}$$

where $\xi_{\text{tip}} \geq 1$ is the crack-tip stress-concentration factor and $\Gamma^{-1} = 1$ when `thirdInvariantDependence=0`. The meridional strength $Y_0(p, D)$ has three pressure ranges,

$$Y_0(p, D) = \begin{cases} Y_1(p, D), & p_0 < p \leq p_1, \\ Y_2(p, D), & p_1 < p < p_2, \\ Y_{\max}, & p \geq p_2, \end{cases} \qquad p_1 = \widehat{Y}_c/3, \qquad p_2 = Y_{\max}/\mu. \tag{2.107}$$

The low-pressure branch Y_1 is linear in pressure and evolves with damage from an intact cohesive surface to a residual frictional slope. The middle branch Y_2 smoothly blends toward the constant high-pressure strength $Y_{\max}$. When enabled, the third-invariant correction modifies this meridional surface through a Lode-angle factor so that the model approximates tensile failure at low confinement and shear-dominated yielding at high confinement. Figure 2.1 shows the resulting surface families.

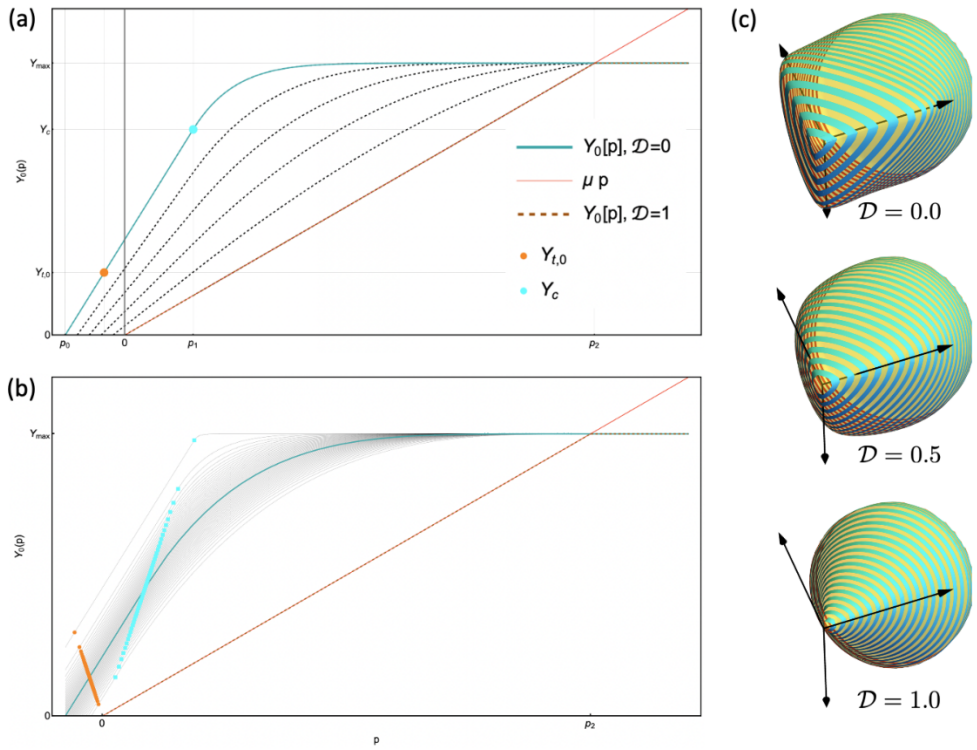


Figure 2.1: Yield surface used by the isotropic ceramic damage model. Panel (a) shows the pressure-dependent meridional strength without third-invariant dependence, evolving from intact to damaged states. Panel (b) shows Weibull/strength-scale perturbations of the tensile and compressive strengths subject to the high-pressure cap. Panel (c) shows the initial principal-stress-space surface for several damage values, illustrating the transition from low-pressure tensile failure toward high-pressure shear strength. Adapted from Fig. 8 of Homel, Crook, and Appleton [32].

Return mapping. The implementation uses a simple radial return in deviatoric stress. After the pressure branch is selected, the trial deviatoric direction is retained and the returned shear magnitude is limited by the damaged strength,

$$q^{n+1} = \min\left(q^{\mathrm{tr}}, Y(p^{\mathrm{tr}}, D, J_2, J_3)/\xi_{\mathrm{tip}}\right), \qquad \boldsymbol{\sigma}^{n+1} = -p^{n+1}\mathbf{I} + \frac{q^{n+1}}{q^{\mathrm{tr}}}\mathbf{s}^{\mathrm{tr}}. \tag{2.108}$$

The elastic strain energy stored at the returned state is evaluated as

$$W_e = \frac{(p^{n+1})^2}{2K_{\mathrm{eff}}} + \frac{(q^{n+1})^2}{6G}, \tag{2.109}$$

which is used by the energy-regularized damage path. The stored `plasticStrain` is a diagnostic/history measure computed from the difference between the elastic strain increment implied by the stress change and the total strain increment; it should not be interpreted as a fully associative plastic-flow solution.

Damage regularization options. Two damage-progression paths are implemented. The older time-to-failure path is selected when `enableEnergyFailureCriterion=0`. If the trial state yields below the brittle-ductile transition pressure p_2, damage increases according to

$$D^{n+1} = \min\left(1, D^n + \frac{\Delta t}{t_f}\right), \qquad t_f = \frac{\ell_{\mathrm{ch}}}{c_f}, \tag{2.110}$$

where $\ell_{\mathrm{ch}} =$ `lengthScale` and $c_f =$ `crackSpeed`. This is computationally inexpensive and useful for simple damage growth studies, but the fracture energy is controlled only indirectly by c_f, ℓ_{ch}, and the stress history.

The energy-regularized path is selected when `enableEnergyFailureCriterion=1`. In that branch the model tracks accumulated stress work W and compares the dissipated work $W - W_e$ to the regularized fracture-energy density G_f/ℓ_{ch}:

$$D^{n+1} \approx \mathrm{clip}\left(\frac{W^{n+1} - W_e^{n+1}}{G_f/\ell_{\mathrm{ch}}}, D^n, 1\right). \tag{2.111}$$

If the strain energy at failure is small enough for a gradual softening process, the update performs a fixed-point iteration on D. If the strain energy at the failure stress already exceeds the target fracture dissipation for the current ℓ_{ch}, the update instead uses a bisection solve for a partially relaxed damage state and sets `surfaceFlag=1` once the dissipated work reaches approximately G_f/ℓ_{ch}. That flag is what makes the option useful with DFG: the continuum law can release only the required amount of energy, while the kinematic enrichment creates a contact-capable fracture surface so the residual elastic energy drives opening or slip rather than being smeared into a continuum porosity field. The regularization concept follows the Homel-Crook-Appleton kinematic-regularization discussion, in which damage, surface insertion, and the energy budget are decoupled when a cell is too large to resolve the fracture process zone [32].

Strength scale and crack-tip scaling. The `strengthScale` field is applied directly to Y_t and Y_c, so Weibull or spatial heterogeneity assigned by PFW affects both tensile and compressive branches while leaving $Y_{\max}$ as the limiting ductile strength. This is the model-specific interpretation of the shared strength-scale field described in Section 2.7.3.1. In DFG fracture calculations, `strengthScale` is commonly assigned by Voronoi/Weibull wrappers so that the initial critical flaws occupy clusters of particles rather than isolated particles.

When crack-tip scaling is enabled, the solver supplies `distanceToCrackTip` from the neighbor-list DFG crack-tip diagnostic. The model computes a fracture-process-zone radius from the nominal intact strength,

$$r_p = \frac{K_{Ic}^2}{2\pi Y_{\text{intact}}^2}, \qquad \xi_{\text{tip}} = \max\left(1, \sqrt{\frac{r}{r_p}}\right), \tag{2.112}$$

and divides the effective strength by ξ_{tip} when evaluating the yield condition and the return surface. The stress that is stored and used in momentum balance is not multiplied by this factor; the factor modifies the damage/fracture criterion to represent under-resolved near-tip stresses. This links the ceramic model to the shared crack-tip modifier in Section 2.7.3.2. The uploaded Homel-Crook-Appleton manuscript describes the same idea: the crack-tip factor affects the propagation criterion, while the final output stress and stress work are computed with the unscaled stress [32].

DFG use and porosity interpretation. The model can run without DFG, but its intended brittle-fracture workflow is `CeramicDamage`+DFG. Without DFG, a fully damaged region remains a continuum carried by the single MPM velocity field and can smear damaged/intact kinematics. With DFG, particles near a sufficiently sharp damage gradient can be split into two velocity fields, and the contact algorithm then supplies frictional slip and separation on the inserted fracture. This is important for ceramics, concrete mesostructures, and brittle granular aggregates, where post-failure deformation is often controlled by relative motion of rough fracture faces or fragments. The `porosity` field in `CeramicDamage` currently scales strength through Eq. (2.102); it should not be used as the primary representation of dilation caused by fracture slip. If the problem requires an evolving continuum porosity, pore-pressure coupling, pore collapse, or compaction-induced void-ratio evolution, use or extend `Geomechanics` (Section 2.7.1.11) rather than interpreting DFG fracture dilation as a scalar porosity increment.

Implementation pseudocode.

Listing 2.48: CeramicDamage update structure.

```
read D, J, lengthScale, strengthScale, porosity, distanceToCrackTip
Yt = strengthScale * (1 - porosity) * tensileStrength
Yc = strengthScale * (1 - porosity) * compressiveStrength
cap Yc by maximumStrength and form Yt0 for optional third-invariant scaling
p_trial = -K_eff(D,J) * log(J)
q_trial, J2, J3, s_hat = deviatoric_invariants(trialStress)
Y_intact = strength(p_trial, D=0, xi=1, J2, J3)
if crack_tip_option and distanceToCrackTip > 0:
  rp = fractureToughness^2 / (2*pi*Y_intact^2)
  xi = max(1, sqrt(distanceToCrackTip / rp))
else:
  xi = 1
Y = strength(p_trial, D, xi, J2, J3)
if p_trial >= (1-D)*p0 and q_trial <= Y:
  accept trial stress
  if energy criterion: accumulatedWork += stress : strainIncrement
else:
  if energy criterion:
    iterate D so (accumulatedWork - elasticEnergy) ~= Gf/lengthScale
    if target dissipation reached: surfaceFlag = 1
  else:
    if p_trial < maximumStrength/frictionSlope:
      D = min(1, D + dt / (lengthScale/crackSpeed))
  radial_return_to_strength_surface(D, xi)
update plasticStrain diagnostic and store stress, D, work, surfaceFlag
```

2.7.1.9 `Chiumenti`

`Chiumenti` is a scalar tensile-damage model built on the `HyperelasticMMS` trial stress. It follows the local continuum damage / crack-band style of Rankine tensile softening used in the work of Cervera and Chiumenti [42]. In GEOS-MPM, it provides a compact hyperelastic-damage option for cases that require tensile degradation with a characteristic length and fracture-energy parameter, but not the full ceramic or geomechanics response.

The model computes a hyperelastic trial stress, obtains principal stresses σ_i, and defines a strength

$$Y_f = s_p Y_{f0}, \tag{2.113}$$

where s_p is the particle `strengthScale`. With Young's modulus E, critical length ℓ_c, and energy-release parameter G_f, the implementation uses a brittleness factor

$$b = \frac{Y_f^2}{2EG_f}, \qquad b_c = \frac{b\ell_c}{1 - b\ell_c}. \tag{2.114}$$

When the maximum tensile principal stress $\sigma_{\max}$ exceeds Y_f, the damage candidate is

$$D_{\text{new}} = (1 + b_c)\left(1 - \frac{Y_f}{\sigma_{\max}}\right), \qquad D^{n+1} = \max\left(D^n, \min(1, \max(0, D_{\text{new}}))\right). \tag{2.115}$$

Positive principal stresses are then degraded by $1 - D$, while compressive principal stresses are left unchanged.

Listing 2.49: Chiumenti tensile-damage update.

```
stress_trial = HyperelasticMMS(F)
if inelasticity disabled: return stress_trial
Yf = failureStrength * strengthScale
principalStress, eigenvectors = eig(stress_trial)
sigmaMax = max(max(principalStress), 0)
if sigmaMax > Yf:
  b = Yf^2 / (2 * E * energyReleaseRate)
  bc = b * criticalLength / (1 - b * criticalLength)
  damage = max(oldDamage, clamp((1 + bc) * (1 - Yf/sigmaMax), 0, 1))
for each principal stress i:
  if principalStress[i] > 0: principalStress[i] *= (1 - damage)
stress = recompose(principalStress, eigenvectors)
```

2.7.1.10 `Graphite`

`Graphite` is an anisotropic damage/plasticity model for graphite-like layered solids. The model assumes that the material has a preferred graphitic or basal-plane normal, stored as the first row of the particle `MaterialDirection` tensor. The weak planes themselves are normal to this vector. The constitutive update therefore treats the particle material direction as a plane normal, not as an ordinary fiber tangent. This distinction matters in large deformation: ordinary line directions are pushed forward with $\mathbf{F}$, while plane normals are pushed forward with the deformation-gradient cofactor $\mathbf{F}^c = J\mathbf{F}^{-T}$. The MPM kinematic update uses this cofactor path for non-fiber material directions, normalizes the transported rows, and then passes the updated material basis to the constitutive model. This is the appropriate graphitic update because the orientation of a basal plane is controlled by the transformed plane normal rather than by the stretch of an in-plane material line; see also the material-direction assignment discussion in Section 3.6.3. The need to distinguish material rotations, basis transformations, and evolving anisotropy is the same tensor-transformation issue emphasized by Brannon's rotation and frame-change treatment [48].

Kinematics and material basis. Let $\mathbf{a}$ be the current unit normal to the graphitic planes and let

$$\mathbf{M} = \mathbf{a} \otimes \mathbf{a}, \qquad \mathbf{P} = \mathbf{I} - \mathbf{M}, \tag{2.116}$$

where $\mathbf{P}$ projects onto the basal plane. In the current implementation the full particle material basis is stored row-wise, but the constitutive update uses only the first row as $\mathbf{a}$. The solver first transports the reference basis by

$$\mathbf{A}^{\text{trial}} = \begin{cases} \mathbf{F}\mathbf{A}_0^T, & \text{fiber-like material directions,} \\ \mathbf{F}^c\mathbf{A}_0^T, & \text{graphitic plane normals and other non-fiber directions,} \end{cases} \qquad \mathbf{F}^c = J\mathbf{F}^{-T}, \tag{2.117}$$

then transposes and normalizes the rows to recover the particle `MaterialDirection`. Inside the constitutive update, the beginning-of-step rotation is removed before evaluating the transversely isotropic elastic and inelastic laws, so the mode decomposition is performed in a corotational material frame. This path is consistent with the general rule that anisotropic constitutive models must evolve their material directions under superposed rigid motion and finite distortion, rather than treating a fixed laboratory axis as a material property [48].

Elastic trial stress. The reviewed code path is a finite-deformation MPM update followed by a corotational transversely isotropic elastic predictor. The implementation is compatible with the

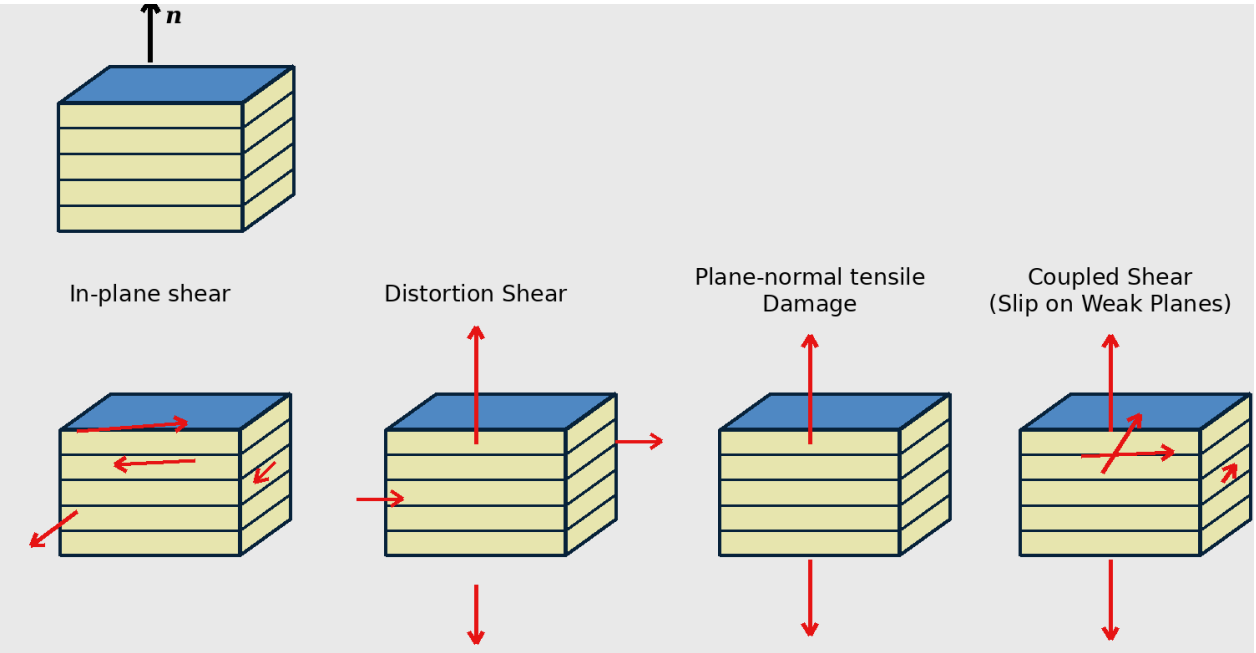


Figure 2.2: Graphite weak-plane deformation modes. The supplied diagram illustrates the basal-plane normal, in-plane shear, distortion shear, plane-normal tensile damage, and coupled shear/slip on weak planes. The assigned material direction **a** is the normal to the weak graphitic planes; in-plane, distortion, plane-normal, and coupled weak-plane stress modes are limited by separate pressure-dependent strength functions.

hyperelastic transversely isotropic interpretation of the model, but the update evaluated here is algorithmically a pressure-dependent rate-form trial stress expressed in the evolving material frame. The pressure-dependent moduli are computed from the beginning-of-step pressure $p^n = -\operatorname{tr}\boldsymbol{\sigma}^n/3$ and the beginning-of-step plane-normal stress $\sigma_n^n = \mathbf{a}\cdot\boldsymbol{\sigma}^n\mathbf{a}$:

$$E_z = \max\left(10^{-3}E_z^0, E_z^0 + E_z'\max\left(0, \frac{p^n - \sigma_n^n}{2}\right)\right), \tag{2.118}$$

$$E_p = \max\left(10^{-3}E_p^0, E_p^0 + E_p'\max(0, p^n)\right), \tag{2.119}$$

$$G_{zp} = \max\left(10^{-3}G_{zp}^0, G_{zp}^0 + G_{zp}'\max(0, p^n)\right), \tag{2.120}$$

$$\nu_{zp} = \min(0.4999, \nu_{zp}^0), \qquad \nu_p = \min(0.4999, \nu_p^0). \tag{2.121}$$

Here z denotes the preferred-axis direction and p denotes the transverse plane. The implementation also forms effective isotropic moduli for wave-speed and stress-control purposes,

$$K_{\text{eff}} = -\frac{E_pE_z}{2E_z(\nu_p + \nu_{zp} - 1) + E_p(2\nu_{zp} - 1)}, \qquad G_{\text{eff}} = 0.6K_{\text{eff}}. \tag{2.122}$$

The elastic stiffness is written using the five transversely isotropic fourth-order basis tensors associated with the symmetry axis **a**. Brannon gives the corresponding TI basis and discusses the physical meaning of these components as axial response, in-plane isotropic response, Poisson coupling, in-plane shear, and

shear involving the symmetry axis [48]. In component form, with a_i denoting the components of $\mathbf{a}$,

$$B^{(1)}_{ijpw} = a_i a_j a_p a_w, \tag{2.123}$$

$$B^{(2)}_{ijpw} = \delta_{ij}\delta_{pw} - a_i a_j \delta_{pw} - \delta_{ij} a_p a_w + a_i a_j a_p a_w, \tag{2.124}$$

$$B^{(3)}_{ijpw} = a_i a_j \delta_{pw} + \delta_{ij} a_p a_w - 2 a_i a_j a_p a_w, \tag{2.125}$$

$$B^{(4)}_{ijpw} = \frac{1}{2}\left(\delta_{ip}\delta_{jw} + \delta_{iw}\delta_{jp}\right) - \frac{1}{2}\left(\delta_{iw} a_j a_p + a_i \delta_{jp} a_w + a_i \delta_{jw} a_p + \delta_{ip} a_j a_w\right) + a_i a_j a_p a_w, \tag{2.126}$$

$$B^{(5)}_{ijpw} = \frac{1}{2}\left(\delta_{ip} a_j a_w + \delta_{iw} a_j a_p + a_i \delta_{jw} a_p + a_i \delta_{jp} a_w\right) - 2 a_i a_j a_p a_w. \tag{2.127}$$

The stiffness used by the trial update is

$$\mathbb{C}_{\mathrm{TI}} = h_1 \mathbb{B}^{(1)} + h_2 \mathbb{B}^{(2)} + h_3 \mathbb{B}^{(3)} + h_4 \mathbb{B}^{(4)} + h_5 \mathbb{B}^{(5)}, \tag{2.128}$$

with

$$h_1 = \frac{E_z^2(\nu_p - 1)}{E_z(\nu_p - 1) + 2E_p\nu_{zp}^2}, \qquad h_2 = -\frac{E_p(E_z\nu_p + E_p\nu_{zp}^2)}{(1+\nu_p)\left[E_z(\nu_p - 1) + 2E_p\nu_{zp}^2\right]}, \tag{2.129}$$

$$h_3 = \frac{E_p E_z \nu_{zp}}{E_z - E_z\nu_p - 2E_p\nu_{zp}^2}, \qquad h_4 = \frac{E_p}{1+\nu_p}, \qquad h_5 = 2G_{zp}. \tag{2.130}$$

The trial stress increment is

$$\Delta\boldsymbol{\sigma}^{\mathrm{tr}} = \mathbb{C}_{\mathrm{TI}} : \left(\mathbf{D} - \boldsymbol{\alpha}\dot{T}\right)\Delta t, \qquad \boldsymbol{\alpha} = (\alpha_L - \alpha_T)\mathbf{a}\otimes\mathbf{a} + \alpha_T\mathbf{I}, \tag{2.131}$$

where $\dot{T} =$ `temperatureRate`. Thus thermal expansion is anisotropic: α_L is applied along the symmetry-axis normal and α_T in the basal plane. In current GEOS-MPM workflows, `temperature` and `temperatureRate` are generally prescribed by global solver events that define a transient thermal response; the graphite update consumes these fields to represent thermal expansion and temperature-dependent properties. A future coupled heat-transfer solver could instead define local particle temperature and temperature-rate fields.

Mode decomposition of stress. After the trial stress is computed, the model decomposes it into four symmetric tensor parts. In the notation of Eq. (2.127),

$$\boldsymbol{\sigma}_1 = \mathbb{B}^{(1)} : \boldsymbol{\sigma}, \qquad \text{plane-normal/axial part}, \tag{2.132}$$

$$\boldsymbol{\sigma}_2 = \mathbb{B}^{(2)} : \boldsymbol{\sigma}, \qquad \text{in-plane isotropic part before the factor } 1/2, \tag{2.133}$$

$$\boldsymbol{\sigma}_4 = \mathbb{B}^{(4)} : \boldsymbol{\sigma}, \qquad \text{in-plane total part}, \tag{2.134}$$

$$\boldsymbol{\sigma}_5 = \mathbb{B}^{(5)} : \boldsymbol{\sigma}, \qquad \text{coupled weak-plane shear part}. \tag{2.135}$$

The in-plane isotropic contribution used for reconstruction is $\boldsymbol{\sigma}_{\mathrm{ip}}^{\mathrm{iso}} = \boldsymbol{\sigma}_2/2$, while the in-plane deviator is

$$\boldsymbol{\sigma}_{\mathrm{ip}}^{\mathrm{dev}} = \boldsymbol{\sigma}_4 - \frac{1}{2}\boldsymbol{\sigma}_2. \tag{2.136}$$

The distortion stress is the combination of the plane-normal and in-plane-isotropic parts,

$$\boldsymbol{\sigma}_{\mathrm{dist}} = \boldsymbol{\sigma}_1 + \frac{1}{2}\boldsymbol{\sigma}_2, \qquad \boldsymbol{\sigma}_{\mathrm{dist}}^{\mathrm{dev}} = \operatorname{dev}\boldsymbol{\sigma}_{\mathrm{dist}}. \tag{2.137}$$

The corresponding mode invariants are evaluated with a Mandel-consistent norm,

$$q_{\mathrm{dist}} = \sqrt{\frac{3}{2}}\,\|\boldsymbol{\sigma}_{\mathrm{dist}}^{\mathrm{dev}}\|, \qquad q_{\mathrm{ip}} = \sqrt{\frac{3}{2}}\,\|\boldsymbol{\sigma}_{\mathrm{ip}}^{\mathrm{dev}}\|, \qquad q_{\mathrm{c}} = \sqrt{\frac{3}{2}}\,\|\boldsymbol{\sigma}_5\|. \tag{2.138}$$

These are the internal stress measures corresponding to distortion shear, in-plane shear, and coupled shear on the graphitic weak planes in Fig. 2.2. Plane-normal tensile opening is handled separately by comparing $\sigma_n = \mathbf{a}\cdot\boldsymbol{\sigma}\mathbf{a}$ with `failureStrength`.

Pressure-dependent point-slope strengths. Each shear mode $m \in \{\mathrm{dist}, \mathrm{ip}, \mathrm{c}\}$ has an independent pressure-dependent strength curve controlled by `*ResponseX2`, `*ResponseY1`, `*ResponseY2`, and `*ResponseM1`. With $x_1 = 0$, $x_2 > 0$, $Y_1 \geq 0$, $Y_2 > Y_1$, and low-pressure tangent slope M_1, the code uses a point-slope form

$$Y_m(p) = \begin{cases} \max\left(0, Y_1 - (x_1 - p)M_1\right), & p < x_1, \\ Y_2 + (Y_1 - Y_2)\left(\dfrac{p - x_2}{x_1 - x_2}\right)^{M_1(x_1 - x_2)/(Y_1 - Y_2)}, & x_1 \leq p < x_2, \\ Y_2, & p \geq x_2. \end{cases} \tag{2.139}$$

This curve passes through (x_1, Y_1) with derivative M_1, passes through (x_2, Y_2), and is tangent to the high-pressure branch, which is a zero-slope plateau in the present form. The elastic domain is convex only if the low-pressure slope is steeper than the secant slope, so the intended parameter regime is

$$M_1 > \frac{Y_2 - Y_1}{x_2 - x_1}. \tag{2.140}$$

After damage and hardening modifications the implementation enforces this condition by increasing M_1 when needed, with a small numerical margin in production inputs preferred over exact equality.

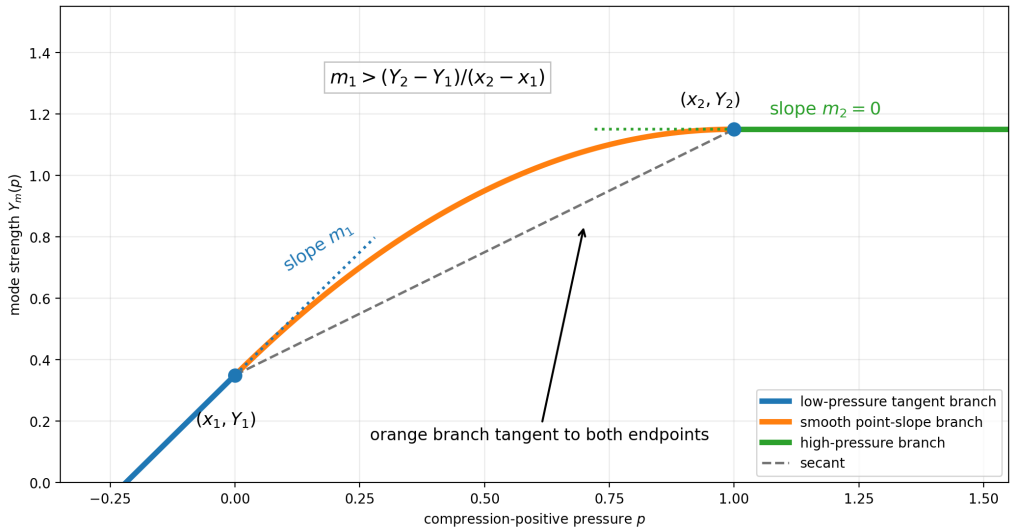


Figure 2.3: Pressure-dependent point-slope strength form used independently for the distortion, in-plane shear, and coupled weak-plane shear modes. The orange transition is constructed to be tangent to the low-pressure branch at (x_1, Y_1) and tangent to the high-pressure plateau at (x_2, Y_2); convexity requires $M_1 > (Y_2 - Y_1)/(x_2 - x_1)$.

Damage, strength scaling, and hardening modify the raw input strengths before Eq. (2.139) is evaluated. Let $s_p =$ `strengthScale`, $D \in [0, 1]$ be scalar damage, $\chi =$ `relaxation` $\in [0, 1]$ be the accumulated plastic-strain hardening coordinate, and

$$S(\chi) = 3\chi^2 - 2\chi^3, \qquad H_m(\chi) = 1 + (C_m - 1)S(\chi). \tag{2.141}$$

Here C_m is one of `distortionStrainHardeningC0`, `inPlaneStrainHardeningC0`, or `coupledStrainHardeningC0`. The low-pressure cohesion-like strength is scaled as

$$Y_1 \leftarrow Y_1^0 s_p (1 - D) H_m, \qquad Y_2 \leftarrow Y_2^0 s_p H_m, \tag{2.142}$$

while the low-pressure slope is interpolated toward the failed-material friction slope,

$$M_1 \leftarrow (1-D)M_1^0 H_m + D\, M_{\text{failed}}. \tag{2.143}$$

The key distinction is that damage removes cohesive low-pressure strength and changes the low-pressure slope, while strain hardening raises the active envelope through the relaxation coordinate. The use of `strengthScale` should be read together with the shared Weibull modifier in Section 2.7.3.1; the graphite model applies the scale to `failureStrength` and to the three pressure-dependent mode strengths, and can optionally scale fracture-energy parameters through `scaleFractureEnergyReleaseRate`.

Mode return and reconstruction. The inelastic correction is a component-wise radial cap in each mode rather than a single closest-point return to a coupled anisotropic yield surface. If $q_m > Y_m(p)$, the corresponding stress part is scaled by Y_m/q_m. After all active mode caps are applied, the stress is reconstructed as

$$\boldsymbol{\sigma}^{n+1} = \boldsymbol{\sigma}_{\text{dist}}^{\text{iso}} + \boldsymbol{\sigma}_{\text{dist}}^{\text{dev,capped}} + \boldsymbol{\sigma}_{\text{ip}}^{\text{dev,capped}} + \boldsymbol{\sigma}_{5}^{\text{capped}}. \tag{2.144}$$

For fully damaged particles, the implementation also removes tensile plane-normal stress and spherical tensile distortion stress, so the damaged material can continue to carry compressive/frictional response but cannot support cohesive opening strength.

Plastic strain, hardening, and damage evolution. When at least one mode cap is active, the model computes a plastic strain increment by subtracting the elastic strain increment implied by the TI compliance from the total symmetric velocity-gradient increment. The TI compliance uses coefficients

$$s_1 = \frac{1}{E_z}, \qquad s_2 = -\frac{\nu_p}{E_p}, \qquad s_3 = -\frac{\nu_{zp}}{E_z}, \qquad s_4 = \frac{1+\nu_p}{E_p}, \qquad s_5 = \frac{1}{2G_{zp}}. \tag{2.145}$$

The relaxation/hardening coordinate is then advanced approximately as

$$\chi^{n+1} = \min\left(1, \chi^n + \frac{\|\Delta\boldsymbol{\epsilon}^p\|}{\epsilon^p_{\max}}\right), \tag{2.146}$$

where $\epsilon^p_{\max} = $ `maximumPlasticStrain`. This history variable is the argument in Eq. (2.141).

Damage can evolve through several mechanisms. First, when `maximumPrincipalStressDamage=1`, the model evaluates the largest principal stress and advances damage if

$$C_{\text{tip}}\, \sigma_{\max} > s_p\, \sigma_f, \tag{2.147}$$

where $\sigma_f = $ `failureStrength` and $C_{\text{tip}} = $ `crackTipStressConcentration`. The crack-tip multiplier is produced by the shared crack-tip modifier described in Section 2.7.3.2. Second, if `crackSpeed` is finite and an inelastic correction has occurred, damage advances over the time-to-failure scale

$$\Delta D = \frac{\Delta t}{\ell/c_f}, \qquad \ell = \texttt{lengthScale}, \quad c_f = \texttt{crackSpeed}. \tag{2.148}$$

Third, the model accumulates basal-plane plastic work and non-basal plastic work. The basal work combines Mode-I basal-plane plastic opening work and basal-plane shear work; the total work path excludes the basal plane-normal and basal shear parts. These work measures are regularized by fracture energy per unit characteristic length,

$$D \leftarrow \max\left(D, \left[\min\left(1, \frac{W_b}{G_b/\ell}\right)\right]^{32}, \left[\min\left(1, \frac{W_t}{G_t/\ell}\right)\right]^{32}\right), \tag{2.149}$$

where $G_b = $ `basalPlaneFractureEnergyReleaseRate` and $G_t = $ `totalFractureEnergyReleaseRate`. If `scaleFractureEnergyReleaseRate=1`, these fracture-energy values are multiplied by `strengthScale` before the work ratio is evaluated. Equation (2.149) makes damage remain near zero until the accumulated work approaches the specified regularized fracture-work threshold, then rapidly ramps it toward one.

Thermal effects. Thermal effects in the current model enter through anisotropic thermal expansion in Eq. (2.131) and through any temperature-dependent parameter paths exposed by the material model. In the current solver workflow, `temperature` and `temperatureRate` are normally imposed by global events, such as a `TemperatureProfile` or `TemperatureRamp`, to represent a prescribed transient thermal response. The graphite constitutive update consumes `temperatureRate` to subtract $\boldsymbol{\alpha}\dot{T}$ from the mechanical rate of deformation. Local temperature fields could instead be supplied in the future by a coupled heat-transfer solver, because temperature is already a particle field shared by PFW initialization, events, output, and constitutive models as described in Section 2.7.3.4.

Update algorithm.

Listing 2.50: Graphite constitutive update structure.

```
read old stress, velocity gradient, F, materialDirection, damage, temperatureRate
transport/normalize material direction; use first row as basal-plane normal a
unrotate L and a into the corotational material frame
compute pressure p, plane-normal stress sigma_n, and pressure-dependent TI moduli
construct TI stiffness from Brannon basis tensors B1...B5
stress_trial = oldStress + C_TI : (D - alpha * temperatureRate) * dt
if inelasticity is disabled: accept stress_trial

decompose stress_trial into sigma1, sigma2, sigma4, sigma5
compute distortion, in-plane, and coupled shear invariants
for each mode:
  compute strength Y_m(p) from the point-slope curve
  apply strengthScale, damage softening, failed slope, and hardening multiplier
  if q_m > Y_m: radially scale that stress part to q_m = Y_m
reconstruct stress from capped distortion, in-plane, and coupled parts
if fully damaged: remove tensile plane-normal and spherical tensile strength

if any mode yielded:
  compute plastic strain increment from total strain increment minus TI elastic increment
  update relaxation/hardening coordinate
  update max-principal, crack-speed, basal-work, and total-work damage paths
store stress, damage, relaxation, plastic strain, plastic work, and diagnostics
```

Capabilities and limitations. The model is useful when the important physics are anisotropic graphitic weak planes, pressure-dependent shear strength, basal-plane slip/opening, thermal expansion anisotropy, and coupling to DFG fracture/contact. It is not a general porous geomaterial model: continuum porosity, pore collapse, pressure-solution creep, and pore-pressure coupling should be handled by the `Geomechanics` family or by a future coupled model. It is also not a fully coupled anisotropic plasticity return algorithm in the sense of a single convex yield surface with a consistent flow rule; instead, it applies radial caps to independent mode stresses and then reconstructs the stress. The strict stored-energy hyperelastic interpretation is likewise limited, because the reviewed code computes a corotational rate-form trial stress. Finally, the constitutive update uses only the first material-direction row as the basal-plane normal, so users should initialize unused material-basis rows sensibly for robust kinematic transport and output, even though they do not directly alter the graphite strength calculation.

Table 2.4: Main `Graphite` input groups. See the generated schema appendix for exact defaults and registration status.

Input group	Purpose

`defaultYoungModulusAxial`
`defaultYoungModulusTransverse`
`defaultShearModulusAxialTransverse`
`defaultPoissonRatioAxialTransverse`
`defaultPoissonRatioTransverse`
	Baseline transversely isotropic elastic moduli along the basal-plane normal, in the basal plane, and in mixed axial-transverse shear.
	Poisson couplings for the TI elastic law.

`*PressureDerivative`	Optional pressure slopes for E_z, E_p, and G_{zp}.

`failureStrength`
`maximumPrincipalStressDamage`
`enableCrackTipStressConcentration`
	Tensile/principal-stress damage controls and crack-tip modifier coupling.

`distortionShearResponse*`	Point-slope parameters for the distortion shear envelope.
`inPlaneShearResponse*`	Point-slope parameters for basal-plane in-plane shear.
`coupledShearResponse*`	Point-slope parameters for coupled shear/slip involving the weak plane normal.

`distortionStrainHardeningC0`
`inPlaneStrainHardeningC0`
`coupledStrainHardeningC0`
`maximumPlasticStrain`
	Plastic-strain hardening controls through the relaxation coordinate.

`basalPlaneFractureEnergyReleaseRate`
`totalFractureEnergyReleaseRate`
`scaleFractureEnergyReleaseRate`
`crackSpeed`
	Damage regularization and rate controls.

`alphaL`
`alphaT`
`temperature`
`temperatureRate`
	Anisotropic thermal expansion path.

`materialDirection`	Particle material basis; the first row is the graphitic plane normal used by the constitutive model.

2.7.1.11 `Geomechanics`

`Geomechanics` is the MPM-connected porous geomaterial model. It is intended for rocks, chalks, cemented granular media, and other pressure-sensitive materials for which the important continuum mechanisms include nonlinear elasticity, pressure-dependent shear strength, a compaction cap, evolving porosity, strain hardening, brittle damage, optional creep, and nonassociative plastic flow. The model lineage follows the Arenisca/Kayenta family of cap-plasticity geomodels and the Homel–Guilkey–Brannon consistency-bisection transformed-space closest-point-return (CB-TSCPR) solution strategy [44, 45, 46]. The Ghareb-chalk formulation adds the creep, hardening, damage, and pressure-dependent shear-modulus features summarized below [47].

The model is distinct from `CeramicDamage`. `CeramicDamage` is primarily a brittle-fracture model intended to work with DFG surfaces and frictional contact, whereas `Geomechanics` treats porosity, compaction, and cap hardening as continuum state variables. Use `Geomechanics` when continuum pore-volume evolution, pressure-sensitive compaction, and creep are part of the material response. Use the DFG/contact machinery with this model only when a separate kinematic representation of fracture or contact is also required.

Stress measures and sign convention. The implementation stores Cauchy stress in the usual GEOS tensor convention and uses

$$I_1 = \operatorname{tr}\boldsymbol{\sigma}, \qquad p = -\frac{1}{3}I_1, \qquad \tau = \sqrt{J_2}, \qquad J_2 = \frac{1}{2}\operatorname{dev}\boldsymbol{\sigma} : \operatorname{dev}\boldsymbol{\sigma}. \tag{2.150}$$

Compressive pressure is therefore positive when $I_1 < 0$. Many calibration plots use the compression-positive invariant $\bar{I}_1 = -I_1 = 3p$. The historical Arenisca notation also allows an isotropic backstress ζ so that the yield surface is evaluated in shifted stress space, $I_1^s = I_1 - \zeta$, or $\bar{I}_1^s = -(I_1 - \zeta)$. In the reviewed GEOS-MPM implementation the fluid/backstress evolution path is disabled by input validation; `fluidBulkModulus` and `fluidInitialPressure` must be zero. The code still retains the shifted-space structure so the formulation can be extended consistently later.

Yield surface and cap. The strength surface is the product of a shear-limit function F_f and a cap function F_c. In compression-positive notation,

$$F_f(\bar{I}_1^s) = a_1 - a_3 \exp(-a_2 \bar{I}_1^s) + a_4 \bar{I}_1^s, \tag{2.151}$$

where the coefficients a_i are computed from user-facing parameters `peakI1`, `fSlope`, `fSlopeFailed`, `stren`, and `ySlope`. The cap reduces shear strength as the stress approaches the hydrostatic compressive strength $\bar{X}$. Let

$$\bar{\kappa} = \bar{I}_{1,\text{peak}} + c_r \left(\bar{X} - \bar{I}_{1,\text{peak}} \right), \qquad 0 < c_r < 1, \tag{2.152}$$

where $\bar{I}_{1,\text{peak}}$ is the tensile vertex in compression-positive notation and c_r is the input `cr`. The cap branch is

$$F_c(\bar{I}_1^s) = \begin{cases} 1, & \bar{I}_1^s \le \bar{\kappa}, \\ \sqrt{1 - \left(\dfrac{\bar{I}_1^s - \bar{\kappa}}{\bar{X} - \bar{\kappa}} \right)^2}, & \bar{\kappa} < \bar{I}_1^s < \bar{X}. \end{cases} \tag{2.153}$$

The elastic domain is approximated by

$$f = \tau - F_f(\bar{I}_1^s) F_c(\bar{I}_1^s) \le 0, \tag{2.154}$$

with special branches for the tensile vertex and the cap apex. Figure 2.4 shows the cap construction, the creep/plastic return path, and the hardening interpretation used in the Ghareb-chalk formulation.

Supported shear-limit parameterizations. The same input fields can represent several useful pressure-dependent limits. The host-side input checks enforce a valid combination.

Table 2.5: Shear-limit surface families selected by `Geomechanics` inputs.

Family	Input pattern and purpose
Linear Drucker–Prager	`fSlope > 0`, `peakI1 >= 0`, and `stren = ySlope = 0`. This gives a linear pressure-dependent shear limit.
Von Mises	`stren > 0` with `fSlope = peakI1 = ySlope = 0`. This gives pressure-independent shear strength.
Zero-vertex transition	`fSlope > 0`, `stren > 0`, and `peakI1 = ySlope = 0`. This connects a zero-pressure vertex to a finite shear plateau.
Nonlinear Drucker–Prager	`fSlope > ySlope > 0`, `stren > ySlope*peakI1`, and `peakI1 >= 0`. This is the full curved form used for many geomaterial fits.

For the nonlinear branch, the code computes

$$a_1 = \texttt{STREN}_h, \tag{2.155}$$

$$a_2 = \frac{\texttt{FSLOPE}_h - \texttt{YSLOPE}_h}{\texttt{STREN}_h - \texttt{YSLOPE}_h \, \texttt{PEAKI1}_h}, \tag{2.156}$$

$$a_3 = (\texttt{STREN}_h - \texttt{YSLOPE}_h \, \texttt{PEAKI1}_h) \exp\left[-a_2 \, \texttt{PEAKI1}_h\right], \tag{2.157}$$

$$a_4 = \texttt{YSLOPE}_h. \tag{2.158}$$

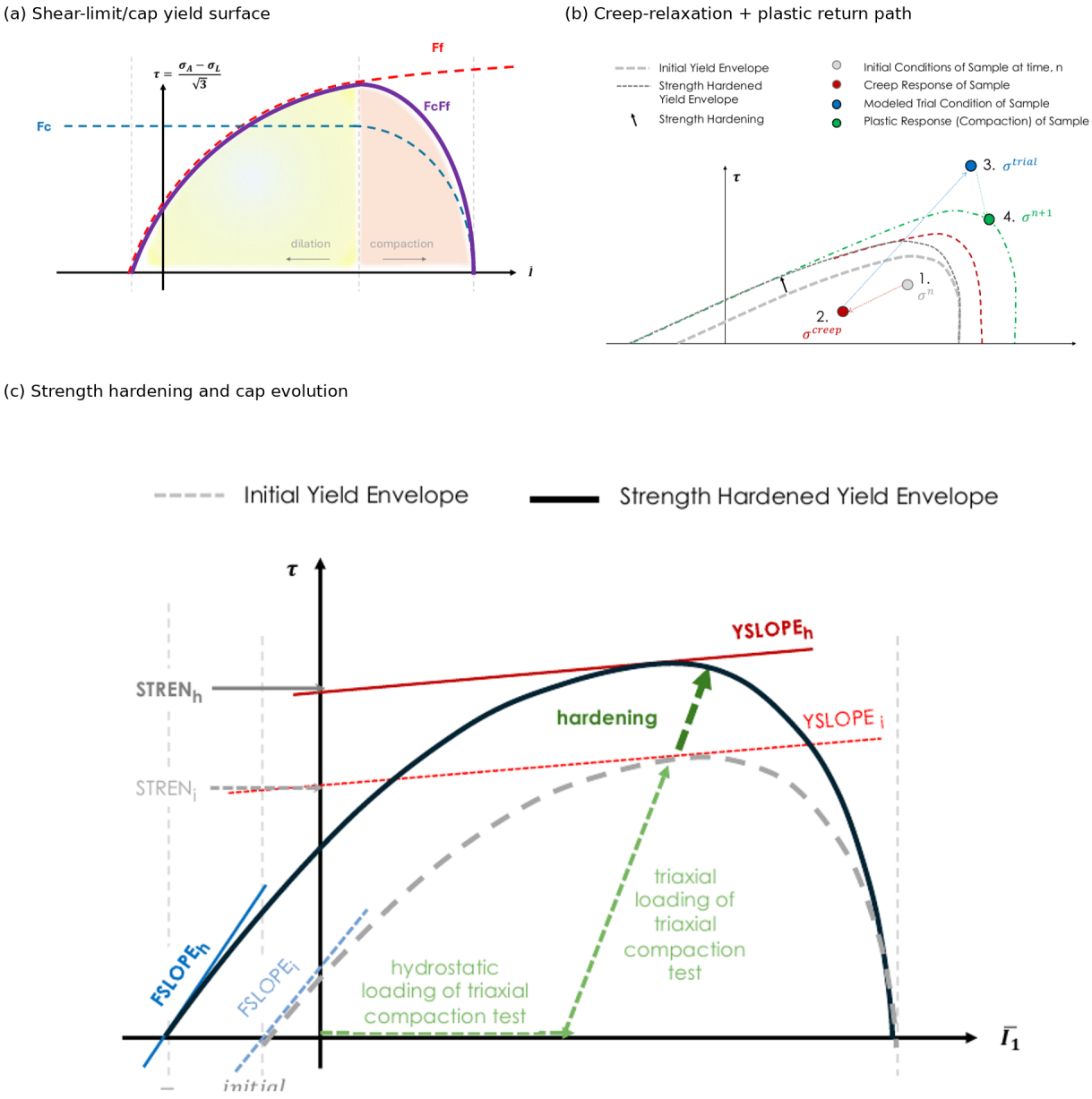


Figure 2.4: Geomechanics formulation diagrams adapted from Malenda, Homel, Kibikas, Choens, Shalev, and Lyakhovsky [47]. (a) The shear-limit function is multiplied by a cap function to obtain a pressure-sensitive strength surface. (b) Creep first relaxes the beginning-of-step state, then an elastic trial stress is computed, and a plastic return is performed to the evolved surface. (c) Strain hardening expands the strength envelope and shifts the triaxial-loading response.

The linear and special-case surfaces are obtained by setting the corresponding subset of a_i to zero, as described in Table 2.5.

Nonlinear elasticity. The tangent bulk modulus is evaluated from the beginning-of-substep stress and volumetric plastic strain. In the dry path used by the reviewed code,

$$K = b_0 + b_1 \exp\left(\frac{b_2}{I_1}\right) - b_3 \exp\left(\frac{b_4}{\epsilon_v^p}\right), \qquad I_1 < 0, \quad \epsilon_v^p < 0, \tag{2.159}$$

with lower bound $K \geq b_0$. The tensile/low-pressure response uses $K = b_0$. The shear modulus defaults to $G = g_0$. If `g1` is nonzero, the model computes a pressure-dependent Poisson ratio,

$$\nu = g_1 + g_2 \exp\left(\frac{g_3}{I_1}\right), \qquad 0 \leq \nu < \frac{1}{2}, \tag{2.160}$$

then sets

$$G = \max\left[g_0, \frac{3K(1-2\nu)}{2(1+\nu)}\right]. \tag{2.161}$$

This is a pragmatic pressure-dependent tangent stiffness for fitting laboratory data; it is evaluated substep-by-substep so the trial predictor follows the local tangent response.

Cap hardening, porosity, and crush curve. The cap position X is computed from volumetric plastic strain $\epsilon_v^p = \operatorname{tr} \boldsymbol{\epsilon}^p$. The implemented dry crush curve is

$$X(\epsilon_v^p) = \begin{cases} \dfrac{p_0 p_1 + \ln\left[(\epsilon_v^p + p_3)/p_3\right]}{p_1}, & \epsilon_v^p \leq 0, \\ p_0 (1 + \epsilon_v^p)^{1/(p_0 p_1 p_3)}, & \epsilon_v^p > 0, \end{cases} \tag{2.162}$$

where `p0 < 0`, `p1 > 0`, and `p3 > 0`. If ϵ_v^p approaches $-p_3$, the cap is pushed to a very large compressive value so the material behaves as fully compacted. The particle porosity stored for plotting and subsequent material response is

$$\phi = 1 + \exp(-\epsilon_v^p)(\phi_i - 1), \qquad \phi_i = 1 - \exp(-p_3). \tag{2.163}$$

Thus the same input `p3` sets the limiting volumetric plastic compaction and the initial unloaded porosity implied by the crush curve.

Strain hardening and damage softening. The equivalent plastic shear strain is computed from the plastic-strain deviator. When `strainHardeningK > 0`, the hardening increment is

$$H = K_h \left(1 - \exp[-n_h \, \bar{\epsilon}_p]\right), \tag{2.164}$$

where $K_h =$ `strainHardeningK` and $n_h =$ `strainHardeningN`. The coherence $c = 1 - D$ softens the surface through c^ψ, where $\psi =$ `fractureSofteningExponent`. In implementation form,

$$\texttt{STREN}_h = b_{\text{buckling}} \left(\texttt{stren} + H \, \texttt{dstrendh}\right), \tag{2.165}$$

$$\texttt{FSLOPE}_h = c^\psi \, \texttt{fSlope} \left(1 + H \, \texttt{dfslopedh}\right) + \left(1 - c^\psi\right) \texttt{fSlopeFailed}, \tag{2.166}$$

$$\texttt{PEAKI1}_h = b_{\text{buckling}} \left(\texttt{peakI1} + H \, \texttt{dpeakI1dh}\right) c^\psi \, s_p, \tag{2.167}$$

$$c_{r,h} = \text{clamp}\left[\texttt{cr}\left(1 + H \, \texttt{dcrdh}/g_0\right), 10^{-10}, 0.99999999999\right], \tag{2.168}$$

where s_p is the particle `strengthScale`. In this model, `strengthScale` is applied to the tensile-vertex/`peakI1` branch and to the fracture-stress threshold; it should not be assumed to scale every coefficient identically. See Section 2.7.3.1 for the shared Weibull-strength discussion.

Damage is optional and active only when `fractureEnergyReleaseRate` is positive. The model stores damage as $D = 1 - c$. Depending on `damageEvolutionCriterion`, damage increments are driven by dilatational plastic work or by a combined dilatational/shear plastic-work measure below a brittle–ductile transition pressure. In schematic form,

$$\Delta D = \frac{\Delta W_p \, \ell_{ch}}{G_f}, \qquad c^{n+1} = \max(0, c^n - \Delta D), \tag{2.169}$$

where $G_f =$ `fractureEnergyReleaseRate` and ℓ_{ch} is the particle length scale. The damage threshold uses `fractureStress*strengthScale`; this is another model-specific use of `strengthScale`. The `damageEvolutionCriterion=2` branch estimates the brittle–ductile transition from the apex of the current surface, while `damageEvolutionCriterion=1` uses the user-provided `brittleDuctileTransition`.

Optional creep relaxation. When `enableCreep=1`, creep is applied once over the full explicit time increment before the plastic substep retry loop. The code uses an Arrhenius-like temperature multiplier

$$A_T = \exp\left[-\frac{Q}{R}\left(\frac{1}{T} - \frac{1}{T_0}\right)\right], \tag{2.170}$$

where $Q =$ `Q`, $T_0 =$ `initialTemperature`, and R is the gas constant. Deviatoric creep uses an equilibrium plastic shear strain and relaxes the deviatoric stress toward it:

$$\gamma_{eq} = C_0 \, \tau^{C_1}, \tag{2.171}$$

$$\Delta\gamma_c = \Delta t \, A_T C_2 \max(\gamma_{eq} - \gamma_p, 0), \tag{2.172}$$

with $C_0 =$ `creepC0`, $C_1 =$ `creepC1`, and $C_2 =$ `creepC2`. The increment is limited by the current elastic shear strain so creep cannot reverse the deviatoric stress in one step.

Volumetric creep uses the current unloaded porosity ϕ_p, an equilibrium porosity ϕ_e, and a pressure-dependent rate:

$$\phi_e = \texttt{creepA} \exp\left[-\left(\frac{p}{\texttt{creepB}}\right)^{\texttt{creepD}}\right] + \texttt{creepE} - 0.0002\langle T - T_0\rangle - 3\times 10^{-6}\langle T - T_0\rangle^2, \tag{2.173}$$

$$\frac{d\phi_p}{dt} = -A_T \left(\frac{p}{\texttt{creepG}}\right)^{\texttt{creepF}} \texttt{creepC}\,(\phi_p - \phi_e)\,, \tag{2.174}$$

where $\langle x\rangle = \max(x, 0)$. The creep-updated porosity is converted back to an equivalent volumetric plastic strain and used to relax the beginning-of-step pressure. As in the Ghareb-chalk implementation, the creep relaxation is intended to represent slow mechanisms in an explicit-dynamics calculation only when creep time scales are separated from elastic-wave and plastic-return time scales [47].

Optional compaction/buckling multiplier. For porous or architected solids, `enableBuckling` applies a prescribed strength multiplier before the cap/plastic update. With `enableBuckling=1`, the strain measure is volumetric, based on $J = \det \mathbf{F}$. With `enableBuckling=2`, the strain measure is directional, based on the stretch along `MaterialDirection`. The multiplier has the form

$$b_{\text{buckling}} = 1 - A_b \sin^2\left[-\left\lceil\frac{\ell_{ch}}{L_b}\right\rceil \pi\hat{\epsilon}\right], \tag{2.175}$$

where $A_b =$ `bucklingAmplitude`, $L_b =$ `bucklingLength`, and $\hat{\epsilon}$ is a normalized compaction strain. This is an empirical multiplier, not a resolved structural-buckling analysis.

Numerical update algorithm. The GEOS-MPM call path is corotational. The velocity gradient is unrotated into the beginning-of-step frame, symmetrized to form the strain-rate tensor, and the old plastic strain is also unrotated. The model then performs the update in that frame and rotates the plastic strain back at the end of the step.

Listing 2.51: Geomechanics update structure.

```
unrotate velocity gradient and plastic strain to beginning-of-step frame
D = sym(L_unrotated)
compute optional buckling multiplier from F and MaterialDirection
compute tangent K, G and conservative wave speed
if inelasticity is disabled:
  sigma = sigma_old + [3 K iso(D) + 2 G dev(D)] dt
else:
  estimate required substeps from trial stress and tangent-stiffness change
  apply optional full-step creep relaxation to sigma_old, ep_old, and X
  for progressively refined subcycle attempts:
    for each substep:
      evaluate K, G from current stress and plastic strain
      sigma_trial = hypoelastic predictor
      compute hardening, damage-modified surface coefficients, and yield state
      if elastic: accept trial state
      else:
        compute nonhardening return by transformed bisection/search
        if cap or backstress evolves:
          solve consistency on volumetric plastic strain by bounded bisection
        update plastic strain, cap X, coherence c, and porosity
    if all substeps succeed: accept attempt
  if all attempts fail: restore state and set constitutiveUpdateFlag = -1
rotate plastic strain back to end-of-step frame and store stress, damage, porosity
```

The nonhardening return uses the CB-TSCPR idea: transform the stress space so the energy-norm return becomes a Euclidean closest-point search, use bisection from an interior point to the trial point, and rotate the candidate point in transformed space until the closest boundary point is found [46]. The nonassociativity parameter `beta` enters this transformation by scaling the deviatoric coordinate,

$$r^* = \beta\sqrt{\frac{3K}{2G}}\, r, \qquad r = \sqrt{2J_2}, \tag{2.176}$$

so the return direction can represent reduced or enhanced dilatation without constructing a separate flow-potential surface. If the cap position changes with volumetric plastic strain, the solver wraps this return in a consistency bisection on $\eta \in [0, 1]$, where

$$\Delta\epsilon_v^p = \eta\, \Delta\epsilon_{v,0}^p. \tag{2.177}$$

Each trial η updates X, coherence, and the shifted-stress state, recomputes the return, and checks that the computed volumetric plastic strain matches the assumed value. This decoupling is the main numerical reason the model can combine nonlinear elasticity, cap hardening, softening, and nonassociativity while retaining a robust update path.

Input parameterization. Table 2.6 summarizes the major input groups. The recommended calibration workflow is to fit `b0-b4` and `p0,p1,p3` from hydrostatic loading/unloading, fit `g0-g3`, `peakI1`, `fSlope`, `stren`, and `ySlope` from triaxial compression/extension data, then fit `strainHardeningK`, `strainHardeningN`, damage, and creep parameters from post-yield, dilation, compaction, and creep histories.

Table 2.6: Main `Geomechanics` input groups. See the generated schema appendix for the exact XML registration status and defaults.

Group	Inputs	Role
Elasticity	`b0,b1,b2,b3,b4`, `g0,g1,g2,g3,g4`	Nonlinear tangent bulk modulus, pressure-dependent Poisson ratio, shear modulus, and conservative wave-speed estimate. `g4` is registered but not active in the reviewed update path.
Cap/crush curve	`p0,p1,p2,p3,p4`, `cr`	Hydrostatic compaction curve, initial porosity relation, cap position, and cap branch point. `p2` and `p4` are registered but currently unused.
Shear limit	`peakI1`, `fSlope`, `fSlopeFailed`, `stren`, `ySlope`, `beta`	Drucker–Prager/Kayenta-style shear surface, residual frictional slope, tensile vertex, and transformed-space nonassociativity.
Hardening	`strainHardeningK`, `strainHardeningN`, `dstrendh`, `dfslopedh`, `dpeakI1dh`, `dcrdh`	Exponential shear hardening and model-specific changes to strength, slope, tensile vertex, and cap branch.
Damage	`fractureEnergyReleaseRate`, `fractureSofteningExponent`, `fractureStress`, `brittleDuctileTransition`, `damageEvolutionCriterion`	Optional work-regularized reduction of coherence, with either user-specified or internally estimated brittle–ductile transition.
Creep and temperature	`enableCreep`, `creepC0`, `creepC1`, `creepC2`, `creepA`, `creepB`, `creepC`, `creepD`, `creepE`, `creepF`, `creepG`, `initialTemperature`, `Q`	Optional volumetric and deviatoric creep relaxation and temperature-scaled rates.
Buckling multiplier	`enableBuckling`, `bucklingLength`, `bucklingAmplitude`, `MaterialDirection`	Optional empirical compaction multiplier using volumetric or directional strain.
Numerical controls	`plasticStrainTolerance`, `stressReturnTolerance`, `maxAllowedSubcycles`, `failedStepResponse`, `disableInelasticity`	Return tolerances, subcycling limits, failure response, and elastic-only test mode. Failed convergence exposes `constitutiveUpdateFlag=-1` to the solver robustness/deletion path.
Currently disabled fluid path	`fluidBulkModulus`, `fluidInitialPressure`	Must be zero in the reviewed implementation. The code retains placeholders for a future persistent pore-pressure/backstress state.

Implementation notes. The model writes `bulkModulus`, `shearModulus`, `porosity`, `damage`, `temperature`, and `constitutiveUpdateFlag` fields. The stored `porosity` is an evolving continuum porosity inferred from volumetric plastic strain, not merely a visualization flag. The stored `constitutiveUpdateFlag` is zero for normal updates, may be one when the model needed additional

subcycling, and is set to -1 when the update fails and the solver should treat the particle through the robustness/deletion logic in Section 2.12. The implementation is presently a dry porous solid model with optional empirical creep; coupled heat transfer, active fluid pressure, and fully coupled pore-pressure/backstress evolution are future extensions rather than active input options.

2.7.2 Fluid models

The reviewed MPM continuum dispatch includes a gas-like continuum model. The PFW and XML infrastructure can assign this model to particles in the same way as solid models, but its stress is purely spherical. A future liquid model can use the same dispatch path once implemented.

2.7.2.1 `Gas`

`Gas` is a simple ideal-gas-style pressure model. It computes a pressure from the reference pressure, the deformation-gradient Jacobian, and a temperature ratio, then returns a spherical compressive stress:

$$p = \frac{p_0}{J}\frac{T}{T_0}, \qquad \boldsymbol{\sigma} = -p\mathbf{I}. \tag{2.178}$$

The wave speed is evaluated as

$$c = \sqrt{\gamma p/\rho}, \qquad \gamma = 1.4 \tag{2.179}$$

for the current implementation.

Listing 2.52: Gas stress update.

```
pressure = referencePressure / jacobian * (temperature / referenceTemperature)
stress = -pressure * I
waveSpeed = sqrt(1.4 * pressure / density)
```

The gas model has special handling in the robustness/deformation-gradient-reset path (Section 2.12) because a gas particle does not require the same persistent deviatoric deformation history as a solid particle.

2.7.2.2 Liquid-model placeholder

A liquid-particle model is not currently documented as an active MPM continuum model in the reviewed snapshot. A future liquid model could be introduced through the same material dispatch interface, for example with a weakly compressible equation of state,

$$p = p_0 + K_f\left(\frac{\rho}{\rho_0} - 1\right), \qquad \boldsymbol{\sigma} = -p\mathbf{I} + 2\eta\,\mathrm{dev}\,\mathbf{D}, \tag{2.180}$$

where K_f is a fluid bulk modulus and η is a dynamic viscosity. This equation is a design placeholder only; it should not be interpreted as an implemented input option until a concrete GEOS-MPM liquid model is registered and verified.

2.7.3 Shared constitutive modifiers and particle state

Several particle fields are not full constitutive models, but they modify how one or more models evaluate strength, damage, heat, or anisotropy. These fields may be painted by geometry wrappers in PFW, updated by solver events, or updated by a constitutive model. Because the same field may be interpreted differently by different models, this section documents the common data path and the expected modeling interpretation; model-specific equations should be added to the detailed material subsections before quantitative calibration.

2.7.3.1 Strength scale and Weibull-type size effects

The particle field `strengthScale` is a multiplicative scalar that can be assigned by PFW geometry wrappers such as `strengthScaleWrapper` and by Voronoi/layered Weibull wrappers. Models that expose a `strengthScale` wrapper then read the value during the constitutive update. In brittle materials this field is often used to represent flaw-controlled size effects or mesoscopic heterogeneity.

A common theoretical basis is Weibull weakest-link scaling [27]. For a reference volume V_0, reference strength σ_0, and Weibull modulus m, the failure probability of a uniformly stressed volume V is often written

$$P_f = 1 - \exp\left[-\frac{V}{V_0}\left(\frac{\sigma}{\sigma_0}\right)^m\right]. \tag{2.181}$$

At fixed failure probability this gives the familiar deterministic size scaling

$$\sigma(V) = \sigma_0\left(\frac{V_0}{V}\right)^{1/m}. \tag{2.182}$$

A stochastic PFW wrapper may instead sample a flaw population and assign a particle-level scale factor s_p so that the material model evaluates a local strength such as $Y_p = s_p Y_0$ or $\sigma_{f,p} = s_p \sigma_f$. The exact placement of s_p is model dependent: for example, some models multiply tensile and compressive strengths, some multiply fracture-energy parameters only when an option is enabled, and some models do not use the field at all. For that reason, `strengthScale` should be documented in each detailed material subsection before it is used for quantitative calibration.

2.7.3.2 Crack-tip stress concentration

The crack-tip correction is a solver/constitutive coupling used by damage models that need an unresolved stress-concentration estimate near a DFG fracture tip, motivated by the Griffith-Irwin crack-tip picture [28, 29]. The solver first uses the neighbor-list damage field to compute a nonlocal damage gradient and a local Hessian-like diagnostic. Particles that satisfy the tip-detection threshold receive a positive `distanceToCrackTip`; other particles receive zero. Models such as `CeramicDamage` and `Graphite` can then compute and store a plotted `crackTipStressConcentration` factor.

The current correction has the form

$$C_{\text{tip}} = \max\left(1, \sqrt{\frac{d_{\text{tip}}}{r_{\text{fpz}}}}\right), \tag{2.183}$$

where d_{tip} is the inverse-kernel estimate of the unresolved distance to the crack tip and r_{fpz} is a process-zone length constructed from fracture toughness or fracture energy and a nominal intact strength. The intent is to avoid over-predicting the resolved particle strength when the MPM/DFG length scale cannot resolve the near-tip stress field. The correction is optional and should be verified against the intended fracture model and neighbor radius.

2.7.3.3 Porosity

`porosity` is a particle field that may be initialized by PFW, painted by wrappers such as `porosityWrapper` or `pointwisePorosityWrapper`, read by a constitutive model, and in some models updated as part of the material response. In the reviewed code path, porous/damage models use porosity differently. For example, `Geomechanics` carries porosity as an evolving material variable, while `CeramicDamage` uses porosity in strength scaling. Solver events and future coupled solvers may also modify porosity, for example to represent healing, compaction, reaction, or pore closure.

Because porosity can be both an input descriptor and a constitutive state variable, users should distinguish the initial particle field from any model-updated value written after the first constitutive

step. Future coupled heat/fluid/chemistry or poromechanics updates should document ownership rules: whether the solver, a flow model, or the solid constitutive model is the authoritative updater for `porosity` at a given step.

2.7.3.4 Temperature

`temperature` and `temperatureRate` are particle fields shared by the solver, events, and thermally sensitive constitutive models. PFW can initialize temperature through geometry wrappers, while events such as `TemperatureProfile` can prescribe domain-wide histories from a solver-level table. The reviewed code also records internal energy and includes constitutive hooks for temperature-dependent parameters. Models such as `StrainHardeningPolymer`, `Graphite`, and `Geomechanics` read temperature-dependent terms; other models may ignore temperature unless later extended.

In a purely mechanical run, temperature may be prescribed externally or inferred internally by a constitutive model from plastic work, strain energy, or an equation of state. In future coupled runs, a heat-transfer solver could become the authoritative updater. Until those ownership rules are formalized for a given application, input files should make clear whether temperature is a fixed initial condition, an event-prescribed history, a material-model output, or a field reserved for later coupling.

2.7.4 External material models

GEOS-MPM can compile continuum material models that live outside the public source tree. This path is intended for site-local, application-specific, or proprietary continuum laws without requiring those files to be committed to the main repository. The build hook is the CMake cache variable `GEOS_EXTERNAL_CONSTITUTIVE_MODELS_DIR`; the `setupMPM` helper described in Section 1.2 forwards this path through the option `-external-material-models`.

A minimal external material-model directory has the form

Listing 2.53: Minimal external material-model directory layout.

```
my_external_materials/
  GEOSExternalConstitutiveModels.cmake
  MyContinuumModel.hpp
  MyContinuumModel.cpp
```

The manifest can also be named `CMakeLists.txt`. For an explicit MPM continuum model, the manifest must list the source/header files and must also add the model type and include to the MPM dispatch lists:

Listing 2.54: External continuum model CMake registration.

```
geos_register_external_constitutive_models(
  BASE_DIR ${CMAKE_CURRENT_LIST_DIR}
  SOURCES
    MyContinuumModel.cpp
  HEADERS
    MyContinuumModel.hpp
  MPM_INCLUDES
    MyContinuumModel.hpp
  MPM_MODELS
    MyContinuumModel )
```

The implementation file must still register the model with the normal GEOS constitutive catalog, for example

Listing 2.55: External model GEOS catalog registration.

```
REGISTER_CATALOG_ENTRY( ConstitutiveBase,
                        MyContinuumModel,
                        std::string const &,
                        Group * const )
```

After the files are in place, configure and build using either the setup helper or a direct CMake configuration:

Listing 2.56: Building with external continuum material models.

```
./setupMPM --dane --external-material-models /path/to/my_external_materials

# Equivalent direct-CMake idea:
cmake -DGEOS_EXTERNAL_CONSTITUTIVE_MODELS_DIR=/path/to/my_external_materials ...
cmake --build <build-dir> --target geosx
```

Once built, the external material is used like any other continuum model: add a corresponding XML tag to the `Constitutive` block, add its name to the relevant `ParticleRegion` material list, and assign the matching material index with PFW geometry objects or wrappers. External cohesive-zone laws use a related but separate registration list and are documented with the cohesive-zone infrastructure in Section 2.10.

2.8 Particle types and interpolation/update options

This section documents the interpolation and particle update choices used by the MPM solver. The user-facing particle-type controls are generated by ParticleFileWriter and are summarized with the other material, group, and particle-type controls in Section 3.6; this section focuses on the internal numerical operators. The reviewed code exposes the particle-type enumeration

$$\texttt{SinglePoint}, \quad \texttt{SinglePointBSpline}, \quad \texttt{CPDI}, \quad \texttt{CPTI}, \quad \texttt{CPDI2}, \tag{2.184}$$

and the velocity/state update enumeration

$$\texttt{FLIP}, \quad \texttt{PIC}, \quad \texttt{XPIC}, \quad \texttt{FMPM}. \tag{2.185}$$

The implemented particle setup paths in the reviewed source are `SinglePoint`, `SinglePointBSpline`, and `CPDI`. The `CPTI` and `CPDI2` identifiers are reserved for future development; attempting to generate those particle types in the reviewed particle setup path raises a not-implemented error. The same particle-to-grid and grid-to-particle mapping data are reused by the force, kinematic, contact, boundary, and diagnostic steps described in Sections 2.5 and 2.9.

2.8.1 Mapping notation and effective support

Let p denote a material point, I a background-grid node, m_p the particle mass, V_p the particle volume, $\boldsymbol{x}_p$ the particle position, $\boldsymbol{v}_p$ the particle velocity, and $\boldsymbol{\sigma}_p$ the Cauchy stress. The solver stores, for each active particle, a sparse set of mapped nodes together with shape values N_{Ip} and gradients $\boldsymbol{\nabla} N_{Ip}$. These quantities define the standard MPM projections

$$m_I = \sum_p m_p N_{Ip}, \qquad \boldsymbol{p}_I = \sum_p m_p \boldsymbol{v}_p N_{Ip}, \qquad \boldsymbol{v}_I = \frac{\boldsymbol{p}_I}{m_I}, \tag{2.186}$$

$$\boldsymbol{f}_I^{\text{int}} = -\sum_p V_p \, \boldsymbol{\sigma}_p \boldsymbol{\nabla} N_{Ip}, \qquad \boldsymbol{L}_p = \sum_I \boldsymbol{v}_I \otimes \boldsymbol{\nabla} N_{Ip}, \tag{2.187}$$

with additional material-field or group labels used when contact and boundary branches need split nodal states. Equations (2.186) and (2.187) are the common PIC/FLIP MPM transfer operators introduced for solid mechanics by Sulsky, Chen, and Schreyer and later generalized through GIMP, CPDI, and related domain methods [1, 19, 2, 3].

The implementation distinguishes a raw geometric support from an effective support. For `SinglePoint`, the raw map has the eight trilinear nodes of the cell containing the particle. For `SinglePointBSpline`, the raw map has the $4^3 = 64$ tensor-product cubic B-spline control nodes surrounding the particle. For `CPDI`, each of the eight particle-domain corners contributes eight trilinear nodes, again giving as many as 64 raw entries. A coalescing pass combines repeated grid nodes, sums their contributions, removes zero entries, and provides the effective map used by most particle-to-grid and grid-to-particle kernels. This detail matters because CPDI corners may share grid nodes after distortion or near grid boundaries, and because B-spline support is broader than linear MPM support even though it is still driven by a single particle center.

2.8.2 Single-point particles with trilinear mapping

The `SinglePoint` type is the classical point-quadrature MPM discretization. The particle characteristic function is a Dirac measure located at $\boldsymbol{x}_p$, and the background basis is the trilinear finite-element basis on the cell containing the particle. If $\boldsymbol{h} = (h_x, h_y, h_z)$ is the local cell spacing and

$$\boldsymbol{s} = (s_x, s_y, s_z), \qquad s_d = \frac{x_{p,d} - x_{0,d}}{h_d} \in [0, 1], \tag{2.188}$$

are the local coordinates measured from the lower cell corner, then the one-dimensional linear weights are

$$\ell_0(s) = 1 - s, \qquad \ell_1(s) = s, \qquad \frac{d\ell_0}{dx} = -\frac{1}{h}, \qquad \frac{d\ell_1}{dx} = \frac{1}{h}. \tag{2.189}$$

For a node with local index $\boldsymbol{a} = (a_x, a_y, a_z)$, $a_d \in \{0, 1\}$,

$$N_{\boldsymbol{a}p} = \ell_{a_x}(s_x)\ell_{a_y}(s_y)\ell_{a_z}(s_z), \tag{2.190}$$

$$\frac{\partial N_{\boldsymbol{a}p}}{\partial x} = \frac{d\ell_{a_x}}{dx}(s_x)\ell_{a_y}(s_y)\ell_{a_z}(s_z), \tag{2.191}$$

with cyclic permutations for y and z. The weights satisfy partition of unity and reproduce linear fields inside a cell, but the gradient is discontinuous at cell boundaries. Consequently, standard point MPM is susceptible to cell-crossing noise and quadrature error when particles move from one background cell to another. These errors motivated the generalized interpolation material point method, smoother basis studies, CPDI, and related projection improvements [2, 13, 14, 11].

In the reviewed implementation the `SinglePoint` generator stores a volume and visualization domain vectors, but the interpolation itself uses only the particle center. The stored domain vectors allow the particle file and plotting tools to display a finite volume, while the map used by Equations (2.186)–(2.187) remains the eight-node point map in Equation (2.190).

2.8.3 Single-point particles with cubic B-spline mapping

The `SinglePointBSpline` type keeps the particle as a point quadrature rule but replaces the trilinear cell basis by a tensor-product cubic cardinal B-spline basis. The support is four nodes in each direction, giving 64 grid nodes in three dimensions. Let

$$u_d = \frac{x_{p,d} - x_d^{\min}}{h_d}, \qquad i_d = \lfloor u_d \rfloor, \qquad s_d = u_d - i_d \in [0, 1), \tag{2.192}$$

and let the first supported node in direction d be $i_d - 1$. For one dimension, the four nonzero cubic B-spline weights used by the solver are

$$b_0(s) = \frac{(1-s)^3}{6}, \qquad b_1(s) = \frac{3s^3 - 6s^2 + 4}{6}, \tag{2.193}$$

$$b_2(s) = \frac{-3s^3 + 3s^2 + 3s + 1}{6}, \qquad b_3(s) = \frac{s^3}{6}. \tag{2.194}$$

Their derivatives with respect to physical position are

$$\frac{db_0}{dx} = -\frac{(1-s)^2}{2h}, \qquad \frac{db_1}{dx} = \frac{1.5s^2 - 2s}{h}, \tag{2.195}$$

$$\frac{db_2}{dx} = \frac{-1.5s^2 + s + 0.5}{h}, \qquad \frac{db_3}{dx} = \frac{s^2}{2h}. \tag{2.196}$$

The three-dimensional basis is the tensor product

$$N_{\boldsymbol{ap}} = b_{a_x}(s_x) b_{a_y}(s_y) b_{a_z}(s_z), \qquad a_d \in \{0, 1, 2, 3\}, \tag{2.197}$$

and the gradient is obtained by differentiating the one-dimensional factor in the selected coordinate direction. The code applies a deterministic closure correction to the final supported node so that the sum of the stored shape values is exactly one and the sum of the stored gradients is exactly zero to roundoff.

The motivation for this option is the same as in smoother-basis MPM studies: broader, smoother basis functions reduce force and velocity projection jumps when particles cross grid-cell boundaries. Steffen et al. analyzed implementation choices and quadrature errors in MPM, and later B-spline MPM work showed that B-spline bases can reduce cell-crossing artifacts and improve continuity of grid fields [13, 14, 21]. The GEOS option should be interpreted as a single-point B-spline quadrature choice, not as a finite particle-domain method: the particle center controls the interpolation, while the stored particle domain vectors are retained for volume accounting and output.

Listing 2.57: Single-point cubic B-spline map construction.

```
for each active particle p:
  for d in x,y,z:
    u = (x_p[d] - grid_min[d]) / h[d]
    cell = floor(u)
    s = u - cell
    base[d] = cell - 1
    compute b[0:4](s) and dbdx[0:4](s)
  for a,b,c in 0..3:
    I = node(base[x]+a, base[y]+b, base[z]+c)
    N_Ip = bx[a] * by[b] * bz[c]
    gradN_Ip = (dbx[a]*by[b]*bz[c],
                bx[a]*dby[b]*bz[c],
                bx[a]*by[b]*dbz[c])
  adjust last entry so sum(N)=1 and sum(gradN)=0
```

2.8.4 CPDI with trilinear background shape functions

The `CPDI` type implements convected particle domain interpolation for a parallelepiped particle domain. CPDI replaces point evaluation of the grid basis by a particle-domain average, thereby reducing cell-crossing noise and improving behavior under large deformation. This follows the convected particle domain interpolation method of Sadeghirad, Brannon, and Burghardt [3]. In GEOS, the particle carries a center $\boldsymbol{x}_p$ and three half-edge vectors $\boldsymbol{r}_1$, $\boldsymbol{r}_2$, and $\boldsymbol{r}_3$. The current domain is

$$\Omega_p = \{\boldsymbol{x}_p + \xi_1 \boldsymbol{r}_1 + \xi_2 \boldsymbol{r}_2 + \xi_3 \boldsymbol{r}_3 \ : \ -1 \le \xi_1, \xi_2, \xi_3 \le 1\}, \tag{2.198}$$

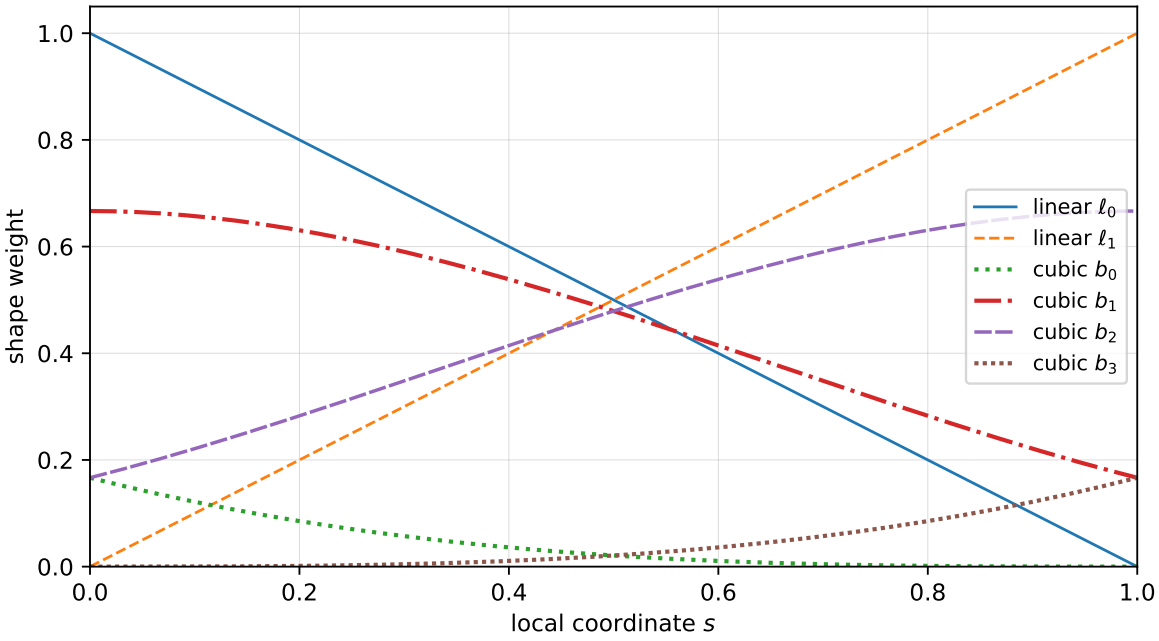


Figure 2.5: One-dimensional shape functions used to build the tensor-product maps. The linear MPM map has two active weights on the host cell, whereas the cubic B-spline map has four active weights with C^2 continuity across cell interfaces. The three-dimensional maps are tensor products of these one-dimensional functions.

with volume

$$V_p = 8\,\boldsymbol{r}_1 \cdot (\boldsymbol{r}_2 \times \boldsymbol{r}_3). \tag{2.199}$$

For the eight corners c, define signs $s_i^c \in \{-1, 1\}$ and corner positions

$$\boldsymbol{x}_p^c = \boldsymbol{x}_p + s_1^c \boldsymbol{r}_1 + s_2^c \boldsymbol{r}_2 + s_3^c \boldsymbol{r}_3. \tag{2.200}$$

The GEOS map evaluates the background trilinear shape functions at each corner and assigns one-eighth of the corner contribution to the particle map,

$$\overline{N}_{Ip} \approx \frac{1}{8} \sum_{c=1}^{8} N_I(\boldsymbol{x}_p^c). \tag{2.201}$$

For gradients, let

$$\boldsymbol{A} = \boldsymbol{r}_2 \times \boldsymbol{r}_3, \qquad \boldsymbol{B} = \boldsymbol{r}_3 \times \boldsymbol{r}_1, \qquad \boldsymbol{C} = \boldsymbol{r}_1 \times \boldsymbol{r}_2. \tag{2.202}$$

The corner gradient coefficient used internally is

$$\boldsymbol{\alpha}_p^c = \frac{s_1^c \boldsymbol{A} + s_2^c \boldsymbol{B} + s_3^c \boldsymbol{C}}{V_p}, \tag{2.203}$$

and the effective gradient is assembled as

$$\boldsymbol{\nabla} \overline{N}_{Ip} \approx \sum_{c=1}^{8} N_I(\boldsymbol{x}_p^c)\, \boldsymbol{\alpha}_p^c. \tag{2.204}$$

Equations (2.201)–(2.204) are stored initially as corner/node raw entries and then coalesced by grid node. A closure adjustment enforces the corner partition of unity and gradient consistency before the raw entries are merged.

Listing 2.58: CPDI map construction for a parallelepiped particle domain.

```
for each active CPDI particle p:
  A = r2 x r3; B = r3 x r1; C = r1 x r2
  V = 8 * dot(r1,A)
  for each corner sign sc in {-1,1}^3:
    x_c = x_p + sc[0]*r1 + sc[1]*r2 + sc[2]*r3
    alpha_c = (sc[0]*A + sc[1]*B + sc[2]*C) / V
    evaluate eight trilinear grid weights N_I(x_c)
    for each corner node I:
      raw_N_Ip     += (1/8) * N_I(x_c)
      raw_grad_Ip  += alpha_c * N_I(x_c)
  coalesce duplicate nodes and enforce consistency corrections
```

CPDI remains a background-grid method: nodal equations are still solved on the rectilinear grid, but the particle basis carries an evolving domain. The particle domain vectors are updated by the deformation history, and the volume used in the stress divergence reflects the convected domain. This is especially important for large tensile strains and rotations, where point MPM may develop severe quadrature error or numerical fracture. The domain-averaged CPDI map trades a larger stencil for smoother transfer and a more physically meaningful particle support [3, 5].

2.8.5 CPDI domain scaling

The code includes an optional CPDI domain-scaling procedure. The intent is to limit excessive growth of convected domains relative to the background grid, which can otherwise delay localization, smear gradients, or contribute to numerical fracture in parallel CPDI calculations. The relevant literature basis is the CPDI domain-scaling work of Homel, Brannon, and Guilkey [5].

The reviewed implementation constructs diagonal-like vectors from the current half-edge vectors and caps their lengths by

$$\ell_{\text{crit}} = 0.49999 \min_d h_d, \tag{2.205}$$

where h_d are the current grid spacings. In plane-strain calculations the two controlled vectors are

$$\boldsymbol{\ell}_0 = \boldsymbol{r}_1 + \boldsymbol{r}_2, \qquad \boldsymbol{\ell}_1 = \boldsymbol{r}_1 - \boldsymbol{r}_2. \tag{2.206}$$

If $\|\boldsymbol{\ell}_i\| > \ell_{\text{crit}}$, the vector is scaled to length ℓ_{crit} and the half-edge vectors are reconstructed as

$$\boldsymbol{r}_1 = \frac{1}{2}(\boldsymbol{\ell}_0 + \boldsymbol{\ell}_1), \qquad \boldsymbol{r}_2 = \frac{1}{2}(\boldsymbol{\ell}_0 - \boldsymbol{\ell}_1). \tag{2.207}$$

In three dimensions the four controlled body diagonals are

$$\boldsymbol{\ell}_0 = \boldsymbol{r}_1 + \boldsymbol{r}_2 + \boldsymbol{r}_3, \qquad \boldsymbol{\ell}_1 = \boldsymbol{r}_1 - \boldsymbol{r}_2 + \boldsymbol{r}_3, \tag{2.208}$$

$$\boldsymbol{\ell}_2 = \boldsymbol{r}_2 - \boldsymbol{r}_1 + \boldsymbol{r}_3, \qquad \boldsymbol{\ell}_3 = \boldsymbol{r}_3 - \boldsymbol{r}_1 - \boldsymbol{r}_2. \tag{2.209}$$

After applying the same length cap, the half-edge vectors are reconstructed as

$$\boldsymbol{r}_1 = \frac{1}{4}(\boldsymbol{\ell}_0 + \boldsymbol{\ell}_1 - \boldsymbol{\ell}_2 - \boldsymbol{\ell}_3), \qquad \boldsymbol{r}_2 = \frac{1}{4}(\boldsymbol{\ell}_0 - \boldsymbol{\ell}_1 + \boldsymbol{\ell}_2 - \boldsymbol{\ell}_3), \tag{2.210}$$

$$\boldsymbol{r}_3 = \frac{1}{4}(\boldsymbol{\ell}_0 + \boldsymbol{\ell}_1 + \boldsymbol{\ell}_2 + \boldsymbol{\ell}_3). \tag{2.211}$$

A particle flag records that scaling occurred. If requested by input options, the solver can still plot unscaled domains by reconstructing the current vectors from the reference vectors and deformation gradient. A separate guard disables surface normals and surface positions for highly distorted, scaled particles when the aspect-ratio threshold used by the code is exceeded. Thus, domain scaling is not a new constitutive model; it is a numerical regularization of the CPDI interpolation domain before subsequent mapping and output operations.

2.8.6 CPTI and CPDI2 placeholders

The `CPTI` and `CPDI2` particle-type names are present in the enumeration but are not implemented in the reviewed particle-generation path. They are retained in this manual as forward links to the methods intended by those names.

CPTI. Convected-particle tetrahedron interpolation represents the particle domain by tetrahedra rather than by a parallelepiped. The method was introduced for mesoscale ceramic calculations by Leavy, Guilkey, Phung, Spear, and Brannon and was motivated by the need to represent complex material geometry while retaining the efficiency of a rectilinear computational grid [22]. A future GEOS implementation would need particle-domain storage for tetrahedral vertices, tetrahedral volume and face-gradient rules, particle-to-grid support construction for each convected tetrahedron, and output conventions for deformed tetrahedral domains.

CPDI2. Second-order CPDI generalizes the original CPDI parallelogram/parallelepiped representation to more flexible quadrilateral or hexahedral particle domains and supports enrichment for weak discontinuities at material interfaces [4]. In the code snapshot reviewed for this manual, the `CPDI2` generator branch is a placeholder. Completing it would require storage for the full convected hexahedral geometry, generalized corner or quadrature-gradient weights, robust coalescing for larger supports, and consistency tests against the published CPDI2 patch and interface examples.

2.8.7 Transfer operators used by update schemes

The update schemes can be described compactly by two transfer operators. Let S denote the particle-to-grid velocity projection

$$(S\boldsymbol{v})_I = \frac{1}{m_I}\sum_p m_p N_{Ip}\boldsymbol{v}_p, \tag{2.212}$$

and let S^+ denote the grid-to-particle interpolation

$$(S^+\boldsymbol{v})_p = \sum_I N_{Ip}\boldsymbol{v}_I. \tag{2.213}$$

With lumped nodal mass, S^+ is an interpolation operator rather than an exact inverse of S. Much of the behavior of PIC, FLIP, XPIC, and FMPM follows from how much filtering is introduced by repeated application of these operators. PIC applies a direct grid velocity interpolation, FLIP updates particles by the grid acceleration increment, XPIC builds higher-order filters from repeated projections, and FMPM approximates the action of a full mass matrix inverse. The projection accuracy and stability consequences of such choices have been studied by Wallstedt and Guilkey, Hammerquist and Nairn, and Nairn and Hammerquist [11, 12, 16, 17].

2.8.8 PIC update

The `PIC` option updates particle velocities by interpolating the current post-force, post-contact grid velocity. In a conventional notation,

$$\boldsymbol{v}_p^{n+1} = \sum_I N_{Ip}\boldsymbol{v}_I^{n+1}, \tag{2.214}$$

$$\boldsymbol{x}_p^{n+1} = \boldsymbol{x}_p^n + \Delta t\sum_I N_{Ip}\left(\boldsymbol{v}_I^{n+1} - \frac{1}{2}\Delta t\,\boldsymbol{a}_I^{n+1}\right), \tag{2.215}$$

$$\boldsymbol{L}_p^{n+1} = \sum_I \boldsymbol{v}_I^{n+1}\otimes\boldsymbol{\nabla}N_{Ip}. \tag{2.216}$$

The position update in Equation (2.215) is the time-centered form used by the reviewed implementation, in which the nodal acceleration term shifts the post-force velocity back to a midpoint-like transport velocity. The velocity gradient used for stress integration is formed from the same grid velocity and the particle map gradients.

PIC is robust because the projection S^+ filters particle velocities through the grid every step, damping grid-scale null modes. The cost of that filtering is numerical diffusion, especially in problems with persistent shear, impact, or oscillatory motion. This tradeoff is the classical motivation for FLIP and for later XPIC filters: PIC damps noise effectively, while FLIP better preserves particle kinetic energy and history variables [19, 20, 16].

Listing 2.59: PIC grid-to-particle update.

```
for each particle p:
  x_inc = 0; v_new = 0; L = 0
  for each mapped node I:
    x_inc += N_Ip * (v_I - 0.5*dt*a_I)
    v_new += N_Ip * v_I
    L     += outer(v_I, gradN_Ip)
  x_p += dt * x_inc
  v_p  = v_new
  L_p  = L
```

2.8.9 FLIP update

The default `FLIP` option updates particle velocities by the nodal acceleration increment while transporting particles with the same time-centered grid velocity used in PIC:

$$\boldsymbol{v}_p^{n+1} = \boldsymbol{v}_p^n + \Delta t \sum_I N_{Ip} \boldsymbol{a}_I^{n+1}, \tag{2.217}$$

$$\boldsymbol{x}_p^{n+1} = \boldsymbol{x}_p^n + \Delta t \sum_I N_{Ip} \left(\boldsymbol{v}_I^{n+1} - \frac{1}{2} \Delta t \, \boldsymbol{a}_I^{n+1} \right), \tag{2.218}$$

$$\boldsymbol{L}_p^{n+1} = \sum_I \boldsymbol{v}_I^{n+1} \otimes \boldsymbol{\nabla} N_{Ip}. \tag{2.219}$$

Because FLIP increments the particle velocity rather than replacing it by $S^+ \boldsymbol{v}_I$, it is much less diffusive than PIC. This is important for dynamic solid mechanics, where over-filtering can erase physically meaningful velocity fluctuations or wave content. The same property can also allow nonphysical particle-grid null-space noise to persist, particularly when particle distributions are irregular or contacts introduce sharp local changes. The stability and accuracy of FLIP-like MPM updates depend strongly on particle density, projection quality, and the chosen particle basis [20, 19, 11, 12].

In the reviewed implementation, FLIP, PIC, and several diagnostic branches share the same effective mapping arrays. For CPU execution, the code uses the coalesced map to avoid repeated grid-node contributions. For device execution, some kernels can recompute or use raw support entries depending on the mapped-field and contact requirements. These implementation choices do not change Equations (2.217)–(2.219); they only control how the sparse sums are evaluated.

Listing 2.60: FLIP grid-to-particle update.

```
for each particle p:
  x_inc = 0; dv = 0; L = 0
  for each mapped node I:
    x_inc += N_Ip * (v_I - 0.5*dt*a_I)
    dv    += N_Ip * a_I
```

```
  L     += outer(v_I, gradN_Ip)
x_p += dt * x_inc
v_p += dt * dv
L_p  = L
```

2.8.10 XPIC update

The `XPIC` option implements the extended PIC method of Hammerquist and Nairn [16]. XPIC introduces an update order m that controls the degree of projection filtering. At $m = 1$, XPIC reduces to PIC. Higher orders apply repeated particle-grid-particle projections in a way that damps the nonphysical null-space modes associated with under-resolved particle distributions without applying as much physical diffusion as PIC. In the limit discussed by Hammerquist and Nairn, the method approaches a modified FLIP behavior while retaining improved stability.

The reviewed GEOS implementation builds XPIC corrections on the grid. It initializes a pre-acceleration velocity-like field

$$\boldsymbol{v}_I^- = \boldsymbol{v}_I^{n+1} - \Delta t\, \boldsymbol{a}_I^{n+1}, \tag{2.220}$$

then repeatedly gathers this field to particles and scatters it back to the grid through the same shape functions and lumped masses used by the rest of the solver. For each order index $r = 2, \ldots, m$, the implementation uses coefficients

$$c_r = \frac{m - r + 1}{r}, \qquad d_r = \frac{m - r}{r}, \qquad \epsilon_r = (-1)^r, \tag{2.221}$$

for the velocity and velocity-difference projection terms. The accumulated filtered grid field is then gathered to particles for the final velocity and position update. The velocity gradient used for stress integration remains based on the physical post-force/contact grid velocity rather than on a separately filtered XPIC velocity gradient; this follows the implementation note in the reviewed solver and avoids introducing a second, inconsistent kinematic field.

Listing 2.61: XPIC update structure.

```
vMinus_I = v_I - dt*a_I
vStar_I  = 0
for r = 2..updateOrder:
  c  = (updateOrder - r + 1) / r
  dc = (updateOrder - r) / r
  gather vMinus_I to particles using N_Ip
  scatter projected particle field back to grid with m_p*N_Ip/m_I
  vStar_I += (-1)^r * projected_v_I
  vMinus_I = projected_v_I
for each particle p:
  gather the XPIC-filtered combination to update x_p and v_p
  L_p = sum_I outer(v_I, gradN_Ip)
```

XPIC is therefore best understood as a controllable compromise between PIC and FLIP. It is appropriate when a calculation is too noisy with FLIP but too diffusive with PIC. The practical input parameter is the update order, exposed as `updateOrder`; Section 3.7 describes how this control is passed through ParticleFileWriter.

2.8.11 FMPM update

The `FMPM` option implements an approximate full-mass-matrix update based on the method of Nairn and Hammerquist [17]. Standard explicit MPM forms a lumped nodal mass m_I and uses m_I^{-1} in the

nodal momentum equation. Lumped mass is simple and robust, but it introduces numerical dissipation and projection errors because the true consistent mass matrix has off-diagonal coupling between grid basis functions. FMPM constructs an iterative approximation to the action of the inverse full mass matrix while preserving the particle/grid data structures used by MPM. The current documentation describes the solver in the incremental-correction interpretation of Nairn's 2026 implementation paper, in which each FMPM loop pass represents the velocity increment $\Delta v^{(\ell)} = v^{+(\ell)} - v^{+(\ell-1)}$ and lumped-mass features are imposed on that increment rather than only on the final filtered velocity [18].

At the operator level, FMPM applies a sequence involving S and S^+. The implementation initializes the cumulative FMPM grid velocity with the constrained post-contact grid velocity and then repeatedly projects the previous increment to particles and back to the grid. Each iteration forms a residual-like increment and adds it to the cumulative velocity. Homogeneous boundary constraints are applied to the increment so that constrained components do not re-enter through the higher-order correction. Multi-field material-contact corrections are accumulated consistently as net contact momentum using the incremental contact correction described by Nairn [18]. The important distinction is that FMPM is intended to work with multi-field contact and with prescribed or moving grid-velocity boundary conditions; the limitation in the reviewed code snapshot is narrower, namely that the specific `boundaryConditionTypes == Contact` wall/contact-boundary bctype remains guarded and should be treated as untested until that branch is validated.

Listing 2.62: FMPM approximate full-mass update.

```
vFmpm_I = v_I
vPrev_I = v_I
for k = 2..updateOrder:
  gather vPrev_I to particles using N_Ip
  scatter gathered particle velocity back to grid with m_p*N_Ip/m_I
  vProj_I = projected grid velocity
  vPrev_I = vPrev_I - vProj_I
  apply homogeneous moving/symmetry boundary constraints to vPrev_I
  apply incremental multi-field material-contact correction if active
  vFmpm_I += vPrev_I
for each particle p:
  v_new = sum_I N_Ip * vFmpm_I
  x_p += 0.5*dt*(v_old + v_new)
  L_p  = sum_I outer(vFmpm_I, gradN_Ip)
  v_p  = v_new
```

Compared with FLIP, FMPM is designed to reduce dissipation associated with lumped-mass projection while retaining an explicit MPM structure. The update order controls how many terms of the approximate inverse are retained. Higher order can improve accuracy but increases cost because each additional term requires another gather/scatter synchronization and another round of boundary/contact consistency operations. For production use, FMPM should be verified with the intended particle basis, contact model, and boundary configuration. In particular, moving velocity boundary conditions and multi-field material contact use the incremental correction path, whereas the contact-wall boundary bctype should remain a separately tracked verification item.

2.8.12 Implementation status summary

Table 2.7 summarizes the status of the particle types and update schemes in the reviewed source snapshot. The table is intended as a code-synchronized checklist for future documentation updates.

Table 2.7: Particle mapping and update status in the reviewed source.

Name	Status	Internal interpretation
`SinglePoint`	implemented	Point particle with eight-node trilinear support. Stored domain vectors are used for volume/output, not for interpolation.
`SinglePointBSpline`	implemented	Point particle with 4^3 cubic B-spline support. The center controls interpolation; domain vectors are retained for output/volume.
`CPDI`	implemented	Convected parallelepiped domain with corner-based trilinear integration, gradient coefficients from particle-domain area vectors, and optional domain scaling.
`CPTI`	placeholder	Reserved for convected tetrahedral particle domains; generator branch is not implemented in the reviewed source.
`CPDI2`	placeholder	Reserved for second-order CPDI with generalized hexahedral/quadrilateral domains; generator branch is not implemented in the reviewed source.
`PIC`	implemented	Replaces particle velocity by interpolated post-force grid velocity; robust but diffusive.
`FLIP`	implemented, default	Increments particle velocity by grid acceleration; less diffusive but can retain null-space noise.
`XPIC`	implemented	Higher-order PIC/FLIP compromise controlled by `updateOrder`; stress kinematics use physical grid velocity.
`FMPM`	implemented with restrictions	Approximate full-mass inverse controlled by `updateOrder`; uses the incremental correction path for moving velocity boundaries and multi-field material contact, while the `Contact` wall-boundary bctype remains guarded/untested in the reviewed branch.

2.9 Boundary conditions, prescribed deformation, and stress control

This section describes what the MPM solver does internally when boundary and domain-control options are active. It deliberately separates the numerical implementation from the user-facing input syntax. The user-facing `pfw_input` controls, including examples of `boundaryConditionTypes`, `fTable`, `bcTable`, transverse boundary velocities, stress control, and periodic flags, are summarized in Section 3.8. The code path described here is the one reached during Step 13 of the explicit solver update, after particle-to-grid projection, MPI synchronization, grid trial dynamics, and optional material-field contact; see Section 2.5.13.

2.9.1 Face convention and nodal data used by the boundary update

The solver stores one boundary type for each axis-aligned face of the background grid. The face order is

$$f = 0, 1, 2, 3, 4, 5 \quad \leftrightarrow \quad x^-, x^+, y^-, y^+, z^-, z^+. \tag{2.222}$$

For face f, the normal coordinate direction is

$$d(f) = \left\lfloor \frac{f}{2} \right\rfloor, \qquad s(f) = f \bmod 2, \qquad \alpha_f = -1 + 2s(f), \qquad \mathbf{n}_f = \alpha_f \mathbf{e}_{d(f)}. \tag{2.223}$$

The two remaining coordinate directions are used as tangential directions. In the code these are computed as

$$d_1 = (d+1) \bmod 3, \qquad d_2 = (d+2) \bmod 3. \tag{2.224}$$

Boundary enforcement operates on grid multi-fields. The field index distinguishes material/contact velocity fields, while the node index distinguishes ordinary grid nodes, boundary nodes, and buffer/ghost-like nodes used to support the interpolation stencil near a face. The principal fields modified by the boundary code are the grid velocity $\mathbf{v}_{I\alpha}$, grid acceleration $\mathbf{a}_{I\alpha}$, and grid velocity increment $\Delta\mathbf{v}_{I\alpha}$ for node I and velocity field α. The boundary update also reads nodal mass, internal force, center-of-volume, surface-position, surface-field-mass, and ghost-rank data when contact or reaction histories are active.

The boundary-node and buffer-node sets are stored in the node-set repository as `boundaryNodes` and `bufferNodes`. For a face f, let $\mathcal{B}_f$ be its boundary nodes and $\mathcal{G}_f$ its buffer nodes. Each buffer node $G \in \mathcal{G}_f$ is paired with an interior node $J(G)$ and, for moving faces, a boundary node $B(G)$ at the same tangential grid coordinates. These pairings are reconstructed from the structured-grid `ijkMap` and the current grid spacing. They provide the even/odd extensions needed by B-spline and other wide-stencil particle-grid transfers near a boundary.

2.9.2 Boundary-type state and time-dependent boundary-type switching

The runtime boundary-type array is `m_boundaryConditionTypes`. If the input does not provide this array, it is initialized to six `Outflow` entries. If it is provided, the solver requires exactly six integer entries and checks that each entry belongs to the boundary-condition enumeration in Table 2.8.

Table 2.8: MPM boundary-condition type enumeration used by `SolidMechanicsMPM`.

Integer	Name	Internal interpretation
0	`Outflow`	Leave grid velocity and acceleration unconstrained on that face.
1	`Symmetry`	Enforce zero normal velocity/acceleration and reflect buffer fields with an odd normal component and even tangential components.
2	`Moving`	Enforce a prescribed affine normal velocity when prescribed boundary deformation or stress control is active; optionally enforce tangential velocities.
3	`Contact`	Treat the face as a wall/contact plane. Incoming nodes are corrected in the face-normal direction and optional boundary friction is applied in the tangential plane.

If `prescribedBcTable` is enabled, Step 4 of the explicit update calls `boundaryConditionUpdate` before particle-grid maps are assembled. The table is a zero-order-hold schedule rather than an interpolated quantity. For a time step beginning at t^n with size Δt, the selected row is the last row whose time satisfies

$$t_{\text{row}} < t^n + \frac{\Delta t}{2}. \tag{2.225}$$

The six boundary-type entries in that row then replace `m_boundaryConditionTypes`. After this update, `updateContactFlagsFromBoundaryConditions` sets `m_hasContact` if either multiple velocity fields are present or at least one face is marked `Contact`. This flag controls whether contact/surface fields are synchronized and whether contact-specific work is performed later in the step.

Listing 2.63: Boundary-type update and contact flag refresh.

```
if prescribedBcTable == 1:
  interval = last row i such that time_n + 0.5*dt > bcTable[i,0]
  for face in 0..5:
    boundaryConditionTypes[face] = bcTable[interval, face+1]

hasContact = (numVelocityFields > 1) or any(boundaryConditionTypes == Contact)
```

2.9.3 Outflow faces

An `Outflow` face is the least intrusive boundary option. In `applyEssentialBCs`, the face is skipped:

$$\mathbf{v}_{I\alpha}^{\text{after}} = \mathbf{v}_{I\alpha}^{\text{before}}, \qquad \mathbf{a}_{I\alpha}^{\text{after}} = \mathbf{a}_{I\alpha}^{\text{before}}, \qquad I \in \mathcal{B}_f. \tag{2.226}$$

No reflective buffer update is applied by the essential-boundary routine for this face. This should not be interpreted as a finite-element natural traction boundary. It means that the explicit MPM grid solve leaves the nodal values produced by particle-to-grid mapping, force assembly, contact, and MPI synchronization unchanged. Particles that advect beyond a nonperiodic global domain are handled later by the out-of-range particle flagging and cleanup path. Consequently, `Outflow` is useful for open-domain style calculations, impact problems where material may leave the computational box, or inactive directions in which the user does not want a symmetry or prescribed-motion constraint.

2.9.4 Symmetry and free-slip reflection

A `Symmetry` face enforces a kinematic free-slip plane. On the boundary nodes, the normal component of the vector field is removed. For velocity and acceleration this gives

$$v_{I\alpha,d} = 0, \qquad a_{I\alpha,d} = 0, \qquad I \in \mathcal{B}_f. \tag{2.227}$$

The corresponding velocity increment is updated so that XPIC/FMPM-style transfer formulas see the boundary-induced correction,

$$\Delta v_{I\alpha,d} \leftarrow \Delta v_{I\alpha,d} - v_{I\alpha,d}^{\text{old}}. \tag{2.228}$$

For buffer nodes, the solver uses an odd extension of the normal component and an even extension of the tangential components from the interior paired node $J(G)$:

$$v_{G\alpha,d} = -v_{J(G)\alpha,d}, \qquad v_{G\alpha,d_1} = v_{J(G)\alpha,d_1}, \qquad v_{G\alpha,d_2} = v_{J(G)\alpha,d_2}, \tag{2.229}$$

$$a_{G\alpha,d} = -a_{J(G)\alpha,d}, \qquad a_{G\alpha,d_1} = a_{J(G)\alpha,d_1}, \qquad a_{G\alpha,d_2} = a_{J(G)\alpha,d_2}. \tag{2.230}$$

This mirrored extension is also applied earlier to mapped surface normals, center-of-mass vectors, and center-of-volume vectors on faces marked `Symmetry` or `Moving`. That earlier pass occurs before the grid dynamics update and ensures that contact and surface quantities remain compatible with the same geometric reflection used for the final velocity and acceleration fields.

The construction is the MPM analogue of imposing an essential normal-velocity condition on the background grid while allowing tangential slip. It is consistent with the grid-based enforcement commonly used in explicit MPM and GIMP calculations, where the particle state is advanced using constrained nodal accelerations and velocities rather than by imposing constraints directly on the particles [1, 2, 12].

Listing 2.64: Symmetry-face enforcement.

```
for field alpha:
  for I in boundaryNodes[face]:
    dVelocity[I,alpha,d] = -velocity[I,alpha,d]
    velocity[I,alpha,d] = 0
```

```
    acceleration[I,alpha,d] = 0

  for G in bufferNodes[face]:
    J = reflected interior node for G
    velocity[G,alpha,d]  = -velocity[J,alpha,d]
    velocity[G,alpha,t1] =  velocity[J,alpha,t1]
    velocity[G,alpha,t2] =  velocity[J,alpha,t2]
    acceleration[G,alpha,d]  = -acceleration[J,alpha,d]
    acceleration[G,alpha,t1] =  acceleration[J,alpha,t1]
    acceleration[G,alpha,t2] =  acceleration[J,alpha,t2]
```

2.9.5 Prescribed domain deformation and F-table interpolation

The F-table machinery produces a diagonal macroscopic deformation history and its associated diagonal velocity-gradient components. The table has one time column and three deformation columns. At the end of the step, $t^{n+1} = t^n + \Delta t$, `interpolateTable` evaluates the active interval and returns $F_i(t^{n+1})$ and $\dot{F}_i(t^{n+1})$. For directions not completely overridden by stress control, the code sets

$$\bar{L}_i(t^{n+1}) = \frac{\dot{F}_i(t^{n+1})}{F_i(t^{n+1})}, \qquad \bar{F}_i(t^{n+1}) = F_i(t^{n+1}). \tag{2.231}$$

The current implementation supports three scalar interpolation kernels. Let y_0, y_1 be consecutive table values, $\tau = (t^{n+1} - t_0)/(t_1 - t_0)$, and $\Delta T = t_1 - t_0$. Linear interpolation uses

$$y(\tau) = (1-\tau)y_0 + \tau y_1, \qquad \dot{y} = \frac{y_1 - y_0}{\Delta T}. \tag{2.232}$$

Cosine interpolation uses

$$y(\tau) = y_0 - \frac{1}{2}(y_1 - y_0)\left[\cos(\pi\tau) - 1\right], \qquad \dot{y} = \frac{\pi}{2\Delta T}(y_1 - y_0)\sin(\pi\tau), \tag{2.233}$$

and `Smoothstep` uses the quintic polynomial

$$y(\tau) = y_0 + (y_1 - y_0)(10\tau^3 - 15\tau^4 + 6\tau^5), \qquad \dot{y} = \frac{y_1 - y_0}{\Delta T}(30\tau^2 - 60\tau^3 + 30\tau^4). \tag{2.234}$$

Values before the first table time are clamped to the first row, and values after the final table time are clamped to the last row.

The two F-table flags route this macroscopic kinematics into different parts of the MPM update. With `prescribedBoundaryFTable`, the diagonal velocity-gradient components become boundary velocities on `Moving` faces during `applyEssentialBCs`. With `prescribedFTable`, the same components are superposed on particle velocity gradients and positions only in directions marked periodic by the spatial partition. The latter path is intended for triply periodic or partially periodic homogeneous-deformation calculations. In both cases, Step 18 later resizes the grid/domain measures using the current `domainL` values.

2.9.6 Moving faces

A `Moving` face is an essential velocity boundary driven by the current domain velocity-gradient state. In the reviewed implementation, the velocity constraint is applied only if either `prescribedBoundaryFTable` is active or at least one `stressControl` entry equals one. This guard is important: marking a face as `Moving` without an active prescribed boundary deformation, or with only the limiting `stressControl` mode two active, does not by itself prescribe a nonzero velocity in `applyEssentialBCs`.

For a moving face f normal to direction d, the code forms an end-of-step grid coordinate

$$X_{I,d}^{n+1,*} = \left(1 + \bar{L}_d \Delta t\right) X_{I,d}^{n}, \tag{2.235}$$

and imposes the normal velocity

$$v_{I\alpha,d}^{\mathrm{bc}} = \bar{L}_d X_{I,d}^{n+1,*}, \qquad I \in \mathcal{B}_f. \tag{2.236}$$

The solver then computes the increment and acceleration correction as

$$\Delta v_{I\alpha,d} \leftarrow v_{I\alpha,d}^{\mathrm{bc}} - v_{I\alpha,d}^{\mathrm{old}}, \qquad a_{I\alpha,d} \leftarrow a_{I\alpha,d} + \frac{v_{I\alpha,d}^{\mathrm{bc}} - v_{I\alpha,d}^{\mathrm{old}}}{\Delta t}, \qquad v_{I\alpha,d} \leftarrow v_{I\alpha,d}^{\mathrm{bc}}. \tag{2.237}$$

Tangential motion is optional. If `enablePrescribedBoundaryTransverseVelocities[f]` is one, the two stored tangential values are enforced on d_1 and d_2 exactly as essential velocity components:

$$v_{I\alpha,d_1} \leftarrow v_{f,d_1}^{\mathrm{presc}}, \qquad v_{I\alpha,d_2} \leftarrow v_{f,d_2}^{\mathrm{presc}}, \tag{2.238}$$

with matching updates to `gridDVelocity` and `gridAcceleration`. If transverse velocities are not enabled, the tangential components are left free on the boundary nodes.

The buffer-node extension is centered on the prescribed boundary value. For the constrained normal component,

$$v_{G\alpha,d} = -v_{J(G)\alpha,d} + 2v_{B(G)\alpha,d}. \tag{2.239}$$

For tangential components, the code chooses either the same moving odd extension when transverse velocities are prescribed or a free-slip even extension otherwise:

$$v_{G\alpha,d_k} = \begin{cases} -v_{J(G)\alpha,d_k} + 2v_{B(G)\alpha,d_k}, & \text{prescribed transverse component active,} \\ v_{J(G)\alpha,d_k}, & \text{transverse component free,} \end{cases} \qquad k = 1, 2. \tag{2.240}$$

The same component-wise extension is applied to the acceleration. This construction gives the interpolation stencil a boundary-consistent field: constrained components are odd about the prescribed control value, while unconstrained tangential components remain even.

Listing 2.65: Moving-face enforcement.

```
for field alpha and boundary node I:
  x_end = (1 + domainL[d]*dt) * gridPosition[I,d]
  v_bc = domainL[d] * x_end

  if transverse velocities enabled on this face:
    set velocity[I,alpha,t1] and velocity[I,alpha,t2]
    update dVelocity and acceleration corrections in t1,t2

  dVelocity[I,alpha,d] = v_bc - velocity[I,alpha,d]
  acceleration[I,alpha,d] += (v_bc - velocity[I,alpha,d]) / dt
  velocity[I,alpha,d] = v_bc

for buffer node G:
  J = reflected interior node
  B = boundary node at same tangential indices
  velocity[G,d] = -velocity[J,d] + 2*velocity[B,d]
  velocity[G,t] = moving extension if tangential constrained, else even extension
```

2.9.7 Boundary contact faces

A face marked `Contact` is handled in the same `Moving/Contact` branch, but with wall-contact detection replacing unconditional prescribed motion. The branch is always entered for a contact face, even if no F-table or stress-control flag is active. For each boundary node and velocity field, the code first estimates a scalar surface position in the face-normal direction. If a particle-mapped surface position is available at the node, it uses `gridSurfacePosition`; otherwise it falls back to `gridCenterOfVolume`. With the sign α_f from Eq. (2.223), the code's contact condition is

$$\alpha_f v_{I\alpha,d} > 0 \quad \text{and} \quad \alpha_f x^{\text{surf}}_{I\alpha,d} > 0. \tag{2.241}$$

When this condition is false, the face does not overwrite the normal velocity. When it is true, the normal boundary target is

$$v^{\text{bc}}_{I\alpha,d} = \bar{L}_d X_{I,d}, \tag{2.242}$$

followed by the same normal velocity-increment, acceleration-correction, reaction-accumulation, and buffer-reflection operations used by a moving face. The registered `boundaryFaceCoefficientsOfRestitution` array is initialized and checked, but the active code path in the reviewed source does not use it in the normal target; a restitution-based line is present only as commented legacy code. Thus boundary contact should currently be interpreted as a sticking/non-penetration style normal constraint with optional tangential friction, not as a coefficient-of-restitution bounce law.

Tangential friction is controlled by `boundaryFaceFrictionCoefficients`. Let

$$\mathbf{v}_t = v_{I\alpha,d_1}\mathbf{e}_{d_1} + v_{I\alpha,d_2}\mathbf{e}_{d_2}, \qquad \mathbf{r}_t = \begin{cases} \mathbf{v}_t/\|\mathbf{v}_t\|, & \|\mathbf{v}_t\| > 10^{-16}, \\ \mathbf{0}, & \text{otherwise}, \end{cases} \tag{2.243}$$

and define the code's scalar friction measure by

$$f_\mu = \mu_f \max\left(-\alpha_f a_{I\alpha,d}, 0\right). \tag{2.244}$$

The code then updates the tangential velocity increment and acceleration fields component-wise as

$$\Delta\mathbf{v}_t \leftarrow f_\mu \Delta t\, \mathbf{r}_t, \qquad \mathbf{v}_t \leftarrow \mathbf{v}_t + \Delta\mathbf{v}_t, \qquad \mathbf{a}_t \leftarrow \mathbf{a}_t - f_\mu \mathbf{r}_t. \tag{2.245}$$

Equation (2.245) is intentionally written to match the reviewed implementation. Because the sign convention combines face orientation, acceleration direction, and the subsequent velocity/acceleration updates, wall-friction verification cases should be used when relying on nonzero boundary friction.

Listing 2.66: Contact-face wall update.

```
for field alpha and boundary node I:
  surfacePosition = gridSurfacePosition if available else gridCenterOfVolume
  incoming = alpha(face)*velocity[I,alpha,d] > 0
  outside  = alpha(face)*surfacePosition[d] > 0

  if incoming and outside:
    v_bc = domainL[d] * gridPosition[I,d]
    apply normal velocity correction as for Moving

  mu = boundaryFaceFrictionCoefficients[face]
  frictionMeasure = mu * max(-alpha(face)*acceleration[I,alpha,d], 0)
  apply implemented tangential velocity/acceleration update
```

2.9.8 Stress-control generated boundary motion

Stress control does not impose tractions directly on grid faces. Instead, it computes a diagonal domain velocity gradient, stores it in `domainL`, and then lets the moving-boundary machinery impose the associated affine boundary velocities. The target stress is obtained from `stressTable` using the same interpolation framework as the F-table. The current stress is computed by `computeBoxMetrics` from particle stresses and material volume in the solver's averaging box. In compact notation,

$$e_i = \sigma_i^{\text{target}} - \sigma_i^{\text{current}}. \tag{2.246}$$

The controller scales the input gains by an estimate of relative density and the maximum bulk or effective bulk modulus over active particles. If $K_{\max}$ is the global maximum bulk modulus and

$$\rho_{\text{rel}} = \min\left(1, \frac{V_{\text{mat}}}{V_{\text{domain}}}\right), \tag{2.247}$$

then the effective gains used by the code are

$$k_p = \frac{\rho_{\text{rel}}\,\texttt{stressControlKp}}{K_{\max}\Delta t}, \qquad k_d = \frac{\rho_{\text{rel}}\,\texttt{stressControlKd}}{K_{\max}}, \qquad k_i = \frac{\rho_{\text{rel}}\,\texttt{stressControlKi}}{K_{\max}\Delta t^2}. \tag{2.248}$$

The integral state and derivative estimate are updated as

$$\mathbf{I}^{n+1} = \mathbf{I}^n + k_i \Delta t\, \mathbf{e}^{n+1}, \tag{2.249}$$

$$\dot{\mathbf{e}}^{n+1} = \frac{\mathbf{e}^{n+1} - \mathbf{e}^n}{\Delta t}, \tag{2.250}$$

and the unconstrained controller output is

$$\bar{\mathbf{L}}_{\text{PID}} = k_p \mathbf{e}^{n+1} + \mathbf{I}^{n+1} + k_d \dot{\mathbf{e}}^{n+1}. \tag{2.251}$$

Each component is clipped so that the implied boundary speed is bounded by the CFL length scale,

$$|\bar{L}_i| \le \frac{\texttt{cflFactor}\, h_i}{\Delta t\,(X_i^{\max} - X_i^{\min})}. \tag{2.252}$$

For `stressControl[i] == 1`, the clipped value replaces `domainL[i]`. For `stressControl[i] == 2`, the direction is treated as a limiting mode that prevents further deformation in the forbidden direction once the stress error changes sign. The updated `domainL` is then converted to `domainF` by the explicit update

$$\bar{F}_i^{n+1} = \bar{F}_i^n + \bar{L}_i \bar{F}_i^n \Delta t. \tag{2.253}$$

Moving faces normal to controlled directions then use Eq. (2.236) when the moving-face branch is active; in practice, stress-controlled boundary-motion cases should use `stressControl[i] == 1` for controlled directions and commonly retain `prescribedBoundaryFTable` with a neutral F-table for robust activation. In this sense, stress control is internally a feedback generator for prescribed kinematics rather than a separate traction-boundary algorithm.

Listing 2.67: Stress-control path to boundary motion.

```
interpolateStressTable(time_n, dt) -> domainStress
computeBoxMetrics() -> currentStress, materialVolume
Kmax = global maximum bulk/effective bulk modulus over active particles
relativeDensity = min(1, materialVolume / domainVolume)
error = targetStress - currentStress
L_pid = scaled_Kp*error + integral + scaled_Kd*(error-lastError)/dt
clip L_pid by CFL boundary speed
for each controlled direction:
  domainL[i] = L_pid[i] or limiter-modified value
  domainF[i] = oldDomainF[i] + domainL[i]*oldDomainF[i]*dt
apply moving-face constraints using domainL
```

2.9.9 Reaction accumulation and reaction-history output

When a moving or contact boundary changes a normal nodal velocity, the solver can accumulate a face reaction. Only nodes owned by the local rank, identified by `gridGhostRank <= -1`, and with nodal mass above `smallMass` contribute. The default reaction calculation is the mass times the velocity correction divided by the time step,

$$R_f^{\text{local}} \mathrel{+}= m_{I\alpha} \frac{v_{I\alpha,d}^{\text{bc}} - v_{I\alpha,d}^{\text{old}}}{\Delta t}. \tag{2.254}$$

If `useInteralForceAsFaceReaction` is enabled, the contribution is instead based on the internal force component,

$$R_f^{\text{local}} \mathrel{-}= f_{I\alpha,d}^{\text{int}}. \tag{2.255}$$

The six local face sums are reduced across MPI ranks with a sum reduction and stored in `m_globalFaceReactions`. If `reactionHistory` is enabled and the write schedule is satisfied, the solver appends a row to `reactionHistory.csv` containing time, domain deformation components, box lengths, the six face reactions, and the current diagonal `domainL` values. This output is therefore a diagnostic of the constraints actually applied by `applyEssentialBCs`; it is not produced for pure outflow or inactive moving faces that do not change normal nodal velocities.

2.9.10 FMPM-specific boundary consistency

The FMPM update stores an uncontacted grid velocity seed before material-contact corrections. After `applyEssentialBCs` has imposed symmetry or moving constraints on `gridVelocity`, the solver copies only the constrained boundary components back into `gridUncontactedVelocity`. Unconstrained tangential components are deliberately left material-contact-free so that the filtered contact update can use them as an uncorrected seed.

Higher-order FMPM velocity increments use a homogeneous version of the boundary constraints. On boundary nodes, constrained components of the increment are set to zero. On buffer nodes, the normal increment is reflected oddly,

$$\Delta v_{G\alpha,d} = -\Delta v_{J(G)\alpha,d}, \tag{2.256}$$

and transverse increments are either reflected oddly when transverse velocities are prescribed or copied evenly when they are free. This is the same incremental compatibility idea emphasized by Nairn [18]: grid-based velocity constraints and lumped-mass contact operations are applied to each FMPM increment so that the final filtered velocity still satisfies the required constraints. Therefore, the FMPM restriction should not be read as a general incompatibility with moving velocity boundaries or multi-field material contact. Those paths use the incremental correction strategy. The code-status caveat is specifically the `boundaryConditionTypes == Contact` wall/contact-boundary bctype, which is guarded in the reviewed branch and should be treated as untested until the corresponding verification case is added.

2.9.11 Periodic boundaries as topology rather than face constraints

Periodic boundaries are not represented by `boundaryConditionTypes`. They are stored in the spatial partition and affect grid and particle topology. During initialization, the solver shifts periodic ghost-node coordinates on end partitions by one global domain extent so that distance-based operations see the closest periodic image. During the topology-preparation step, inactive ghost particle centers are wrapped across periodic boundaries for neighbor operations. During repartitioning and cleanup, active particle centers are wrapped back into the global domain by a modulo operation,

$$x_{p,i} \leftarrow \operatorname{mod}(x_{p,i} - X_i^{\min}, L_i) + X_i^{\min} \qquad \text{for periodic direction } i. \tag{2.257}$$

Out-of-range deletion checks are skipped in periodic directions. If `prescribedFTable` is active, the superposed velocity-gradient update is applied to particles only in periodic directions, which is the internal distinction between domain-periodic homogeneous deformation and boundary-driven deformation on moving faces.

2.10 Cohesive zone implementation

This section describes the solver-side cohesive-zone implementation used by GEOS-MPM. It is intentionally distinct from the user-facing event and PFW controls: the event-level inputs that activate a cohesive-zone interface are summarized in Section 2.6.5, while PFW-side input convenience and generated XML controls are summarized in Chapter 3. Here the focus is how the initialized interface is represented internally, how nodal cohesive tractions are computed, and which cohesive constitutive models are available. The implementation follows the reference-configuration cohesive-zone strategy described by Crook and Homel [23]: cohesive forces are not carried by separate massless cohesive particles. Instead, particles on a tagged surface store surface metadata and reference grid mappings. During each explicit step, the solver reconstructs jumps in displacement across the two material fields, evaluates a cohesive law at reference cohesive nodes, converts tractions to nodal forces using current surface area measures, and maps those forces back to the ordinary MPM particles.

2.10.1 Reference-interface representation

For a cohesive interface, the key geometric objects are the two material fields adjacent to the interface, denoted A and B, a reference surface normal $\mathbf{N}^0$, a current surface normal $\mathbf{n}$, and a reference surface area A^0. GEOS-MPM stores these data in `CohesiveZoneRegion` fields and in particle-level surface fields. The region owns a list of cohesive nodes, their reference positions, reference partitioning normals, side-specific reference normals, and side-specific reference areas. Cohesive particles store the reference grid nodes and reference shape-function weights that couple the particle to those cohesive nodes.

The central design choice is that cohesive integration uses a reference grid mapping. If I denotes a cohesive node and p a particle whose initial particle domain intersects the cohesive surface, the solver stores

$$N_{Ip}^0 = N_I(\mathbf{X}_p), \qquad \mathcal{S}_p^0 = \{I : N_{Ip}^0 \neq 0\}, \tag{2.258}$$

where $\mathbf{X}_p$ is a reference particle position or reference particle-domain quadrature location, depending on the particle representation. The same reference weights are reused during cohesive-law enforcement. This is the mechanism emphasized by Crook and Homel: by keeping the cohesive mapping tied to the reference interface, the cohesive law can remain stable during large deformation without introducing a separate surface-particle discretization [23].

The implementation currently assumes two velocity fields for cohesive-zone enforcement. During initialization, particles with a cohesive surface flag and a matching `CZTag` are partitioned onto fields A and B using the explicit surface normal and the particle's damage-field partitioning. Nodes receiving mass from both fields become cohesive nodes. This two-field assumption is checked explicitly when the cohesive reference configuration is initialized.

2.10.2 Initialization algorithm

Cohesive zones are initialized by the `CohesiveZone` MPM event. The event names one or more particle regions, assigns one cohesive constitutive model to each region, and associates each region with an integer `czTag`. A particle is eligible for a region only when its surface flag marks it as cohesive and its particle `CZTag` matches the region tag. The high-level initialization is:

Listing 2.68: Cohesive-zone reference-configuration initialization.

```
reset cohesive surface flags on particles
project particle surface normals to the grid
for each CohesiveZoneRegion R that is not initialized:
    select particles with SurfaceFlag == Cohesive and CZTag == R.tag
    split selected particle contributions onto fields A and B
    mark grid nodes that receive mass from both fields as cohesive nodes
    collect cohesive-node global ids across MPI ranks
    store cohesive-node reference positions and reference partition normals
    initialize nodal cohesive state: damage, temperature, and max jump history
    for each selected particle:
        store reference cohesive nodes and reference shape-function values
        store reference deformation-gradient and reference surface data
    compute side-specific reference nodal areas
    mark R.initialized = 1
```

The reference nodal area can be computed by the mesh-based area integration path or by the brute-force particle-area path. When `czVolumeNormalization` is enabled, the computed nodal surface area is multiplied by a local volume-normalization factor that limits the area contribution in partially filled grid cells. This option is intended to make the discrete cohesive area less sensitive to boundary cells and trimmed particle domains.

Two options control how particle surface data enter the reference configuration. If `computeNormalsAndPositions` is enabled, the solver computes particle surface normals and positions before initializing the cohesive mapping. The `normalsAndPositionsMethod` option selects the reconstruction method; the reviewed source registers `LogisticRegression` as the default. The `czSurfaceDisplacementUpdate` option then selects how particle surface displacements are reconstructed during the step. `TypeA` uses stored surface-position information; `TypeB`, the default, reconstructs the cohesive surface displacement from the particle-domain geometry and deformation gradients.

2.10.3 Step-level cohesive force update

At the beginning of the cohesive update, particle cohesive forces are zeroed. If any cohesive region has not yet been initialized, the initialization algorithm above is executed. The solver then enforces each cohesive law on the reference cohesive nodes. For every cohesive particle, a side-specific surface displacement $\mathbf{u}_p^s$ is computed from the particle motion and the selected surface-displacement update.

For the `TypeA` update, the stored reference surface-position vector is mapped through the current deformation gradient and compared with the reference position. A schematic form is

$$\mathbf{u}_p^{s,A} = (\mathbf{x}_p - \mathbf{X}_p) + \left(\mathbf{F}_p \mathbf{F}_{p,cz}^{-1} \mathbf{s}_p^0 - \mathbf{s}_p^0\right), \tag{2.259}$$

where $\mathbf{F}_{p,cz}$ is the deformation gradient stored at cohesive initialization and $\mathbf{s}_p^0$ is the particle-to-surface reference vector. For the `TypeB` update, the particle-domain surface vector is reconstructed from the particle geometry, mapped by the current and cohesive-reference deformation gradients, and differenced. Both forms have the same purpose: they estimate the displacement of the material surface point, not merely the particle centroid.

The particle surface displacements are projected to each cohesive node using the stored reference weights. For side $\alpha \in \{A, B\}$,

$$\bar{\mathbf{u}}_{I\alpha} = \frac{\sum_{p\in\mathcal{P}_{I\alpha}} m_p N_{Ip}^0 \, \mathbf{u}_p^s}{\sum_{p\in\mathcal{P}_{I\alpha}} m_p N_{Ip}^0}, \qquad \bar{\mathbf{n}}_{I\alpha} = \frac{\sum_{p\in\mathcal{P}_{I\alpha}} m_p N_{Ip}^0 \, \mathbf{n}_p}{\left\| \sum_{p\in\mathcal{P}_{I\alpha}} m_p N_{Ip}^0 \, \mathbf{n}_p \right\|}. \tag{2.260}$$

The implementation also maps the cofactor of the deformation gradient, $\operatorname{cof}\mathbf{F}$, so that reference area vectors can be advanced to the current configuration.

The effective interface normal is a mass-weighted difference of the two side normals,

$$\mathbf{n}_{AB} = \frac{m_A \bar{\mathbf{n}}_{IA} - m_B \bar{\mathbf{n}}_{IB}}{\|m_A \bar{\mathbf{n}}_{IA} - m_B \bar{\mathbf{n}}_{IB}\|}, \tag{2.261}$$

with the out-of-plane component suppressed for plane-strain runs. The opening and slip measures passed to the cohesive constitutive law are

$$\delta_n = -\mathbf{n}_{AB} \cdot (\bar{\mathbf{u}}_{IA} - \bar{\mathbf{u}}_{IB}), \tag{2.262}$$

$$\boldsymbol{\delta}_t = (\bar{\mathbf{u}}_{IA} - \bar{\mathbf{u}}_{IB}) + \delta_n \mathbf{n}_{AB}, \qquad \delta_t = \|\boldsymbol{\delta}_t\|. \tag{2.263}$$

The cohesive constitutive wrapper returns a normal traction T_n and shear traction magnitude T_t from these jump measures and the nodal state variables. When `preventCZInterpenetration` is enabled and the constitutive law would return a tensile-sign compression penalty inconsistent with the contact branch, the normal cohesive stress is clipped to avoid cohesive interpenetration forces.

The traction is converted to a nodal cohesive force by multiplying by the current area measure. In schematic form,

$$\mathbf{a}_{I\alpha} = \operatorname{cof} \mathbf{F}_{I\alpha}\, A^0_{I\alpha} \mathbf{N}^0_{I\alpha}, \qquad A_I = \mathbf{n}_{AB} \cdot \mathbf{a}_{I,AB}, \qquad \mathbf{f}^{cz}_{IA} = A_I \left(T_n \mathbf{n}_{AB} + T_t \mathbf{t}_{AB}\right), \tag{2.264}$$

where $\mathbf{t}_{AB} = \boldsymbol{\delta}_t / \delta_t$ when $\delta_t > 0$ and the side-B force is equal and opposite. The code stores these nodal tractions in temporary cohesive-grid arrays and finally scatters them back to particles through the same reference mapping,

$$\mathbf{f}^{cz}_p \mathrel{+}= \sum_{I \in \mathcal{S}^0_p} N^0_{Ip} \frac{m_p}{m_{I\alpha}} \mathbf{f}^{cz}_{I\alpha}. \tag{2.265}$$

Fully damaged cohesive nodes are propagated back to their supporting particles. Affected particles have their cohesive flag disabled for future cohesive-force calculations and their surface flag is changed to the damaged-cohesive state so that the interface can be handled by the ordinary multi-field contact machinery. This transition is one of the practical advantages of the Crook–Homel formulation: a weak or failed cohesive interface can degrade into a contact interface without changing the particle discretization [23].

2.10.4 Available cohesive constitutive models

GEOS-MPM dispatches cohesive laws through `ConstitutivePassThruCohesiveZone`. The reviewed source registers four executable cohesive-zone model classes: `UncoupledCohesiveZone`, `CoupledCohesiveZone`, `BicrystalCohesiveZone`, and `PolymerCohesiveZone`. All inherit the base cohesive state interface, which stores normal and shear stresses and copies converged stresses to old stresses at the end of the update.

`UncoupledCohesiveZone`. This is a linear uncoupled spring law with independent normal and tangential failure thresholds. Required inputs are `normalForceConstant`, `shearForceConstant`, `maxNormalDisplacement`, and `maxTangentialDisplacement`. The law is

$$T_n = -k_n \delta_n (1 - D), \qquad T_t = -k_t \delta_t (1 - D), \tag{2.266}$$

where D is set to one when either displacement threshold is exceeded. This model is useful for verification problems in which a simple elastic interface with abrupt failure is desired.

CoupledCohesiveZone. This law implements a coupled exponential cohesive response of the Needleman–Xu family [24]. Required inputs are `characteristicNormalDisplacement`, `characteristicTangentialDisplacement`, `defaultMaxNormalStress`, and `defaultMaxShearStress`. Optional displacement cutoffs `maxNormalDisplacement` and `maxTangentialDisplacement` default to very large values. The code forms work-of-separation scales

$$\phi_n = e\,\sigma_n^{max}\delta_n^c, \qquad \phi_t = \sqrt{e/2}\,\tau^{max}\delta_t^c, \tag{2.267}$$

then evaluates coupled normal and shear stresses from normalized jumps $\bar{\delta}_n = \delta_n/\delta_n^c$ and $\bar{\delta}_t = \delta_t/\delta_t^c$. In the reviewed implementation the coupling constants are fixed at $q = 1$ and $r = 0$, and complete damage is triggered when either optional maximum displacement cutoff is exceeded.

BicrystalCohesiveZone. This model extends `CoupledCohesiveZone` by scaling the nodal peak normal and shear strengths using a grain-boundary misorientation measure. The strength scale follows a Read–Shockley-like low-angle boundary form [25]: for angles below a cutoff θ_m the reduction depends on $(\theta/\theta_m)\,[1 - \log(\theta/\theta_m)]$, and above the cutoff the reduction saturates. The model inherits the coupled cohesive inputs listed above. In the reviewed source, the misorientation array is registered as model state; users should verify that the intended setup path populates this state before relying on misorientation-dependent strengths.

PolymerCohesiveZone. The polymer law represents a finite-thickness cohesive layer with bulk and shear stiffness, temperature-dependent strength and softening factors, plastic-strain-like history, and a maximum stretch criterion. Required inputs are `thickness`, the bulk-modulus parameters `bulkModulus`, `bulkModulusA`, `bulkModulusB`, `bulkModulusT0`, the shear-modulus parameters `shearModulus`, `shearModulusA`, `shearModulusB`, `shearModulusT0`, yield-strength parameters `yieldStrength0`, `yieldStrengthA`, `yieldStrengthB`, `yieldStrengthT0`, rate/softening parameters `r0`, `r0A`, `r0B`, `r0T0`, `r1`, `r2`, energy-like parameters `Gr`, `GrA`, `GrB`, `GrT0`, and stretch-limit parameters `maximumStretch`, `maximumStretchA`, `maximumStretchB`, `maximumStretchT0`. The model computes an effective stretch

$$\lambda = \frac{\sqrt{(h + \delta_n)^2 + \delta_t^2}}{h}, \tag{2.268}$$

where h is the layer thickness, updates temperature-dependent moduli and strengths, computes a yield-limited traction scale, and tracks previous stretch and plastic-strain-like history. Damage is activated when the stretch exceeds the temperature-adjusted maximum stretch.

Table 2.9: Cohesive-zone constitutive model summary.

Model	Principal required inputs	Notes
`Uncoupled CohesiveZone`	Normal and shear force constants; normal and tangential displacement limits.	Independent linear normal/tangential springs with abrupt complete damage.
`Coupled CohesiveZone`	Normal and tangential characteristic displacements; peak normal and shear stresses; optional displacement cutoffs.	Needleman–Xu-style coupled exponential cohesive law.

Model	Principal required inputs	Notes
`Bicrystal CohesiveZone`	Inherits `CoupledCohesiveZone` inputs; requires a valid misorientation state for strength scaling.	Coupled law with misorientation-dependent peak-strength scaling.
`Polymer CohesiveZone`	Layer thickness; temperature-dependent bulk, shear, yield, rate/softening, `Gr`, and maximum-stretch parameters.	Finite-thickness polymer-interface law with thermal softening and stretch-based failure.

2.10.5 Input controls and current limitations

The minimum event-level controls are `regionNames`, `constitutiveModels`, and `czTags`; the arrays must have the same nonzero length. Optional controls are `czVolumeNormalization`, `computeNormalsAndPositions`, `normalsAndPositionsMethod`, and `czSurfaceDisplacementUpdate`. These controls are documented with the other event inputs in Section 2.6.5. PFW-side setup usually also requires particle fields for surface flag, cohesive-zone tag, surface normal, and surface position. The default PFW particle-field list already includes `SurfaceFlag`, `CZTag`, `SurfaceNormal`, and `SurfacePosition` in cohesive-capable cases.

The principal implementation limitations in the reviewed snapshot are: cohesive-zone enforcement assumes two velocity fields; initialization depends on consistent particle surface flags and CZ tags; misorientation-dependent bicrystal strengths require a valid misorientation state; and once cohesive damage reaches completion the interface is handed off to contact rather than continuing to carry cohesive traction. These limitations should be reflected in verification tests for new cohesive-zone use cases.

2.11 Contact options: fields, normals, gap closure, and overlap control

This section documents the solver-side contact machinery. It intentionally focuses on the internal fields, algorithms, and option-dependent branches. User-facing input controls are collected in Section 3.8, and the generated attribute inventory in Appendix A should be used as the authoritative list of currently registered solver keys. At the algorithmic level, GEOS-MPM contact follows the multi-velocity-field MPM contact family of Bardenhagen, Brackbill, Sulsky, Guilkey, and coworkers [8, 9]. GEOS-MPM then provides several complementary ways to create the contact fields and contact geometry. Prescribed multi-field contact uses user-assigned particle groups. Dynamic same-material fracture/contact uses the damage-field-gradient, or DFG, partition of Homel and Herbold [6]. Logistic-regression contact reconstructs a separating plane from the local point cloud following Nairn, Hammerquist, and Smith [10]. In addition, the cohesive-zone implementation described in Section 2.10 can provide exact surface-contact data: stored surface positions and normals define the gap plane for curved boundaries, so the contact normal and gap are not forced to follow stair-step particle domains. This removes the particle-domain stair-step discretization error in the contact geometry for those cohesive-zone-derived surfaces, while the usual grid-transfer, time-integration, and constitutive-discretization errors remain [23].

2.11.1 Nodal contact state and activation

GEOS-MPM treats material contact as a nodal, pairwise correction between velocity fields. At a grid node I, each velocity field α carries field-specific mass $m_{I\alpha}$, momentum $\mathbf{q}_{I\alpha}$, velocity $\mathbf{v}_{I\alpha}$, material volume $V_{I\alpha}$, damage summaries, center of mass or center of volume, and surface geometry fields such as $\mathbf{n}_{I\alpha}$ and $\mathbf{x}^s_{I\alpha}$. Contact is active only when both the global contact switch is on and the run actually has

multiple velocity fields or contact-wall boundaries,

$$\texttt{hasContact} = (N_{\text{fields}} > 1) \vee (\exists\, \text{face with } \texttt{boundaryConditionTypes} = \texttt{Contact}) \,. \tag{2.269}$$

For material-material contact, the contact update loops over each nodal pair (A, B) with $A < B$, checks that both field masses exceed the small-mass cutoff, constructs an effective pair normal $\hat{\mathbf{n}}_{AB}$, evaluates whether the pair is separable, and then applies equal-and-opposite force or impulse corrections. The ordinary lumped update uses a force form, while the FMPM path uses the same kinematics in impulse form so that the cumulative contact impulse can be used by the incremental full-mass-matrix correction.

Listing 2.69: Solver-level material contact loop.

```
for node I:
    for fields A < B:
        if m[I,A] <= smallMass or m[I,B] <= smallMass:
            continue
        nAB = pair_normal(I,A,B, contactNormalType)
        if norm(nAB) is tiny:
            nAB = n[I,A]
        if planeStrain:
            nAB.z = 0
        nAB = normalize(nAB)
        separable = evaluate_separability(I,A,B)
        apply_pairwise_contact(I,A,B,nAB,separable,
                               contactGapCorrection,
                               overlapCorrection,
                               frictionCoefficientTable)
```

2.11.2 Prescribed multi-field contact groups

A prescribed multi-field contact calculation begins with the particle field `particleGroup`. During initialization, the solver finds the maximum group index over all active particle subregions and sets

$$N_g = 1 + \max_p g_p, \tag{2.270}$$

where g_p is the stored particle group. A protective check rejects cases with more than 100 prescribed groups, which is intended to catch accidental large group identifiers before they create excessive nodal storage. If DFG partitioning is disabled, the field index is simply

$$\alpha = g_p, \qquad N_{\text{fields}} = N_g. \tag{2.271}$$

Thus, particles in different groups scatter mass, momentum, internal force, damage summaries, and surface fields to different velocity fields at the same grid node. Pairwise contact between fields with different group indices is considered a prescribed material-material interface. The pairwise friction coefficient is taken from `frictionCoefficientTable` by reducing the field indices modulo the prescribed group count,

$$\mu_{AB} = \mu_{A \bmod N_g,\, B \bmod N_g}. \tag{2.272}$$

This modulo rule is important when DFG is also enabled, because it keeps friction material-pair-based even after each prescribed group is split into damage-side subfields.

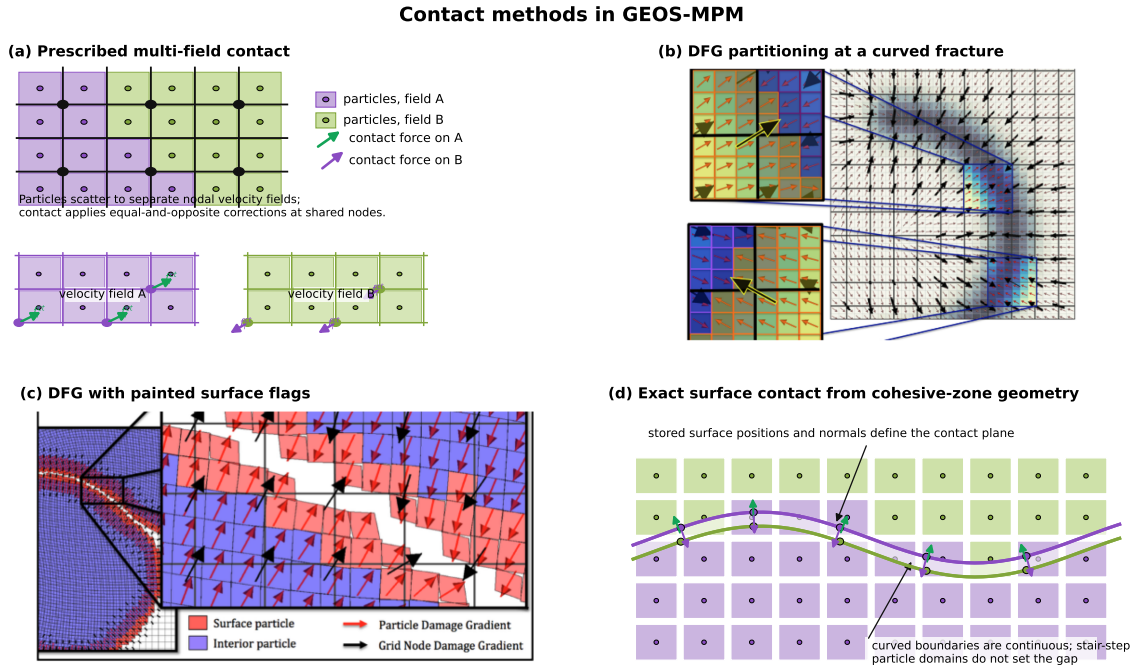


Figure 2.6: Composite schematic of the contact-method families used by GEOS-MPM. Panel (a) is redrawn from the multi-velocity-field contact concept of Bardenhagen, Guilkey, and coworkers. Panels (b) and (c) are adapted from the DFG fracture/contact figures of Homel and Herbold. Panel (d) summarizes the exact surface-position/normal path used by cohesive-zone-derived contact, where stored surface geometry rather than stair-step particle domains defines the gap and normal [9, 6, 23].

2.11.3 Damage-field-gradient partitioning

When `damageFieldPartitioning=1`, each prescribed group is split into two possible DFG flags. The total number of nodal velocity fields becomes

$$N_{\text{fields}} = 2N_g, \tag{2.273}$$

with field index

$$\alpha = c_p(I)N_g + g_p, \qquad c_p(I) \in \{0, 1\}. \tag{2.274}$$

The flag $c_p(I)$ is recomputed for each particle-grid mapping. Let $\mathbf{g}_I$ be the grid damage gradient at node I. Let the particle mapping normal be

$$\mathbf{m}_p = \begin{cases} \nabla D_p, & \|\mathbf{n}_p^s\| \approx 0, \\ \mathbf{n}_p^s, & \|\mathbf{n}_p^s\| > 0, \end{cases} \tag{2.275}$$

where ∇D_p is the particle damage gradient and $\mathbf{n}_p^s$ is the explicit particle surface normal. The reviewed implementation assigns

$$c_p(I) = \begin{cases} 1, & \texttt{damageFieldPartitioning} = 1 \text{ and } \mathbf{g}_I \cdot \mathbf{m}_p < 0, \\ 0, & \text{otherwise.} \end{cases} \tag{2.276}$$

This sign test is the practical DFG operation: particles on opposite sides of a damage-gradient interface scatter to different velocity fields at the same node, so the solver can represent a velocity discontinuity without explicitly meshing a crack surface.

This has three important consequences. First, DFG enables automatic self-contact for bodies with painted surface flags or evolving damage fields without requiring a priori assignment of a separate contact group to every potential contact partner. This is especially useful in porous, fragmented, or granular configurations where the set of contacts is not known before the calculation. Second, DFG is a kinematic enrichment for continuum damage models: once a damage band becomes sufficiently localized and passes the separability criteria, the two sides can acquire independent velocities, slip, and separation. In this sense the method automatically inserts a contact-capable fracture surface into an otherwise continuum damage calculation. Third, for many-body same-material problems, DFG can be cheaper than prescribed one-field-per-body contact because it requires only two DFG velocity fields per prescribed contact group, independent of how many potential same-material contacts appear within that group.

DFG and prescribed multi-field contact are compositional. Prescribed groups determine the material or body identity g_p, while DFG determines the damage-side flag $c_p(I)$. In a two-group calculation, for example, group 0 can occupy fields 0 and 2, while group 1 can occupy fields 1 and 3. Contact between different prescribed groups is separable by construction. Contact between the two DFG sides of the same prescribed group is allowed only when the damage and surface-quality criteria described in Section 2.11.9 are satisfied.

2.11.4 Surface normals and surface positions

Each contact pair requires a normal direction and, for the more restrictive gap-closure options, a surface position. GEOS-MPM can use explicit particle surface geometry supplied in the particle file, implicit geometry inferred from particle volume gradients, logistic-regression reconstruction from the local point cloud, or exact surface data created by cohesive-zone initialization. The exact cohesive-zone path is a special case of the explicit-geometry path: the particles carry surface positions and normals tied to a smooth reference interface, so the later contact calculation can use that stored surface rather than the visible stair-step particle domain.

The grid surface normal accumulated during particle-to-grid mapping has the form

$$\widetilde{\mathbf{n}}_{I\alpha} = \sum_{p \in \mathcal{P}_{I\alpha}} V_p \, \nabla N_{Ip} + \eta \sum_{p \in \mathcal{P}^s_{I\alpha}} V_p N_{Ip} \, \mathbf{n}^s_p, \tag{2.277}$$

where $\mathcal{P}_{I\alpha}$ is the set of particles mapped to node I and field α, $\mathcal{P}^s_{I\alpha}$ is the subset with explicit surface normals, and $\eta =$ `explicitSurfaceNormalInfluence`. The first term is the implicit volume-gradient normal used by DFG-style surface reconstruction. The second term lets explicitly supplied particle surface normals dominate the grid normal when the influence factor is chosen large enough. The normalized vector is

$$\hat{\mathbf{n}}_{I\alpha} = \frac{\widetilde{\mathbf{n}}_{I\alpha}}{\|\widetilde{\mathbf{n}}_{I\alpha}\|}, \tag{2.278}$$

with the z component removed in plane-strain calculations.

When a particle has an explicit surface normal and surface position, the solver also scatters a nodal surface-position vector. The scalar signed distance from node I to the particle surface along the particle normal is

$$s_{Ip} = \left(\mathbf{x}^s_p + \mathbf{x}_p - \mathbf{x}_I \right) \cdot \mathbf{n}^s_p. \tag{2.279}$$

The corresponding grid contribution is

$$\mathbf{x}^s_{I\alpha} \leftarrow \mathbf{x}^s_{I\alpha} + m_p N_{Ip} \, s_{Ip} \, \mathbf{n}^s_p, \tag{2.280}$$

with a separate mass-like normalization stored in `gridSurfaceFieldMass`. When `useSurfacePositionForContact=1`, these surface positions are used by the gap criterion if both fields have sufficient surface-position mass; otherwise the contact law falls back to field centers with a spacing correction.

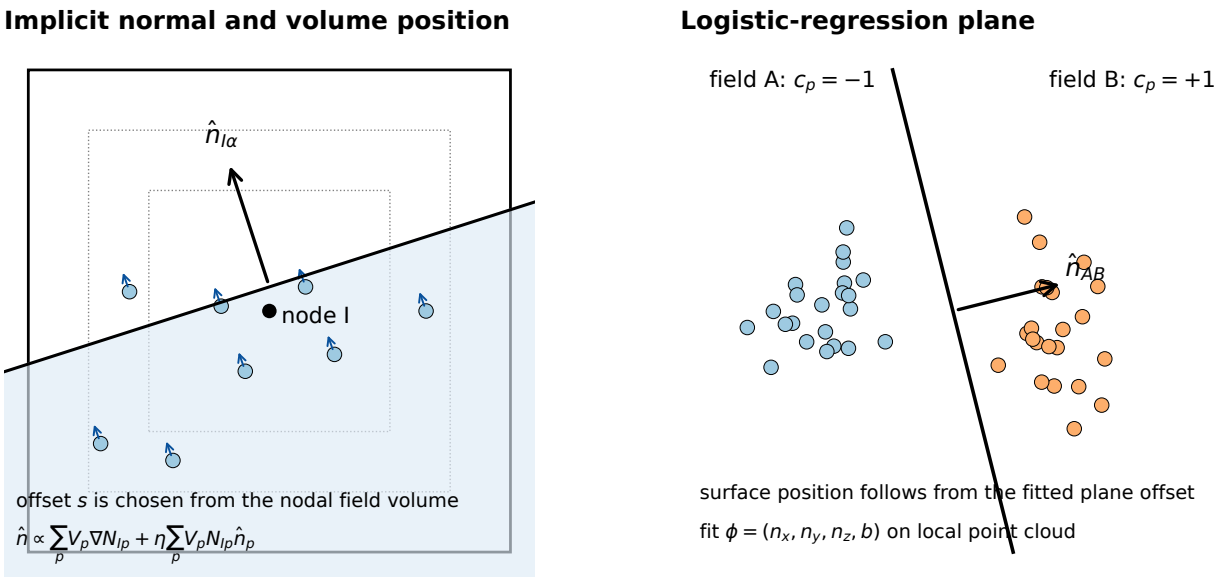


Figure 2.7: Schematic of the two automatic surface-geometry families. Left: an implicit normal from mapped particle volume gradients, optionally biased by explicit particle normals, plus a volume-matched plane position. Right: a logistic-regression plane fitted to the two local field point clouds. The figure is newly drawn for this manual and is conceptually adapted from DFG and regression-based MPM contact descriptions [6, 10].

2.11.5 Automatic particle normal and position reconstruction

The option `computeParticleSurfaceNormalsAndPositions` invokes a one-time reconstruction pass before the particle-to-grid stage of the step in which the flag is seen. The pass first scatters temporary mass, volume, and the raw normal in Eq. (2.277). It then branches on `normalAndPositionMethod`:

`DFGAndVolumeIntegration`. The normalized DFG or volume-gradient normal is kept as the grid-based normal. The grid-based surface position is recovered by matching the mapped nodal field volume to the volume of a weighted half-space, as described in Section 2.11.7.

`LogisticRegression`. For each active node and each active field pair, the solver fits a plane to the local two-field point cloud, stores opposite normals and offsets for the two fields, and then maps those normals and positions back to the particles.

After either branch, `mapSurfaceNormalsAndPositionsToParticles` transfers the reconstructed geometry back to particle fields and updates the corresponding reference normal and reference surface-position fields. The temporary grid mass, volume, and surface-normal arrays used by the reconstruction are zeroed at the end of the pass so that subsequent particle-to-grid operations start from a clean state.

2.11.6 Logistic-regression surface reconstruction

The logistic-regression method follows the point-cloud philosophy described by Nairn, Hammerquist, and Smith: rather than deriving the interface solely from grid data, it estimates the most probable local plane separating two material point clouds at a contact node [10]. For a node I and field pair (A, B), the reviewed implementation constructs feature vectors

$$\boldsymbol{\xi}_p = \begin{bmatrix} x_{p,1} - x_{I,1} & x_{p,2} - x_{I,2} & x_{p,3} - x_{I,3} & 1 \end{bmatrix}^T, \tag{2.281}$$

using neighboring particles that partition into either field A or field B. The class label is

$$c_p = \begin{cases} -1, & p \mapsto A, \\ +1, & p \mapsto B. \end{cases} \tag{2.282}$$

For parameter vector $\boldsymbol{\phi} = (n_1, n_2, n_3, b)^T$, the code uses the logistic score

$$\widehat{c}_p(\boldsymbol{\phi}) = \frac{2}{1 + \exp(-\boldsymbol{\xi}_p \cdot \boldsymbol{\phi})} - 1. \tag{2.283}$$

The nonlinear least-squares update is regularized on the normal components,

$$\min_{\boldsymbol{\phi}} \sum_p w_p \left(c_p - \widehat{c}_p(\boldsymbol{\phi})\right)^2 + \lambda \left(n_1^2 + n_2^2 + n_3^2\right), \tag{2.284}$$

with $w_p = 1$ and $\lambda = 10^{-7}\|\mathbf{h}\|^2$ in the reviewed code. The initial normal is the mass-weighted pair normal, $m_A\mathbf{n}_A - m_B\mathbf{n}_B$. Iteration stops when the normalized surface-position change is below `LRtolerance`, or when `maxLRIterations` is reached. If the local 4×4 matrix is singular or the normal magnitude collapses, the code falls back to the previous normal and surface position.

The fitted normal is the normalized first three components of $\boldsymbol{\phi}$. The implemented surface offset is

$$s = \frac{-\log(1/3) - b}{\|\mathbf{n}\|}, \qquad \mathbf{x}^s_{I,AB} = s\,\hat{\mathbf{n}}_{AB}. \tag{2.285}$$

This is used both as an automatic surface-position reconstruction option and, when the requested pair normal type is `LogisticRegression`, as an on-demand pair-normal calculation inside the contact loop.

2.11.7 Volume method for contact surface position

The `DFGAndVolumeIntegration` position method assumes that a partially filled nodal support can be approximated by a fully dense half-space cut by a plane with known unit normal $\hat{\mathbf{n}}_{I\alpha}$. The unknown is the plane offset s from the grid node. Let $\mathbf{r} = \mathbf{x} - \mathbf{x}_I$ and define the trilinear support weight

$$W_I(\mathbf{r}) = \left(1 - \frac{|r_1|}{h_1}\right)\left(1 - \frac{|r_2|}{h_2}\right)\left(1 - \frac{|r_3|}{h_3}\right) \tag{2.286}$$

inside the nodal support and zero outside. The volume predicted by an offset s is

$$\mathcal{V}(s) = \int_{\mathrm{supp}(I)} H(s - \hat{\mathbf{n}}_{I\alpha} \cdot \mathbf{r})\, W_I(\mathbf{r})\, dV. \tag{2.287}$$

The solver performs bisection over the admissible offset interval and chooses s such that $\mathcal{V}(s)$ matches the mapped grid material volume $V_{I\alpha}$ to a tolerance of 10^{-3} cell volumes. The numerical integration currently uses a structured 200^3 quadrature over the nodal support. Cells with zero mapped volume and cells with mapped volume greater than or equal to the cell volume are ignored; the latter limitation matters when contact overlap or densification produces overfilled nodal supports.

2.11.8 Pair-normal weighting between fields

The field normals $\mathbf{n}_A$ and $\mathbf{n}_B$ must be reduced to a single pair normal $\hat{\mathbf{n}}_{AB}$ oriented outward from field A toward field B. GEOS-MPM exposes the following `contactNormalType` branches:

Option	Solver operation
`Difference`	Uses $\mathbf{n}_{AB} = \mathbf{n}_A - \mathbf{n}_B$ before normalization. This is the simplest symmetric difference of the two outward normals.
`MassWeighted`	Uses $\mathbf{n}_{AB} = m_A\mathbf{n}_A - m_B\mathbf{n}_B$. This is the constructor default in the reviewed source.
`LargerMass`	Uses $\mathbf{n}_A$ if $m_A > m_B$, otherwise uses $-\mathbf{n}_B$. This can be useful when a heavy or well-resolved field should dominate the pair normal.
`Mixed`	Compares field densities $\rho_A = m_A/V_A$ and $\rho_B = m_B/V_B$. If they are similar, it uses the mass-weighted normal. Otherwise it uses the normal of the denser field.
`Aligned`	Uses explicit-normal alignment weights to suppress poorly aligned field normals. The current code computes a smoothstep weight above a fixed threshold of 0.9 and forms the weighted difference. The attribute `contactNormalExponent` is registered, but the active branch uses the fixed smoothstep weighting rather than the commented exponentiation lines.
`LogisticRegression`	Re-fits a logistic-regression plane for the specific field pair at the contact node and uses the fitted plane normal as $\mathbf{n}_{AB}$.

For the `Aligned` option, the alignment weights are accumulated during particle-to-grid mapping from particles with explicit normals. In abstract notation, the weight is a mass-normalized average of the particle/grid normal agreement,

$$w_{I\alpha} = \frac{\sum_{p \in \mathcal{P}^s_{I\alpha}} m_p N_{Ip} \left(\hat{\mathbf{n}}_{I\alpha} \cdot \hat{\mathbf{n}}^s_p\right)}{\sum_{p \in \mathcal{P}^s_{I\alpha}} m_p N_{Ip}}. \tag{2.288}$$

The pair normal is normalized after the option-specific construction, and the solver stores opposite normals back to the two grid fields for consistency.

2.11.9 Separability criteria

The contact law first decides whether two fields are bonded together at the node or represent separable surfaces. If they are not separable, the nodal correction drives both fields toward the center-of-mass velocity in both normal and tangential directions, which is effectively a no-slip compatibility correction. If they are separable, the normal and Coulomb-like tangential contact law is applied only when the gap-closure criterion is satisfied.

The reviewed separability criterion is

$$\begin{aligned}\text{separable} = \Big[\big(\max(D_A^{\max}, D_B^{\max}) \geq 0.9999 \big) \wedge (D_A \geq D_{\min}) \\ \wedge (D_B \geq D_{\min}) \wedge (Q_s > Q_{\min}) \Big] \\ \vee (A \bmod N_g \neq B \bmod N_g),\end{aligned} \tag{2.289}$$

where $D_{\min}$ = `separabilityMinDamage` and $Q_{\min}$ = `surfaceQualityThreshold`. The second term means that different prescribed contact groups are separable regardless of damage. The first term is the same-group DFG/fracture branch: at least one field must be essentially fully damaged, both fields must exceed the minimum damage threshold, and the local damage-gradient/surface alignment must pass the quality threshold.

Two additional guards can suppress separability. If `treatFullyDamagedAsSingleField=1` and the grid damage-gradient magnitude is very small, the code treats the fields as inseparable to avoid arbitrary internal separation planes in fully damaged or undamaged material. If `thinFeatureDFGThreshold` is set below its large default value, a same-material DFG pair with very small field-center spacing relative to the neighbor radius is also forced to be inseparable. This prevents a thin strip of material, whose two physical surfaces are closer than the grid resolution, from being spuriously split into internal slip surfaces.

2.11.10 Gap closure and frictional contact

For a field pair, the center-of-mass velocity is

$$\mathbf{v}_{AB} = \frac{\mathbf{q}_A + \mathbf{q}_B}{m_A + m_B}. \tag{2.290}$$

The normal correction needed to bring field A to the center-of-mass normal velocity is proportional to

$$f_n = \frac{m_A}{\Delta t} \left(\mathbf{v}_{AB} - \mathbf{v}_A \right) \cdot \hat{\mathbf{n}}_{AB} \tag{2.291}$$

for the force form, or to the same expression without $1/\Delta t$ for the impulse form. The solver uses an orthonormal basis $\{\hat{\mathbf{n}}_{AB}, \hat{\mathbf{s}}_1, \hat{\mathbf{s}}_2\}$ to compute tangential sticking forces or impulses. For separable contact, the normal force is multiplied by a gap-closure factor $\chi \in [0, 1]$, and the tangential force magnitude is limited by

$$|f_t| \leq \mu_{AB} |f_n|. \tag{2.292}$$

If cohesive tangential forces are active for the pair, the frictional tangential contribution is set to zero so that the cohesive law supplies the tangential resistance.

The gap length scale used by the contact criterion is the grid-normal support length,

$$g_0 = \left(\sum_{i=1}^{d} \frac{\hat{n}_{AB,i}^2}{h_i^2} \right)^{-1/2}, \tag{2.293}$$

with $d = 2$ for plane strain and $d = 3$ otherwise. If both fields have usable surface positions and `useSurfacePositionForContact=1`, the solver uses those positions in the gap,

$$g = (\mathbf{x}_B^s - \mathbf{x}_A^s) \cdot \hat{\mathbf{n}}_{AB}. \tag{2.294}$$

If one or both fields lack surface positions, the corresponding field center is used and the code subtracts $0.5g_0$ for each missing side. In the general implemented form,

$$g = (\mathbf{x}_B^* - \mathbf{x}_A^*) \cdot \hat{\mathbf{n}}_{AB} - \gamma g_0, \qquad \gamma \in \{0, 0.5, 1\}, \tag{2.295}$$

where $\mathbf{x}^*$ is either a surface position or a field center.

The approach test is

$$a = (\mathbf{v}_A - \mathbf{v}_{AB}) \cdot \hat{\mathbf{n}}_{AB}. \tag{2.296}$$

The `contactGapCorrection` option selects the closure factor:

$$\chi = \begin{cases} H(a), & \texttt{Simple}, \\ H(a)H(-g), & \texttt{Implicit}, \\ H(a), & \texttt{Softened} \text{ and } g \le 0, \\ H(a)\left(1 - g/g_0\right), & \texttt{Softened} \text{ and } 0 < g < g_0, \\ 0, & \texttt{Softened} \text{ and } g \ge g_0, \end{cases} \tag{2.297}$$

where H is the Heaviside step function. `Simple` is purely velocity based and can close gaps before geometric overlap is detected. `Implicit`, the current default, requires both approach and negative gap. `Softened` ramps the normal correction over one grid-normal support length to avoid a discontinuous on/off transition near first contact.

2.11.11 Global and group-dependent friction coefficients

The tangential contact limit uses a coefficient μ_{AB} chosen from a solver-side square table. The contact loop does not distinguish whether the table was supplied directly or expanded from a scalar; it simply evaluates

$$\mu_{AB} = \texttt{frictionCoefficientTable}\left[A \bmod N_g,\, B \bmod N_g\right], \tag{2.298}$$

where A and B are velocity-field indices and N_g is the number of prescribed contact groups. The modulo reduction is important when DFG is active: the DFG side flag changes the velocity-field index, but the friction lookup still refers to the underlying prescribed group.

If the input provides only the scalar `frictionCoefficient`, `initializeFrictionCoefficients` expands it to a uniform $N_g \times N_g$ table,

$$\mu_{ij} = \mu_{\text{global}}, \qquad i, j = 0, \ldots, N_g - 1. \tag{2.299}$$

A scalar value of `-1` is interpreted as unspecified and is converted to zero friction before the table is built; negative values after that conversion are rejected. This global mode is concise for homogeneous interfaces, DFG self-contact with a single material, and verification problems where all material pairs should share the same Coulomb limit.

If `frictionCoefficientTable` is supplied, it is used directly after validation. The table must be square, its number of rows and columns must equal N_g, and off-diagonal entries must be symmetric. Off-diagonal entries control prescribed group-group interfaces, while diagonal entries control same-group contact, including DFG self-contact or post-failure contact between the two sides of a same-group fracture. A `FrictionCoefficientSwap` event can later replace either the scalar coefficient or the table; after the event the same initialization logic is called, so the material contact loop always sees a validated table.

2.11.12 Overdensification and overlap correction

Contact, DFG, CPDI domain changes, and high compression can create nodal or particle states that are too dense relative to the grid or to a nonlocal particle-volume estimate. GEOS-MPM provides several overlap-control branches. These should be viewed as stabilization or correction options rather than replacements for a well-resolved contact surface.

`Off`. No additional overlap correction is applied beyond the contact law.

`NormalForce`. During pairwise nodal contact, the code estimates an overlap length from the sum of the pair volumes and the grid support geometry. In 3D,

$$\ell_{\mathrm{ov}} = \frac{V_A + V_B - V_{\mathrm{cell}}}{A_\perp}, \qquad A_\perp = \frac{V_{\mathrm{cell}}}{\sum_i |\hat{n}_i| h_i}. \tag{2.300}$$

In plane strain the implementation uses the corresponding half-cell support convention. If ℓ_{ov} lies between the two threshold levels `overlapThreshold1` and `overlapThreshold2`, an additional compressive normal force or impulse is added. The correction is ramped from zero to full strength across the threshold interval and is limited by a CFL-like velocity bound proportional to $0.05 \min(m_A, m_B) v_{\max}/\Delta t$ in force form.

`SPH`. Post-input initialization forces `computeSPHJacobian=1`. Each step computes a nonlocal SPH estimate of the particle Jacobian, J_{SPH}, from neighboring particles. After the standard deformation-gradient update, the correction compares the constitutive-particle Jacobian $J = \det \mathbf{F}$ to the nonlocal value through

$$r = \frac{J}{J_{\mathrm{SPH}}}. \tag{2.301}$$

If $r >$ `overlapThreshold1`, a smoothstep ramp toward the SPH Jacobian is applied. The deformation gradient is scaled by

$$s = \begin{cases} \left(1 + \alpha(r^{-1} - 1)\right)^{1/2}, & \text{plane strain}, \\ \left(1 + \alpha(r^{-1} - 1)\right)^{1/3}, & \text{3D}, \end{cases} \tag{2.302}$$

with $s \geq 0.99$ so the density correction cannot change too much in a single step. The code also updates `particleFDot` and adjusts the velocity gradient consistently with the scaled deformation gradient. Comments in the source note that the SPH estimate can be noisy near free and symmetry boundaries.

`Volume`. During grid-to-particle transfer, the solver estimates the nodal material volume divided by the nodal support volume. If this overlap measure exceeds `overlapThreshold1`, the diagonal particle velocity-gradient components are reduced by a small compressive correction proportional to $(\text{overlap} - 1)/\Delta t$. The current implementation is deliberately gentle, targeting roughly a 100-step correction time. Because the reviewed code sums a group-indexed subset of grid material volumes in this branch, DFG multi-flag cases should be verified before using `Volume` as the primary overlap-control mechanism.

The separate flag `directionalOverlapCorrection` registers a particle `particleSPHF` tensor and activates neighbor-list construction. The reviewed source also contains a kernel-based `computeSphF` helper that estimates a deformation-like tensor from reference and current neighbor positions. In the current explicit-step sequence, however, this directional path is not wired into the main overlap-correction loop in the same way as `SPH`, `Volume`, and `NormalForce`. Treat it as a development hook unless a dedicated verification case exercises it.

2.11.13 Summary examples and verification targets

The contact options above can be viewed as a hierarchy of increasingly geometric contact descriptions. Prescribed multi-field contact supplies the velocity fields directly through particle groups. DFG supplies

the velocity-field split dynamically from the damage-gradient or painted surface data. Logistic regression reconstructs a local separating surface from the two-field point cloud. The cohesive-zone surface path supplies exact surface positions and normals from the initialized cohesive interface and therefore keeps curved contact geometry independent of the stair-step outline of the particle domains. Verification should exercise each of these descriptions separately and in combination with FMPM, boundary motion, frictional slip, gap closure, and overdensification control.

Figure 2.8 records two representative high-value verification targets. The first is the large-deformation DFG contact example represented by Fig. 9 of Homel and Herbold's field-gradient partitioning paper, where a dynamically evolving damage/fracture surface must split the material into contact-capable sides without pre-defining every potential contact pair [6]. The second is indentation against an explicitly described surface from the Crook-Homel cohesive-zone formulation. In that case, the relevant contact test is not merely that particles do not interpenetrate; it is that the gap, normal, and tangential slip response remain controlled by the stored smooth surface positions and normals after a cohesive interface transitions into ordinary contact [23].

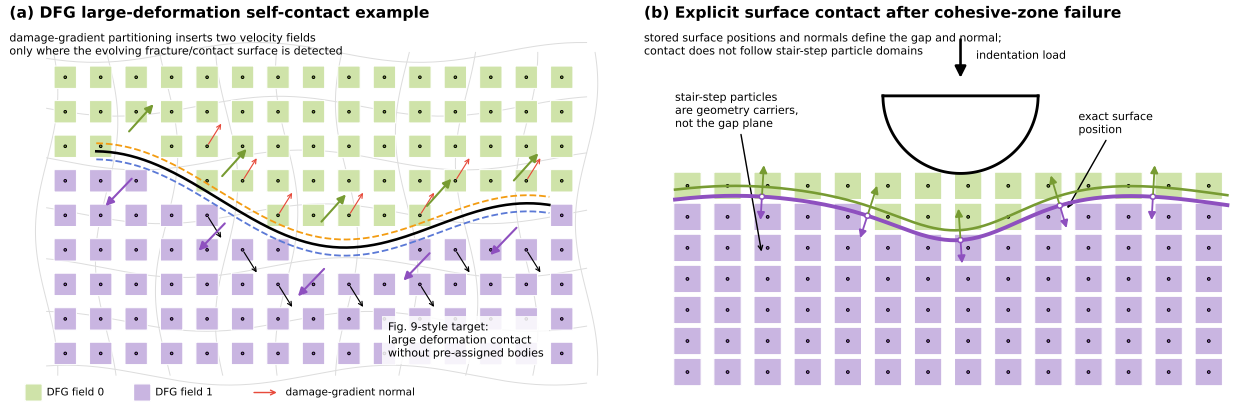


Figure 2.8: Representative contact verification targets. Panel (a) is an original schematic inspired by the large-deformation DFG contact demonstration, Fig. 9 of Homel and Herbold [6]. Panel (b) is an original schematic inspired by the indentation/explicit-surface contact capability in the Crook-Homel cohesive-zone formulation [23].

A useful verification matrix is: (i) prescribed two-body contact with a scalar global friction coefficient, (ii) prescribed multi-body contact with a symmetric group-dependent friction table, (iii) DFG same-material self-contact with a nonzero diagonal friction entry, (iv) DFG plus prescribed groups where field indices use the modulo friction-table lookup, (v) logistic-regression contact on a curved or oblique two-material interface, (vi) cohesive-zone-derived exact surface contact after cohesive failure, and (vii) each of the above under `Simple`, `Implicit`, and `Softened` gap closure. Overlap correction should then be tested as an optional stabilization layer rather than as the primary mechanism for enforcing contact.

2.11.14 Input controls and cross references

Table 2.11 summarizes the solver attributes most directly tied to the contact algorithms above. These are solver-side controls. Section 3.8 should describe how PFW exposes or forwards them from `pfw_input`, and Appendix A provides the generated attribute inventory.

Table 2.11: Internal contact controls discussed in Section 2.11.

Attribute	Internal effect
`enableContact`	Master switch for material contact once `hasContact` is true.
`damageFieldPartitioning`	Enables DFG two-flag splitting inside each prescribed contact group.
`contactNormalType`	Selects the pair-normal construction branch: `Difference`, `MassWeighted`, `LargerMass`, `Mixed`, `Aligned`, or `LogisticRegression`.
`contactGapCorrection`	Selects `Simple`, `Implicit`, or `Softened` closure in Eq. (2.297).
`frictionCoefficient`	Scalar global coefficient expanded to a uniform group-group table when no explicit table is supplied.
`frictionCoefficientTable`	Defines pairwise group-dependent friction coefficients; must be square, must match the number of contact groups, and is indexed by A mod N_g and B mod N_g when DFG adds fields.
`computeParticleSurface NormalsAndPositions`	Triggers automatic particle normal/position reconstruction.
`normalAndPositionMethod`	Chooses `LogisticRegression` or `DFGAndVolumeIntegration` for automatic reconstruction.
`explicitSurfaceNormalInfluence`	Controls the relative strength of explicit particle normals in the grid normal scatter.
`useSurfacePositionForContact`	Allows surface-position fields to replace center-of-volume positions in the gap calculation when available.
`LRtolerance`, `maxLRIterations`	Control convergence and iteration limit of the logistic-regression fit.
`surfaceQualityThreshold`, `separabilityMinDamage`	Control same-group DFG separability.
`thinFeatureDFGThreshold`	Suppresses separability for very thin same-material DFG features.
`treatFullyDamagedAsSingleField`	Suppresses arbitrary splitting in fully damaged regions with small damage-gradient magnitude.
`overlapCorrection`	Selects `Off`, `NormalForce`, `SPH`, or `Volume`.
`overlapThreshold1`, `overlapThreshold2`	Define ramp thresholds for overlap-correction branches.
`computeSPHJacobian`	Computes nonlocal SPH Jacobian data; automatically enabled by `overlapCorrection=SPH`.
`directionalOverlapCorrection`	Development hook for neighbor-based directional overlap data.

2.12 Robustness controls: deletion, deformation-gradient reset, and particle splitting

This section documents solver-side mechanisms that deliberately alter the active particle set or the particle kinematic state to keep large-deformation MPM calculations well posed. These controls are not substitutes for mesh/time-step convergence studies or physically calibrated constitutive regularization, but they are useful guardrails for impact, fragmentation, contact, pore collapse, and highly damaged calculations. The controls described here act after the ordinary explicit transfer/update sequence described in Section 2.5 and are related to, but distinct from, the contact and cohesive-zone algorithms described in Sections 2.11 and 2.10. User-facing parameters are listed in the generated solver attribute appendix and, when they are written by ParticleFileWriter, in the PFW controls of Chapter 3.

2.12.1 Particle deletion and active-particle compaction

GEOS-MPM uses a deferred deletion model. During the step, several kernels or events set a per-particle integer flag,

$$\texttt{particleDeleteFlag}_p = 1, \tag{2.303}$$

when particle p should no longer participate in subsequent active-particle loops. The physical erase is performed later by `deleteBadParticles`, which moves particle-field wrappers to host memory, builds a set of indices with the delete flag set, erases those entries from the particle subregion, and reconstructs the active-particle index set. Deleting is therefore a topology-changing operation; it removes the particle mass, history variables, constitutive state, and any diagnostic fields attached to the particle.

The principal automatic deletion criteria are:

1. **Constitutive failure flag.** If a material wrapper exposes `constitutiveUpdateFlag`, `updateSolverDependencies` copies it to the solver field `particleConstitutiveUpdateFlag`. A negative value at the first material point flags the particle for deletion. This path lets a constitutive model request deletion after an unrecoverable update, for example failed return mapping or a material-specific erosion condition.
2. **Deformation-gradient determinant limits.** During `particleKinematicUpdate`, the solver computes
$$J_p = \det \mathbf{F}_p. \tag{2.304}$$
The particle is flagged if
$$J_p \le J_{\min} \quad \text{or} \quad J_p \ge J_{\max}, \tag{2.305}$$
where $J_{\min} = \texttt{minParticleJacobian}$ and $J_{\max} = \texttt{maxParticleJacobian}$. In the reviewed source snapshot the constructor defaults are $J_{\min} = 0.1$ and $J_{\max} = 10$. This criterion removes particles that have inverted, collapsed, or expanded beyond the range for which the mapped particle volume and constitutive response are trustworthy.
3. **Velocity overflow and maximum velocity.** The kinematic update also checks whether any component is so large that squaring it would overflow a floating-point value. Such particles are flagged before kinetic-energy or speed calculations can fail. The registered user parameter `maxParticleVelocity` sets the intended speed limit through `maxParticleVelocitySquared`. In the reviewed code, the overflow guard is active; the speed-squared accumulation for the ordinary maximum-speed comparison appears inside the overflow branch after a `break`, so that ordinary `maxParticleVelocity` deletion should be verified before relying on it as an active erosion rule.
4. **Out-of-domain particles.** At the end-of-step cleanup stage, `flagOutOfRangeParticles` removes particles that cannot be mapped safely on the next step. For `SinglePoint` particles the center must remain inside the global domain plus one buffer-cell layer, excluding periodic directions. For `SinglePointBSpline` particles the center must remain inside the physical global domain because the cubic B-spline basis already needs the available buffer support. For `CPDI` particles, all eight current-domain corners generated from the center and three $\mathbf{r}$ vectors must remain in range. Periodic directions are skipped because particle centers are wrapped during repartitioning.
5. **Event-driven machining.** The `MachineSample` event sets delete flags for particles outside an idealized dogbone, cylinder, or Brazilian-disk sample geometry. This is a constructive-geometry deletion mechanism rather than a numerical-failure guard.

A compact view of the deletion path is:

Listing 2.70: Deferred particle deletion in GEOS-MPM.

```
for particle p:
    if constitutiveUpdateFlag[p,0] < 0:
        particleDeleteFlag[p] = 1
    J = det(F[p])
```

```
    if J <= minParticleJacobian or J >= maxParticleJacobian:
        particleDeleteFlag[p] = 1
    if any velocity component would overflow when squared:
        particleDeleteFlag[p] = 1

# Later, after grid resize/domain cleanup:
for particle p:
    if particle center or CPDI corner cannot map to available nodes:
        particleDeleteFlag[p] = 1

erase all particles with particleDeleteFlag == 1
rebuild activeParticleIndices
```

Because particle deletion is non-conservative unless the removed material has negligible mass or represents intentional erosion, production studies should report the deletion thresholds, the number or mass of deleted particles, and whether deletion occurs at outflow boundaries, from constitutive failure, or from numerical pathologies.

2.12.2 Deformation-gradient reset

The deformation-gradient reset is a kinematic regularization for selected particles. It preserves the particle volume ratio and approximate rotation while removing deviatoric stretch from the stored deformation gradient. It is applied in the grid-cleanup stage after particle deletion and repartitioning and before optional plotting of unscaled CPDI domains.

For a selected particle, the solver computes the polar rotation $\mathbf{R}_p$ of the current deformation gradient $\mathbf{F}_p$ and its determinant $J_p = \det \mathbf{F}_p$. It then forms an isotropic stretch with the same determinant,

$$\mathbf{U}_p^{\mathrm{iso}} = \begin{cases} \mathrm{diag}(J_p^{1/2}, J_p^{1/2}, 1), & \texttt{planeStrain} = 1, \\ J_p^{1/3}\mathbf{I}, & \texttt{planeStrain} = 0, \end{cases} \tag{2.306}$$

then replaces

$$\mathbf{F}_p \leftarrow \mathbf{R}_p \mathbf{U}_p^{\mathrm{iso}}. \tag{2.307}$$

This operation leaves J_p unchanged but removes the shape distortion stored in $\mathbf{F}_p$. The source comments note that this is appropriate mainly for cases whose deviatoric response is hypoelastic or otherwise not meant to store permanent finite stretch in `particleDeformationGradient`. The gas material path also uses this reset.

The reviewed implementation has three activation routes:

Gas materials. If the subregion constitutive model catalog name is `Gas`, eligible active particles are reset every cleanup pass.

Fully damaged scaled particles. If `resetDefGradForFullyDamagedParticles=1`, CPDI-domain-scaled particles with `particleDamage > 0.9999999` and `particleDomainScaledFlag == 1` are reset. This keeps fully failed, scaled particle domains from retaining excessive deviatoric distortion.

Scaled surface particles and events. If `resetDefGradForScaledSurfaceParticles=1`, CPDI-domain-scaled particles with surface flags 3 or 4 are reset. The `ResetDeformationGradient` MPM event sets this flag for one subsequent cleanup pass and then the solver resets the flag to zero.

For particles with surface geometry, the reset also updates reference surface data so that the current physical surface normal and position are preserved under the new $\mathbf{F}_p$. For surface particles, the solver computes

$$\mathbf{n}_p^{s,\mathrm{ref}} \leftarrow \frac{\mathrm{cof}(\mathbf{F}_p)^{-1}\mathbf{n}_p^s}{\left\|\mathrm{cof}(\mathbf{F}_p)^{-1}\mathbf{n}_p^s\right\|}, \qquad \mathbf{x}_p^{s,\mathrm{ref}} \leftarrow \mathbf{F}_p^{-1}\mathbf{x}_p^s. \tag{2.308}$$

This detail is important for contact and cohesive-zone calculations: the reset regularizes the CPDI domain but does not intentionally move an already stored surface in the current configuration.

Listing 2.71: Deformation-gradient reset.

```
for active particle p:
    if Gas material or selected damaged/scaled-surface CPDI particle:
        R = polar_rotation(F[p])
        J = det(F[p])
        if planeStrain:
            Uiso = diag(sqrt(J), sqrt(J), 1)
        else:
            Uiso = J**(1/3) * I
        F[p] = R @ Uiso
        if particle has surface geometry:
            reference normal and surface position are recomputed
resetDefGradForScaledSurfaceParticles = 0
```

2.12.3 Particle splitting

Particle splitting is controlled by `subdivideParticles`. It is called during the topology-preparation stage, after events and before particle maps, ghosts, and active-particle indices are refreshed. The intent is to prevent finite-size CPDI-style particles from spanning too much of the background grid after large deformation. In the reviewed source, the code comment notes that splitting should only be an option for finite-domain particle types such as CPDI, CPTI, and CPDI2; the actual implementation uses the particle $\mathbf{r}$ vectors and is therefore meaningful for particles with finite domain vectors.

The splitting test uses a critical half-length based on the smallest grid spacing,

$$\ell_{\text{crit}} = \begin{cases} 0.49999 \min(h_x, h_y), & \texttt{planeStrain} = 1, \\ 0.49999 \min(h_x, h_y, h_z), & \texttt{planeStrain} = 0. \end{cases} \tag{2.309}$$

For each active particle, the solver tests each active domain vector. A split direction d is selected when

$$\|\mathbf{r}_{p,d}\|^2 > \ell_{\text{crit}}^2. \tag{2.310}$$

If k directions are selected, the parent particle is replaced by 2^k children: one child reuses the original particle slot and $2^k - 1$ new particles are appended to the subregion. Mass, current volume, and reference volume are divided by 2^k. The selected current and reference $\mathbf{r}$ vectors are halved. Current and reference centers are shifted by all combinations of $\pm$ the halved vectors in the split directions. All other particle fields and constitutive-model fields are copied from the parent using `particleCopyFlag`, except for fields explicitly recomputed during the split such as mass, volume, center, reference center, and $\mathbf{r}$ vectors.

$$m_{p_i} = \frac{m_p}{2^k}, \qquad V_{p_i} = \frac{V_p}{2^k}, \qquad V_{p_i}^0 = \frac{V_p^0}{2^k}. \tag{2.311}$$

For a split direction d, the child domain vector is

$$\mathbf{r}_{p_i,d} = \frac{1}{2}\mathbf{r}_{p,d}, \tag{2.312}$$

while unsplit directions retain the original vector. The child center is

$$\mathbf{x}_{p_i} = \mathbf{x}_p + \sum_{d \in \mathcal{S}_p} s_{i,d}\mathbf{r}_{p_i,d}, \qquad s_{i,d} \in \{-1, +1\}, \tag{2.313}$$

with the analogous operation applied to the reference center using reference $\mathbf{r}$ vectors.

Listing 2.72: Particle splitting.

```
lcrit = 0.49999 * min(active grid spacings)
for particle p:
    split_dirs = [d for d in active_dims if norm(r[p,d]) > lcrit]
    if split_dirs is empty:
        continue
    k = len(split_dirs)
    factor = 2**k
    create factor-1 appended children
    for each child, including modified parent:
        mass, volume, referenceVolume /= factor
        halve r-vectors and reference r-vectors in split_dirs
        shift current and reference centers by +/- halved r-vectors
        copy all remaining particle and constitutive fields from parent
rebuild activeParticleIndices
```

Splitting improves mapping quality when CPDI domains stretch across grid cells, but it also changes the particle discretization and can increase particle count rapidly in problems with persistent extension. It should therefore be reported with the split threshold implied by the grid spacing and monitored through the emitted log message, which reports the number of particles generated by subdivision.

Chapter 3

ParticleFileWriter

3.1 Purpose and generated products

The material point method solver tracks material state on Lagrangian particles and solves the equations of motion on an Eulerian background grid. In GEOS-MPM, the background grid is constructed by the GEOS internal mesh generator, but the particles are created externally through a set of Python tools collectively referred to as the ParticleFileWriter, or *PFW*. As its name implies, the ParticleFileWriter creates the particle file: a plain-text description of the position, velocity, and geometry of each material point "particle," along with auxiliary variables that define material type, contact group, surface state, particle type, and other optional properties. This pre-processor exists as a separate Python toolset so that users can create custom configurations, geometries, and loading paths without editing or rebuilding the GEOS C++ source code.

PFW is therefore both a particle generator and a GEOS input generator. Starting from a Python case definition, it writes

- the particle file read by the `ParticleMeshGenerator`;
- the GEOS XML input containing the `InternalMesh`, `ParticleRegion`, constitutive blocks, solver block, finite-element space, events, diagnostics, and output blocks;
- optional run scripts and scheduler settings; and
- optional data or Python dependencies staged into the run directory.

The division of labor is intentional: GEOS owns the background grid and time integration, while PFW owns the initial particle state and the user-facing construction logic for nontrivial geometries.

3.2 The `pfw_input` file

A PFW case file is a Python file that is imported during case creation. It can define any local Python variables, helper functions, classes, or objects needed to build the case. Only data that must be passed to the PFW pre-processor, however, should be placed in the dictionary named `pfw`. The dictionary is the contract between the case script and `particleFileWriter.py`. Values outside the dictionary remain ordinary Python workspace variables used by the input script; values inside `pfw` are interpreted as PFW controls, particle-generation controls, or solver XML attributes.

The import-based design makes a `pfw_input` file more flexible than a static keyword file. Users commonly compute dimensions from parameters, loop over material definitions, read tabulated data, construct lists of geometry objects, or select between load cases before filling the final `pfw` dictionary. PFW recognizes a set of predefined keys and also passes many unrecognized keys through as attributes on `SolidMechanics_MPM`, after filtering known PFW-only metadata. This pass-through behavior lets new solver controls be exercised before a dedicated PFW wrapper is added.

Listing 3.1: Minimal shape of a PFW input file.

```
import numpy as np
import pfw_geometryObjects as geom
import pfw_materials as mat

# Local variables are allowed. They are not PFW inputs unless assigned to pfw.
refine = 2
cpp = 12
length = 1.0

pfw = {}
pfw["xmin"], pfw["xmax"] = -0.5*length, 0.5*length
pfw["ymin"], pfw["ymax"] = -0.5*length, 0.5*length
pfw["zmin"], pfw["zmax"] = -0.5*length, 0.5*length
pfw["xpar"], pfw["ypar"], pfw["zpar"] = refine, refine, refine
pfw["nI"], pfw["nJ"], pfw["nK"] = pfw["xpar"]*cpp, pfw["ypar"]*cpp, pfw["zpar"]*cpp
pfw["ppc"] = 2
pfw["endTime"] = 1.0e-5
pfw["outputType"] = "silo"       # or "vtk"
pfw["materials"] = [mat.elasticIsotropic("solid", rho=1000.0, E=1.0e6, nu=0.25)]
pfw["objects"] = [geom.box("block", mat=0, x0=[-0.4,-0.4,-0.4], x1=[0.4,0.4,0.4])]
```

The input file is imported as a module, so normal Python caveats apply. Expensive work at module import time will be repeated whenever PFW imports the file. For large geometry construction, users can put object generation in helper functions or use the `make_objects` pattern supported by the current PFW script so that geometry construction can be restricted to the rank slice being generated.

3.3 Input dependencies

PFW supports two related dependency mechanisms: ordinary Python imports and explicit run-directory staging. Ordinary imports are used for code that must be available while the `pfw_input` file is executed; explicit staging is used for files that must be copied beside the generated case or made importable in the run directory.

Python and PFW helper imports. Most inputs import standard libraries, numerical packages, and PFW helper modules directly:

Listing 3.2: Typical Python imports in a PFW input file.

```
import math
import numpy as np
import pfw_geometryObjects as geom
import pfw_materials as mat
from pfw_tracerPoints import disk, plane_grid, set_tracers
```

These imports are evaluated when `particleFileWriter.py` imports the input module. The PFW run directory and the original input-file directory are inserted into `sys.path`, so local helper modules can also be imported if they are present or staged.

PFW dependency comments. Input files can request explicit file staging with comments of the form `#[pfw_dependency]`. The current parser accepts paths relative to the original input directory, paths relative to the PFW directory, absolute paths, and optional destination paths inside the run directory:

Listing 3.3: Examples of PFW dependency staging comments.

```
#[pfw_dependency] pfw_materials.py
#[pfw_dependency] input:tables/strength_table.csv
#[pfw_dependency] pfw:pfw_geometryObjects.py
#[pfw_dependency] /absolute/path/to/local_geometry.stl
#[pfw_dependency] input:tables/load_history.csv => tables/load_history.csv
```

A relative dependency first resolves relative to the original input-file directory and then falls back to the configured PFW directory for compatibility with older examples. The `input:` prefix forces input-directory lookup, and the `pfw:` prefix forces lookup relative to the PFW installation directory. A destination following `=>` is always relative to the run directory and cannot be absolute or contain `..`. This is useful for material tables, STL files, tabulated load histories, Python helper modules, bitmap/voxel data, and post-processing scripts that should travel with a generated case.

Generated dependency list. After staging, PFW appends the staged paths to `pfw["dependencies"]`. That list is workflow metadata and is filtered so that it does not become a solver XML attribute. Rank zero stages files before the input module is imported on the other ranks, avoiding races on shared file systems.

3.4 Background grid

PFW writes a GEOS `InternalMesh` for the background grid. The main controls are the physical domain bounds `xmin/xmax`, `ymin/ymax`, `zmin/zmax`; the total cell counts `nI/nJ/nK`; periodic flags `periodic`; partition counts `xpar/ypar/zpar`; and particle-density controls `ppc`, `ppcx`, `ppcy`, and `ppcz`. The generated mesh uses uniform Cartesian cells and the `C3D8` element type, even for plane-strain cases, because the MPM solver and particle mesh operate on the GEOS three-dimensional mesh data structures.

3.4.1 Parallel grid-generation constraints

The practical challenge in parallel PFW generation is to keep each MPI partition balanced while preserving a uniform grid and a simple particle-generation loop. In typical production inputs, each partition is assigned the same number of cells in each active direction. This means `nI`, `nJ`, and `nK` should be chosen consistently with `xpar`, `ypar`, and `zpar`. Periodic and non-periodic directions differ slightly: non-periodic directions include one ghost cell on each side in the total `InternalMesh` count, whereas periodic directions do not add those two extra cells. Consequently, PFW computes interior cell counts as

$$n_x^{\mathrm{int}} = \begin{cases} N_x, & \text{periodic in } x, \\ N_x - 2, & \text{non-periodic in } x, \end{cases} \tag{3.1}$$

and analogously for y and z, except that plane strain always uses the reduced through-thickness treatment described below. The interior spacings are then

$$\Delta x = \frac{x_{\max} - x_{\min}}{n_x^{\mathrm{int}}}, \qquad \Delta y = \frac{y_{\max} - y_{\min}}{n_y^{\mathrm{int}}}, \qquad \Delta z = \frac{z_{\max} - z_{\min}}{n_z^{\mathrm{int}}}. \tag{3.2}$$

The appropriate number of particles per partition is hardware and model dependent. A common CPU target is below roughly 4×10^4 particles per MPI partition for interactive development and moderate material complexity. On GPU-enabled workflows, substantially larger partitions can be practical; values approaching 10^6 particles per partition may be usable for simple kernels and sufficiently large devices. These are operational guidelines rather than solver limits: constitutive cost, contact fields, neighbor lists, output variables, and available memory can change the useful range by orders of magnitude.

3.4.2 Common `cpp`–`refine`–`ppc` pattern

Most suite and example inputs use a small set of derived variables rather than entering every mesh count directly. The pattern is

$$\texttt{cpp} = \text{cells per partition in a coordinate direction}, \tag{3.3}$$

$$\texttt{refine} = \text{integer or real resolution multiplier}, \tag{3.4}$$

$$\texttt{xpar} = \max(N_x^{\text{pbc}}, \text{round}(\texttt{refine}\,\texttt{xpar0})), \tag{3.5}$$

$$\texttt{nI} = \texttt{xpar}\,\texttt{cpp}, \tag{3.6}$$

with analogous definitions in y and z. Here N_x^{pbc} is usually 3 in a periodic direction and 1 otherwise, so that periodic directions have enough partitions for periodic neighbor communication and representative partitioning. The particle density is then set with `ppc` or direction-specific values. With `ppc=2`, a full three-dimensional cell initially contains $2^3 = 8$ candidate particle points before geometry filtering, while a plane-strain cell contains one through-thickness layer and $2^2 = 4$ in-plane candidate points.

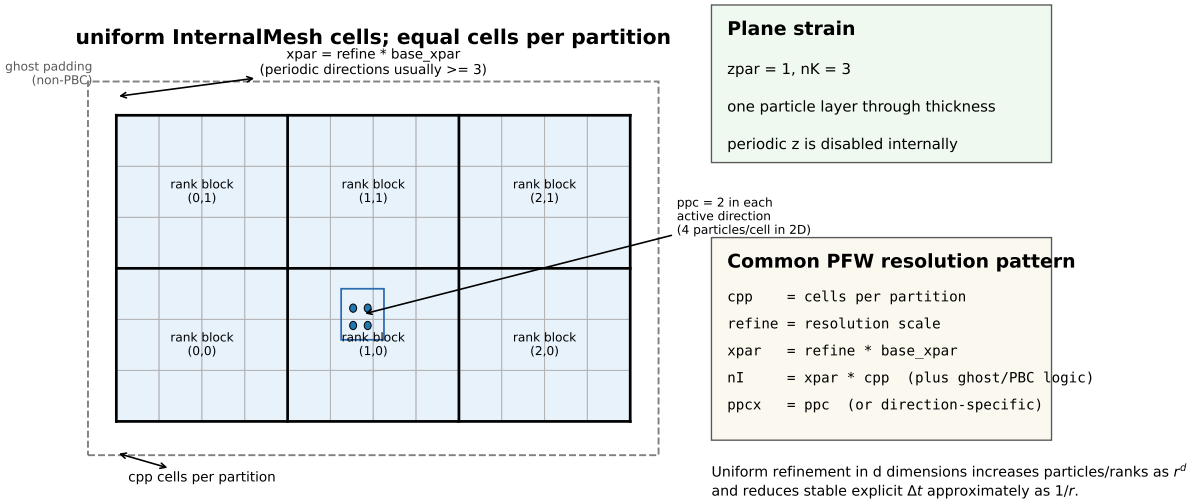


Figure 3.1: Typical PFW resolution pattern. Users select a cells-per-partition count `cpp`, scale the number of partitions with `refine`, and choose particles per cell with `ppc`. The resulting `InternalMesh` cells are uniform; non-periodic directions include ghost-cell corrections, while periodic directions generally require at least three partitions in production examples.

A representative input fragment is

Listing 3.4: Resolution variables used to populate PFW mesh controls.

```
refine = 3
cpp = 12
base = [1, 1, 1]
pbc_min = [3 if p else 1 for p in pfw["periodic"]]

pfw["xpar"] = max(pbc_min[0], int(round(refine*base[0])))
pfw["ypar"] = max(pbc_min[1], int(round(refine*base[1])))
```

```
pfw["zpar"] = max(pbc_min[2], int(round(refine*base[2])))
pfw["nI"] = pfw["xpar"] * cpp
pfw["nJ"] = pfw["ypar"] * cpp
pfw["nK"] = pfw["zpar"] * cpp
pfw["ppc"] = 2
pfw["mCores"] = pfw["xpar"] * pfw["ypar"] * pfw["zpar"]
```

3.4.3 Plane-strain meshes

Plane-strain cases are represented with a three-dimensional GEOS mesh but a reduced particle layer. The current PFW checks

$$\texttt{planeStrain} = 1 \quad \Rightarrow \quad \texttt{zpar} = 1, \qquad \texttt{nK} = 3. \tag{3.7}$$

PFW then treats the through-thickness direction as one active interior cell plus ghost padding, creates a single layer of particles through thickness, sets through-thickness particle velocities to zero, and passes `planeStrain` to the solver. In-plane controls `ppcx` and `ppcy` still set the in-plane particle density; `ppcz` does not create multiple through-thickness layers in a plane-strain run. The generated GEOS XML also disables periodicity in the suppressed z direction when `planeStrain`=1.

This representation keeps the same particle-file and mesh infrastructure as three-dimensional calculations while reducing the particle count and enforcing the intended kinematics. Users should still choose a physical z-extent consistent with the constitutive model and any diagnostic interpretation of force, stress, or reaction quantities.

3.4.4 Refinement and cost scaling

For explicit MPM, the stable time step scales with the cell size through the CFL condition. If a fixed physical domain is refined by a factor r in each active direction, then $\Delta x \sim r^{-1}$ and the number of steps required to reach a fixed physical time grows approximately as r. With fixed `ppc`, the particle count grows as r^d, where $d = 2$ for plane-strain or other effectively two-dimensional calculations and $d = 3$ for full three-dimensional calculations. The total explicit work to reach a fixed time therefore scales roughly as

$$W(r) \propto r^{d+1}, \tag{3.8}$$

ignoring changes in contact, neighbor-list, constitutive, and communication overheads. Thus a two-dimensional refinement sequence typically grows like r^3 in total core work, and a three-dimensional sequence like r^4. If `cpp` is held fixed while `refine` increases the number of partitions, the per-rank particle count can remain approximately constant, but the number of time steps still grows as r and the total number of ranks grows as r^d. This is why PFW examples tend to expose `refine`, `cpp`, and `ppc` explicitly: the variables make accuracy, memory, and run-time tradeoffs visible in the input file.

3.5 Geometry objects

PFW geometry objects are the particle-creation layer between a user-defined model and the plain-text particle file. During particle generation, PFW loops over the candidate particle centers implied by the background grid and particle-density controls. Each candidate point is tested against an ordered list of geometry objects. The first object that accepts the point supplies the particle type, material index, contact group, initial velocity, and any optional fields requested in `particleFileFields`. Objects can therefore create solid regions, leave voids, tag surface particles, assign different materials or contact groups, paint damage or porosity, prescribe material directions, and provide exact surface normals or surface positions for contact and cohesive-zone calculations.

3.5.1 Creating objects and assembling the object list

The most direct workflow is to construct objects in the input file and assign the ordered list to `pfw["objects"]`. The object order matters. PFW tests objects sequentially and stops at the first match, so later objects do not overwrite particles created by earlier objects unless the earlier object rejects the particle point. This priority rule is useful for inserting inclusions, defects, voids, or wrappers with controlled precedence.

Listing 3.5: Assembling an explicit PFW object list.

```
import pfw_geometryObjects as geom

matrix = geom.box("matrix", x0=[-1.0,-1.0,-1.0], x1=[1.0,1.0,1.0], mat=0, group=0)
inclusion = geom.sphere("inclusion", x0=[0.0,0.0,0.0], r=0.25, mat=1, group=1)
void = geom.sphere("void", x0=[0.4,0.0,0.0], r=0.10, mat=-1, group=0)

# First accepted object wins.  Put small, high-priority features first.
pfw["objects"] = [void, inclusion, matrix]
```

When geometry construction is expensive or rank-dependent, the input module can instead define a function named `make_objects`. The current PFW script uses this function only when `pfw["objects"]` is absent. A zero-argument `make_objects()` function is the legacy form and constructs the full list on every PFW rank. A two-argument form, `make_objects(rankxmin, rankxmax)`, receives the approximate x-extent assigned to the current particle-generation rank and can construct only objects whose bounding boxes overlap that slice. Some older notes and user inputs may refer to this pattern as a `makeObjects` option; the active Python hook in the reviewed code is `make_objects`.

Listing 3.6: Rank-sliced object construction with `make_objects`.

```
def make_objects(rankxmin, rankxmax):
    objs = []
    for center, radius, mat, group in particle_database:
        if center[0] + radius < rankxmin:
            continue
        if center[0] - radius > rankxmax:
            continue
        objs.append(geom.sphere("grain", x0=center, r=radius, mat=mat, group=group))
    return objs
```

The rank-sliced form is particularly useful for granular beds, voxel-derived objects, Voronoi constructions, and other cases in which constructing all objects on every rank would dominate preprocessing time or memory. It does not change the meaning of the final particle file; it only reduces the number of candidate objects that a rank constructs and tests.

3.5.2 Object sorting in parallel PFW runs

The `sortObjects` control is a second acceleration mechanism. If `sortObjects=True`, PFW filters the object list at each candidate x-slice by evaluating

$$x_{\min}^{(a)} \le x \le x_{\max}^{(a)}, \tag{3.9}$$

where x is the candidate particle center and $x_{\min}^{(a)}$, $x_{\max}^{(a)}$ are returned by object a's `xMin()` and `xMax()` methods. Only the filtered slice list is then searched for that x coordinate. The original order of the retained objects is preserved, so first-match priority is unchanged.

This option is intended for many-object geometries with compact bounding boxes, such as packed granular beds or collections of inclusions. It is not enabled by default because some objects are unbounded,

use infinite default bounds, or have only approximate `xMin`/`xMax` implementations. Such objects remain correct when `sortObjects=False`; they simply do not benefit from x-slice filtering. The two acceleration mechanisms are complementary: `make_objects(rankxmin, rankxmax)` reduces object construction, while `sortObjects` reduces per-point object testing after the list has been constructed.

3.5.3 Geometry-object interface and particle fields

All PFW geometry objects are expected to provide at least an inside/outside query and enough metadata to assign particle fields. The base `Geometry` constructor stores common attributes including `name`, `vel`, `mat`, `group`, `particleType`, `damage`, `porosity`, `temperature`, and `surfaceTraction`. Objects can supply these as constants, callables, or explicit getter methods.

The central query is

$$f_a(\mathbf{x}_p, h_s) = \texttt{object.isInterior(pt, skinDepth)}, \tag{3.10}$$

where $\mathbf{x}_p$ is the candidate particle center and h_s is the surface-search depth computed from the cell and particle spacing. A negative return value rejects the particle point. A nonnegative return value accepts it and is interpreted as the particle's surface flag when the `SurfaceFlag` field is enabled. The most common values are `0` for an interior particle, `1` for a fully damaged particle, `2` for a ordinary surface particle, `3` for a cohesive-zone surface particle, and `4` for a damaged cohesive particle. Older comments and user discussions sometimes refer to an `isSurface` method; in the current particle-generation path the surface/interior decision is carried by the integer returned from `isInterior`.

After a candidate point is accepted, PFW queries optional object methods in the order required by the particle file. Important methods and fallback attributes are summarized in Table 3.1. If a method is absent, PFW falls back to an attribute of the same concept, and if that attribute is callable it is evaluated at the particle point. This pattern allows simple constant-valued objects and spatially varying user-defined objects to share the same interface.

The distinction between object geometry and object metadata is important. The inside/outside test decides whether a particle exists; the getter methods decide how the accepted particle is interpreted by GEOS-MPM. A geometry object can therefore be used as a solid inclusion, a void, a contact body, a material-orientation map, a cohesive surface generator, or a local state-field painter depending on which getters or wrappers are attached.

3.5.4 Primitive analytic objects

`box`. Creates an axis-aligned rectangular solid from two opposite corners. It is the most common object for blocks, platens, test coupons, and matrix domains. Optional surface flags can enable or suppress selected faces for contact/cohesive workflows.

`box2`. A box variant used primarily for contact testing. It follows the same rectangular membership test as `box`, but uses mixed corner-normal behavior so that contact-normal handling can be exercised at edges and corners.

`notchedBar`. Creates a rectangular bar with an edge notch cut into the positive-y face. It is useful for fracture and localization examples that need a prescribed geometric flaw. The source notes that notch surface normals are not fully defined, so exact-surface contact or cohesive workflows should verify this object before relying on those normals.

`sphere`. Creates a sphere from center and radius, returning surface flags and surface-normal/position data near the spherical boundary. In two-dimensional or plane-strain examples it is commonly used as a disk-like cross-section when the through-thickness grid has one active layer.

Table 3.1: Common PFW geometry-object attributes and methods.

Attribute or method	**Role in particle generation**
`isInterior(pt, skinDepth)`	Accepts or rejects a candidate particle center. Nonnegative return values become surface flags when the surface-flag particle field is written.
`xMin()`, `xMax()`	Bounding interval used by `sortObjects`; finite bounds improve sorting efficiency.
`getParticleType(pt)` or `particleType`	Selects single-point, B-spline, CPDI, CPTI, or CPDI2 particle type.
`getMat(pt)` or `mat`	Selects the material index written to the particle file. A negative material index is used as a void/deletion convention and suppresses particle creation.
`getGroup(pt)` or `group`	Selects the contact group used by multi-field contact and related grouping options.
`getVelocity(pt)` or `vel`	Provides the initial particle velocity. In plane strain, PFW forces the through-thickness component to zero.
`getDamage`, `getPorosity`, `getTemperature`	Assign optional initial state fields used by damage, porosity, and thermo-mechanical workflows.
`getSurfaceNormal(pt)`	Provides an explicit outward surface normal for contact, DFG, or cohesive-zone workflows when the particle is surface flagged.
`getSurfacePosition(pt)`	Provides the vector from the particle center to the exact or reconstructed surface point.
`getSurfaceTraction(pt)`	Provides an optional initial/external surface-traction vector for surface particles.
`getMatDir(pt)`	Provides a local material-direction triad for anisotropic materials, crystals, fibers, or layered strength fields.
`getStrengthScale(pt)`	Provides spatial strength scaling, often for Weibull or layered heterogeneity.
`getCZTag(pt)`	Provides cohesive-zone tags for surfaces that use cohesive constitutive models.
`getSubregions()`	Reports material/particle-type pairs so PFW can construct compatible `ParticleRegion` subregions.

sphericalInclusion. Creates a spherical inclusion-like region but suppresses ordinary surface flagging. This is useful when the inclusion should assign material or group data without creating a separate contact surface at the inclusion boundary.

shell. Creates a spherical shell between inner and outer radii. It marks particles lying between the two spherical surfaces and provides inward/outward surface data near the shell boundaries.

ellipsoid. Creates a grid-aligned ellipsoid from a center and three semi-axis lengths. It is useful for inclusions, pores, or idealized grains with anisotropic aspect ratios.

cone. Creates a cone from two axis points and a base radius. Surface normals distinguish the side wall and base, making the object useful for idealized tooling or conical inclusions.

cylinder. Creates a solid or hollow cylinder from two axis points, an outer radius, and optionally an inner radius. The object supports face/wall surface-flag controls, a slanted top plane through `n2`, and an axis-based material-direction triad. Cylinders are used for disks, rods, pores, tools, and plane-strain circular samples.

toroid. Creates a toroidal annulus from a center, major radius, and minor radius. It is useful for ring-like structures or curved voids.

polygon. Creates a two-dimensional polygon from ordered vertices. PFW uses point-in-polygon and edge-distance operations to determine interior particles and surface flags. The polygon is generally used for plane-strain domains or extruded cross-sections.

porousPolygon. Extends `polygon` by inserting two-dimensional pores according to a porosity and pore-size specification. It is used for plane-strain porous mesostructures where the outer boundary is polygonal.

fill. Accepts all candidate particle points. This is a convenience object for filling the full background domain or for constructing a lowest-priority background material beneath higher-priority objects.

3.5.5 Tooling, sample, and test-fixture objects

indentor. Creates a faceted, grid-aligned indentation tool from an apex/location, number of facets, and included angle. Three facets approximate a Berkovich-style geometry and four facets approximate a Vickers-style geometry.

spherical_indenter. Creates an indenter with a spherical tip. It is intended for indentation examples in which a curved contact surface is more appropriate than a faceted tip.

flatpunch_indenter. Creates a flat-punch indenter with a specified radius and angle. It is useful for verification of moving tools, boundary contact, and explicit contact surfaces.

simple_tensile_gripper. Creates one side of a simple gripper claw for micromechanical tensile tests. Dimensions define the inside width, grip height, gap, outer width, and thickness.

tensile_gripper. Creates a more detailed tensile gripper based on micromechanical test hardware dimensions. It is used when the gripper itself is represented by particles rather than only by boundary conditions.

pillar. Creates a micropillar-style compression specimen with a cylindrical gauge length and a wider base connected by a smooth radial profile. The object carries material-direction information and is intended for microcompression or strength-size-effect studies. The source notes that its curved-surface normal approximation should be checked for exact-surface contact use.

pillarBase. Creates a supporting base geometry for pillar simulations. It is typically paired with `pillar` to represent a substrate or pedestal beneath the gauge section.

whiskers. Creates whisker- or fiber-like inclusions. It is used for reinforced microstructure examples where slender embedded features need separate material directions or contact groups.

3.5.6 Mesostructure and inclusion objects

crystal. Generates a faceted crystal analogue from a center, axis, height, and minimum/maximum face offsets. It is useful for synthetic crystal-like inclusions or grains with flat facets.

foam. Creates a box with user-specified spherical pores. The object removes particles inside the pore list and returns surface flags near the pore and outer-box boundaries.

poissonDiskFoam. Creates a grid-aligned foam block with pores sampled by a Poisson-disk-style algorithm. It supports two-dimensional and three-dimensional modes, periodic sampling options, minimum-ligament constraints, and explicit surface-normal/position information.

packedSphericalBed. Represents a packed bed of spherical grains inside a box. The object stores grain centers, radii, material indices, contact groups, and particle types, and uses a cell-list search to answer point queries efficiently. It can generate random sequential-addition-style packings or consume precomputed grain data provided through the input script.

twoMaterialBox. Creates a box in which accepted particles randomly select between two material indices. This is a simple stochastic mixture helper rather than a resolved microstructure generator.

twoFieldBox. Creates a box intended for two-contact-field or two-group testing. It is a legacy/test helper for random or mixed contact group assignments; users should verify the current constructor before relying on it in production inputs.

polygonInclusions. Assigns particles to polygonal inclusions embedded in a surrounding fill group. Polygons should be provided with counter-clockwise point ordering. The object is useful for two-dimensional composite or mesoscale inclusion studies.

3.5.7 Cohesive-zone and prill objects

czSphericalPrill. Creates a spherical prill containing Voronoi-like crystals separated by cohesive-zone surfaces. It can assign contact groups, crystal material directions, surface positions, and cohesive tags needed by the cohesive-zone workflow.

explicitBinderSphericalPrill. Creates a two-dimensional spherical prill with explicit grain and binder regions. It assigns grain and binder material indices separately and is used when binder ligaments should be represented by particles rather than only by cohesive interfaces.

czCylindricalPrill. Creates a cylindrical prill with Voronoi-like internal crystals and cohesive-zone surfaces. It is appropriate for cylindrical or disk-like prill geometries where cohesive interfaces are inserted between neighboring grains.

czPrill. Creates a box-shaped prill with internal Voronoi crystals, cohesive-zone tags, and configurable bonded/neat surface fractions. It is a general prill mesostructure object for cohesive fracture and contact studies.

3.5.8 Voxel, image, and surface-import objects

stl. Imports a closed triangulated STL surface. The object supports ASCII and binary STL files, translation, scaling, optional centering, normal flipping, ray-casting inside/outside tests, and nearest-triangle surface projection. It is useful for CAD-derived or segmented geometries that are not easily represented by analytic primitives.

VCCTL. Creates particles from a VCCTL-style voxelized dataset. Integer voxel phase values are mapped to material indices through a user-supplied map, allowing cementitious or other voxel-resolved microstructures to be converted into particles.

CT. Creates particles from a three-dimensional computed-tomography voxel dataset. It is similar to `VCCTL` but kept separate for compatibility and specialization of CT-generated arrays and phase maps.

bitmap. Creates a two-dimensional particle geometry from an image or binary array. Pixel or label values map to material indices, making this object useful for plane-strain image-based mesostructures.

GrainStack. Creates particles from a stacked voxel or labeled-array representation of grains. It provides bounding-box and index-casting utilities for converting array coordinates into model-space particles.

3.5.9 Periodic and architected-structure objects

spinodal. Creates a spinodal-like porous or bicontinuous structure from a random field and target density controls. It returns surface data by projecting to the implicit surface and is useful for synthetic porous materials.

tpms. Creates triply periodic minimal surface or related periodic architectures using a selected implicit surface type, target density, cell size, and offset. It is used for gyroid-like, Schwarz-like, spinodal, or other periodic mesostructures.

3.5.10 Boolean set operations

The Boolean objects combine two existing geometry objects while preserving the PFW query interface. They are useful for creating holes, clipped inclusions, intersected tool shapes, and domains assembled from reusable primitives.

union. Accepts a particle point if either sub-object accepts it. Surface data are inherited from the sub-object that controls the accepted point according to the operation's priority rules.

`intersection`. Accepts a particle point only if both sub-objects accept it. This is useful for clipping one object by another, such as retaining only the portion of a cylinder inside a box.

`difference`. Accepts particles that are inside object `A` and outside object `B`. This is the standard tool for cutting voids, pores, holes, or notches from an otherwise solid object. The current implementation includes surface-normal and surface-position logic to decide whether the nearest retained boundary belongs to `A` or to the removed object `B`.

3.5.11 Geometry wrappers and field painters

Wrappers take a sub-object and override or augment one part of its response. They are the preferred way to add field data without reimplementing an inside/outside test. All wrappers forward unmodified queries to the wrapped object unless they specifically override that field.

Basic field wrappers. The wrappers `materialDirectionWrapper`, `surfaceFlagWrapper`, `czTagWrapper`, `surfaceNormalWrapper`, `surfacePositionWrapper`, `shrinkageFlagWrapper`, `strengthScaleWrapper`, `damageWrapper`, `porosityWrapper`, and `temperatureWrapper` assign a corresponding constant or callable value to particles accepted by the wrapped object.

Deletion and masking wrappers. `functionalDeletionWrapper` calls a user-supplied deletion function and rejects points that should be removed. `pointwisePorosityWrapper` randomly suppresses a fraction of points to represent unresolved porosity, but does not construct deterministic pore surfaces or surface flags.

Voronoi, Weibull, and material-direction wrappers. `voronoiWrapperWithLayeredStrengthScale`, `layeredVoronoiWeibullWrapper`, `voronoiWeibullBoxWrapper`, `voronoiMatDirBoxWrapper`, `sizeEffectVoronoiWeibullBoxWrapper`, and `layeredStrengthScaleBoxWrapper` assign spatially varying material directions, strength scales, flaws, or layered Weibull fields. These wrappers are used for heterogeneous strength distributions, grain-level anisotropy, and size-effect studies.

Box-local and crack-like wrappers. `surfaceFlagBoxWrapper` and `damageBoxWrapper` override surface flag or damage only inside a specified box. `pennyShapedCrackWrapper` paints a penny-shaped damaged region defined by two points and a radius, making it useful for seeded-crack or pre-damage studies.

Coordinate-transform wrapper. `transform` maps query points through a user-supplied transform before delegating to the sub-object, and transforms returned normals or vectors as needed. It allows an analytic or imported object to be translated, rotated, or otherwise mapped without creating a new object class.

3.6 Materials, groups, and particle types

This section describes how PFW assigns the particle-level integers and optional orientation data that connect a generated particle file to the solver-side material, contact, and interpolation algorithms. Constitutive-model equations and integration details are described in Section 2.7; particle interpolation and update schemes are described in Section 2.8; and multi-field, DFG, and explicit-surface contact options are described in Section 2.11. The purpose of this PFW section is narrower: it documents how the input script and geometry objects choose the values that are written into the particle file.

PFW can generate particle files for several solver material-model families. The current source-derived catalogue includes the models listed in Table 3.2. Only the model names are listed here because the physics, parameters, and calibration guidance belong in the constitutive-model documentation and linked material-model reports.

Table 3.2: Solver material-model families visible to PFW in the reviewed source tree. See Section 2.7 and Appendix E for the generated catalogue.

Family	Model names
Elastic and anisotropic elastic	`ElasticIsotropic`, `ElasticTransverseIsotropic`, `ElasticTransverseIsotropicPressureDependent`.
Plasticity and hyperelasticity	`VonMisesJ`, `PerfectlyPlastic`, `Hyperelastic`, `HyperelasticMMS`.
Damage, polymers, graphite, and geomaterials	`CeramicDamage`, `Chiumenti`, `StrainHardeningPolymer`, `Graphite`, `Geomechanics`.
Cohesive-zone materials	`BicrystalCohesiveZone`, `CoupledCohesiveZone`, `PolymerCohesiveZone`; see Section 2.10.

3.6.1 Particle-level assignments made by geometry objects

Each accepted particle receives three core classification values:

$$m_p^{\mathrm{PFW}} = \texttt{MaterialType}, \tag{3.11}$$

$$g_p^{\mathrm{PFW}} = \texttt{ContactGroup}, \tag{3.12}$$

$$q_p^{\mathrm{PFW}} = \texttt{ParticleType}. \tag{3.13}$$

The first two are ordinary particle-file columns when `MaterialType` and `ContactGroup` are present in `particleFileFields`; the particle type is used by PFW while building particle blocks and by the generated `ParticleMesh` XML entry. In the reviewed PFW code, the standard particle-type integers are

Integer	Solver particle type
0	`SinglePoint`
1	`SinglePointBSpline`
2	`CPDI` (current PFW default)
3	`CPTI` placeholder
4	`CPDI2` placeholder

The `mat` value is a zero-based index into `pfw["materials"]`. If `pfw["materials"] = ["matrix", "inclusion"]`, then `mat=0` assigns the particle to the `matrix` material and `mat=1` assigns it to `inclusion`. PFW groups particles with the same material index into separate generated `ParticleRegion` entries whose `materialList` names come from `pfw["materials"]`. The corresponding material XML snippets must be present in `pfw["materialPropertyString"]`; Section 3.6.4 gives examples using `pfw_materials.py`.

The `group` value is the contact-group or material-field label used by the solver contact algorithms. For prescribed multi-field contact, particles in different groups can map to different nodal velocity fields and interact through the contact update described in Section 2.11. For DFG contact, the prescribed group is the base field label and the damage-field-gradient split creates additional side fields internally. The group label is therefore a kinematic/contact classification, not a constitutive material selection; it is common for two particles to share the same material index but have different contact groups, or to have different material indices but the same contact group when no material-field separation is intended.

The `particleType` value selects the mapping and domain representation described in Section 2.8. It can be used uniformly for an entire object, or varied by object to mix, for example, CPDI particles in deformable solids with single-point particles in simple fixtures. Mixing particle types should be deliberate because the particle type controls stencil size, surface/domain geometry, and allowable future options.

Listing 3.7: Constant material, group, and particle-type assignments on geometry objects.

```
import pfw_geometryObjects as geom

# Particle type integers: 0 SinglePoint, 1 SinglePointBSpline,
# 2 CPDI, 3 CPTI placeholder, 4 CPDI2 placeholder.
matrix = geom.box("matrix",
                  x0=[-1.0,-1.0,-1.0], x1=[1.0,1.0,1.0],
                  mat=0, group=0, particleType=2)

inclusion = geom.sphere("inclusion",
                        x0=[0.0,0.0,0.0], r=0.25,
                        mat=1, group=1, particleType=2)

pfw["objects"] = [inclusion, matrix]
pfw["materials"] = ["matrixElastic", "inclusionCeramic"]
```

PFW obtains these values by querying the accepted object. If an object provides `getMat(pt)`, `getGroup(pt)`, or `getParticleType(pt)`, those methods are used. Otherwise PFW falls back to the object attributes `mat`, `group`, and `particleType`; callables are evaluated at the particle point where supported by the current code path. Objects that can emit multiple material/type combinations should also implement `getSubregions()` so PFW can build the complete `ParticleMesh` block list before the particle file is written. The default implementation returns one pair, `[(mat, particleType)]`; multi-material objects such as packed beds, voxel imports, and stochastic mixtures override this to enumerate all possible (`mat`, `particleType`) pairs.

Listing 3.8: Spatially varying material and group assignment through object methods.

```
class LayeredBox(geom.box):
    def getMat(self, pt):
        return 0 if pt[1] < 0.0 else 1

    def getGroup(self, pt):
        return 10 if pt[0] < 0.0 else 11

    def getSubregions(self):
        # PFW needs all material/type pairs before particle generation.
        return [(0, self.particleType), (1, self.particleType)]

obj = LayeredBox("layered", x0=[-1,-1,-1], x1=[1,1,1], particleType=2)
pfw["objects"] = [obj]
pfw["materials"] = ["lowerLayer", "upperLayer"]
```

3.6.2 Wrappers and field painters used for assignment

Most simple material, group, and particle-type assignments are made directly in the geometry-object constructor. The wrapper system described in Section 3.5 is used when the same inside/outside geometry should be reused with modified particle fields. The current `BaseWrapper` forwards unmodified queries to its sub-object and also checks whether the wrapper itself defines `mat`, `group`, `particleType`, `matDir`,

`damage`, `porosity`, `temperature`, `surfaceNormal`, `surfacePosition`, `surfaceTraction`, or related fields. This means custom wrappers can be used as field painters without duplicating geometry tests.

The standard field-painting wrappers most relevant to material assignment are summarized in Table 3.3. There is no special-purpose `materialWrapper` in the reviewed source; for material or group changes the usual choices are to pass `mat`/`group`/`particleType` at object construction time, to implement `getMat`/`getGroup`/`getParticleType`, or to write a small `BaseWrapper`-derived class that sets those fields and forwards all other queries.

Table 3.3: PFW wrappers and field painters related to material, group, particle-type, and constitutive metadata assignment.

Wrapper or pattern	**Use**
`materialDirectionWrapper`	Assigns a constant vector or full orthonormal material-direction triad for anisotropic or direction-dependent material models.
`strengthScaleWrapper`, layered/Voronoi strength wrappers	Assign strength-scale factors for stochastic or spatially varying strength fields used by compatible material models.
`damageWrapper`, `damageBoxWrapper`, `pennyShapedCrackWrapper`	Paint initial damage or seeded crack-like damaged regions.
`porosityWrapper`, `pointwisePorosityWrapper`, `temperatureWrapper`	Assign auxiliary material-state fields when the particle file includes the corresponding columns.
`surfaceFlagWrapper`, `surfaceNormalWrapper`, `surfacePositionWrapper`, `czTagWrapper`	Assign surface, cohesive-zone, and explicit-surface data that feed contact and cohesive-zone algorithms.
Custom `BaseWrapper` subclass	Override `mat`, `group`, `particleType`, or the corresponding getter methods while forwarding the rest of the geometry interface.

Listing 3.9: Minimal custom wrapper that changes contact group while reusing a geometry.

```
class groupWrapper(geom.BaseWrapper):
    def __init__(self, name, subObject, group):
        super().__init__(name, subObject)
        self.group = group

left_grain = groupWrapper("left_grain", grain_geometry, group=0)
right_grain = groupWrapper("right_grain", grain_geometry, group=1)
```

3.6.3 Material directions for vector or tensor directions

The optional `MaterialDirection` particle-file field stores a local material basis for each particle. PFW writes this as nine columns, `MaterialDirectionXX` through `MaterialDirectionZZ`, which form a row-wise 3×3 direction matrix. The identity matrix is used when no object supplies a material direction. Direction-dependent material models can use this matrix to interpret vector-valued or tensor-valued material axes in the global coordinate system. For example, a transverse-isotropic material may use the first material axis as the fiber or bedding direction, while other models may use the triad to rotate a preferred damage, strength, or elastic tensor basis.

For simple cases, `materialDirectionWrapper` can be applied to any geometry object. If the supplied value is a three-component vector, PFW normalizes it and constructs an orthonormal triad with that vector as the first row. If the supplied value is already a 3×3 array, PFW writes it directly. Users

should supply unit vectors or proper rotation matrices; non-orthogonal matrices can make anisotropic constitutive response difficult to interpret even if the input is accepted.

Listing 3.10: Assigning a material-direction vector or full material triad.

```
# A single vector: PFW builds an orthonormal triad with this as axis 1.
fiber_block = geom.materialDirectionWrapper("fiber_block", matrix,
                                            matDir=[1.0, 1.0, 0.0])

# A full row-wise triad: useful for verification or crystallographic axes.
crystal_axes = [[1.0, 0.0, 0.0],
                [0.0, 0.0, 1.0],
                [0.0, 1.0, 0.0]]
crystal = geom.materialDirectionWrapper("crystal", inclusion,
                                        matDir=crystal_axes)

pfw["particleFileFields"] = ["Velocity", "MaterialType", "ContactGroup",
                             "SurfaceFlag", "MaterialDirection"]
```

Material directions are particle data, not material definitions. Two particles can use the same material model and material parameters while carrying different local bases. This is the appropriate mechanism for grains, fibers, bedding planes, or spatially varying anisotropy when the constitutive model reads the `MaterialDirection` field.

3.6.4 The `pfw_materials.py` material library

The file `pfw_materials.py` is a Python material-card library for PFW inputs. It provides dictionary-compatible material entries with named parameters, a `name`, a GEOS `model`, and a generated `materialString`. The current library includes engineering metals, engineering ceramics, graphite materials, engineering polymers, explicit-solid examples, example-suite materials, verification materials, and a `materialDatabase` dictionary keyed by material name. Appendix E lists the preset counts discovered in the reviewed source tree.

PFW uses two separate pieces of information when writing the GEOS XML:

- `pfw["materials"]` is the ordered list of material names used by `ParticleRegion` entries. Geometry-object `mat` indices refer to this list.
- `pfw["materialPropertyString"]` is the concatenated XML text for those material cards inside the generated `Constitutive` block.

The material library helps keep those two entries consistent.

Listing 3.11: Using two material entries from `pfw_materials.py`.

```
# [pfw_dependency] pfw_materials.py
import importlib
matdb = importlib.import_module("pfw_materials")

matrix = matdb.elasticDemo.copy()
matrix["name"] = "matrixElastic"

inclusion = matdb.verificationQuartz.copy()
inclusion["name"] = "inclusionQuartz"

pfw["materials"] = [matrix["name"], inclusion["name"]]
pfw["materialPropertyString"] = "\n".join([
    matrix["materialString"],
    inclusion["materialString"],
])

matrix_object = geom.box("matrix", [-1,-1,-1], [1,1,1], mat=0, group=0)
inclusion_object = geom.sphere("inclusion", [0,0,0], 0.25, mat=1, group=1)
pfw["objects"] = [inclusion_object, matrix_object]
```

The material entries are instances of `MaterialProperties`, a `dict` subclass that refreshes the XML card when `materialString` is accessed. It is still good practice to copy a library entry before editing it, because the imported module-level object may be reused elsewhere in the same input file.

Listing 3.12: Modifying a material parameter and regenerating the XML string.

```
import importlib
matdb = importlib.import_module("pfw_materials")

soft_matrix = matdb.elasticDemo.copy()
soft_matrix["name"] = "softMatrix"
soft_matrix["defaultYoungModulus"] = 0.50
soft_matrix["defaultPoissonRatio"] = 0.30

# Accessing materialString refreshes the XML from the current dictionary.
xml_card = soft_matrix["materialString"]

# An explicit refresh is also available and can be clearer in scripted edits.
soft_matrix.refreshMaterialString()

pfw["materials"] = [soft_matrix["name"]]
pfw["materialPropertyString"] = soft_matrix["materialString"]
```

When creating a material dictionary from scratch rather than copying a library entry, use the model-specific required parameter names from Section 2.7 and the generated material catalogue, then call `matdb.finalizeMaterialEntry(custom)` to attach the appropriate generator. This keeps the user-defined dictionary compatible with the same `materialString` workflow used by the built-in entries.

Listing 3.13: Finalizing a new material dictionary for PFW use.

```
custom = {
    "name": "customElastic",
    "version": 1,
    "model": "ElasticIsotropic",
    "defaultDensity": 1.0,
    "defaultYoungModulus": 2.0,
    "defaultPoissonRatio": 0.25,
}
custom = matdb.finalizeMaterialEntry(custom)

pfw["materials"] = [custom["name"]]
pfw["materialPropertyString"] = custom["materialString"]
```

3.7 Solver controls

The PFW parameter catalogue in Appendix C lists all keys discovered in `particleFileWriter.py`; Appendix A lists the corresponding solver-side wrappers discovered from `SolidMechanics_MPM`. This section gives copyable solver-control fragments from the verification, validation, and example inputs, but it intentionally avoids re-documenting controls that are treated in detail elsewhere in this chapter. Geometry construction is described in Sections 3.5 and 3.6; boundary, F-table, and stress-control inputs are described in Section 3.8; diagnostics are described in Section 3.9; and output controls are described in Section 3.10. The examples below are therefore fragments to be placed into a complete `pfw_input` file after the grid, material, and object definitions have been supplied.

PFW has two solver-control paths. Keys that appear in its internal parameter table and are marked as solver attributes are written directly to the generated `SolidMechanics_MPM` XML node. Unknown `pfw` keys are also written as solver attributes, after a warning and typo-suggestion pass, so newly registered solver options can often be exercised without modifying PFW. Because this pass-through behavior can also expose spelling mistakes, always inspect the generated `mpm_*.xml` file when introducing a new key.

3.7.1 Core time integration and update controls

Most production and V&V examples use explicit dynamics. The time-integration enumeration is described in Appendix A; the particle/grid update algorithms are described in Section 2.8. A minimal explicit-dynamic block is

Listing 3.14: Minimal explicit-dynamic solver-control block.

```
stop_time = 1.0e-3

pfw["timeIntegrationOption"] = "ExplicitDynamic"
pfw["updateMethod"] = "FLIP"       # FLIP, PIC, XPIC, or FMPM
pfw["updateOrder"] = 2             # used by XPIC and FMPM
pfw["cflFactor"] = 0.25
pfw["initialDt"] = 1.0e-16

pfw["endTime"] = stop_time
pfw["plotInterval"] = stop_time / 100.0
pfw["restartInterval"] = 2.0 * stop_time
pfw["solverProfiling"] = 0
```

`cflFactor` limits the stable explicit step computed by the solver, while `initialDt` is usually chosen very small so that the first step is accepted even before wave-speed and cell-size estimates have settled.

`updateMethod` should be chosen together with the particle type: CPDI examples commonly use `FLIP` or `FMPM`, single-point B-spline examples commonly use `FMPM`, and highly dissipative smoke tests may use `PIC`. `XPIC` and `FMPM` use `updateOrder` as the order of the projection/filter approximation.

Example pattern	Representative suite input	Solver-control emphasis
Elastic or hyperelastic contact	`examples/elasticDisk/pfw_input_elasticDisk.py`	Explicit dynamics, FMPM, domain deformation, basic contact and robustness guards.
Contact-normal and gap tests	`verification/contact/pfw_input_symmetricInterface_explicit.py` and `verification/contact/pfw_input_symmetricInterface_implicit.py`	Explicit vs implicit surface data, contact gap closure, DFG contact.
CPDI damage benchmark	`examples/brazilianDisk_FLIP/pfw_input_brazilianDisk_FLIP.py`	CPDI domain scaling, FLIP, neighbor list, damage and DFG.
B-spline / FMPM benchmark	`examples/brazilianDisk_BSpline/pfw_input_brazilianDisk_BSpline.py` and `examples/brazilianDisk_FMPM/pfw_input_brazilianDisk_FMPM.py`	B-spline particles, FMPM order, DFG contact.
Notched-bar damage	`verification/sizeEffect/pfw_input_notchedBar.py`	Damage localization, fracture/contact enrichment, crack-tip controls.
Cohesive-zone tests	`verification/cohesiveZones/pfw_input_cz_flip.py` and `verification/cohesiveZones/pfw_input_multi_cz.py`	Cohesive events, explicit surface data, CZ tags, cohesive constitutive cards.

Table 3.4: Representative source inputs used to construct the compact solver-control examples in this section.

3.7.2 Example: hyperelastic single-field contact

A single velocity field gives the classical MPM no-slip, non-penetrating contact behavior for particles that map to the same grid nodes. In PFW this is obtained by assigning the contacting bodies to the same contact group, usually `group=0`, and leaving `damageFieldPartitioning` disabled. The material assignment itself follows Section 3.6; the example below uses a hyperelastic material entry from `pfw_materials.py` and two objects in the same group.

Listing 3.15: Single-field hyperelastic contact.

```
import pfw_materials as matdb
import pfw_geometryObjects as geom

rubber = matdb.hyperelasticNaturalRubber.copy()
rubber["name"] = "rubber"
rubber.refreshMaterialString()

pfw["materials"] = [rubber["name"]]
pfw["materialPropertyString"] = rubber["materialString"]

# Same group -> one velocity field when DFG is off.
left_disk  = geom.cylinder("leftDisk",  [-0.25, 0.0, zmin], [-0.25, 0.0, zmax], 0.2,
```

```
                          mat=0, group=0, particleType="CPDI", dim=2)
right_disk = geom.cylinder("rightDisk", [ 0.25, 0.0, zmin], [ 0.25, 0.0, zmax], 0.2,
                          mat=0, group=0, particleType="CPDI", dim=2)
pfw["objects"] = [left_disk, right_disk]

pfw["timeIntegrationOption"] = "ExplicitDynamic"
pfw["updateMethod"] = "FLIP"
pfw["cpdiDomainScaling"] = 1
pfw["damageFieldPartitioning"] = 0
pfw["frictionCoefficient"] = 0.0       # not used between particles in one field
pfw["particleFileFields"] = ["Velocity", "MaterialType", "ContactGroup",
                             "SurfaceFlag", "RVector"]
```

Use this pattern for compact smoke tests where the goal is to verify the constitutive response or deformation kinematics without material-contact separation. If bodies should slip or separate, assign different groups and use the contact examples below, or enable DFG contact for self-contact after damage localization.

3.7.3 Example: two-material frictional contact with group-dependent coefficients

The global `frictionCoefficient` is convenient when all contact pairs share one Coulomb coefficient. For mixed materials, assign each object a contact group and provide `frictionCoefficientTable`. Section 2.11 describes the solver-side contact algorithm and the modulo indexing used when DFG adds additional velocity fields.

Listing 3.16: Group-dependent friction and explicit surface contact.

```
# Geometry objects are assigned group=0 for matrix and group=1 for inclusion/tool.
# The table is indexed by contact group, not material ID.
pfw["frictionCoefficient"] = 0.25      # fallback/global value
pfw["frictionCoefficientTable"] = [
    [0.00, 0.15],     # group 0 with groups 0,1
    [0.15, 0.45],     # group 1 with groups 0,1
]
pfw["frictionCoefficientRuleOfMixtures"] = 0

# Contact surface reconstruction and gap closure; see Section 2.11.
pfw["contactGapCorrection"] = "Implicit"   # Simple, Implicit, or Softened
pfw["contactNormalType"] = "Aligned"        # pass-through solver key
pfw["contactNormalExponent"] = 2.0          # pass-through solver key
pfw["useSurfacePositionForContact"] = 1
pfw["explicitSurfaceNormalInfluence"] = 1000.0

pfw["particleFileFields"] = ["Velocity", "MaterialType", "ContactGroup",
                             "SurfaceFlag", "RVector",
                             "SurfaceNormal", "SurfacePosition"]
```

If explicit surface normals or positions are requested but the corresponding particle-file columns are absent, PFW adds `SurfaceNormal` and `SurfacePosition` automatically and emits a warning. It is better to list them explicitly in production inputs so that the intent is visible in version control.

3.7.4 Example: CPDI, FLIP, domain scaling, and damage-enabled DFG

The Brazilian-disk FLIP example is representative of CPDI damage simulations. CPDI domain scaling improves contact/fracture behavior for distorted domains; DFG contact introduces two fields when

the damage-gradient criteria are met; and the neighbor list is typically required by damage models, field-gradient partitioning, or surface reconstruction.

Listing 3.17: CPDI/FLIP damage and DFG control block.

```
pfw["updateMethod"] = "FLIP"
pfw["cpdiDomainScaling"] = 1
pfw["damageFieldPartitioning"] = 1
pfw["needsNeighborList"] = 1

# DFG and damaged-particle behavior.
pfw["separabilityMinDamage"] = 0.5
pfw["treatFullyDamagedAsSingleField"] = 1
pfw["resetDefGradForFullyDamagedParticles"] = 1
pfw["disableSurfaceNormalsAndPositionsOnCPDIScaling"] = 1
pfw["useDamageAsSurfaceFlag"] = 0

# Useful output during development of CPDI scaling and DFG contact.
pfw["plotUnscaledParticles"] = 1
pfw["particleFileFields"] = ["Velocity", "MaterialType", "ContactGroup",
                             "SurfaceFlag", "Damage", "StrengthScale",
                             "RVector"]
```

Set `useDamageAsSurfaceFlag=1` only when the initial `Damage` field is intended to seed surface flags. For many continuum-damage runs, surface flags are painted geometrically while the evolving damage field controls DFG separability.

3.7.5 Example: notched bar with crack-tip correction controls

The notched-bar verification input constructs an initial notch by painting damage and surface data on particles. Solver-side crack-tip controls are direct pass-through keys: they are registered by `SolidMechanics_MPM` even though they are not part of the older PFW default table. This is the normal pattern for newly added solver controls.

Listing 3.18: Notched-bar crack-tip and DFG controls.

```
pfw["updateMethod"] = "FMPM"
pfw["updateOrder"] = 2
pfw["cpdiDomainScaling"] = 1
pfw["damageFieldPartitioning"] = 1
pfw["needsNeighborList"] = 1

# Direct solver pass-through keys for crack-tip aware damage/contact runs.
pfw["useCrackTipDetection"] = 1
pfw["crackTipDetectionThreshold"] = 0.75
pfw["surfaceQualityThreshold"] = 0.2
pfw["thinFeatureDFGThreshold"] = 1.5
pfw["useDamageAsSurfaceFlag"] = 1

pfw["particleFileFields"] = ["Velocity", "MaterialType", "ContactGroup",
                             "SurfaceFlag", "Damage", "StrengthScale",
                             "RVector", "SurfaceNormal", "SurfacePosition"]
```

The numerical values above are starting values, not universal defaults. The thresholds should be verified against the chosen grid spacing, damage regularization length, notch radius, and constitutive damage law.

3.7.6 Example: B-spline particles, FMPM, and DFG contact

B-spline particles are selected through the geometry object's `particleType`, not by a top-level PFW key. The solver-control part is the FMPM/DFG combination. The B-spline examples in the example suite use `cpdiDomainScaling=0` because the particles are single-point B-spline particles rather than CPDI domains.

Listing 3.19: B-spline/FMPM/DFG solver controls.

```
# Geometry example, shown only to emphasize where the particle type is assigned.
sample = geom.cylinder("sample", [0.0, cy, zmin], [0.0, cy, zmax], radius,
                       mat=0, group=0, dim=2,
                       particleType="SinglePointBSpline")
pfw["objects"] = [sample]

pfw["updateMethod"] = "FMPM"
pfw["updateOrder"] = 2
pfw["cpdiDomainScaling"] = 0
pfw["damageFieldPartitioning"] = 1
pfw["needsNeighborList"] = 1

# Optional normal/position reconstruction controls for contact studies.
pfw["normalAndPositionMethod"] = "LogisticRegression"
pfw["maxLRIterations"] = 25
pfw["LRtolerance"] = 1.0e-8
pfw["contactGapCorrection"] = "Implicit"
pfw["frictionCoefficient"] = 0.25

pfw["particleFileFields"] = ["Velocity", "MaterialType", "ContactGroup",
                             "SurfaceFlag", "Damage"]
```

For FMPM, `updateOrder` controls the number of projection/filter passes. Higher order can reduce lumped-mass projection error but costs additional particle-grid transfers. The boundary/contact caveat for the specific wall-contact boundary condition is described in Sections 2.8 and 2.9; multi-field contact and moving velocity boundaries use the incremental correction path described there.

3.7.7 Example: cohesive-zone run controls

Cohesive-zone runs require both a cohesive constitutive card in `materialPropertyString` and a `CohesiveZone` event in `mpmEventsString`. Section 2.10 describes the internal cohesive algorithm; Section 3.6 describes the material-string workflow.

Listing 3.20: Cohesive-zone PFW controls with explicit surface data.

```
pfw["updateMethod"] = "FMPM"
pfw["updateOrder"] = 2
pfw["needsNeighborList"] = 1
pfw["useEvents"] = 1

# Surface columns are required by the cohesive initialization/contact geometry.
pfw["particleFileFields"] = [
    "Velocity", "MaterialType", "ContactGroup", "SurfaceFlag", "CZTag",
    "RVector", "SurfaceNormal", "SurfacePosition",
]

pfw["useSurfacePositionForContact"] = 1
pfw["explicitSurfaceNormalInfluence"] = 1000.0
```

```
pfw["cohesiveFieldPartitioning"] = 1
pfw["preventCZInterpenetration"] = 1

# Section 3.6 shows how to build or edit cohesive material cards.
# Newline-separated XML attributes avoid visible-space artifacts in this listing.
cz_card = "\n".join([
    "<CoupledCohesiveZone",
    "name='cz'",
    "maxNormalStress='0.01'",
    "maxShearStress='0.01'",
    "characteristicNormalDisplacement='0.05'",
    "characteristicTangentialDisplacement='0.05'/>",
])
pfw["materialPropertyString"] = (
    matdb.elasticDemo["materialString"] + "\n" + cz_card
)

pfw["mpmEventsString"] = "\n".join([
    "<CohesiveZone",
    "name='czEvent'",
    "startTime='0.0'",
    "constitutiveModel='cz'",
    "czVolumeNormalization='1'/>",
])
```

Older cohesive-failure style controls such as `enableCohesiveFailure`, `cohesiveLaw`, `maxCohesiveNormalStress`, and displacement-scale parameters are still registered through the solver interface and appear in some legacy V&V inputs. New cohesive-zone examples should prefer explicit cohesive constitutive cards and `CohesiveZone` events so that the material law, tags, and initialization event are all visible in the generated XML.

3.7.8 Example: body force, manufactured solution, and miscellaneous pass-through controls

A few solver controls are used in specialized verification or method-development runs. PFW will write recognized keys whose table entry emits a solver attribute and will also pass through unknown solver keys. This makes it possible to keep the PFW input and the GEOS XML input synchronized while a new option is being developed.

Listing 3.21: Specialized solver pass-through examples.

```
# Uniform body acceleration or source term; see the BodyForceUpdate event for
# time-dependent body-force changes.
pfw["bodyForce"] = [0.0, -9.81e-3, 0.0]

# Manufactured-solution or algorithm-development flags.
pfw["generalizedVortexMMS"] = 1
pfw["neighborRadius"] = 2.5
pfw["useAPIC"] = 0
pfw["useReferenceVectorsForParticleUpdate"] = 1
pfw["plotGridFields"] = 1
```

The solver registers `FSubcycles` for deformation-gradient subcycling, but in the reviewed PFW table it is marked as a local/non-emitted key. If a case needs this control, inspect the generated XML and either update the PFW parameter table so `FSubcycles` emits as a solver attribute or add the attribute in a post-generation XML edit until the PFW table is corrected.

3.7.9 Robustness guards commonly paired with solver controls

The deletion and reset algorithms are documented in Section 2.12. The following guards are often included in solver-control examples so that unstable particles are removed before they corrupt the grid state.

Listing 3.22: Common robustness guard parameters.

```
pfw["maxParticleVelocity"] = 10.0
pfw["minParticleJacobian"] = 0.01
pfw["maxParticleJacobian"] = 10.0
pfw["logLevel"] = 1
```

These values are unit-system and problem dependent. They should be set wide enough that valid material response is not clipped, but tight enough to catch failed particles, bad initial overlap, or runaway constitutive states early in a calculation.

3.7.10 Coverage map for solver-control examples

Table 3.5 summarizes where a user can find a PFW usage example for the main solver-control keys that are not already treated as primary topics in Sections 3.8-3.10. The generated appendices remain the authoritative inventory of all registered keys.

Table 3.5: PFW solver-control example coverage in Section 3.7.

Control family	Keys shown here	Example subsection
Time integration and output schedule	`timeIntegrationOption`, `cflFactor`, `initialDt`, `endTime`, `plotInterval`, `restartInterval`, `solverProfiling`	Core explicit-dynamic block
Particle/grid update	`updateMethod`, `updateOrder`	Core block; B-spline/FMPM example
Single-field contact	`damageFieldPartitioning=0`, same object `group`	Hyperelastic single-field example
Group contact and friction	`frictionCoefficient`, `frictionCoefficientTable`, `frictionCoefficientRuleOfMixtures`	Two-material friction example
Contact normals and gap closure	`contactGapCorrection`, `contactNormalType`, `contactNormalExponent`, `useSurfacePositionForContact`, `explicitSurfaceNormalInfluence`	Two-material friction example; B-spline/FMPM example
CPDI and DFG damage	`cpdiDomainScaling`, `damageFieldPartitioning`, `needsNeighborList`, `separabilityMinDamage`, `treatFullyDamagedAsSingleField`	CPDI/FLIP damage example
Damage-derived surfaces	`useDamageAsSurfaceFlag`, `surfaceQualityThreshold`, `thinFeatureDFGThreshold`	Notched-bar example
Crack-tip controls	`useCrackTipDetection`, `crackTipDetectionThreshold`	Notched-bar example
Logistic-regression surface reconstruction	`normalAndPositionMethod`, `maxLRIterations`, `LRtolerance`	B-spline/FMPM example

Continued on next page

Control family	Keys shown here	Example subsection
Cohesive-zone event controls	`useEvents`, `mpmEventsString`, `cohesiveFieldPartitioning`, `preventCZInterpenetration`	Cohesive-zone example
Legacy cohesive-failure controls	`enableCohesiveFailure`, `cohesiveLaw`, cohesive strength-/displacement scales	Cohesive-zone notes; see Section 2.10
Special verification controls	`bodyForce`, `generalizedVortexMMS`, `neighborRadius`, `useAPIC`, `useReferenceVectorsForParticleUpdate`	Miscellaneous pass-through example
Robustness guards	`maxParticleVelocity`, `minParticleJacobian`, `maxParticleJacobian`, `logLevel`	Robustness guard block

3.8 Boundary conditions, F-table, and stress control

This section describes how users control the boundary and related domain-motion features through a `pfw_input` file. The internal solver algorithms are described in Section 2.9. PFW writes most of these keys directly as attributes on the generated `SolidMechanics_MPM` XML node, so new solver-side attributes can often be exercised through PFW without adding a special PFW wrapper.

3.8.1 Face ordering and boundary-condition types

Boundary-condition arrays use the face order

$$\{x^-, x^+, y^-, y^+, z^-, z^+\}. \tag{3.14}$$

The primary PFW key is `boundaryConditionTypes`. Its values are the integer enum values used by the solver: `0` for `Outflow`, `1` for `Symmetry`, `2` for `Moving`, and `3` for `Contact`. For example, a two-dimensional compression-style input often uses moving boundaries in the loading direction, outflow or free sides in the lateral direction, and symmetry through the suppressed z direction:

Listing 3.23: PFW boundary face order and type selection.

```
# Face order: x-, x+, y-, y+, z-, z+
pfw["boundaryConditionTypes"] = [0, 0, 2, 2, 1, 1]
```

If this key is omitted, the solver initializes all six faces to `Outflow`.

3.8.2 Time-dependent boundary-type switching

To switch boundary types during a run, enable `prescribedBcTable` and provide `bcTable`. Each row contains a time followed by six boundary-type integers in the same face order. The solver uses zero-order hold selection at approximately the time-step midpoint, not interpolation.

Listing 3.24: PFW boundary-condition table.

```
pfw["prescribedBcTable"] = 1
pfw["bcTable"] = [
  [0.0, 0, 0, 2, 2, 1, 1],
  [1.0e-3, 3, 3, 2, 2, 1, 1],
]
```

The table must have seven entries per row. The first column is time; the remaining columns are the six boundary types.

3.8.3 Boundary-driven F-table deformation

Moving boundaries require a source of domain motion. The common PFW pattern is to set `prescribedBoundaryFTable = 1`, choose an interpolation type, and provide an `fTable`. The table columns are time, F_{xx}, F_{yy}, and F_{zz}. The first row should normally start at time zero with unit stretches.

Listing 3.25: Boundary-driven diagonal deformation history.

```
pfw["boundaryConditionTypes"] = [2, 2, 2, 2, 1, 1]
pfw["prescribedBoundaryFTable"] = 1
pfw["fTableInterpType"] = "Cosine"   # Linear, Cosine, or Smoothstep
pfw["fTable"] = [
  [0.0,     1.00, 1.00, 1.00],
  [1.0e-3, 1.00, 0.90, 1.00],
]
```

PFW normalizes legacy integer interpolation values `0`, `1`, and `2` to `Linear`, `Cosine`, and `Smoothstep`. A face marked `Moving` is only an active velocity boundary when prescribed boundary deformation or stress control is active.

3.8.4 Periodic homogeneous deformation

Use `prescribedFTable = 1` for a superposed homogeneous velocity-gradient update in periodic directions. Periodicity itself is controlled by the background-grid key `periodic`, not by `boundaryConditionTypes`. A periodic homogeneous compression example has the form

Listing 3.26: Periodic F-table control.

```
pfw["periodic"] = [True, True, True]
pfw["prescribedFTable"] = 1
pfw["fTableInterpType"] = "Smoothstep"
pfw["fTable"] = [
  [0.0,     1.0, 1.0, 1.0],
  [5.0e-4, 0.9, 0.9, 0.9],
]
```

This is distinct from `prescribedBoundaryFTable`: the periodic path modifies particle velocity gradients and wraps particles through the periodic spatial partition, while the boundary path imposes velocities on selected physical grid faces.

3.8.5 Prescribed tangential velocities on moving faces

Tangential velocity components on moving faces are free unless explicitly enabled. The enable flag is a six-entry face array; the velocity array has shape `6 x 2`, with the two entries on each face corresponding to the two tangential coordinate directions used internally by the solver.

Listing 3.27: Tangential velocities on moving faces.

```
pfw["enablePrescribedBoundaryTransverseVelocities"] = [0, 0, 1, 0, 0, 0]
pfw["prescribedBoundaryTransverseVelocities"] = [
  [0.0, 0.0], [0.0, 0.0],
  [0.0, -0.1],   # y- face tangential values
```

```
  [0.0, 0.0],
  [0.0, 0.0], [0.0, 0.0],
]
```

This feature is used for shear, peel, and tool-like moving-boundary setups where the normal motion comes from `fTable` or stress control but a tangential sliding speed must also be prescribed. In the current PFW script, `prescribedBoundaryTransverseVelocities` has a standard default, while `enablePrescribedBoundaryTransverseVelocities` is passed through to the solver as an XML attribute when the user supplies it.

3.8.6 Boundary contact-wall controls

Boundary contact walls are selected with boundary type `3`. Optional per-face friction coefficients are supplied with `boundaryFaceFrictionCoefficients`. The solver also registers `boundaryFaceCoefficientsOfRestitution`, but the reviewed active code path does not use this coefficient in the normal wall update.

Listing 3.28: Contact-wall boundary setup.

```
pfw["boundaryConditionTypes"] = [3, 3, 3, 3, 1, 1]
pfw["boundaryFaceFrictionCoefficients"] = [0.0, 0.0, 0.2, 0.2, 0.0, 0.0]
# Registered by the solver, but not active in the current normal-contact update:
pfw["boundaryFaceCoefficientsOfRestitution"] = [1, 1, 1, 1, 1, 1]
```

Contact-wall calculations can use particle-mapped surface positions when they are available, otherwise the active boundary-contact code falls back to center-of-volume information at the grid node. Cases that require explicit surface-position or surface-normal fields should include those fields directly, or enable the related explicit-contact or cohesive-zone options that make PFW add `SurfaceNormal` and `SurfacePosition` automatically.

3.8.7 Stress-control inputs

Stress control is configured with a three-entry `stressControl` flag, a `stressTable`, an interpolation type, and PID-like gains. The table columns are time followed by target normal stresses in the grid-aligned directions. Stress control generates `domainL`; moving faces then impose the associated boundary velocities when the moving-face branch is active, which in the reviewed code means `prescribedBoundaryFTable` is active or at least one `stressControl` entry equals one.

Listing 3.29: Stress-control input pattern.

```
pfw["boundaryConditionTypes"] = [2, 2, 2, 2, 2, 2]
pfw["prescribedBoundaryFTable"] = 1
pfw["fTable"] = [[0.0, 1.0, 1.0, 1.0], [stopTime, 1.0, 1.0, 1.0]]
pfw["stressControl"] = [1, 1, 1]
pfw["stressTableInterpType"] = "Smoothstep"
pfw["stressTable"] = [
  [0.0,      -1.0e6, -1.0e6, -1.0e6],
  [stopTime, -5.0e6, -5.0e6, -5.0e6],
]
pfw["stressControlKp"] = 0.01
pfw["stressControlKi"] = 0.0
pfw["stressControlKd"] = 0.001
```

A practical pattern is to include a neutral `fTable` with unit stretches when using stress control on moving faces, then let the controller determine the actual velocity-gradient components.

3.9 Diagnostics

PFW exposes several lightweight CSV diagnostics that are separate from plot-file output. They are commonly used for verification plots, automated suite checks, and rapid inspection of a run before opening Silo or VTK files. Each diagnostic has an enable flag and its own write schedule. Time-based schedules use a write interval in simulation time, while cycle-based schedules, when available, use an integer cycle interval. For GEOS string-array inputs such as `profileVariables` and `tracerVariables`, either pass the GEOS brace syntax directly or use the tracer helpers in `pfw_tracerPoints.py`.

Diagnostic	Principal PFW controls
Reaction history	`reactionHistory`, `reactionWriteInterval`, `useInteralForceAsFaceReaction`
Box averages	`boxAverageHistory`, `boxAverageWriteInterval`, `boxAverageMin`, `boxAverageMax`, `boxAverageResizeWithDomain`
Profiles	`profileHistory`, `profileDirection`, `profileVariables`, `profileNumSlices`, `profileWriteInterval`, `profileCycleInterval`
Tracers	`tracerHistory`, `tracerCoordinates`, `tracerVariables`, `tracerWriteInterval`, `tracerCycleInterval`, `tracerOutputPrefix`

Table 3.6: PFW diagnostic controls summarized in Section 3.9.

3.9.1 Reaction history

Boundary reactions are enabled with `reactionHistory`. The write cadence is controlled by `reactionWriteInterval`; if this interval is zero or omitted, the solver can write every step. The output is `reactionHistory.csv`. Section 2.9.9 describes the internal accumulation algorithm, and Section 4.5 summarizes the reaction file layout and post-processing conventions.

Listing 3.30: Reaction-history controls using the default velocity-correction measure.

```
pfw["reactionHistory"] = 1
pfw["reactionWriteInterval"] = stopTime / 1000.0

# Default behavior: reaction comes from the imposed nodal velocity correction.
pfw["useInteralForceAsFaceReaction"] = 0
```

The default reaction measure is the momentum correction required to enforce the normal velocity on a moving or contact boundary. At a fixed-velocity boundary, the particle-to-grid velocity mapping can place a nonzero normal velocity on boundary nodes before the essential boundary condition is applied. The subsequent correction may be tiny in displacement but large when divided by a very small time step, which can create narrow spikes in a reaction-stress plot. When these spikes obscure the trend of interest, `useInteralForceAsFaceReaction=1` uses the internal-force component at the boundary node instead of the velocity-correction impulse. The name preserves the current solver spelling.

Listing 3.31: Reaction-history controls using the boundary internal-force component.

```
pfw["reactionHistory"] = 1
pfw["reactionWriteInterval"] = stopTime / 2000.0

# Spelling matches the current GEOS solver attribute.
pfw["useInteralForceAsFaceReaction"] = 1
```

The internal-force option is a diagnostic correction; it does not change the boundary condition. An APIC-consistent boundary mapping would address the same issue more directly, because the affine part

of the particle velocity field would be represented during the boundary projection rather than being corrected only after the velocity has been mapped to the grid.

The six reaction values are forces, not stresses. To interpret a face reaction as an engineering stress, divide by the actual material surface area that carries load, not necessarily by the nominal area of the background-grid face. This distinction matters for porous specimens, trimmed objects, cylindrical or curved boundaries embedded in a box, damaged surfaces, and cases where only part of a boundary face is occupied by material. A typical post-processing calculation is

$$\sigma_{\mathrm{eng}}(t) = \frac{R_{y^+}(t)}{A_{\mathrm{material}}(t)}, \tag{3.15}$$

where A_{material} should match the specimen area used in the experiment or analytical comparison.

3.9.2 Box-average history

Setting `boxAverageHistory` writes `boxAverageHistory.csv`. The output contains averages of stress, density, damage, internal energy, kinetic energy, strain, material volume, temperature, and the diagonal domain-deformation components. If `boxAverageMin` and `boxAverageMax` are omitted, the solver initializes the averaging box from the full computational domain. This full-domain default is the common choice for homogeneous verification problems, triaxial tests, and bulk compression runs.

Listing 3.32: Full-domain box-average history.

```
pfw["boxAverageHistory"] = 1
pfw["boxAverageWriteInterval"] = stopTime / 1000.0

# Omit boxAverageMin and boxAverageMax to use the full initial domain.
```

A fixed subregion is selected with `boxAverageMin` and `boxAverageMax`. These are solver attributes and may be passed through the `pfw` dictionary. A fixed box is useful for excluding platens, buffer layers, free surfaces, voids, or boundary artifacts.

Listing 3.33: Fixed Eulerian sub-box average.

```
Lx = pfw["xmax"] - pfw["xmin"]
Ly = pfw["ymax"] - pfw["ymin"]

pfw["boxAverageHistory"] = 1
pfw["boxAverageWriteInterval"] = stopTime / 500.0
pfw["boxAverageMin"] = [-0.25 * Lx, -0.25 * Ly, pfw["zmin"]]
pfw["boxAverageMax"] = [ 0.25 * Lx,  0.25 * Ly, pfw["zmax"]]
pfw["boxAverageResizeWithDomain"] = 0
```

When a moving grid, D-table style domain update, or F-table boundary condition changes the domain length, the volume used in volume-normalized quantities changes as well. If the diagnostic should follow a deforming specimen or the full current domain, set `boxAverageResizeWithDomain=1` so the box limits scale with the diagonal domain stretch. If the diagnostic should remain a fixed Eulerian measurement window, keep `boxAverageResizeWithDomain=0` and provide explicit limits. In either case, interpret density, stress, and energy histories with the selected averaging volume in mind.

3.9.3 Profiles

Profile outputs reduce particle data into one-dimensional spatial bins and write one CSV file per requested variable. Files are named `profile_<direction>_<variable>.csv`. The direction can be `x`, `y`, or `z`. Supported variables include `density`, `damage`, `temperature`, `internalEnergy`, `kineticEnergy`, `velocityX`,

`velocityY`, `velocityZ`, `volumeFraction`, `plasticStrainMagnitude`, the six stress components, and the six plastic-strain components.

The following example requests two x-profiles. With `profileNumSlices=0`, the solver uses the background-grid resolution in the profile direction. The write interval is time-based.

Listing 3.34: X-profiles of density and axial stress using grid-cell spacing.

```
pfw["profileHistory"] = 1
pfw["profileDirection"] = "x"
pfw["profileVariables"] = "{ density, stressXX }"
pfw["profileNumSlices"] = 0          # use grid resolution in x
pfw["profileWriteInterval"] = stopTime / 200.0
pfw["profileCycleInterval"] = 0
```

The next example uses a sparse y-profile and a sparse temporal schedule. This pattern is useful for long calculations where full plot files are too expensive to write frequently, but low-dimensional diagnostics are needed for convergence or stability checks.

Listing 3.35: Sparse Y-profile of several variables.

```
pfw["profileHistory"] = 1
pfw["profileDirection"] = "y"
pfw["profileVariables"] = (
    "{ density, volumeFraction, velocityY, damage, "
    "stressXX, stressYY, plasticStrainMagnitude }"
)
pfw["profileNumSlices"] = 64
pfw["profileWriteInterval"] = 0.0
pfw["profileCycleInterval"] = 100
```

Use either `profileWriteInterval` or `profileCycleInterval` for a given run. Keeping the inactive interval at zero makes the intended scheduling mode explicit in the generated XML.

3.9.4 Tracers

Tracer histories select the nearest active particle to each requested coordinate during initialization, store that particle ID, and then follow the same particle through time. Each tracer file is named from `tracerOutputPrefix`; rows contain `t,x,y,z` followed by the requested particle variables. The helper module `pfw_tracerPoints.py` is recommended because it formats `tracerCoordinates` and `tracerVariables` using the GEOS array syntax expected by the XML parser.

A single point at the center of the two-disk collision domain is useful for checking symmetry and the time at which the two disks first interact.

Listing 3.36: Single tracer at the center of the colliding-disks domain.

```
import pfw_tracerPoints as tracers

center_point = [
    0.5 * (pfw["xmin"] + pfw["xmax"]),
    0.5 * (pfw["ymin"] + pfw["ymax"]),
    0.0,
]

tracers.set_tracers(
    pfw,
    points=[center_point],
    variables=["particleID", "speed", "velocityX", "velocityY", "stressXX", "stressYY"],
```

```
    write_interval=stopTime / 400.0,
    output_prefix="collidingDisks_center",
)
```

For the copper-foam compression example, two transverse planes of tracers at $0.1L$ and $0.9L$ along the impact direction provide a compact way to monitor compaction-wave arrival, velocity dispersion, and stress heterogeneity.

Listing 3.37: Two planes of tracers in the copper-foam compression example.

```
import pfw_tracerPoints as tracers

L = domainSize["z"]
plane_count = 9
plane_span_x = domainSize["x"]
plane_span_y = domainSize["y"]

front_plane = tracers.plane_grid(
    center=[0.0, 0.0, 0.1 * L], axis1="x", axis2="y",
    span1=plane_span_x, span2=plane_span_y,
    count1=plane_count, count2=plane_count,
)
back_plane = tracers.plane_grid(
    center=[0.0, 0.0, 0.9 * L], axis1="x", axis2="y",
    span1=plane_span_x, span2=plane_span_y,
    count1=plane_count, count2=plane_count,
)

tracers.set_tracers(
    pfw,
    points=front_plane + back_plane,
    variables=["particleID", "density", "speed", "stressZZ", "plasticStrainMagnitude"],
    write_interval=stopTime / 1000.0,
    output_prefix="foamPlaneTracer",
)
```

For a plane-strain borehole, a ring of tracer points on the initial inner radius tracks borehole-surface motion. The helper is named `disk`; setting one radial ring and `include_center=False` produces only the circular ring. Increase `radial_count` if a full disk of interior monitoring points is desired.

Listing 3.38: Tracer ring on the borehole inner surface.

```
import pfw_tracerPoints as tracers

borehole_center = [0.0, 0.0, 0.0]
inner_radius = 0.5 * boreholeDiameter
inner_surface_points = tracers.disk(
    center=borehole_center,
    radius=inner_radius,
    normal_axis="z",
    radial_count=1,
    angular_count=64,
    include_center=False,
)

tracers.set_tracers(
    pfw,
```

```
    points=inner_surface_points,
    variables=["particleID", "velocityX", "velocityY", "stressXX", "stressYY", "damage"],
    write_interval=stopTime / 1000.0,
    output_prefix="boreholeInnerSurface",
)
```

When tracer coordinates lie in void or outside the active particle cloud, the nearest-particle selection may choose an unintended particle. For surface tracking, place tracers slightly inside the material side of the surface or use generated geometry information to construct points on the material boundary.

3.10 Silo and VTK output controls

GEOS-MPM plot output is selected in PFW with `pfw["outputType"]`. The two supported output blocks are `"vtk"`, which is the ParaView-oriented path, and `"silo"`, which is the VisIt-oriented path. Both options use the same GEOS event schedule: PFW writes a `PeriodicEvent` named `outputs` with `timeFrequency=pfw["plotInterval"]`, targeting either `/Outputs/vtkOutput` or `/Outputs/siloOutput`. Restart output is independent of the plotting format and is scheduled with `pfw["restartInterval"]`. In practice, every production `pfw_input.py` should set both intervals explicitly, because they control file count, post-processing cost, and restart cadence.

The plotting controls are intentionally separate from the CSV diagnostics in Section 3.9. Plot files are full-state visualization products for ParaView or VisIt, while reaction, box-average, profile, and tracer histories are compact time-series or reduced spatial summaries. For detailed CSV file interpretation, see Section 4.5.

3.10.1 PFW output-control keys

Table 3.7 summarizes the PFW keys most commonly used to control plot output. Some keys write attributes on the GEOS output block; others write attributes on the `SolidMechanics_MPM` solver so that the solver adjusts wrapper plot levels before the output manager writes a plot file.

Table 3.7: PFW keys relevant to VTK/Silo plot output.

PFW key	Typical values	Effect
`outputType`	`"vtk"`, `"silo"`	Selects the generated GEOS output block. The default in PFW is `"vtk"`.
`plotInterval`	positive time	Time frequency for plot-file writes. This controls the `outputs` event.
`restartInterval`	positive time	Time frequency for restart writes. This is independent of VTK/Silo selection.
`plotGridFields`	`0`, `1`	Solver flag that enables background-grid wrappers for plot output, primarily for VTK workflows. The solver default is off.
`plottableFields`	GEOS name array	Optional solver-side field filter. Use it to reduce VTK file sizes by retaining only selected particle and, when `plotGridFields=1`, grid wrappers.
`gridFieldNames`	list or GEOS name array	Silo-only alias for requested background-grid fields on the `Silo` output block.
`siloGridFieldNames`	list or GEOS name array	Equivalent Silo-only alias for `gridFieldNames`.

Continued on next page

PFW key	Typical values	Effect
siloGridFields	False, True, "common", "all", list	Convenience Silo-only control. True and "common" request the standard debug set; "all" requests every grid field known to PFW and should be used sparingly.
plotUnscaledParticles	0, 1	Plot CPDI/domain-scaled particles without drawing the scaled particle domain. This is useful when scaled domains obscure particle-center fields.

The solver also registers plottableFields directly. Current PFW versions pass unknown pfw keys through to the generated solver XML after printing a warning, so pfw["plottableFields"] can be used even if it is not present in the built-in PFW parameter table. Inspect the generated XML when using this pass-through mechanism.

3.10.2 VTK output for ParaView

When pfw["outputType"] = "vtk", PFW emits a GEOS block of the form

Listing 3.39: Output block generated by PFW for VTK output.

```
<VTK
  name="vtkOutput"
  format="ascii"/>
```

The corresponding output event targets /Outputs/vtkOutput. The VTK path is generally the most convenient choice for ParaView-oriented inspection of particle fields, material-state variables, and optional background-grid fields. By default, the MPM solver does not plot background-grid wrappers. Set pfw["plotGridFields"] = 1 to plot grid quantities such as gridMass, gridVelocity, gridInternalForce, gridContactForce, and contact-surface reconstruction fields. Since grid fields are defined on the background mesh and often include one or more velocity-field dimensions, enabling them can substantially increase file size.

Use plottableFields to control VTK file size. If this array is omitted, the solver uses the default plot levels registered by each wrapper. If it is supplied, wrappers not listed are set to NOPLOT; when grid output is desired, include both plotGridFields=1 and the grid wrapper names in plottableFields.

3.10.3 Silo output for VisIt

When pfw["outputType"] = "silo", PFW emits a GEOS block of the form

Listing 3.40: Output block generated by PFW for Silo output.

```
<Silo
  name="siloOutput"
  plotFileRoot="mpm_cpdi"
  plotLevel="1"
  parallelThreads="..."
  writeCellElementMesh="1"
  writeFaceElementMesh="0"
  writeEdgeMesh="0"
  writeFEMFaces="0"/>
```

If Silo grid fields are requested, PFW adds a gridFieldNames attribute to this block. The accepted PFW inputs include a Python list, a GEOS-style braced string, a comma-separated string, True, "common", and "all". The "common" preset expands to gridMass, gridVelocity, gridInternalForce,

and `gridDamageGradient`. The `"all"` preset expands to the complete PFW-known list in Appendix C; it is intended for focused debugging because it can produce very large plot files.

The Silo block sets `parallelThreads` automatically from the run configuration when not specified. PFW does not expose the low-level Silo mesh toggles as first-class user controls; by default, the generated Silo block writes the cell-element mesh and does not write face, edge, or FEM-face meshes.

3.10.4 Minimal plot-output examples

A minimal VTK setup writes particle-level plot files at a moderate cadence and disables background-grid fields. The `plottableFields` filter is optional, but is useful for reducing file size when a case contains many material, damage, cohesive, or diagnostic fields.

Listing 3.41: Minimal VTK/ParaView plot output.

```
pfw["outputType"] = "vtk"
pfw["plotInterval"] = endTime / 100.0
pfw["restartInterval"] = endTime / 10.0
pfw["plotGridFields"] = 0

# Optional direct solver field filter for smaller VTK files.
pfw["plottableFields"] = [
    "particleCenter",
    "particleVelocity",
    "particleStress",
    "particleDamage",
]
```

A minimal Silo setup is similar, except that Silo-specific grid-field requests are left unset or explicitly disabled. This is the smallest VisIt-oriented plot configuration and is appropriate when grid-contact diagnostics are not needed.

Listing 3.42: Minimal Silo/VisIt plot output.

```
pfw["outputType"] = "silo"
pfw["plotInterval"] = endTime / 100.0
pfw["restartInterval"] = endTime / 10.0

# No optional Silo grid diagnostics.
pfw["siloGridFields"] = False
```

3.10.5 Diagnostic grid-output examples

For VTK, enable grid plotting in the solver and include the desired grid wrappers in the field filter. The following example is useful for debugging momentum transfer, boundary reactions, contact forces, and DFG or explicit-surface contact reconstruction.

Listing 3.43: VTK output with selected particle and grid diagnostics.

```
pfw["outputType"] = "vtk"
pfw["plotInterval"] = endTime / 200.0
pfw["restartInterval"] = endTime / 20.0
pfw["plotGridFields"] = 1

pfw["plottableFields"] = [
    # Particle fields.
    "particleCenter",
```

```
    "particleVelocity",
    "particleStress",
    "particleDamage",
    "particleVolume",

    # Background-grid transfer and force diagnostics.
    "gridMass",
    "gridVelocity",
    "gridMomentum",
    "gridInternalForce",
    "gridExternalForce",
    "gridContactForce",

    # DFG/contact-surface diagnostics.
    "gridDamage",
    "gridDamageGradient",
    "gridSurfaceNormal",
    "gridSurfacePosition",
]
```

For Silo, request additional grid fields through the Silo aliases. The compact diagnostic preset is adequate for many cases where the goal is to verify mass mapping, velocity mapping, internal force balance, and the DFG direction.

Listing 3.44: Silo output with the common grid-diagnostic preset.

```
pfw["outputType"] = "silo"
pfw["plotInterval"] = endTime / 200.0
pfw["restartInterval"] = endTime / 20.0

# Expands to gridMass, gridVelocity, gridInternalForce,
# and gridDamageGradient.
pfw["siloGridFields"] = "common"
```

For contact, cohesive-zone, or boundary-condition debugging, use an explicit list rather than `"all"`. This keeps files smaller and makes the intent of the input easier to review.

Listing 3.45: Silo output with explicit contact/cohesive grid diagnostics.

```
pfw["outputType"] = "silo"
pfw["plotInterval"] = endTime / 200.0

pfw["gridFieldNames"] = [
    "gridMass",
    "gridVelocity",
    "gridInternalForce",
    "gridExternalForce",
    "gridContactForce",
    "gridDamage",
    "gridDamageGradient",
    "gridSurfaceNormal",
    "gridSurfacePosition",
    "gridCohesiveForce",
    "gridCohesiveArea",
]
```

The most aggressive Silo diagnostic mode is

Listing 3.46: One-line request for all PFW-known Silo grid fields.

```
pfw["siloGridFields"] = "all"
```

Use this only for short debugging runs or single-output snapshots, because it requests every grid field known to the PFW preset table.

3.10.6 Practical output guidance

For routine V&V or parameter studies, prefer particle-only VTK or Silo output plus the CSV diagnostics from Section 3.9. Increase the plot cadence only for short windows of interest, because plot-file writes can dominate wall time and storage. For contact and boundary-condition debugging, enable grid diagnostics for a small number of output frames and include only the grid fields needed to answer the question. For CPDI/domain-scaling visualization, `pfw["plotUnscaledParticles"] = 1` can make particle-center fields easier to interpret when scaled domains overlap visually.

3.11 Event controls

This section describes how a PFW input file writes solver events. The solver-side algorithms and required/optional event inputs are documented in Section 2.6, and the generated schema-derived event catalogue is in Appendix B. The PFW control layer is intentionally simple: users enable events with `pfw["useEvents"] = 1` and place XML event child nodes in `pfw["mpmEventsString"]`. PFW writes the surrounding `<MPMEvents>` block inside the generated `SolidMechanics_MPM` solver block.

Use current event attributes, `startTime` and `endTime`, in new inputs. Older PFW examples sometimes include an explicit `<MPMEvents>` wrapper or use `time` and `interval`; the current PFW normalization layer accepts many of those legacy snippets, strips the wrapper, and maps legacy attributes to current event names. New files should avoid that compatibility path so that the generated GEOS XML is explicit and easy to review.

Listing 3.47: General PFW event block pattern.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<InitializeStress
  startTime="0.0"
  endTime="1.0e-6"
  targetRegion="all"
  pressure="10.0e6"/>
"""
```

Events are evaluated by the solver event manager near the beginning of a time step, before the particle-to-grid transfer sequence, except for the late `TransformParticles` pass described in Section 2.6.19. Multiple event nodes may be placed in one string. Event order in the input should follow the intended physical sequence, especially when two events share the same activation time and modify related state.

3.11.1 Examples by event type

The following fragments show the typical PFW input form for every event type registered in the reviewed source snapshot. They are intentionally minimal: a production input usually combines these snippets with the boundary controls in Section 3.8, the solver controls in Section 3.7, the material assignments in Section 3.6, and the output/diagnostic controls in Sections 3.9 and 3.10.

3.11.1.1 `Anneal`

`Anneal` acts on a particle region and relaxes the deviatoric part of the stored material stress history while preserving the hydrostatic component. It is most often used after a consolidation or compaction stage to remove residual shear stress before a subsequent loading branch.

Listing 3.48: Anneal event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<Anneal
  startTime="1.0e-3"
  endTime="1.2e-3"
  targetRegion="all"/>
"""
```

3.11.1.2 `BodyForceUpdate`

`BodyForceUpdate` overwrites the solver-level body-force vector at the event time. The updated vector is then used by later particle-to-grid body-force projection.

Listing 3.49: Body-force update event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<BodyForceUpdate
  startTime="0.0"
  bodyForce="{0.0, -9.81, 0.0}"/>
"""
```

3.11.1.3 `BoreholePressure`

`BoreholePressure` writes a pressure-like background stress in a circular borehole region aligned with the z axis. The pressure is ramped between the start and end values using `interpType`: `0` for linear, `1` for cosine, and `2` for smoothstep interpolation.

Listing 3.50: Borehole-pressure event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<BoreholePressure
  startTime="0.0"
  endTime="2.0e-4"
  boreholeRadius="0.015"
  startPressure="5.0e6"
  endPressure="25.0e6"
  interpType="1"/>
"""
```

3.11.1.4 `CohesiveZone`

`CohesiveZone` creates cohesive-zone regions at run time and associates each region with a cohesive constitutive model and a cohesive-zone tag. New inputs should use explicit arrays for `regionNames`, `constitutiveModels`, and `czTags`; the arrays must have identical length. The surface-normal and surface-position options are described in Sections 2.10 and 2.11.

Listing 3.51: Cohesive-zone insertion event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<CohesiveZone
  startTime="0.0"
  endTime="1.0"
  regionNames="{czInterface}"
  constitutiveModels="{czLaw}"
  czTags="{1}"
  czVolumeNormalization="1"
  computeNormalsAndPositions="1"
  normalsAndPositionsMethod="LogisticRegression"
  czSurfaceDisplacementUpdate="TypeB"/>
"""
```

3.11.1.5 `ConfiningPressure`

`ConfiningPressure` applies a virtual confining pressure over a rectangular box. It is commonly paired with `InitializeStress` so that particles begin with a compatible hydrostatic stress and the boundary/background pressure is then held or ramped over a short start-up interval.

Listing 3.52: Confining-pressure event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<InitializeStress
  startTime="0.0"
  endTime="1.0e-6"
  targetRegion="all"
  pressure="10.0e6"/>
<ConfiningPressure
  startTime="0.0"
  endTime="1.0e-3"
  confiningPressureBoxMin="{-0.02, -0.02, -0.001}"
  confiningPressureBoxMax="{ 0.02,  0.02,  0.001}"
  startPressure="10.0e6"
  endPressure="10.0e6"
  interpType="1"/>
"""
```

3.11.1.6 `CrystalHeal`

`CrystalHeal` performs a crystal-oriented healing operation on a target particle region. The `healType` input is an integer selector used by the solver. In current source, `0` is the default branch, while other integer values activate more restrictive or specialized healing choices; users should pair this event with a verification problem before using a new `healType` in production.

Listing 3.53: Crystal-healing event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<CrystalHeal
  startTime="1.20e-3"
  endTime="1.25e-3"
  targetRegion="all"
```

```
  healType="0"/>
"""
```

3.11.1.7 DeformationUpdate

`DeformationUpdate` switches prescribed-deformation and stress-control modes during a run. Use it when one stage of a simulation is driven by an F-table or boundary F-table and a later stage should change to a stress-control mode, or vice versa. The actual deformation table and stress-control gains are solver/boundary inputs described in Section 3.8; the event changes which mode is active.

Listing 3.54: Deformation-control update event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<DeformationUpdate
  startTime="0.0"
  endTime="1.0e-3"
  prescribedFTable="1"
  stressControl="{0, 0, 0}"/>
<DeformationUpdate
  startTime="1.0e-3"
  endTime="2.0e-3"
  prescribedFTable="1"
  stressControl="{1, 0, 1}"/>
"""
```

3.11.1.8 FrictionCoefficientSwap

`FrictionCoefficientSwap` changes the scalar friction coefficient and/or the group-pair friction table for later contact calculations. Use a scalar coefficient for a uniform Coulomb coefficient and a table when different contact-group pairs should have different coefficients. See Section 2.11.11 for the group-pair lookup convention.

Listing 3.55: Friction-coefficient swap event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<FrictionCoefficientSwap
  startTime="5.0e-4"
  frictionCoefficient="0.10"/>
<FrictionCoefficientSwap
  startTime="1.0e-3"
  frictionCoefficientTable="{ {0.00, 0.30}, {0.30, 0.05} }"/>
"""
```

3.11.1.9 Heal

`Heal` resets selected damage-related particle state in a target region. It is the generic healing event, distinct from the polymer- and crystal-specific variants. It is useful for staged calculations where a preparation step intentionally removes damage before the next loading stage.

Listing 3.56: Generic heal event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
```

```
<Heal
  startTime="1.20e-3"
  endTime="1.25e-3"
  targetRegion="all"/>
"""
```

3.11.1.10 `InitializeStress`

`InitializeStress` initializes the stress state of a particle region to a hydrostatic pressure. This is typically used in geomechanics, periodic-cell preparation, and pre-stressed packing examples before the main load path begins.

Listing 3.57: Hydrostatic stress initialization event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<InitializeStress
  startTime="0.0"
  endTime="1.0e-6"
  targetRegion="all"
  pressure="10.0e6"/>
"""
```

3.11.1.11 `InsertPeriodicContactSurfaces`

`InsertPeriodicContactSurfaces` inserts contact-capable surfaces on periodic boundaries. It is useful after a densification or healing stage when the subsequent loading branch should treat lateral periodic interfaces as contact surfaces rather than purely topological periodic copies.

Listing 3.58: Periodic-contact-surface insertion event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<InsertPeriodicContactSurfaces
  startTime="1.25e-3"/>
"""
```

3.11.1.12 `MachineSample`

`MachineSample` deletes particles to create a test-fixture shape from an initially simpler particle set. Current options include dogbone-style, Brazilian-disk-style, and cylindrical machining branches; the relevant optional dimensions depend on `sampleType`.

Listing 3.59: Sample-machining events.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<MachineSample
  startTime="0.0"
  sampleType="dogbone"
  gaugeLength="0.020"
  gaugeRadius="0.003"
  filletRadius="0.002"/>
<MachineSample
  startTime="0.0"
```

```
  sampleType="brazilianDisk"
  diskRadius="0.010"/>
"""
```

3.11.1.13 MaterialSwap

`MaterialSwap` transfers particles from one particle region to another. This changes the constitutive model and particle-region membership while retaining the particle state copied by the solver's swap operation. Define both regions in PFW, even if one begins empty, so the generated XML contains valid source and destination particle regions.

Listing 3.60: Material-region swap event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<MaterialSwap
  startTime="2.0e-4"
  sourceRegion="ParticleRegion1"
  destinationRegion="ParticleRegion2"/>
"""
```

3.11.1.14 PolymerHeal

`PolymerHeal` is the polymer-material healing variant. It acts on the named target region and is usually scheduled after unloading, annealing, or temperature conditioning in polymer verification problems.

Listing 3.61: Polymer-healing event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<PolymerHeal
  startTime="1.20e-3"
  endTime="1.25e-3"
  targetRegion="all"/>
"""
```

3.11.1.15 ResetDeformationGradient

`ResetDeformationGradient` requests one solver cleanup pass in which the deformation gradient of scaled surface particles is reset as described in Section 2.12. It is useful after geometry or contact-surface operations that should restart the local finite-domain particle shape from a well-conditioned state.

Listing 3.62: Reset-deformation-gradient event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<ResetDeformationGradient
  startTime="1.0e-3"/>
"""
```

3.11.1.16 TemperatureProfile

`TemperatureProfile` copies the solver's domain temperature table value to active particles and cohesive-zone points during the active event window. The event itself only needs the time window; the temperature

history is supplied through the solver/domain temperature-table controls used by the thermal material model.

Listing 3.63: Temperature-profile event.

```
pfw["useEvents"] = 1
# The domain temperature table is supplied by the thermal solver controls.
pfw["temperatureTable"] = "{ {0.0, 293.0}, {1.0, 373.0} }"
pfw["mpmEventsString"] = """
<TemperatureProfile
  startTime="0.0"
  endTime="1.0"/>
"""
```

3.11.1.17 `TemperatureRamp`

`TemperatureRamp` is registered as an event input class and takes a start temperature, end temperature, and interpolation type. In the reviewed solver snapshot, the event is not dispatched by the active event loop, so it should be treated as a reserved/future event until a regression test demonstrates run-time execution. Use `TemperatureProfile` for current temperature-table-driven particle temperatures.

Listing 3.64: Registered but currently not dispatched temperature-ramp event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<TemperatureRamp
  startTime="0.0"
  endTime="1.0"
  startTemperature="293.0"
  endTemperature="373.0"
  interpType="2"/>
"""
```

3.11.1.18 `TransformParticles`

`TransformParticles` is handled by a later event pass after the deformation-gradient update. It is used by cohesive-zone and finite-domain particle workflows that need to transform stored particle geometry after an initial configuration has been established.

Listing 3.65: Transform-particles event.

```
pfw["useEvents"] = 1
pfw["mpmEventsString"] = """
<TransformParticles
  startTime="1.0"/>
"""
```

3.11.2 Sequential staged event example

A staged load path often combines boundary-control tables with event controls. The following example sketches a common sequence: compress a sample using the active F-table or moving-boundary controls, anneal the deviatoric stress, heal the damage state, insert lateral periodic contact surfaces, and then unload/reload in tension under uniaxial-stress control. The event block only schedules state changes. The actual compression and tensile strain histories must be supplied by the F-table or boundary controls in Section 3.8.

Listing 3.66: Sequential events for compression, annealing, healing, periodic-contact insertion, and tensile loading.

```
# Stage times.
t_compress_end = 1.00e-3
t_anneal_end   = 1.10e-3
t_heal_end     = 1.20e-3
t_periodic     = 1.20e-3
t_tension_end  = 2.50e-3

# Boundary/F-table controls are defined elsewhere in the pfw input.
# For example, the F-table may compress during [0,t_compress_end]
# and then prescribe axial extension during [t_heal_end,t_tension_end].
pfw["prescribedFTable"] = 1
pfw["stressControl"] = [0, 0, 0]

pfw["useEvents"] = 1
pfw["mpmEventsString"] = f"""
<DeformationUpdate
  startTime="0.0"
  endTime="{t_compress_end}"
  prescribedFTable="1"
  stressControl="{{0, 0, 0}}"/>

<Anneal
  startTime="{t_compress_end}"
  endTime="{t_anneal_end}"
  targetRegion="all"/>

<Heal
  startTime="{t_anneal_end}"
  endTime="{t_heal_end}"
  targetRegion="all"/>

<InsertPeriodicContactSurfaces
  startTime="{t_periodic}"/>

<DeformationUpdate
  startTime="{t_heal_end}"
  endTime="{t_tension_end}"
  prescribedFTable="1"
  stressControl="{{1, 0, 1}}"/>
"""
```

In this sketch the first `DeformationUpdate` leaves all three directions prescribed by the deformation table. The second update keeps the axial deformation history prescribed but turns on stress control in the two lateral directions, producing a uniaxial-stress tensile branch when the boundary/F-table and stress-control parameters are chosen consistently. For polymer workflows, replace `Heal` with `PolymerHeal`; for crystal workflows, replace it with `CrystalHeal` and supply the desired integer `healType`.

3.11.3 Event-control checklist

Before running a staged event input, check the generated XML and verify the following points:

- `pfw["useEvents"] = 1` is set and `pfw["mpmEventsString"]` is not empty.
- The string contains event child nodes only. PFW writes the surrounding `<MPMEvents>` block.

- New inputs use `startTime` and `endTime`; legacy `time`/`interval` snippets have been replaced.
- Region names such as `ParticleRegion1`, `all`, or a custom region name match the generated `ParticleRegion` names.
- Cohesive-zone event arrays `regionNames`, `constitutiveModels`, and `czTags` have the same length, and the named cohesive constitutive models are present in the constitutive block.
- Deformation and stress-control events are consistent with the boundary, D-table, or F-table inputs in Section 3.8.
- Events that modify contact behavior are paired with plot or CSV diagnostics from Sections 3.9 and 3.10 for at least one short verification run.

3.12 Restart controls

Restart output is controlled by a GEOS `Restart` output object and a `PeriodicEvent` that targets that output object. PFW always writes the restart output block

Listing 3.67: GEOS restart output block written by PFW.

```
<Outputs>
  ...
  <Restart name="restartOutput"/>
</Outputs>
```

and writes the associated event

Listing 3.68: GEOS restart event written by PFW.

```
<PeriodicEvent
  name="restart"
  timeFrequency="..."
  target="/Outputs/restartOutput"/>
```

where the time frequency is supplied by `pfw["restartInterval"]`. This plot-independent restart stream is separate from the VTK/Silo output controls in Section 3.10; changing `outputType` does not change the restart format.

A GEOS restart captures the data-repository state of objects and wrappers whose restart flags permit writing. For MPM this includes the particle mesh state, active particle fields needed to continue the calculation, and restart-enabled solver state. Output-only diagnostics and many temporary grid wrappers are intentionally not restart data. After restarting, the particle file is no longer the source of the material state; it is retained by PFW mostly for bookkeeping and because the generated input directory is expected to remain self-contained.

3.12.1 Restart interval and disabling restarts

The primary user control is `restartInterval`. It is a simulation-time interval, not a wall-clock interval. A smaller value gives more recent checkpoints but writes larger and more frequent restart files. A larger value reduces I/O overhead but increases the amount of simulation time that may be lost if the job stops before completion.

For short examples and deterministic verification tests, it is common to disable effective restart writing by setting the restart interval beyond the end time:

Listing 3.69: Effectively disabling restart writes for a short run.

```
stopTime = 1.0e-4
pfw["endTime"] = stopTime
```

```
pfw["plotInterval"] = stopTime / 100.0

# PFW still writes the GEOS Restart block, but this event is never reached
# before the simulation completes.
pfw["restartInterval"] = 2.0 * stopTime
```

For production runs, choose a restart interval based on the cost of losing work and the file-system cost of restart I/O:

Listing 3.70: Periodic restart output during a production run.

```
stopTime = 2.0e-3
pfw["endTime"] = stopTime
pfw["plotInterval"] = stopTime / 200.0

# Keep at most about 1 percent of the simulated time between checkpoints.
pfw["restartInterval"] = stopTime / 100.0
```

Restart cadence should usually be coarser than plot cadence for small examples and finer than plot cadence for long production jobs where plots are infrequent. For contact, cohesive-zone, damage, or machining calculations, restart files can be large because the particle state is large; verify restart-write cost in a short run before using very frequent checkpoints on a large calculation.

3.12.2 Manual continuation from a known restart

PFW can generate a continuation run from a previous restart with the `runContinuation` controls. In this mode PFW skips particle-file generation, copies the selected restart directory and root file into the new run directory, and adds the GEOS command-line restart flag `-r`. The selected restart is constructed from the previous run-directory name and a zero-padded cycle number:

$$\texttt{mpm\_}\langle\texttt{restartJobDir name}\rangle\texttt{\_restart\_}\langle\texttt{cycle number}\rangle. \tag{3.16}$$

Listing 3.71: Manual continuation from a previous PFW run directory.

```
pfw["runContinuation"] = True
pfw["restartJobDir"] = "/p/lustre1/user/runs/240603123456_myCase"
pfw["restartCycleNum"] = 350000

# In continuation mode PFW sets generateParticleFile to False internally.
# The new run may extend endTime, change output cadence, or add later events.
pfw["endTime"] = 4.0e-3
pfw["plotInterval"] = 2.0e-5
pfw["restartInterval"] = 1.0e-5
```

When using manual continuation, keep the mesh partitioning controls consistent with the original run: `xpar`, `ypar`, `zpar`, mesh dimensions, material-region names, and solver layout should match the restart. Changing output frequency, diagnostic flags, or later event schedules is usually safe; changing geometry, particle density, contact-group meaning, or constitutive-model ordering is not a restart continuation and should be treated as a new calculation.

3.12.3 Automatic restart with `pfw_check`

The PFW automatic-restart path is a scheduler wrapper around the same GEOS restart files. A submitted job writes periodic restarts. When `pfw["autoRestart"] = True` and `pfw["mSubmitJobs"] = True`, PFW submits a second Slurm job that runs `pfw_check.py` after the GEOS job finishes. The

check script scans the Slurm output. If it sees `Job complete`, it stops. If it sees `Job exited early`, `TIME LIMIT`, or `NODE FAILURE`, it finds the most recent `*_restart_*` directory, submits a restart job with `-r`, and schedules another check job.

Listing 3.72: Automatic Slurm restart workflow.

```
pfw["mSubmitJobs"] = True
pfw["autoRestart"] = True
pfw["mWallTime"] = "02:00:00"
pfw["runCheckTime"] = "00:02:00"
pfw["mPartition"] = "pdebug"      # or a production partition
pfw["mBank"] = "my_bank"

# Restart cadence is still simulation-time based.
pfw["restartInterval"] = stopTime / 100.0
```

The current PFW auto-restart wrapper is Slurm-oriented. The reviewed PFW script prints that auto-restart is not supported for the Flux scheduler path and does not submit the check job in that case. The suite runner can set the same option from the command line with `-auto-restart`; internally it appends `pfw['autoRestart'] = True` to the generated suite input.

The automatic path depends on log messages and file naming. If a job exits for a reason not recognized by `pfw_check.py`, the checker reports an unknown interruption and stops rather than blindly resubmitting. If the job stops before any restart file has been written, the checker cannot continue and reports that the restart interval should be reduced.

3.12.4 Stopping before wall time ends

PFW writes a GEOS `HaltEvent` so that a submitted job exits before the scheduler kills it at the batch wall-time limit. The input is not a user-written event in `mpmEventsString`; it is generated automatically from the batch controls:

Listing 3.73: PFW-generated wall-clock halt event.

```
<HaltEvent
  maxRuntime="..."/>
```

The `maxRuntime` is computed from `pfw["mWallTime"]` and `pfw["lastRestartBufferInSeconds"]`. The intent is to leave enough wall-clock time for GEOS to exit cleanly, flush logs, and allow the automatic restart checker to detect `Job exited early`. A typical production setting is

Listing 3.74: Wall-time buffer for a restartable batch job.

```
pfw["mWallTime"] = "08:00:00"
pfw["lastRestartBufferInSeconds"] = 120
pfw["autoRestart"] = True
pfw["mSubmitJobs"] = True
```

The wall-clock halt does not itself force a new restart file at the instant the job stops. It only prevents scheduler termination from interrupting GEOS abruptly. To have a recent checkpoint available when the halt event fires, choose `restartInterval` small enough that at least one restart is expected during the wall-time allocation. For jobs with rapidly changing time step or expensive output, test a short run and estimate both the simulated time advanced per wall-clock hour and the time required to write a restart. Then choose the restart interval and buffer so that

1. a valid restart is written well before the halt event can fire;
2. the buffer is longer than the observed restart-write and shutdown time; and
3. the checkpoint frequency does not dominate the total run time or overwhelm the file system.

For very long campaigns, a robust pattern is to set `restartInterval` to a small fraction of the simulated time expected in one batch allocation and set `lastRestartBufferInSeconds` to a conservative I/O buffer. If a checkpoint must be taken at a precise physical time, use `restartInterval` or a segmented run plan so that the desired time is an integer multiple of the restart cadence; the current PFW path does not provide a separate wall-clock-triggered restart writer.

3.12.5 Restart-control checklist

Before using restart in a production run, verify the following in the generated XML and run directory:

- The `Events` block contains a `PeriodicEvent` named `restart` targeting `/Outputs/restartOutput`.
- `restartInterval` is less than `endTime` when restart files are desired, and greater than `endTime` only when restart output is intentionally disabled.
- A short smoke run produces a restart directory and matching `.root` file that can be read by GEOS with `-r`.
- Manual continuation uses the same mesh partitioning and compatible constitutive, contact, and particle-field definitions as the original run.
- Automatic restart is only enabled on scheduler paths that support the generated `pfw_check.py` job.
- The wall-time buffer is large enough for clean shutdown and does not assume that `HaltEvent` will force an additional restart write.

Chapter 4

Post-processing

4.1 Output families

GEOS-MPM cases commonly produce plot files, restart files, PFW logs, XML input, particle files, scheduler scripts, and optional CSV histories. Plot output is selected with the PFW `outputType` key or directly in XML.

4.2 VisIt

Silo output is the standard VisIt pathway. Open the `output_*` file set and add mesh plots for the grid or particles, or pseudocolor/vector plots for particle and requested grid variables. The suite workflow includes `pfw_visit_render.py`, a command-line renderer for initial/final frames after a case has produced Silo or VTK output.

Listing 4.1: VisIt batch rendering helper from the suite workflow.

```
visit -nowin -cli -s pfw_visit_render.py \
  --run-dir /p/lustre1/$USER/geos-mpm-suite-runs/verification/Ftable/elasticBlockUni \
  --variable Damage \
  --view xy
```

4.3 ParaView

VTK output is the ParaView pathway. Configure and build GEOS with VTK output enabled, then set `pfw["outputType"] = "vtk"`. ParaView can load the generated VTK file sequence for particle and mesh variables. Use VTK output for ParaView-oriented exploration; use Silo output for workflows that require the current VisIt scripts or Silo-native grouped vector variables.

4.4 Analysis scripts

The PFW tree contains suite-level and case-specific analysis scripts. They include suite post-processing/reporting, rendered-frame generation, verification/validation plotting, reaction-history plots, material-model checks, and example-specific post-processors. Appendix G lists the discovered Python and shell scripts in the current archive.

4.5 CSV histories

Reaction, box-average, profile, and tracer histories are intentionally simple CSV outputs. They are useful for quick Python, MATLAB, Julia, or shell-based checks and for suite status scans. Because file names and headers are controlled by the solver, downstream scripts should validate header names rather than rely only on column positions.

The reaction file `reactionHistory.csv` has the header `time, F00, F11, F22, length_x, length_y, length_z, Rx-, Rx+, Ry-, Ry+, Rz-, Rz+, L00, L11, L22`. The reaction columns are integrated face forces in the solver force units. To compare to a measured or analytical stress, divide the appropriate reaction by the actual material area of the loaded boundary. The PFW controls that enable this file and select the velocity-correction or internal-force reaction measure are described in Section 3.9.1.

The box-average file `boxAverageHistory.csv` reports volume- or mass-weighted aggregate quantities over the selected averaging box. Profile files use the pattern `profile_<direction>_<variable>.csv`. Tracer files use the configured `tracerOutputPrefix` and write `t,x,y,z` followed by the requested particle fields for the selected particle ID.

Chapter 5

Linked reports and suite organization

5.1 Verification report

The source archive includes a generated verification report:

Verification suite report PDF

The verification suite currently spans boundary conditions, prescribed deformation/F-tables, annealing, ceramic damage, cohesive zones, contact, expanding bars/rings, geomechanics, periodic boundaries, polymer cohesive-zone behavior, shrinkage, size effects, stress control, temperature tables, triply periodic geometry, and von Mises material behavior. Appendix F lists the discovered input cases.

5.2 Validation report

The source archive includes a generated validation report:

Validation suite report PDF

The validation cases in the reviewed archive are organized under geomechanics and polymer families. Appendix F lists the discovered input cases.

5.3 Examples report

The source archive includes a generated examples report:

Examples suite report PDF

The examples include STL import, borehole collapse, Brazilian disk variants, colliding disks, foam compression, elastic disk, PDC-like examples, powder compression, and other demonstration cases.

5.4 LLNL-specific material models and test suites

The manual source bundle contains a placeholder linked report:

LLNL-specific material models and test suites placeholder

Replace this file with the eventual out-of-tree/internal material-model report. The build pathway for these models should remain documented through `GEOS_EXTERNAL_CONSTITUTIVE_MODELS_DIR` and the external registration hook.

Chapter 6

Maintaining and extending the documentation

6.1 LaTeX source organization

The manual source is organized so narrative chapters and generated tables can evolve independently:

- `geos_mpm_manual.tex`: top-level LaTeX driver.
- `sections/*.tex`: human-maintained narrative chapters.
- `generated/*.tex`: code-derived appendices.
- `generated/geos_mpm_extracted.json`: intermediate structured extraction data.
- `tools/update_generated_tables.py`: extraction script copied from the generation run; refresh this script as the source layout changes.
- `linked_reports/*.pdf`: reports linked from the manual but maintained separately.

6.2 Regenerating the PDF

Run the following from the manual source directory:

Listing 6.1: Manual build commands.

```
python3 make_manual.py
pdflatex -interaction=nonstopmode geos_mpm_manual.tex
makeindex geos_mpm_manual
pdflatex -interaction=nonstopmode geos_mpm_manual.tex
pdflatex -interaction=nonstopmode geos_mpm_manual.tex
```

6.3 Keeping tables consistent with code

The generated content is intended to be refreshed from a current checkout. A robust production workflow should:

1. run the extraction script against the checkout being documented;
2. fail CI if the generated JSON or generated LaTeX/RST changes are not committed;
3. build the PDF and Sphinx HTML/PDF outputs;
4. render key pages to images to catch clipped tables, broken glyphs, or missing links;
5. publish linked suite reports as versioned artifacts.

6.4 Sphinx migration path

The reviewed repository already includes schema-to-RST documentation machinery under `src/coreComponents/schema/docs` and `scripts/SchemaToRSTDocumentation.py`, plus a Read the Docs configuration. To migrate this manual online:

- keep the chapter hierarchy in `src/docs/sphinx/mpm/`;
- emit generated tables to both LaTeX and RST or MyST from the same JSON source;
- use Sphinx `toctree` pages matching this manual: getting started, theory, PFW, post-processing, linked reports, generated API/reference appendices, and index;
- preserve hyperlinks to generated verification, validation, example, and LLNL-specific reports as downloadable artifacts;
- cross-link solver attributes to schema-generated pages and source-file references where possible.

Chapter 7

References and source inventory

This manual cites the reviewed codebase directly and uses the following MPM literature as the initial formal reference set for Chapter 2. The list emphasizes the requested Homel-Brannon-Guilkey, Nairn, Bardenhagen, and Sulsky literature families. Future versions should move these entries to BibTeX or BibLaTeX so the LaTeX report and later Sphinx documentation can share one reference database.

Bibliography

[1] D. Sulsky, Z. Chen, and H. L. Schreyer. A particle method for history-dependent materials. *Computer Methods in Applied Mechanics and Engineering*, 118(1–2):179–196, 1994. https://doi.org/10.1016/0045-7825(94)90112-0.

[2] S. G. Bardenhagen and E. M. Kober. The generalized interpolation material point method. *Computer Modeling in Engineering & Sciences*, 5(6):477–496, 2004. https://doi.org/10.3970/cmes.2004.005.477.

[3] A. Sadeghirad, R. M. Brannon, and J. Burghardt. A convected particle domain interpolation technique to extend applicability of the material point method for problems involving massive deformations. *International Journal for Numerical Methods in Engineering*, 86(12):1435–1456, 2011. https://doi.org/10.1002/nme.3110.

[4] A. Sadeghirad, R. M. Brannon, and J. E. Guilkey. Second-order convected particle domain interpolation (CPDI2) with enrichment for weak discontinuities at material interfaces. *International Journal for Numerical Methods in Engineering*, 95(11):928–952, 2013. https://doi.org/10.1002/nme.4526.

[5] M. A. Homel, R. M. Brannon, and J. E. Guilkey. Controlling the onset of numerical fracture in parallelized implementations of the material point method with convective particle domain interpolation domain scaling. *International Journal for Numerical Methods in Engineering*, 107(1):31–48, 2016. https://doi.org/10.1002/nme.5151.

[6] M. A. Homel and E. B. Herbold. Field-gradient partitioning for fracture and frictional contact in the material point method. *International Journal for Numerical Methods in Engineering*, 109(7), 2016. https://doi.org/10.1002/nme.5317.

[7] J. A. Nairn. Material point method calculations with explicit cracks. *Computer Modeling in Engineering & Sciences*, 4(6):649–663, 2003. https://doi.org/10.3970/cmes.2003.004.649.

[8] S. G. Bardenhagen, J. U. Brackbill, and D. Sulsky. The material-point method for granular materials. *Computer Methods in Applied Mechanics and Engineering*, 187(3–4):529–541, 2000. https://doi.org/10.1016/S0045-7825(99)00338-2.

[9] S. G. Bardenhagen, J. E. Guilkey, K. M. Roessig, J. U. Brackbill, W. M. Witzel, and J. C. Foster. An improved contact algorithm for the material point method and application to stress propagation in granular material. *Computer Modeling in Engineering & Sciences*, 2(4):509–522, 2001. https://doi.org/10.3970/cmes.2001.002.509.

[10] J. A. Nairn, C. C. Hammerquist, and G. D. Smith. New material point method contact algorithms for improved accuracy, large-deformation problems, and proper null-space filtering. *Computer Methods in Applied Mechanics and Engineering*, 362:112859, 2020. https://doi.org/10.1016/j.cma.2020.112859.

[11] P. C. Wallstedt and J. E. Guilkey. Improved velocity projection for the material point method. *Computer Modeling in Engineering & Sciences*, 19(3):223–232, 2007.

[12] P. C. Wallstedt and J. E. Guilkey. An evaluation of explicit time integration schemes for use with the generalized interpolation material point method. *Journal of Computational Physics*, 227(22):9628–9642, 2008.

[13] M. Steffen, R. M. Kirby, and M. Berzins. Analysis and reduction of quadrature errors in the material point method. *International Journal for Numerical Methods in Engineering*, 76(6):922–948, 2008. https://doi.org/10.1002/nme.2360.

[14] M. Steffen, P. C. Wallstedt, J. E. Guilkey, R. M. Kirby, and M. Berzins. Examination and analysis of implementation choices within the material point method. *Computer Modeling in Engineering & Sciences*, 31(2):107–128, 2008. https://doi.org/10.3970/cmes.2008.031.107.

[15] J. E. Guilkey and J. A. Weiss. Implicit time integration for the material point method: quantitative and algorithmic comparisons with the finite element method. *International Journal for Numerical Methods in Engineering*, 57:1323–1338, 2003. https://doi.org/10.1002/nme.729.

[16] C. C. Hammerquist and J. A. Nairn. A new method for material point method particle updates that reduces noise and enhances stability. *Computer Methods in Applied Mechanics and Engineering*, 318:724–738, 2017. https://doi.org/10.1016/j.cma.2017.01.035.

[17] J. A. Nairn and C. C. Hammerquist. Material point method simulations using an approximate full mass matrix inverse. *Computer Methods in Applied Mechanics and Engineering*, 377:113667, 2021. https://doi.org/10.1016/j.cma.2021.113667.

[18] J. A. Nairn. Improved implementation of approximate full mass matrix inverse methods into material point method simulations. arXiv:2604.07307, 2026. https://doi.org/10.48550/arXiv.2604.07307.

[19] D. Sulsky, S.-J. Zhou, and H. L. Schreyer. Application of a particle-in-cell method to solid mechanics. *Computer Physics Communications*, 87(1–2):236–252, 1995. https://doi.org/10.1016/0010-4655(94)00170-7.

[20] J. U. Brackbill, D. B. Kothe, and H. M. Ruppel. FLIP: a low-dissipation, particle-in-cell method for fluid flow. *Computer Physics Communications*, 48:25–38, 1988. https://doi.org/10.1016/0010-4655(88)90020-3.

[21] Y. Gan, Z. Sun, Z. Chen, X. Zhang, and Y. Liu. Enhancement of the material point method using B-spline basis functions. *International Journal for Numerical Methods in Engineering*, 113(3):411–431, 2018. https://doi.org/10.1002/nme.5620.

[22] R. B. Leavy, J. E. Guilkey, B. R. Phung, A. D. Spear, and R. M. Brannon. A convected-particle tetrahedron interpolation technique in the material-point method for the mesoscale modeling of ceramics. *Computational Mechanics*, 64, 2019. https://doi.org/10.1007/s00466-019-01670-x.

[23] C. M. Crook and M. A. Homel. A cohesive zone treatment for the material point method involving problems of large deformation and damage. *Computer Methods in Applied Mechanics and Engineering*, 448:118399, 2026. https://doi.org/10.1016/j.cma.2025.118399.

[24] X.-P. Xu and A. Needleman. Numerical simulations of fast crack growth in brittle solids. *Journal of the Mechanics and Physics of Solids*, 42(9):1397–1434, 1994. https://doi.org/10.1016/0022-5096(94)90003-5.

[25] W. T. Read and W. Shockley. Dislocation models of crystal grain boundaries. *Physical Review*, 78:275–289, 1950. https://doi.org/10.1103/PhysRev.78.275.

[26] F. H. Harlow. The particle-in-cell computing method for fluid dynamics. In *Methods in Computational Physics*, volume 3, pages 319–343. Academic Press, 1964.

[27] W. Weibull. A statistical distribution function of wide applicability. *Journal of Applied Mechanics*, 18:293–297, 1951.

[28] A. A. Griffith. The phenomena of rupture and flow in solids. *Philosophical Transactions of the Royal Society of London. Series A*, 221:163–198, 1921.

[29] G. R. Irwin. Analysis of stresses and strains near the end of a crack traversing a plate. *Journal of Applied Mechanics*, 24:361–364, 1957.

[30] M. A. Homel, J. E. Guilkey, and R. M. Brannon. Mesoscale validation of simplifying assumptions for modeling the plastic deformation of fluid-saturated porous material. *Journal of Dynamic Behavior of Materials*, 3:23–44, 2017. https://doi.org/10.1007/s40870-017-0092-8.

[31] M. A. Homel and E. B. Herbold. Mesoscale modeling of porous materials using new methodology for fracture and frictional contact in the material point method. In *Dynamic Behavior of Materials, Volume 1*, Conference Proceedings of the Society for Experimental Mechanics Series, pages 97–102. Springer, 2017. https://doi.org/10.1007/978-3-319-62956-8_17.

[32] M. A. Homel, C. M. Crook, and J. Appleton. Emergent size effects in continuum damage modeling of brittle fracture with under-resolved crack-tip stresses. Manuscript in preparation, 2026.

[33] M. A. Homel, J. Iyer, S. J. Semnani, and E. B. Herbold. Mesoscale model and X-ray computed micro-tomographic imaging of damage progression in ultra-high-performance concrete. *Cement and Concrete Research*, 157:106799, 2022. https://doi.org/10.1016/j.cemconres.2022.106799.

[34] L. E. Malvern. *Introduction to the Mechanics of a Continuous Medium*. Prentice-Hall, 1969.

[35] R. W. Ogden. *Non-Linear Elastic Deformations*. Dover Publications, 1997. Originally published by Ellis Horwood, 1984.

[36] G. A. Holzapfel. *Nonlinear Solid Mechanics: A Continuum Approach for Engineering*. Wiley, 2000.

[37] S. G. Lekhnitskii. *Theory of Elasticity of an Anisotropic Elastic Body*. Holden-Day, 1963.

[38] J. C. Simo and T. J. R. Hughes. *Computational Inelasticity*. Springer, 1998. https://doi.org/10.1007/b98904.

[39] P. J. Roache. *Verification and Validation in Computational Science and Engineering*. Hermosa Publishers, 1998.

[40] M. C. Boyce, D. M. Parks, and A. S. Argon. Large inelastic deformation of glassy polymers. Part I: rate dependent constitutive model. *Mechanics of Materials*, 7(1):15–33, 1988. https://doi.org/10.1016/0167-6636(88)90003-8.

[41] E. M. Arruda and M. C. Boyce. A three-dimensional constitutive model for the large stretch behavior of rubber elastic materials. *Journal of the Mechanics and Physics of Solids*, 41(2):389–412, 1993. https://doi.org/10.1016/0022-5096(93)90013-6.

[42] M. Cervera and M. Chiumenti. Mesh objective tensile cracking via a local continuum damage model and a crack tracking technique. *Computer Methods in Applied Mechanics and Engineering*, 196(1–3):304–320, 2006. https://doi.org/10.1016/j.cma.2006.04.008.

[43] R. I. Borja. *Plasticity: Modeling and Computation*. Springer, 2013. https://doi.org/10.1007/978-3-642-38547-6.

[44] R. M. Brannon, A. F. Fossum, and O. E. Strack. *Kayenta: Theory and User's Guide*. Sandia National Laboratories Report SAND2009-2282, 2009.

[45] M. A. Homel, A. Sadeghirad, D. Austin, J. Colovos, R. M. Brannon, and J. E. Guilkey. *ARENISCA: Theory and User's Guide*. University of Utah, 2014.

[46] M. A. Homel, J. E. Guilkey, and R. M. Brannon. Numerical solution for plasticity models using consistency bisection and a transformed-space closest-point return: a nongradient solution method. *Computational Mechanics*, accepted manuscript, 2015.

[47] M. G. Malenda, M. A. Homel, W. M. Kibikas, C. Choens, E. Shalev, and V. Lyakhovsky. A continuum-scale approach to predicting wellbore stability in Ghareb chalk for safe nuclear waste disposal. In *Proceedings of the 59th U.S. Rock Mechanics/Geomechanics Symposium*, ARMA 25-340, 2025.

[48] R. M. Brannon. *Rotation, Reflection, and Frame Changes with Applications in Computational Continuum Mechanics*. University of Utah, 2017.

7.1 Code source inventory

The source inventory in Table 7.1 lists the principal paths reviewed or extracted from the current archive.

Table 7.1: Primary source files and directories used to generate this manual.

Component	Path
MPM solver	`src/coreComponents/physicsSolvers/solidMechanics/SolidMechanicsMPM.[ch]pp`
MPM kernels	`src/coreComponents/physicsSolvers/solidMechanics/kernels/ExplicitMPM.hpp`
MPM solver fields	`src/coreComponents/physicsSolvers/solidMechanics/MPMSolverFields.hpp`
MPM events	`src/coreComponents/events/mpmEvents/*`
Particle mesh and regions	`src/coreComponents/mesh/Particle*.{hpp,cpp}, src/coreComponents/mesh/particleGenerators/*`
Schema-generated docs	`src/coreComponents/schema/docs/SolidMechanics_MPM.rst and related MPM docs`
ParticleFileWriter	`scripts/preProcessing/materialPointMethodParticleFileWriter/particleFileWriter.py`
PFW geometry objects	`scripts/preProcessing/materialPointMethodParticleFileWriter/pfw_geometryObjects.py`
PFW material library	`scripts/preProcessing/materialPointMethodParticleFileWriter/pfw_materials.py`
PFW suite tools	`scripts/preProcessing/materialPointMethodParticleFileWriter/pfw_suite.py and mpm_vv_*.py`

Continued on next page

Component	Path
Canonical XML examples	`inputFiles/materialPointMethod/*`

Chapter A

Generated solver and schema reference

This appendix is generated from solver wrapper registrations and schema-generated RST files. It should be regenerated whenever input wrappers, defaults, or schema documentation change.

Table A.1: Solver enumerations discovered in SolidMechanicsMPM.hpp.

Enumeration	Values
`TimeIntegrationOption`	QuasiStatic, ImplicitDynamic, ExplicitDynamic
`UpdateMethodOption`	FLIP, PIC, XPIC, FMPM
`BoundaryConditionOption`	Outflow, Symmetry, Moving, Contact
`SurfaceFlag`	Interior, FullyDamaged, Surface, Cohesive, DamagedCohesive
`InterpolationOption`	Linear, Cosine, Smoothstep
`ContactNormalTypeOption`	Difference, MassWeighted, LargerMass, Mixed, Aligned, LogisticRegression
`ContactGapCorrectionOption`	Simple, Implicit, Softened
`AreaIntegrationOption`	BruteForce, Mesh
`OverlapCorrectionOption`	Off, NormalForce, SPH, Volume
`CohesiveLawOption`	Uncoupled, NeedlemanXu, Polymer
`CohesiveSurfaceDisplacementUpdateOption`	TypeA, TypeB
`GPUSchemeOption`	Atomics, NoAtomics, RandomMix, MinimalAtomics, Reduction, Colors
`NormalsAndPositionsMethodOption`	LogisticRegression, DFGAndVolumeIntegration

Table A.2: User-facing SolidMechanics_MPM wrapper registrations from the current source.

Attribute	Input	Default	Description
`FSubcycles`	`OPTIONAL`	`m_FSubcycles`	Number of sub cycles to more accurately integrate the deformation gradient
`LBar`	`OPTIONAL`	`m_LBar`	Used to choose the Lbar method. 0 = do nothing, 1 = run trD through the SPH kernel, 2 = straight-up volume average
`LBarScale`	`OPTIONAL`	`m_LBarScale`	Scales the contribution of volume-averaged trD to particle L, alleviates checkerboarding
`LRtolerance`	`OPTIONAL`	`m_LRtolerance`	Tolerance for logistic regression calculations
`areaIntegrationMethod`	`OPTIONAL`	`m_areaIntegrationMethod`	Method for performing nodal area integration
`artificialViscosityQ0`	`OPTIONAL`	`m_artificialViscosityQ0`	Artificial viscosity q0 parameter
`artificialViscosityQ1`	`OPTIONAL`	`m_artificialViscosityQ1`	Artificial viscosity q1 parameter
`bcTable`	`OPTIONAL`	-	Array that stores time-dependent bc types on x-, x+, y-, y+, z- and z+ faces.
`binSizeMultiplier`	`OPTIONAL`	`m_binSizeMultiplier`	Multiplier for setting bin size, used to speed up particle neighbor sorting
`bodyForce`	`OPTIONAL`	-	Array that stores uniform body force
`boundaryConditionTypes`	`OPTIONAL`	-	Boundary conditions on x-, x+, y-, y+, z- and z+ faces. Options are:; * ; *
`boundaryFaceCoefficientsOfRestitution`	`OPTIONAL`	-	Boundary face coefficients of restitution for BC type = 3 on x-, x+, y-, y+, z- and z+ faces.
`boundaryFaceFrictionCoefficients`	`OPTIONAL`	-	Boundary face fricition coefficients for BC type = 3 on x-, x+, y-, y+, z- and z+ faces.

Continued on next page

Attribute	Input	Default	Description
boxAverageHistory	OPTIONAL	m_boxAverageHistory	Flag for whether to output box average history
boxAverageMax	OPTIONAL	-	Maximum corner position of box average
boxAverageMin	OPTIONAL	-	Minimum corner position of box average
boxAverageResizeWithDomain	OPTIONAL	m_boxAverageResizeWithDomain	Flag for whether to output box average integration domain updates with domainL
boxAverageWriteInterval	OPTIONAL	m_boxAverageWriteInterval	Interval between writing box averages to files
computeInternalEnergyAndTemperature	OPTIONAL	m_computeInternalEnergyAnd Temperature	Deprecated compatibility flag. MPM particle thermal state is updated during the constitutive stress update.
computeNodalArea	OPTIONAL	m_computeNodalArea	Option to compute nodal area from grid surface normals and positions (results may be inaccurate if implicit surface normals and positiosn are used)
computeParticleSurfaceNormalsAnd Positions	OPTIONAL	m_computeParticleSurfaceNormals AndPositions	Flag to compute particle surface normals and positions
computeSPHJacobian	OPTIONAL	m_computeSPHJacobian	Overlap correction flag (0=off, 1=increase normal force to remove gap. (not fully developed), 2=compute the SPH Jacobian and use it to scale particle density to improve the overlap.)
contactGapCorrection	OPTIONAL	m_contactGapCorrection	Flag for mitigating contact gaps
contactNormalExponent	OPTIONAL	m_contactNormalExponent	Exponent value for surface normal weights in contact normal type aligned
contactNormalType	OPTIONAL	m_contactNormalType	Flag for contact normal type
cpdiDomainScaling	OPTIONAL	m_cpdiDomainScaling	Option for CPDI domain scaling
crackTipDetectionThreshold	OPTIONAL	m_crackTipDetectionThreshold	Set threshold for crack-tip detection
damageFieldPartitioning	OPTIONAL	m_damageFieldPartitioning	Flag for using the gradient of the particle damage field to partition material into separate velocity fields
directionalOverlapCorrection	OPTIONAL	m_directionalOverlapCorrection	Flag for mitigating pile-up of particles at contact interfaces
disableSurfaceNormalsAndPositionsOn CPDIScaling	OPTIONAL	m_disableSurfaceNormalsAnd PositionsOnCPDIScaling	Option for disabling explicit surface normals and positions when a particle has severely deformed
disableSurfaceNormalsAndPositionsOn Damage	OPTIONAL	m_disableSurfaceNormalsAnd PositionsOnDamage	Option for disabling explicit surface normals and positions when a particle has been severely damaged
enableContact	OPTIONAL	m_enableContact	Flag to enable contact between velocity fields
enablePrescribedBoundaryTransverse Velocities	OPTIONAL	-	Array storing a flag to enable transverse velocity boundary conditions for each face of domain
enableSurfaceTension	OPTIONAL	m_enableSurfaceTension	Flag to enable surface tension forces on particles
exactJIntegration	OPTIONAL	m_exactJIntegration	Will force integration of F to have an exact integral of J.
explicitSurfaceNormalInfluence	OPTIONAL	m_explicitSurfaceNormalInfluence	Determines the relative weighting for implicit and explicit surface normals
fTable	OPTIONAL	-	Array that stores time-dependent grid-aligned stretches interpreted as a gloabl background grid F read from the XML file.
fTableInterpType	OPTIONAL	m_fTableInterpType	The type of F table interpolation. Options are 0 (linear), 1 (cosine), 2 (quintic polynomial).
frictionCoefficient	OPTIONAL	m_frictionCoefficient	Coefficient of friction, currently assumed to be the same everywhere
frictionCoefficientTable	OPTIONAL	-	Friction coefficient table for different groups
generalizedVortexMMS	OPTIONAL	m_generalizedVortexMMS	Option for CPDI domain scaling
gpuScheme	OPTIONAL	m_gpuScheme	GPU Scheme (Atomics, No atomics, Random shuffling, Minimal atomics)
logMomentum	OPTIONAL	-	Flag for logging momentum balance.
logStartCycle	OPTIONAL	-	Cycle to start logging output for debugging
maxLRIterations	OPTIONAL	m_maxLRIterations	Maximum number of iterations for logistic regression calculations
maxParticleJacobian	OPTIONAL	m_maxParticleJacobian	Jacobian value above which particles are deleted
maxParticleVelocity	OPTIONAL	m_maxParticleVelocity	Velocity above which particles are deleted
minParticleJacobian	OPTIONAL	m_minParticleJacobian	Jacobian value below which particles are deleted
needsNeighborList	OPTIONAL	m_needsNeighborList	Flag for whether to construct neighbor list
needsNodalNeighborList	OPTIONAL	m_needsNodalNeighborList	Flag for whether to construct nodal neighbor list
neighborRadius	OPTIONAL	m_neighborRadius	Neighbor radius for SPH-type calculations
normalAndPositionMethod	OPTIONAL	m_normalAndPositionMethod	Method for computing particle surface normals and positions
numSurfaceIntegrationPoints	OPTIONAL	m_numSurfaceIntegrationPoints	Number of surface integration points for cohesive grid nodes
overlapCorrection	OPTIONAL	m_overlapCorrection	Overlap correction flag (0=off, 1=increase normal force to remove gap. (not fully developed), 2=compute the SPH Jacobian and use it to scale particle density to improve the overlap.)
overlapThreshold1	OPTIONAL	m_overlapThreshold1	Overlap correction threshold 1
overlapThreshold2	OPTIONAL	m_overlapThreshold2	Overlap correction threshold 2
overwriteExistingNormalsAndPositions	OPTIONAL	m_overwriteExistingNormalsAnd Positions	Flag to that determines if existing particle normals and positions are overriden
particleDataWriteInterval	OPTIONAL	m_particleDataWriteInterval	Interval to write particle data to file
planeStrain	OPTIONAL	m_planeStrain	Flag for performing plane strain calculations
plotGridFields	OPTIONAL	m_plotGridFields	Flag to enable plotting of grid fields in vtk output
plotUnscaledParticles	OPTIONAL	m_plotUnscaledParticles	Flag for plotting particles without CPDI domain scaling
plottableFields	OPTIONAL	-	If specified, determines which fields are written to vtk. Helpful for reducing the vtk output file sizes
prescribedBcTable	OPTIONAL	m_prescribedBcTable	Flag for whether to have time-dependent boundary condition types
prescribedBoundaryFTable	OPTIONAL	-	Flag for whether to have time-dependent boundary conditions described by a global background grid F
prescribedBoundaryTransverse Velocities	OPTIONAL	-	Array storing the transverse velocity boundary conditions for each face of domain

Continued on next page

Attribute	Input	Default	Description
prescribedFTable	OPTIONAL	-	Flag for whether to have time-dependent superimposed velocity gradient for triply periodic simulations
preventCZInterpenetration	OPTIONAL	m_preventCZInterpenetration	Flag to use contact normal forces instead of CZ forces in compression
profileCycleInterval	OPTIONAL	m_profileCycleInterval	Cycle interval for MPM profile output. Use either profileCycleInterval or profileWriteInterval. A value of 0 means no cycle-based interval.
profileDirection	OPTIONAL	m_profileDirection	Direction for MPM profile output. Valid values are x, y, and z.
profileHistory	OPTIONAL	m_profileHistory	Flag to enable MPM profile CSV output. One CSV file is written per requested profile variable.
profileNumSlices	OPTIONAL	m_profileNumSlices	Number of profile slices. A value of 0 uses the background-grid resolution in profileDirection.
profileVariables	OPTIONAL	-	Variables to write to MPM profile CSV files. Supported names include density, damage, temperature, internalEnergy, kineticEnergy, velocityX, velocityY, velocityZ, volumeFraction, plasticStrainMagnitude, stressXX, stressYY, stressZZ, stressYZ, stressXZ, stressXY, and plasticStrainXX/YY/ZZ/YZ/XZ/XY.
profileWriteInterval	OPTIONAL	m_profileWriteInterval	Time interval for MPM profile output. Use either profileWriteInterval or profileCycleInterval. A value of 0 means no time-based interval.
reactionHistory	OPTIONAL	0	Flag for whether to output face reaction history
reactionWriteInterval	OPTIONAL	m_reactionWriteInterval	Interval between writing reactions to files
resetDefGradForFullyDamagedParticles	OPTIONAL	m_resetDefGradForFullyDamaged Particles	Flag for resetting deformation gradient of fully damaged particles
resetDefGradForScaledSurfaceParticles	OPTIONAL	m_resetDefGradForScaledSurface Particles	Flag for resetting deformation gradient of CPDI scaled surface particles
separabilityMinDamage	OPTIONAL	m_separabilityMinDamage	Damage threshold for field separability
setDomainTemperature	OPTIONAL	-	Flag that activates domain temperature interpolation from table.
setDomainTemperatureRate	OPTIONAL	-	Flag that activates domain temperature rateinterpolation from table.
smallMass	OPTIONAL	m_smallMass	Optional small mass threshold for ignoring extremely low-mass nodes. If omitted, an automatic threshold is computed from the global minimum particle mass.
solverProfiling	OPTIONAL	-	Flag for timing subroutines in the solver
stressControl	OPTIONAL	-	Flag for whether stress control using box averages is enabled
stressControlKd	OPTIONAL	-	Derivative gain of stress PID controller
stressControlKi	OPTIONAL	-	Integral gain of stress PID controller
stressControlKp	OPTIONAL	-	Proportional gain of stress PID controller
stressTable	OPTIONAL	-	Array that stores the time-depended grid aligned stresses
stressTableInterpType	OPTIONAL	m_stressTableInterpType	The type of stress table interpolation. Options are 0 (linear), 1 (cosine), 2 (quintic polynomial).
subdivideParticles	OPTIONAL	m_subdivideParticles	Option for splitting particles when they span more than a grid cell (prevents numerical fracture)
surfaceDetection	OPTIONAL	m_surfaceDetection	Flag for automatic surface detection on the 1st cycle
surfaceNormalAndPositionDamage Threshold	OPTIONAL	m_surfaceNormalAndPositionDamage Threshold	Sets the damage threshold at which explicit particle surface normals and positions are disabled
surfaceQualityThreshold	OPTIONAL	m_surfaceQualityThreshold	Particle-Grid DFG alignment threshold for separability
surfaceTensionCoefficient	OPTIONAL	m_surfaceTensionCoefficient	Stores the value of the surface tension coefficient
temperatureTable	OPTIONAL	-	Array that stores the time-dependent domain temperature
temperatureTableInterpType	OPTIONAL	m_temperatureTableInterpType	The type of temperature table interpolation. Options are 0 (linear), 1 (cosine), 2 (quintic polynomial).
thinFeatureDFGThreshold	OPTIONAL	m_thinFeatureDFGThreshold	Threshold to treat relatively thin (compared to grid spacing) damaged material to avoid spurious slip surfaces
timeIntegrationOption	OPTIONAL	m_timeIntegrationOption	Time integration method. Options are:; * ; *
totalBinderVolume	OPTIONAL	m_totalBinderVolume	Total volume of binder
tracerCoordinates	OPTIONAL	-	Coordinates used to initialize MPM lagrangian tracers. Each row is x,y,z; each tracer follows the nearest particle selected during initialization.
tracerCycleInterval	OPTIONAL	m_tracerCycleInterval	Cycle interval for MPM lagrangian tracer CSV output. Use either tracerCycleInterval or tracerWriteInterval. A value of 0 means no cycle-based interval.
tracerHistory	OPTIONAL	m_tracerHistory	Flag to enable MPM lagrangian tracer CSV output. One CSV file is written per tracer point.
tracerOutputPrefix	OPTIONAL	m_tracerOutputPrefix	Filename prefix for MPM lagrangian tracer CSV files.
tracerVariables	OPTIONAL	-	Particle variables to write to each MPM lagrangian tracer CSV file. Supported names include particleID, mass, volume, materialType, porosity, density, damage, temperature, internalEnergy, kineticEnergy, speed, velocityX, velocityY, velocityZ, plasticStrainMagnitude, stressXX, stressYY, stressZZ, stressYZ, stressXZ, stressXY, and plasticStrainXX/YY/ZZ/YZ/XZ/XY.
tracerWriteInterval	OPTIONAL	m_tracerWriteInterval	Time interval for MPM lagrangian tracer CSV output. Use either tracerWriteInterval or tracerCycleInterval. A value of 0 means no time-based interval.
treatFullyDamagedAsSingleField	OPTIONAL	m_treatFullyDamagedAsSingleField	Whether to consolidate fully damaged fields into a single field. Nice for modeling damaged mush.
updateMethod	OPTIONAL	m_updateMethod	Update method. Options are:; * ; *
updateOrder	OPTIONAL	m_updateOrder	Order for update method, only applies to XPIC and FMPM
useAPIC	OPTIONAL	m_useAPIC	Will use APIC in particle to grid mapping of momentum
useArtificialViscosity	OPTIONAL	m_useArtificialViscosity	Flag to enable artificial viscosity

Continued on next page

Attribute	Input	Default	Description
useCrackTipDetection	OPTIONAL	m_useCrackTipDetection	Set to activate crack-tip detection and distance calc
useDamageAsSurfaceFlag	OPTIONAL	m_useDamageAsSurfaceFlag	Indicates whether particle damage at the beginning of the simulation should be interpreted as a surface flag
useEvents	OPTIONAL	m_useEvents	Enable events
useInteralForceAsFaceReaction	OPTIONAL	m_useInteralForceAsFaceReaction	Will use internal force component at boundary node as reaction
useNodePosForArea	OPTIONAL	m_useNodePosForArea	Option to compute nodal area from grid surface normals and positions (results may be inaccurate if implicit surface normals and positiosn are used)
useReferenceVectorsForParticleUpdate	OPTIONAL	m_useReferenceVectorsForParticle Update	Flag for determining the integration scheme of particle vector attributes
useSurfacePositionForContact	OPTIONAL	m_useSurfacePositionForContact	Flag for to use particle mapped surface positions in contact calculations
writeParticleData	OPTIONAL	m_writeParticleData	Flag to enable writing particle data to file

Internal/non-input wrappers. The following registered wrappers were marked with `InputFlags::FALSE` in the reviewed source and are therefore treated as internal state rather than direct input controls: boreholeRadius, boreholeStress, confiningPressureBoxMax, confiningPressureBoxMin, confiningStress, domainExtent, domainF, domainL, domainStress, domainTemperature, domainTemperatureRate, elementSize, enableBoreholePressure, enableConfiningPressure, globalFaceReactions, globalMaximum, globalMinimum, localMaximum, localMaximumNoGhost, localMinimum, localMinimumNoGhost, m_ijkMap, maxNodalNeighbors, maxParticleVelocitySquared, nextBoxAverageWriteTime, nextParticleDataWriteTime, nextProfileWriteTime, nextReactionWriteTime, nextTracerWriteTime, numContactFlags, numContactGroups, numDims, numElements, numVelocityFields, numberOfSubRegions, partitionExtent, plottableFieldsSorted, stressControlITerm, stressControlLastError, tracerParticleIDs.

A.1 Schema-generated reference tables

The following tables reproduce the existing schema-generated documentation snippets found for MPM-specific schema groups.

Table A.3: Schema documentation table for SolidMechanics_MPM.

Name	Type	Default	Description
boundaryConditionTypes	integer_array	{0}	| Boundary conditions on x-, x+, y-, y+, z- and z+ faces. Options are:
boxAverageHistory	integer	0	Flag for whether to output box average history
cflFactor	real64	0.5	Factor to apply to the ‘CFL condition <http://en.wikipedia.org/wiki/Courant-Friedrichs-Lewy_condition>‘_ when calculating the maximum allowable time step. Values should be in the interval (0,1]
contactGapCorrection	integer	0	Flag for mitigating contact gaps
cpdiDomainScaling	integer	0	Option for CPDI domain scaling
damageFieldPartitioning	integer	0	Flag for using the gradient of the particle damage field to partition material into separate velocity fields
discretization	groupNameRef	required	Name of discretization object (defined in the :ref:‘NumericalMethodsManager‘) to use for this solver. For instance, if this is a Finite Element Solver, the name of a :ref:‘FiniteElement‘ should be specified. If this is a Finite Volume Method, the name of a :ref:‘FiniteVolume‘ discretization should be specified.
fTableInterpType	integer	0	The type of F table interpolation. Options are 0 (linear), 1 (cosine), 2 (quintic polynomial).
fTablePath	path	Path to f-table	
frictionCoefficient	real64	0	Coefficient of friction, currently assumed to be the same everywhere
initialDt	real64	1e+99	Initial time-step value required by the solver to the event manager.

Continued on next page

Name	Type	Default	Description
logLevel	integer	0	Log level
name	groupName	required	A name is required for any non-unique nodes
needsNeighborList	integer	0	Flag for whether to construct neighbor list
neighborRadius	real64	-1	Neighbor radius for SPH-type calculations
planeStrain	integer	0	Flag for performing plane strain calculations
prescribedBcTable	integer	0	Flag for whether to have time-dependent boundary condition types
prescribedBoundary FTable	integer	0	Flag for whether to have time-dependent boundary conditions described by a global background grid F
profileDirection	string	x	Direction for MPM profile output. Valid values are x, y, and z.
profileCycleInterval	integer	0	Cycle interval for MPM profile output. Use either profileCycleInterval or profileWriteInterval. A value of 0 means no cycle-based interval.
profileHistory	integer	0	Flag to enable MPM profile CSV output. One CSV file is written per requested profile variable.
profileNumSlices	integer	0	Number of profile slices. A value of 0 uses the background-grid resolution in profileDirection.
profileVariables	string_array	{}	Variables to write to MPM profile CSV files. Supported names include density, damage, temperature, internalEnergy, kineticEnergy, velocityX, velocityY, velocityZ, volumeFraction, plasticStrainMagnitude, stressXX, stressYY, stressZZ, stressYZ, stressXZ, stressXY, and plasticStrainXX/YY/ZZ/YZ/XZ/XY.
profileWriteInterval	real64	0	Time interval for MPM profile output. Use either profileWriteInterval or profileCycleInterval. A value of 0 means no time-based interval.
reactionHistory	integer	0	Flag for whether to output face reaction history
separabilityMinDamage	real64	0.5	Damage threshold for field separability
solverProfiling	integer	0	Flag for timing subroutines in the solver
surfaceDetection	integer	0	Flag for automatic surface detection on the 1st cycle
targetRegions	groupNameRef_array	required	Allowable regions that the solver may be applied to. Note that this does not indicate that the solver will be applied to these regions, only that allocation will occur such that the solver may be applied to these regions. The decision about what regions this solver will beapplied to rests in the EventManager.
treatFullyDamagedAs SingleField integer	1	Whether to consolidate fully damaged fields into a s...	
useDamageAsSurfaceFlag	integer	0	Indicates whether particle damage at the beginning of the simulation should be interpreted as a surface flag
writeLinearSystem	integer	0	Write matrix, rhs, solution to screen (= 1) or file (= 2).
LinearSolverParameters	node	unique	:ref:‘XML_LinearSolverParameters‘
NonlinearSolver Parameters	node	unique	:ref:‘XML_NonlinearSolverParameters‘

Table A.4: Schema documentation table for ParticleMesh.

Name	Type	Default	Description
headerFile	path	required path to the header file	
name	groupName	required A name is required for any non-unique nodes	
particleFile	path	required path to the particle file	

Table A.5: Schema documentation table for ParticleRegion.

Name	Type	Default	Description
meshBody	string	Mesh body that contains this region	
name	groupName	required A name is required for any non-unique nodes	

Table A.6: Schema documentation table for MPMEvents.

Name	Type	Default	Description

Chapter B

Generated MPM events catalogue

Table B.1: MPM event classes discovered in the current source.

Event	Class	Description	Source
Anneal	AnnealMPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/AnnealMPMEvent.hpp
BodyForceUpdate	BodyForceUpdateMPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/BodyForceUpdateMPMEvent.hpp
BoreholePressure	BoreholePressureMPMEvent	This class implements a virtual fluid boundary condition for a region in the X-Y plane defined by a borehole radius.	src/coreComponents/events/mpmEvents/BoreholePressureMPMEvent.hpp
CohesiveZone	CohesiveZoneMPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/CohesiveZoneMPMEvent.hpp
ConfiningPressure	ConfiningPressureMPMEvent	This class implements a virtual fluid boundary condition for a region in the X-Y plane defined by a borehole radius.	src/coreComponents/events/mpmEvents/ConfiningPressureMPMEvent.hpp
CrystalHeal	CrystalHealMPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/CrystalHealMPMEvent.hpp
DeformationUpdate	DeformationUpdateMPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/DeformationUpdateMPMEvent.hpp
FrictionCoefficientSwap	FrictionCoefficientSwapMPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/FrictionCoefficientSwapMPMEvent.hpp
Heal	HealMPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/HealMPMEvent.hpp
InitializeStress	InitializeStressMPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/InitializeStressMPMEvent.hpp
InsertPeriodicContactSurfaces	InsertPeriodicContactSurfacesMPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/InsertPeriodicContactSurfacesMPMEv hpp

Continued on next page

Event	Class	Description	Source
MachineSample	MachineSampleMPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/MachineSampleMPMEvent.hpp
MaterialSwap	MaterialSwapMPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/MaterialSwapMPMEvent.hpp
PolymerHeal	PolymerHealMPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/PolymerHealMPMEvent.hpp
ResetDeformation Gradient	ResetDeformation GradientMPMEvent	This class resets the deformation gradient of cpid scaled particles regardless of damage	src/coreComponents/events/mpmEvents/ResetDeformationGradientMPMEvent.hpp
TemperatureProfile	TemperatureProfile MPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/TemperatureProfileMPMEvent.hpp
TemperatureRamp	TemperatureRampMPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/TemperatureRampMPMEvent.hpp
TransformParticles	TransformParticles MPMEvent	This class implements the material swap mpm event for the solid mechanics material point method solver	src/coreComponents/events/mpmEvents/TransformParticlesMPMEvent.hpp

Table B.2: MPM event attributes, excluding internal non-input state.

Event	Attribute	Input	Default	Description
Anneal	startTime	REQUIRED	-	Time at which event starts
Anneal	endTime	OPTIONAL	DBL_MAX	Time at which event ends
Anneal	targetRegion	REQUIRED	-	Particle region to perform anneal on
BodyForceUpdate	startTime	REQUIRED	-	Time at which event starts
BodyForceUpdate	endTime	OPTIONAL	DBL_MAX	Time at which event ends
BodyForceUpdate	bodyForce	OPTIONAL	-	Body force vector
BoreholePressure	startTime	REQUIRED	-	Time at which event starts
BoreholePressure	endTime	OPTIONAL	DBL_MAX	Time at which event ends
BoreholePressure	boreholeRadius	REQUIRED	-	Starting temperature to ramp from
BoreholePressure	startPressure	REQUIRED	-	Starting temperature to ramp from
BoreholePressure	endPressure	REQUIRED	-	End temperature to ramp to
BoreholePressure	interpType	OPTIONAL	m_interpType	Interpolation scheme: 0 (Linear), 1 (Cosine), 2 (Smooth-step)
CohesiveZone	startTime	REQUIRED	-	Time at which event starts
CohesiveZone	endTime	OPTIONAL	DBL_MAX	Time at which event ends
CohesiveZone	regionNames	REQUIRED	-	Region names for cohesive zones
CohesiveZone	constitutiveModels	REQUIRED	-	Constitutive model names for cohesive zones
CohesiveZone	czTags	REQUIRED	-	Tag IDs for cohesive zones
CohesiveZone	czVolume Normalization	OPTIONAL	m_czVolume Normalization	Flag to perform volume normalization of cohesive nodal area for partially filled grid cells
CohesiveZone	computeNormalsAnd Positions	OPTIONAL	m_computeNormals AndPositions	Flag to automicatlly compute particle surface normals and positions
CohesiveZone	normalsAndPositions Method	OPTIONAL	m_normalsAnd PositionsMethod	Method for computing particle surface normals and positions
CohesiveZone	czSurface DisplacementUpdate	OPTIONAL	m_czSurface Displacement Update	Cohesive surface displacement update method. TypeA uses the stored surface-position vector; TypeB uses the CPDI particle-face vector. Options are:; * ; *

Continued on next page

Event	Attribute	Input	Default	Description
ConfiningPressure	startTime	REQUIRED	-	Time at which event starts
ConfiningPressure	endTime	OPTIONAL	DBL_MAX	Time at which event ends
ConfiningPressure	confiningPressure BoxMin	REQUIRED	-	Min corner of box defining confining pressure region
ConfiningPressure	confiningPressure BoxMax	REQUIRED	-	Max corner of box defining confining pressure region
ConfiningPressure	startPressure	REQUIRED	-	Starting pressure to ramp from
ConfiningPressure	endPressure	REQUIRED	-	End pressure to ramp to
ConfiningPressure	interpType	OPTIONAL	m_interpType	Interpolation scheme: 0 (Linear), 1 (Cosine), 2 (Smooth-step)
CrystalHeal	startTime	REQUIRED	-	Time at which event starts
CrystalHeal	endTime	OPTIONAL	DBL_MAX	Time at which event ends
CrystalHeal	targetRegion	REQUIRED	-	Particle region to perform heal on
CrystalHeal	healType	REQUIRED	-	Type of heal algorithm to perform
DeformationUpdate	startTime	REQUIRED	-	Time at which event starts
DeformationUpdate	endTime	OPTIONAL	DBL_MAX	Time at which event ends
DeformationUpdate	prescribedFTable	OPTIONAL	0	Flag to enable prescribed F table
DeformationUpdate	prescribedFTable	OPTIONAL	0	Flag to enable prescribed boundary F table
DeformationUpdate	stressControl	OPTIONAL	-	Flags to enable stress control in x, y and z directions
FrictionCoefficient Swap	startTime	REQUIRED	-	Time at which event starts
FrictionCoefficient Swap	endTime	OPTIONAL	DBL_MAX	Time at which event ends
FrictionCoefficient Swap	frictionCoefficient	OPTIONAL	-1	Coefficient of friction, currently assumed to be the same everywhere
FrictionCoefficient Swap	frictionCoefficient Table	OPTIONAL	-	Friction coefficient table for different groups
Heal	startTime	REQUIRED	-	Time at which event starts
Heal	endTime	OPTIONAL	DBL_MAX	Time at which event ends
Heal	targetRegion	REQUIRED	-	Particle region to perform heal on
InitializeStress	startTime	REQUIRED	-	Time at which event starts
InitializeStress	endTime	OPTIONAL	DBL_MAX	Time at which event ends
InitializeStress	pressure	REQUIRED	-	Starting temperature to ramp from
InitializeStress	targetRegion	REQUIRED	-	Particle region to perform anneal on
InsertPeriodic ContactSurfaces	startTime	REQUIRED	-	Time at which event starts
InsertPeriodic ContactSurfaces	endTime	OPTIONAL	DBL_MAX	Time at which event ends
MachineSample	startTime	REQUIRED	-	Time at which event starts
MachineSample	endTime	OPTIONAL	DBL_MAX	Time at which event ends
MachineSample	sampleType	REQUIRED	-	Type of sample to machine
MachineSample	filletRadius	OPTIONAL	-1	Fillet radius for machining dogbone sample
MachineSample	gaugeLength	OPTIONAL	-1	Gauge length for machining dogbone sample
MachineSample	gaugeRadius	OPTIONAL	-1	Gauge radius for machining dogbone sample
MachineSample	diskRadius	OPTIONAL	-1	Disk radius for machining brazil disk sample
MaterialSwap	startTime	REQUIRED	-	Time at which event starts
MaterialSwap	endTime	OPTIONAL	DBL_MAX	Time at which event ends
MaterialSwap	sourceRegion	REQUIRED	-	Particle region to transfer particles from
MaterialSwap	destinationRegion	REQUIRED	-	Particle region to transfer particles to
PolymerHeal	startTime	REQUIRED	-	Time at which event starts
PolymerHeal	endTime	OPTIONAL	DBL_MAX	Time at which event ends
PolymerHeal	targetRegion	REQUIRED	-	Particle region to perform polymer heal on
ResetDeformation Gradient	startTime	REQUIRED	-	Time at which event starts
ResetDeformation Gradient	endTime	OPTIONAL	DBL_MAX	Time at which event ends
TemperatureProfile	startTime	REQUIRED	-	Time at which event starts
TemperatureProfile	endTime	OPTIONAL	DBL_MAX	Time at which event ends
TemperatureRamp	startTime	REQUIRED	-	Time at which event starts

Continued on next page

Event	Attribute	Input	Default	Description
TemperatureRamp	endTime	OPTIONAL	DBL_MAX	Time at which event ends
TemperatureRamp	startTemperature	REQUIRED	-	Starting temperature to ramp from
TemperatureRamp	endTemperature	REQUIRED	-	End temperature to ramp to
TemperatureRamp	interpType	OPTIONAL	m_interpType	Interpolation scheme: 0 (Linear), 1 (Cosine), 2 (Sigmoid)
TransformParticles	startTime	REQUIRED	-	Time at which event starts
TransformParticles	endTime	OPTIONAL	DBL_MAX	Time at which event ends

Chapter C

Generated ParticleFileWriter reference

Table C.1: ParticleFileWriter parameter keys and defaults from particleFileWriter.py.

PFW key	Default	Emits solver attribute
runDebug	False	no
runCheckTime	'00:02:00'	no
outputType	'vtk'	no
gridFieldNames	[]	no
siloGridFieldNames	None	no
siloGridFields	None	no
generateParticleFile	True	no
runContinuation	False	no
restartJobDir	'.'	no
restartCycleNum	0	no
mCores	1	no
mNodes	1	no
mBank	None	no
mWallTime	'00:30:00'	no
mBatch	True	no
mSubmitJobs	False	no
autoRestart	False	no
mPartition	'pbatch'	no
periodic	[False, False, False]	no
xpar	1	no
ypar	1	no
zpar	1	no
nI	5	no
nJ	5	no
nK	5	no
ppc	2	no
ppcx	None	no
ppcy	None	no
ppcz	None	no
xmin	-0.5	no
xmax	0.5	no
ymin	0.0	no
ymax	1.0	no
zmin	-0.5	no
zmax	0.5	no
lastRestartBufferInSeconds	10	no
objects	None	no

Continued on next page

PFW key	Default	Emits solver attribute
sortObjects	False	no
timeIntegrationOption	None	yes
tracerHistory	None	yes
tracerCoordinates	None	yes
tracerVariables	None	yes
tracerWriteInterval	None	yes
tracerCycleInterval	None	yes
tracerOutputPrefix	None	yes
updateMethod	None	yes
updateOrder	None	yes
cflFactor	None	yes
initialDt	None	yes
solverProfiling	None	yes
cpdiDomainScaling	None	yes
damageFieldPartitioning	None	yes
planeStrain	False	yes
needsNeighborList	None	yes
reactionHistory	None	yes
reactionWriteInterval	None	yes
boxAverageHistory	None	yes
boxAverageWriteInterval	None	yes
prescribedBcTable	None	yes
prescribedFTable	None	yes
prescribedBoundaryFTable	None	yes
fTable	None	yes
fTableInterpType	None	yes
FSubcycles	None	no
resetDefGradForFullyDamagedParticles	None	yes
treatFullyDamagedAsSingleField	None	yes
plotUnscaledParticles	None	yes
contactGapCorrection	None	yes
explicitSurfaceNormalInfluence	None	yes
useSurfacePositionForContact	None	yes
disableSurfaceNormalsAndPositionsOnCPDIScaling	None	yes
separabilityMinDamage	0.5	yes
stressControl	None	yes
stressTable	None	yes
stressControlKp	None	yes
stressControlKi	None	yes
stressControlKd	None	yes
stressTableInterpType	None	yes
boundaryConditionTypes	None	yes
bcTable	None	yes
useEvents	None	yes
bodyForce	None	yes
generalizedVortexMMS	None	yes
frictionCoefficient	None	yes
frictionCoefficientTable	None	yes
frictionCoefficientRuleOfMixtures	None	yes
useDamageAsSurfaceFlag	False	yes
neighborRadius	None	yes
minParticleJacobian	None	yes
maxParticleJacobian	None	yes
logLevel	None	yes
materials	None	no
materialPropertyString	None	no
endTime	1.0	no

Continued on next page

PFW key	Default	Emits solver attribute
plotInterval	None	no
restartInterval	None	no
plotGridFields	None	yes
useSinusoidalDamageField	False	no
wavyCrack	False	no
particleRefinement	None	no
mpmEventsString	"	no
maxParticleVelocity	10.0	yes
cohesiveFieldPartitioning	0	yes
enableCohesiveFailure	0	yes
maxCohesiveNormalStress	0.01	yes
maxCohesiveShearStress	0.01	yes
characteristicNormalDisplacement	0.01	yes
characteristicTangentialDisplacement	0.01	yes
maxCohesiveNormalDisplacement	0.01	yes
maxCohesiveTangentialDisplacement	0.01	yes
prescribedBoundaryTransverseVelocities	[[0.0, 0.0], [0.0, 0.0], [0.0, 0.0], [0.0, 0.0], [0....	yes
particleFileFields	['Velocity', 'MaterialType', 'Contact Group', 'Surfac...	no

C.1 Silo grid field presets

Common preset. gridMass, gridVelocity, gridInternalForce, gridDamageGradient.

All preset. gridCohesiveNode, gridSurfaceMass, gridSurfaceFieldMass, gridExplicitSurfaceNormal, gridPrincipalExplicitSurfaceNormal, gridCohesiveFieldFlag, gridMaxMappedParticleID, gridCohesiveArea, gridCohesiveForce, gridReferenceSurfacePosition, gridReferenceMaterialVolume, gridReferenceAreaVector, gridDisplacement, gridCenterOfVolume, gridParticleMappedSurfaceNormal, gridMass, gridActive, gridMaterialVolume, gridUncontactedVelocity, gridVelocity, gridDVelocity, gridMomentum, gridAcceleration, gridExternalForce, gridSurfaceTensionForce, gridInternalForce, gridContactForce, gridDamage, gridDamageGradient, gridMaxDamage, gridFieldGradientAlignment.

Chapter D

Generated particle-file reference

Table D.1: Particle type enum values in the current source.

Enum value	Particle type
0	SinglePoint
1	SinglePointBSpline
2	CPDI
3	CPTI
4	CPDI2

Table D.2: Particle file field ordering and defaults from ParticleMeshGenerator.

Column	Name	Default	Required if absent
	ID	required	no
	PositionX	required	no
	PositionY	required	no
	PositionZ	required	no
	VelocityX	0.0 if omitted	no
	VelocityY	0.0 if omitted	no
	VelocityZ	0.0 if omitted	no
	MaterialType	0.0 if omitted	no
	ParticleType	2.0 (CPDI) if omitted	no
	ContactGroup	0.0 if omitted	no
	SurfaceFlag	required	no
	Damage	0.0 if omitted	no
	Porosity	0.0 if omitted	no
	Temperature	300.0 if omitted	no
	TemperatureRate	0.0 if omitted	no
	StrengthScale	1.0 if omitted	no
	CZTag	0.0 if omitted	no
	RVectorXX	required	no
	RVectorXY	required	no
	RVectorXZ	required	no
	RVectorYX	required	no
	RVectorYY	required	no
	RVectorYZ	required	no
	RVectorZX	required	no

Continued on next page

Column	Name	Default	Required if absent
	RVectorZY	required	no
	RVectorZZ	required	no
	MaterialDirectionXX	1.0 if omitted	no
	MaterialDirectionXY	0.0 if omitted	no
	MaterialDirectionXZ	0.0 if omitted	no
	MaterialDirectionYX	0.0 if omitted	no
	MaterialDirectionYY	1.0 if omitted	no
	MaterialDirectionYZ	0.0 if omitted	no
	MaterialDirectionZX	0.0 if omitted	no
	MaterialDirectionZY	0.0 if omitted	no
	MaterialDirectionZZ	1.0 if omitted	no
	SurfaceNormalX	0.0 if omitted	no
	SurfaceNormalY	0.0 if omitted	no
	SurfaceNormalZ	0.0 if omitted	no
	SurfacePositionX	0.0 if omitted	no
	SurfacePositionY	0.0 if omitted	no
	SurfacePositionZ	0.0 if omitted	no
	SurfaceTractionX	0.0 if omitted	no
	SurfaceTractionY	0.0 if omitted	no
	SurfaceTractionZ	0.0 if omitted	no

Chapter E

Generated geometry and material catalogue

Table E.1: Geometry objects discovered in pfw_geometryObjects.py.

Geometry object	Docstring summary
`shell`	Spherical shell between inner and outer radii, with surface information on both spherical boundaries.
`sphere`	Sphere defined by center and radius, with surface flags, normals, and surface positions near the boundary.
`sphericalInclusion`	Sphere-like inclusion helper that assigns material/group data while suppressing ordinary surface flagging.
`czSphericalPrill`	Spherical prill with Voronoi-like crystals and cohesive-zone interfaces between neighboring grains.
`explicitBinderSphericalPrill`	Spherical prill with explicit grain and binder regions, mainly for two-dimensional prill/binder mesostructures.
`czCylindricalPrill`	Cylindrical prill with Voronoi-like internal crystals and cohesive-zone surfaces.
`czPrill`	Box-shaped prill with Voronoi crystals, cohesive-zone tags, and configurable bonded/neat surfaces.
`ellipsoid`	Grid-aligned ellipsoid defined by center and three semi-axis lengths.
`crystal`	Faceted crystal analogue defined by center, axis, height, and face-offset controls.
`indentor`	Faceted indentation tool parameterized by number of facets and angle.
`spherical_indenter`	Indentation tool with a spherical tip.
`flatpunch_indenter`	Flat-punch indentation tool with a specified radius and orientation angle.
`simple_tensile_gripper`	Single gripper-claw geometry for micromechanical tensile examples.
`tensile_gripper`	Detailed micromechanical tensile gripper geometry.
`box`	Axis-aligned rectangular solid defined by two opposite corners.
`notchedBar`	Axis-aligned bar with a 45-degree edge notch in the positive-y face.
`box2`	Box variant with mixed corner-normal behavior for contact testing.
`polygon`	Two-dimensional polygon from ordered vertices, typically used in plane strain.
`foam`	Box with user-specified spherical pores.
`poissonDiskFoam`	Box with Poisson-disk sampled pores, including surface-normal and position reconstruction.
`packedSphericalBed`	Packed bed of spherical grains with cell-list queries and per-grain material/group/type data.
`twoMaterialBox`	Box that randomly assigns accepted particles to one of two material indices.
`twoFieldBox`	Legacy/test box intended for two-group or two-field contact testing.
`cone`	Cone defined by two axis points and a base radius.
`stl`	Closed triangulated STL import with ray-casting inside/outside tests and nearest-triangle surface projection.
`cylinder`	Solid or hollow cylinder defined by axis endpoints and radii, with configurable face/wall surface flags.
`pillar`	Micropillar compression specimen with cylindrical gauge region and wider base.

Continued on next page

Geometry object	Docstring summary
pillarBase	Base/substrate companion geometry for pillar simulations.
whiskers	Whisker or fiber-like inclusion geometry.
toroid	Toroidal annulus/ring geometry.
spinodal	Implicit spinodal-like porous or bicontinuous architecture.
tpms	Triply periodic minimal-surface or related periodic architecture object.
polygonInclusions	Two-dimensional polygonal inclusions embedded in a fill group.
fill	Accepts all candidate particle points to fill the background domain.
VCCTL	Voxelized VCCTL-style phase dataset mapped to material indices.
GrainStack	Stacked voxel or labeled-array grain geometry.
CT	Computed-tomography voxel dataset mapped to material indices.
bitmap	Two-dimensional image or label map converted to particles and materials.
porousPolygon	Polygon with randomly generated two-dimensional pores.
union	Boolean union of two geometry objects.
intersection	Boolean intersection of two geometry objects.
difference	Boolean set difference A minus B, including surface data for cut boundaries.

Table E.2: Geometry wrappers discovered in pfw_geometryObjects.py.

Wrapper	Docstring summary
materialDirectionWrapper	Overrides or assigns the local material-direction triad.
surfaceFlagWrapper	Overrides surface-flag response for the wrapped object.
czTagWrapper	Assigns cohesive-zone tags to cohesive surface particles.
functionalDeletionWrapper	Rejects points according to a user-supplied deletion/masking function.
surfaceNormalWrapper	Assigns explicit surface normals.
surfacePositionWrapper	Assigns explicit vectors from particles to surface positions.
shrinkageFlagWrapper	Assigns shrinkage flags for compatible workflows.
strengthScaleWrapper	Assigns strength-scale values.
damageWrapper	Assigns initial damage values.
porosityWrapper	Assigns initial porosity values.
temperatureWrapper	Assigns initial temperature values.
voronoiWrapperWithLayered StrengthScale	Voronoi and layered-strength wrapper for grain-scale heterogeneity.
layeredVoronoiWeibullWrapper	Voronoi-layered Weibull strength-scale wrapper.
voronoiWeibullBoxWrapper	Box-local Voronoi Weibull strength wrapper.
pointwisePorosityWrapper	Random pointwise porosity mask without deterministic pore surfaces.
voronoiMatDirBoxWrapper	Box-local Voronoi material-direction assignment.
sizeEffectVoronoiWeibullBox Wrapper	Size-effect and flaw-field strength wrapper.
surfaceFlagBoxWrapper	Overrides surface flags inside a box.
damageBoxWrapper	Overrides damage inside a box.
pennyShapedCrackWrapper	Paints a penny-shaped damaged/cracked region.
layeredStrengthScaleBoxWrapper	Assigns layered strength-scale fields inside a box.
transform	Applies coordinate transformations before delegating to a sub-object.

Table E.3: Selected geometry utility functions discovered in pfw_geometryObjects.py.

Utility	Docstring summary
maxFlawRadius	
maxFlawRadius2D	
Grid	
Geometry	Base class and interface for PFW geometry objects.
BaseWrapper	Base forwarding wrapper for geometry-object field overrides.
SetOperation	Base class for Boolean geometry operations.

Table E.4: Material helper functions discovered in pfw_materials.py.

Material helper	Docstring summary
BicrystalCohesiveZone	
CeramicDamage	
Chiumenti	
CoupledCohesiveZone	
ElasticIsotropic	
ElasticTransverseIsotropic	
ElasticTransverseIsotropicPressureDependent	
Geomechanics	
Graphite	
Hyperelastic	
HyperelasticMMS	
PerfectlyPlastic	
PolymerCohesiveZone	
StrainHardeningPolymer	
VonMisesJ	

Table E.5: Material preset dictionary counts in pfw_materials.py.

PFW material preset family	Preset count
CeramicDamage	21
Chiumenti	3
ElasticIsotropic	3
ElasticTransverseIsotropic	1
Geomechanics	1
Graphite	3
Hyperelastic	1
HyperelasticMMS	1
PerfectlyPlastic	1
StrainHardeningPolymer	8
VonMisesJ	30

Chapter F

Generated suite case catalogue

Table F.1: Discovered ParticleFileWriter suite input cases. Leading comment-block descriptions are kept in the linked suite reports.

Suite	Family	Case	Input path
examples	benchy	benchy/benchy	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/benchy/pfw_input_benchy.py
examples	borehole	borehole/boreholeCollapse2D_Ghareb_borehole	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/borehole/pfw_input_boreholeCollapse2D_Ghareb_borehole.py
examples	brazilianDisk_BSpline	brazilianDisk_BSpline/brazilianDisk_BSpline	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_BSpline/pfw_input_brazilianDisk_BSpline.py
examples	brazilianDisk_FLIP	brazilianDisk_FLIP/brazilianDisk_FLIP	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_FLIP/pfw_input_brazilianDisk_FLIP.py
examples	brazilianDisk_FMPM	brazilianDisk_FMPM/brazilianDisk_FMPM	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_FMPM/pfw_input_brazilianDisk_FMPM.py
examples	brazilianDisk_PIC	brazilianDisk_PIC/brazilianDisk_PIC	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_PIC/pfw_input_brazilianDisk_PIC.py
examples	brazilianDisk_XPIC	brazilianDisk_XPIC/brazilianDisk_XPIC	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_XPIC/pfw_input_brazilianDisk_XPIC.py
examples	collidingDisks	collidingDisks/collidingDisks	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/collidingDisks/pfw_input_collidingDisks.py
examples	copperFoamCompression	copperFoamCompression/copperFoamCompression	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/copperFoamCompression/pfw_input_copperFoamCompression.py
examples	copperFoamCompression	copperFoamCompression/copperFoamCompression2D	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/copperFoamCompression/pfw_input_copperFoamCompression2D.py
examples	elasticDisk	elasticDisk/elasticDisk	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/elasticDisk/pfw_input_elasticDisk.py
examples	pdc	pdc/pdc	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/pdc/pfw_input_pdc.py

Continued on next page

Suite	Family	Case	Input path
examples	powderCompression	powderCompression/mixedPowderCompression2D	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/powderCompression/pfw_input_mixedPowderCompression2D.py
examples	powderCompression	powderCompression/mixedPowderCompression3D	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/powderCompression/pfw_input_mixedPowderCompression3D.py
verification	Ftable	Ftable/elasticBlockUni	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/Ftable/pfw_input_elasticBlockUni.py
verification	anneal	anneal/annealCompression	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/anneal/pfw_input_annealCompression.py
verification	anneal	anneal/annealTension	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/anneal/pfw_input_annealTension.py
verification	bcCheck	bcCheck/bcCheck	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/bcCheck/pfw_input_bcCheck.py
verification	ceramic	ceramic/ceramicSmoke	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/ceramic/pfw_input_ceramicSmoke.py
verification	ceramic	ceramic/ceramicTXC	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/ceramic/pfw_input_ceramicTXC.py
verification	ceramic	ceramic/ceramicTXE	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/ceramic/pfw_input_ceramicTXE.py
verification	cohesiveZones/bicrystal	cohesiveZones/bicrystal/cz_bicrystal	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/bicrystal/pfw_input_cz_bicrystal.py
verification	cohesiveZones/bicrystal	cohesiveZones/bicrystal/cz_bicrystal_0deg	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/bicrystal/pfw_input_cz_bicrystal_0deg.py
verification	cohesiveZones/bicrystal	cohesiveZones/bicrystal/cz_bicrystal_15deg	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/bicrystal/pfw_input_cz_bicrystal_15deg.py
verification	cohesiveZones/bicrystal	cohesiveZones/bicrystal/cz_bicrystal_45deg	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/bicrystal/pfw_input_cz_bicrystal_45deg.py
verification	cohesiveZones/bicrystal	cohesiveZones/bicrystal/cz_bicrystal_7p5deg	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/bicrystal/pfw_input_cz_bicrystal_7p5deg.py
verification	cohesiveZones/bicrystal	cohesiveZones/bicrystal/cz_bicrystal_90deg	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/bicrystal/pfw_input_cz_bicrystal_90deg.py
verification	cohesiveZones	cohesiveZones/cz_45deg	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/pfw_input_cz_45deg.py
verification	cohesiveZones	cohesiveZones/cz_flip	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/pfw_input_cz_flip.py
verification	cohesiveZones	cohesiveZones/cz_gridAligned	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/pfw_input_cz_gridAligned.py
verification	cohesiveZones	cohesiveZones/cz_overlapStart	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/pfw_input_cz_overlapStart.py
verification	cohesiveZones	cohesiveZones/cz_reinitialize	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/pfw_input_cz_reinitialize.py
verification	cohesiveZones	cohesiveZones/multi_cz	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/pfw_input_multi_cz.py

Continued on next page

Suite	Family	Case	Input path
verification	cohesiveZones	cohesiveZones/shearPeel	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/pfw_input_shearPeel.py
verification	cohesiveZones	cohesiveZones/spinning_control	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/pfw_input_spinning_control.py
verification	cohesiveZones	cohesiveZones/spinning_cz	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/pfw_input_spinning_cz.py
verification	cohesiveZones	cohesiveZones/spinning_cz_matDir	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/pfw_input_spinning_cz_matDir.py
verification	cohesiveZones	cohesiveZones/uniaxial_cylinder_cz	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cohesiveZones/pfw_input_uniaxial_cylinder_cz.py
verification	contact	contact/asymmetricInterface	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/contact/pfw_input_asymmetricInterface.py
verification	contact	contact/contactGapCorrection	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/contact/pfw_input_contactGapCorrection.py
verification	contact	contact/symmetricInterface_explicit	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/contact/pfw_input_symmetricInterface_explicit.py
verification	contact	contact/symmetricInterface_implicit	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/contact/pfw_input_symmetricInterface_implicit.py
verification	contactBC	contactBC/ballSmack	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/contactBC/pfw_input_ballSmack.py
verification	contactBC	contactBC/bouncyBall	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/contactBC/pfw_input_bouncyBall.py
verification	cubic	cubic/elasticCubic_100	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cubic/pfw_input_elasticCubic_100.py
verification	cubic	cubic/elasticCubic_110	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cubic/pfw_input_elasticCubic_110.py
verification	cubic	cubic/elasticCubic_111	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/cubic/pfw_input_elasticCubic_111.py
verification	diskThruPartitions	diskThruPartitions/ballsThruPartitions	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/diskThruPartitions/pfw_input_ballsThruPartitions.py
verification	diskThruPartitions	diskThruPartitions/diskThruPartitions	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/diskThruPartitions/pfw_input_diskThruPartitions.py
verification	diskThruPartitions	diskThruPartitions/diskThruPartitionsMultiField	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/diskThruPartitions/pfw_input_diskThruPartitionsMultiField.py
verification	diskThruPartitions	diskThruPartitions/diskThruSpace	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/diskThruPartitions/pfw_input_diskThruSpace.py
verification	diskThruPartitions	diskThruPartitions/diskThruSpace_rXXXX	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/diskThruPartitions/pfw_input_diskThruSpace_rXXXX.py
verification	expandingBar	expandingBar/expandingBar_chiumenti_noWeibull_sigma0p3_Gf1e-05_v00p1	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/expandingBar/pfw_input_expandingBar_chiumenti_noWeibull_sigma0p3_Gf1e-05_v00p1.py

Continued on next page

Suite	Family	Case	Input path
verification	expandingRing	expandingRing/expandingRing_chiumenti_noWeibull_sigma0p3_Gf1e-05_v00p016	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/expandingRing/pfw_input_expandingRing_chiumenti_noWeibull_sigma0p3_Gf1e-05_v00p016.py
verification	explicitContact	explicitContact/explicitContactTest	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/explicitContact/pfw_input_explicitContactTest.py
verification	explicitContact	explicitContact/extremeDeformation	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/explicitContact/pfw_input_extremeDeformation.py
verification	explicitContact	explicitContact/spinningNormals	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/explicitContact/pfw_input_spinningNormals.py
verification	gas	gas/dilatingDomain	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/gas/pfw_input_dilatingDomain.py
verification	gas	gas/expandingGas	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/gas/pfw_input_expandingGas.py
verification	generalizedVortex	generalizedVortex/generalizedVortexMMS	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/generalizedVortex/pfw_input_generalizedVortexMMS.py
verification	geomechanics	geomechanics/geoModel_FTable	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/geomechanics/pfw_input_geoModel_FTable.py
verification	geomechanics	geomechanics/geoModel_HYD	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/geomechanics/pfw_input_geoModel_HYD.py
verification	geomechanics	geomechanics/geoModel_TXC	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/geomechanics/pfw_input_geoModel_TXC.py
verification	geomechanics	geomechanics/geoModel_Tension	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/geomechanics/pfw_input_geoModel_Tension.py
verification	geomechanics	geomechanics/geoModel_anisotropicBuckling_unconfinedCompression	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/geomechanics/pfw_input_geoModel_anisotropicBuckling_unconfinedCompression.py
verification	geomechanics	geomechanics/geoModel_isotropicBuckling_unconfinedCompression	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/geomechanics/pfw_input_geoModel_isotropicBuckling_unconfinedCompression.py
verification	geomechanics	geomechanics/geoModel_isotropicBuckling_uniaxial	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/geomechanics/pfw_input_geoModel_isotropicBuckling_uniaxial.py
verification	implicitFluid	implicitFluid/fluidPressure	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/implicitFluid/pfw_input_fluidPressure.py
verification	materialSwap	materialSwap/materialSwap	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/materialSwap/pfw_input_materialSwap.py
verification	momentumTest	momentumTest/preStressed4DisksCompressedPBC5x2MomentumLogging	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/momentumTest/pfw_input_preStressed4DisksCompressedPBC5x2MomentumLogging.py
verification	momentumTest	momentumTest/preStressed4DisksPBC5x1MomentumLogging	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/momentumTest/pfw_input_preStressed4DisksPBC5x1MomentumLogging.py

Continued on next page

Suite	Family	Case	Input path
verification	momentumTest	momentumTest/preStressed4 DisksPBC5x2MomentumLogging	scripts/preProcessing/ materialPointMethodParticleFileWriter/ verification/momentumTest/pfw_input_ preStressed4DisksPBC5x2MomentumLogging.py
verification	momentumTest	momentumTest/preStressed MovingBlockPBC5x1Momentum Logging	scripts/preProcessing/ materialPointMethodParticleFileWriter/ verification/momentumTest/pfw_input_ preStressedMovingBlockPBC5x1MomentumLogging.py
verification	momentumTest	momentumTest/preStressed MovingBlockPBC5x3Momentum Logging	scripts/preProcessing/ materialPointMethodParticleFileWriter/ verification/momentumTest/pfw_input_ preStressedMovingBlockPBC5x3MomentumLogging.py
verification	momentumTest	momentumTest/preStressed MovingMultiMatBlockPBC5x3 MomentumLogging	scripts/preProcessing/ materialPointMethodParticleFileWriter/ verification/momentumTest/pfw_input_ preStressedMovingMultiMatBlockPBC5x3MomentumLogging. py
verification	momentumTest	momentumTest/preStressed SingleParticlePBC3x1Momentum Logging	scripts/preProcessing/ materialPointMethodParticleFileWriter/ verification/momentumTest/pfw_input_ preStressedSingleParticlePBC3x1MomentumLogging.py
verification	periodicBoundaries	periodicBoundaries/ballThru PBC	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ periodicBoundaries/pfw_input_ballThruPBC.py
verification	periodicBoundaries	periodicBoundaries/diskThru PBC	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ periodicBoundaries/pfw_input_diskThruPBC.py
verification	polymerCZ	polymerCZ/polymerCZ	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ polymerCZ/pfw_input_polymerCZ.py
verification	polymerHeal	polymerHeal/polymerHeal	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ polymerHeal/pfw_input_polymerHeal.py
verification	shrinkage	shrinkage/pillarCalcination	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ shrinkage/pfw_input_pillarCalcination.py
verification	shrinkage	shrinkage/shrinkingSphere	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ shrinkage/pfw_input_shrinkingSphere.py
verification	sizeEffect	sizeEffect/notchedBar	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ sizeEffect/pfw_input_notchedBar.py
verification	spinningDisk	spinningDisk/spinningDisk_ FLIP	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ spinningDisk/pfw_input_spinningDisk_FLIP.py
verification	spinningDisk	spinningDisk/spinningDisk_ FMPM2	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ spinningDisk/pfw_input_spinningDisk_FMPM2.py
verification	spinningDisk	spinningDisk/spinningDisk_ PIC	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ spinningDisk/pfw_input_spinningDisk_PIC.py
verification	stressControl	stressControl/stressControl	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ stressControl/pfw_input_stressControl.py
verification	stressControl	stressControl/stressControl2	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ stressControl/pfw_input_stressControl2.py
verification	temperatureTable	temperatureTable/geoModel_ tempTable	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ temperatureTable/pfw_input_geoModel_tempTable.py
verification	triplyPeriodic	triplyPeriodic/triply Periodic	scripts/preProcessing/ materialPointMethodParticleFileWriter/verification/ triplyPeriodic/pfw_input_triplyPeriodic.py

Continued on next page

Suite	Family	Case	Input path
verification	vonMisesJ	vonMisesJ/vonMisesJ	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/vonMisesJ/pfw_input_vonMisesJ.py
validation	geomechanics	geomechanics/geoOHCrD_2024_ISR08_V5	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/geomechanics/pfw_input_geoOHCrD_2024_ISR08_V5.py
validation	geomechanics	geomechanics/geomechanicsHydrostaticCreep_HP03	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/geomechanics/pfw_input_geomechanicsHydrostaticCreep_HP03.py
validation	geomechanics	geomechanics/geomechanicsTriaxialCompression_ISR03	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/geomechanics/pfw_input_geomechanicsTriaxialCompression_ISR03.py
validation	geomechanics	geomechanics/geomechanicsTriaxialCompression_TP1	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/geomechanics/pfw_input_geomechanicsTriaxialCompression_TP1.py
validation	geomechanics	geomechanics/geomechanicsTriaxialCompression_TP2	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/geomechanics/pfw_input_geomechanicsTriaxialCompression_TP2.py
validation	geomechanics	geomechanics/geomechanicsTriaxialCompression_TP3	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/geomechanics/pfw_input_geomechanicsTriaxialCompression_TP3.py
validation	geomechanics	geomechanics/geomechanicsTriaxialCreep_TC02	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/geomechanics/pfw_input_geomechanicsTriaxialCreep_TC02.py
validation	polymer	polymer/polymerCompression	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/polymer/pfw_input_polymerCompression.py
validation	polymer	polymer/polymerTension	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/polymer/pfw_input_polymerTension.py

Chapter G

Generated post-processing script catalogue

Table G.1: Post-processing and suite analysis scripts discovered under ParticleFileWriter.

Script	Path
postProcess_benchy_FMPM	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/benchy/postProcess_benchy_FMPM
visitRender_benchy.py	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/benchy/visitRender_benchy.py
visitRender_borehole_Ghareb.py	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/borehole/visitRender_borehole_Ghareb.py
plotReactions_brazilianDisk_BSpline.py	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_BSpline/plotReactions_brazilianDisk_BSpline.py
postProcess_brazilianDisk_BSpline	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_BSpline/postProcess_brazilianDisk_BSpline
plotReactions_brazilianDisk_FLIP.py	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_FLIP/plotReactions_brazilianDisk_FLIP.py
postProcess_brazilianDisk_FLIP	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_FLIP/postProcess_brazilianDisk_FLIP
plotReactions_brazilianDisk_FMPM.py	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_FMPM/plotReactions_brazilianDisk_FMPM.py
postProcess_brazilianDisk_FMPM	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_FMPM/postProcess_brazilianDisk_FMPM
plotReactions_brazilianDisk_PIC.py	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_PIC/plotReactions_brazilianDisk_PIC.py
postProcess_brazilianDisk_PIC	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_PIC/postProcess_brazilianDisk_PIC
plotReactions_brazilianDisk_XPIC.py	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_XPIC/plotReactions_brazilianDisk_XPIC.py
postProcess_brazilianDisk_XPIC	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/brazilianDisk_XPIC/postProcess_brazilianDisk_XPIC
visitRender_collidingDisks.py	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/collidingDisks/visitRender_collidingDisks.py
visitRender_copperFoamCompression.py	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/copperFoamCompression/visitRender_copperFoamCompression.py

Continued on next page

Script	Path
visitRender_copperFoamCompression2D.py	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/copperFoamCompression/visitRender_copperFoamCompression2D.py
visitRender_elasticDisk.py	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/elasticDisk/visitRender_elasticDisk.py
mpm_example_postprocess.py	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/mpm_example_postprocess.py
visitRender_pdc.py	scripts/preProcessing/materialPointMethodParticleFileWriter/examples/pdc/visitRender_pdc.py
mpm_vv_plot_tools.py	scripts/preProcessing/materialPointMethodParticleFileWriter/mpm_vv_plot_tools.py
mpm_vv_postprocess.py	scripts/preProcessing/materialPointMethodParticleFileWriter/mpm_vv_postprocess.py
pfw_analysis.py	scripts/preProcessing/materialPointMethodParticleFileWriter/pfw_analysis.py
pfw_visit_render.py	scripts/preProcessing/materialPointMethodParticleFileWriter/pfw_visit_render.py
plotReactions_geoOHCrD_2024_ISR08.py	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/geomechanics/plotReactions_geoOHCrD_2024_ISR08.py
plotReactions_geomechanicsHydrostaticCreep.py	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/geomechanics/plotReactions_geomechanicsHydrostaticCreep.py
plotReactions_geomechanicsTriaxialCompression_ISR03.py	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/geomechanics/plotReactions_geomechanicsTriaxialCompression_ISR03.py
plotReactions_geomechanicsTriaxialCompression_TP1_TP2_TP3.py	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/geomechanics/plotReactions_geomechanicsTriaxialCompression_TP1_TP2_TP3.py
plotReactions_geomechanicsTriaxialCreep.py	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/geomechanics/plotReactions_geomechanicsTriaxialCreep.py
plotReactions_polymer.py	scripts/preProcessing/materialPointMethodParticleFileWriter/validation/polymer/plotReactions_polymer.py
plotReactions_elasticBlockUni.py	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/Ftable/plotReactions_elasticBlockUni.py
plotReactions_ceramicTXC.py	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/ceramic/plotReactions_ceramicTXC.py
plotReactions_ceramicTXE.py	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/ceramic/plotReactions_ceramicTXE.py
plotReactions_geomechanicsBuckling.py	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/geomechanics/plotReactions_geomechanicsBuckling.py
plotReactions_geomechanicsDemo.py	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/geomechanics/plotReactions_geomechanicsDemo.py
plotReactions_geomechanicsHYD.py	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/geomechanics/plotReactions_geomechanicsHYD.py
plotReactions_geomechanicsTXC.py	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/geomechanics/plotReactions_geomechanicsTXC.py
plotReactions_geomechanicsTension.py	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/geomechanics/plotReactions_geomechanicsTension.py
plotReactions_geomechanicsUnconfinedBuckling.py	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/geomechanics/plotReactions_geomechanicsUnconfinedBuckling.py
plotReactions_polymerCZ.py	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/polymerCZ/plotReactions_polymerCZ.py
plotReactions_stressControl.py	scripts/preProcessing/materialPointMethodParticleFileWriter/verification/stressControl/plotReactions_stressControl.py

Index